

BỘ GIÁO DỤC ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ VIỄN THÔNG

Đề tài:

THIẾT KẾ VÀ TỐI ƯU KIẾN TRÚC PIPELINE CHO LỖI

AES-128

GVHD: PGS.TS. Phạm Ngọc Sơn

Sinh viên thực hiện:

Mai Nguyễn Hà Phương

MSSV: 22161301

Nguyễn Thị Hải Ngọc

MSSV: 22161296

Thành phố Hồ Chí Minh, tháng 6 năm 2026

BỘ GIÁO DỤC ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ VIỄN THÔNG

Đề tài:

THIẾT KẾ VÀ TỐI ƯU KIẾN TRÚC PIPELINE CHO LỖI

AES-128

GVHD: PGS.TS. Phạm Ngọc Sơn

Sinh viên thực hiện:

Mai Nguyễn Hà Phương

MSSV: 22161301

Nguyễn Thị Hải Ngọc

MSSV: 22161296

Thành phố Hồ Chí Minh, tháng 6 năm 2026

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế và tối ưu kiến trúc pipeline cho lõi AES-128

Sinh viên: + Mai Nguyễn Hà Phương

MSSV: 22161301

+ Nguyễn Thị Hải Ngọc

MSSV: 22161296

Hướng dẫn: PGS.TS. Phạm Ngọc Sơn

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

- Đặt vấn đề hợp lý trong bối cảnh nhu cầu bảo mật dữ liệu cho hệ thống nhúng, SoC, IoT và các nền tảng xử lý tốc độ cao ngày càng cấp thiết. Mục tiêu thiết kế lõi AES-128 dùng Verilog HDL theo kiến trúc pipeline rõ ràng, có ý nghĩa khoa học trong việc vận dụng kiến thức mật mã, thiết kế số và kiến trúc phần cứng, đồng thời có ý nghĩa thực tiễn vì hướng đến cải thiện thông lượng xử lý cho các ứng dụng mã hóa/giải mã thời gian thực.

2. Phương pháp thực hiện/ phân tích/ thiết kế:

- Phương pháp tiếp cận phù hợp: nghiên cứu cơ sở lý thuyết AES-128, phân tích các phép biến đổi SubBytes, ShiftRows, MixColumns, AddRoundKey, Key Expansion; sau đó thiết kế theo hướng topdown ở mức RTL bằng Verilog HDL. Kiến trúc hệ thống được phân rã rõ thành tuyến mã hóa, tuyến giải mã, khối sinh khóa, thanh ghi pipeline và khối điều khiển FSM. Cách tổ chức pipeline 10 tầng cho phép xử lý nhiều khối dữ liệu nối tiếp.

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

- Kết quả thực hiện cơ bản đáp ứng mục tiêu đề tài. Báo cáo đã trình bày thiết kế các module chức năng của lõi AES-128 và kiểm chứng bằng mô phỏng trên Xilinx Vivado với các test case ở cấp module và cấp hệ thống. Các kịch bản kiểm thử như mã hóa/giải mã theo vector chuẩn FIPS, dữ liệu người dùng, hold pipeline bằng enable, reset giữa quá trình xử lý và kiểm tra throughput đều đạt. Đặc biệt, sau khi pipeline được lấp đầy, thiết kế cho khả năng xuất một khối dữ liệu 128-bit ở mỗi chu kỳ xung nhịp, chứng minh ưu điểm về thông lượng của kiến trúc pipeline.

4. Kết luận và đề xuất:

- Kết luận phù hợp với mục tiêu ban đầu: thiết kế và mô phỏng thành công lõi IP AES-128 có cả mã hóa và giải mã, quản lý luồng dữ liệu bằng FSM và xác minh tính đúng đắn bằng mô hình tham chiếu. Các hướng phát triển được nêu có tính thực tế, bao gồm tối ưu tài nguyên phần cứng, bổ sung cơ chế quản lý/trao đổi khóa an toàn, chống tấn công kênh kề và tích hợp vào hệ thống SoC hoặc giao tiếp chuẩn để tăng khả năng ứng dụng.

5. Hình thức trình bày và bố cục báo cáo:

- Báo cáo có bố cục rõ ràng gồm tổng quan, cơ sở lý thuyết, thiết kế hệ thống, mô phỏng - đánh giá, kết luận và hướng phát triển. Hình vẽ, bảng biểu, mô tả tín hiệu và

kết quả mô phỏng được trình bày khá chi tiết, giúp người đọc theo dõi được quá trình từ thuật toán đến hiện thực RTL.

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

- Khả năng làm việc nhóm hiệu quả. Nhóm sinh viên tự nghiên cứu tài liệu, phân chia công việc, thiết kế module phần cứng, xây dựng testbench và phân tích kết quả mô phỏng.

7. Tài liệu trích dẫn

- Các tài liệu tham khảo có liên quan đến chuẩn AES, mật mã học, kiến trúc phần cứng và thiết kế pipeline và được trích dẫn.

8. Đánh giá về sự trùng lặp của đề tài

- Chưa phát hiện trùng lặp với các đề tài trong nước.

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

- Một số nội dung có thể được rút ngắn, đặc biệt nội dung không liên quan trực tiếp đến công việc trong đề tài.
- Nhóm sinh viên cần bổ sung đầy đủ thông tin nguồn cho các tài liệu trực tuyến và đảm bảo danh mục tài liệu tham khảo được định dạng nhất quán.
- Cần rà soát toàn bộ báo cáo và chỉnh sửa lỗi chính tả, thống nhất thuật ngữ, đánh số hình/bảng đầy đủ.

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

- Sinh viên có thái độ học tập và nghiên cứu nghiêm túc, chủ động tìm hiểu kiến thức liên ngành giữa mật mã học, thiết kế số và FPGA. Nhóm đã hoàn thành khối lượng công việc tương đối lớn, thể hiện sự kiên trì trong quá trình mô phỏng, kiểm thử và hoàn thiện báo cáo. Tinh thần hợp tác nhóm tốt, có trách nhiệm với sản phẩm đồ án và nghiêm túc tiếp thu, chỉnh sửa theo góp ý của giảng viên hướng dẫn.

Đề nghị của giảng viên hướng dẫn

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư, và được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 25 tháng 06 năm 2026

Người nhận xét (Ký và ghi rõ họ tên)



PGS.TS. Phạm Ngọc Sơn

LỜI CẢM ƠN

Để hoàn thành tốt đề án này, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy **PGS.TS. Phạm Ngọc Sơn** – người đã luôn tận tình chỉ bảo, hướng dẫn và hỗ trợ chúng em trong suốt quá trình học tập và thực hiện đề tài.

Chúng em vô cùng biết ơn sự tâm huyết và những kiến thức chuyên môn quý báu mà Thầy đã truyền đạt. Nhờ sự chỉ dẫn tận tình của Thầy, chúng em không chỉ hoàn thành được các mục tiêu của đề án mà còn tích lũy thêm nhiều kinh nghiệm thực tế, góp phần xây dựng hành trang vững chắc cho sự nghiệp tương lai.

Chúng em xin kính chúc Thầy luôn dồi dào sức khỏe, giữ mãi ngọn lửa đam mê với nghề và gặt hái được nhiều thành công hơn nữa trong sự nghiệp trồng người cao quý.

Chúng em xin chân thành cảm ơn!

TÓM TẮT

Đồ án tập trung nghiên cứu và thiết kế lõi IP AES-128 tốc độ cao trên nền tảng FPGA dựa trên kiến trúc Pipeline nhằm nâng cao hiệu năng xử lý dữ liệu và đáp ứng yêu cầu bảo mật trong các hệ thống nhúng hiện đại. Hệ thống được hiện thực bằng ngôn ngữ Verilog HDL và mô phỏng trên phần mềm Xilinx Vivado.

Kiến trúc đề xuất sử dụng mô hình Fully Pipelined với nhiều tầng xử lý nối tiếp tương ứng với các vòng của thuật toán AES-128, cho phép nhiều khối dữ liệu được xử lý đồng thời nhằm cải thiện đáng kể thông lượng hệ thống. Đồ án xây dựng đầy đủ các khối chức năng của AES bao gồm Key Expansion, SubBytes, ShiftRows, MixColumns, AddRoundKey và các phép biến đổi nghịch phục vụ giải mã. Ngoài ra, hệ thống còn hỗ trợ đồng thời cả hai chế độ mã hóa và giải mã thông qua hai luồng xử lý dữ liệu độc lập.

Kết quả mô phỏng cho thấy hệ thống hoạt động đúng theo chuẩn AES-128 của NIST, đảm bảo khả năng mã hóa và giải mã chính xác dữ liệu 128-bit. Sau khi pipeline được lấp đầy, hệ thống có khả năng tạo ra một khối dữ liệu đầu ra mới tại mỗi chu kỳ xung nhịp, góp phần nâng cao thông lượng xử lý và cải thiện hiệu năng tổng thể của hệ thống. Đồ án là cơ sở cho việc phát triển các hệ thống bảo mật phần cứng tốc độ cao trên FPGA và các nền tảng nhúng trong tương lai.

MỤC LỤC

LỜI CẢM ƠN.....	iii
TÓM TẮT.....	iv
DANH MỤC HÌNH ẢNH.....	v
DANH MỤC BẢNG.....	xi
CHƯƠNG 1: TỔNG QUAN	1
1.1. Lý do chọn đề tài.....	1
1.2. Mục tiêu nghiên cứu	2
1.3. Đối tượng và phạm vi nghiên cứu.....	2
1.4. Phương pháp nghiên cứu	3
1.5. Bố cục đề tài.....	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	6
2.1. Tổng quan về mật mã học.....	6
2.1.1. Các khái niệm cơ bản.....	6
2.1.2. Mật mã học đối xứng và bất đối xứng	6
2.1.3. Ứng dụng trong hệ thống nhúng và SoC	7
2.2. Thuật toán mã hóa AES (Rijndael).....	7
2.2.1. Cấu trúc tổng quan và quy trình mã hóa AES-128	7
2.2.2. Các phép biến đổi trong mã hóa AES-128.....	12
2.2.3. Quá trình sinh khóa (Key Expansion).....	21
2.3. Giải mã trong thuật toán AES-128.....	24
2.3.1. Quy trình giải mã AES-128	24
2.3.2. Các phép biến đổi trong quá trình giải mã.....	25
2.4. Kiến trúc phần cứng cho AES	33
2.4.1. Iterative architecture	33
2.4.2. Fully unrolled architecture	34
2.4.3. Kiến trúc Pipeline	34
2.4.4 Kiến trúc pipeline cho các vòng mã hóa AES	35
2.4.5 Kiến trúc pipeline cho các vòng giải mã AES	36
2.4.6 Tiêu chí đánh giá hiệu năng	36
2.5. Công cụ và nền tảng thiết kế.....	38

2.5.1. Ngôn ngữ mô tả phần cứng (Verilog).....	38
2.5.2. Công cụ mô phỏng và tổng hợp.....	39
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	41
3.1. Tổng quan hệ thống	41
3.1.1. Yêu cầu thiết kế	41
3.1.2. Sơ đồ khối tổng thể	41
3.2. Thiết kế kiến trúc pipeline AES	43
3.2.1. Mô hình và phân chia pipeline	43
3.2.2. Tổ chức pipeline và luồng dữ liệu	43
3.2.3. Thiết kế khối thanh ghi đường ống (Pipeline Registers)	44
3.3. Thiết kế khối mã hóa AES-128	46
3.3.1. Thiết kế các khối xử lý vòng.....	46
3.3.2. Thiết kế chi tiết các module mã hóa	52
3.4. Thiết kế khối sinh khóa (Key Expansion)	61
3.4.1. Kiến trúc sinh khóa đồng bộ Pipeline.....	62
3.4.2. Thiết kế module sinh khóa đơn vòng	63
3.5. Thiết kế khối giải mã AES	66
3.5.1. Thiết kế các khối xử lý vòng giải mã.....	67
3.5.2. Thiết kế chi tiết các module giải mã	71
3.6. Tích hợp hệ thống	79
3.6.1. Thiết kế khối điều khiển trung tâm (FSM Control)	79
3.6.2. Thiết kế khối aes_core	82
CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ.....	86
4.1. Tổng quan phương pháp kiểm thử và cấu hình.....	86
4.2. Kết quả mô phỏng khối sinh khóa (key expansion).....	87
4.3. Kết quả mô phỏng bộ mã hóa AES-128	89
4.3.1. Kết quả mô phỏng vòng mã hóa tiêu chuẩn (Round 1)	89
4.3.2. Mô phỏng vòng mã hóa cuối cùng.....	95
4.4. Kết quả mô phỏng bộ giải mã AES-128	97
4.4.1. Kết quả mô phỏng vòng giải mã tiêu chuẩn (Round 1)	97
4.4.2. Mô phỏng vòng giải mã cuối cùng	103
4.5. Kết quả mô phỏng khối thanh ghi đường ống (Pipeline Registers).....	105

4.6. Kết quả mô phỏng khối điều khiển trung tâm (FSM Control).....	105
4.7. Kết quả mô phỏng khối tích hợp hệ thống (aes_core)	111
4.7.1. Mục tiêu kiểm thử.....	111
4.7.2. Các kịch bản kiểm thử hệ thống tổng thể	112
4.7.2.1. Kết quả mô phỏng TC1 – Mã hóa AES-128 với vector chuẩn FIPS	115
4.7.2.2. Kết quả mô phỏng TC2 – Giải mã AES-128 với vector chuẩn FIPS	121
4.7.2.3. Kết quả mô phỏng TC3 – Mã hóa AES-128 với dữ liệu người dùng	124
4.7.2.4. Kết quả mô phỏng TC4 – Giải mã AES-128 với dữ liệu cá nhân	128
4.7.2.5. Kết quả mô phỏng TC5 – Kiểm tra chức năng hold pipeline bằng enable	131
4.7.2.6. Kết quả mô phỏng TC6 – Kiểm tra reset giữa quá trình pipeline.....	133
4.7.2.7. Kết quả mô phỏng TC7 – Kiểm tra throughput của pipeline.....	135
4.7.3. Đánh giá hiệu năng, thông lượng (Throughput) và tần số vận hành tối đa	136
4.7.4. Tổng hợp kết quả các test case.....	140
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	141
5.1. Kết luận.....	141
5.2. Hạn chế	141
5.3. Hướng phát triển	141
TÀI LIỆU THAM KHẢO	143

DANH MỤC HÌNH ẢNH

Hình 2.1: Đầu vào, ma trận trạng thái và đầu ra	9
Hình 2.2: Khóa và khóa mở rộng.....	9
Hình 2.3: Minh họa cấu trúc tổng thể của quá trình mã hóa AES.	11
Hình 2.4: Minh họa cấu trúc của một vòng mã hóa hoàn chỉnh.	13
Hình 2.5: Minh họa phép biến đổi AddRoundKey	14
Hình 2.6: Phép thay thế byte.....	15
Hình 2.7: S - box của mã hóa AES	16
Hình 2.8: Minh họa phép biến đổi SubBytes	17
Hình 2.9: Minh họa quá trình dịch hàng.	18
Hình 2.10: Quá trình giải mã của thuật toán AES-128	24
Hình 2.11: Minh họa phép biến đổi InvShiftRows, trong đó ba hàng cuối của ma trận trạng thái được dịch vòng sang phải với các độ dịch khác nhau.	27
Hình 2.12: Bảng Inverse S-box, thể hiện các giá trị thay thế tương ứng với mỗi byte đầu vào được biểu diễn ở dạng thập lục phân.....	29
Hình 2.13: Quá trình xử lý AES theo kiến trúc pipeline	35
Hình 2.14: Giao diện phần mềm thiết kế và mô phỏng Vivado	40
Hình 3.1: Sơ đồ khối mức RTL của hệ thống AES-128 Fully Pipelined	42
Hình 3.2: Giảm đồ thời gian mô tả luồng dữ liệu và thông số của kiến trúc AES-128 Pipeline	43
Hình 3.3: Sơ đồ khối RTL của khối thanh ghi đường ống pipeline_regs.....	45
Hình 3.4: Lưu đồ thuật toán điều khiển của khối pipeline_regs	46
Hình 3.5: Sơ đồ khối RTL của khối khởi tạo enc_ark0	47
Hình 3.6: Lưu đồ thuật toán của module vòng khởi tạo	48
Hình 3.7: Kiến trúc RTL nội bộ của khối vòng tiêu chuẩn aes_round	48
Hình 3.8: Lưu đồ thuật toán của module vòng tiêu chuẩn.....	50
Hình 3.9: Kiến trúc RTL của module vòng kết thúc aes_final_round.....	50

Hình 3.10: Lưu đồ thuật toán của module vòng kết thúc.....	52
Hình 3.11: Sơ đồ khối RTL của module sub_bytes và kiến trúc 16 S-Box song song	53
Hình 3.12: Kiến trúc nội bộ RTL_ROM bên trong một module sbx đơn lẻ.....	54
Hình 3.13: Lưu đồ thuật toán và luồng phân tách dữ liệu của module sub_bytes.....	54
Hình 3.14: Sơ đồ định tuyến tín hiệu RTL của module shift_rows	55
Hình 3.15: Lưu đồ thuật toán của module shift_rows	56
Hình 3.16: Sơ đồ khối RTL của module mix_columns.	57
Hình 3.17: Kiến trúc RTL nội bộ của khối mix_single_column hiện thực hóa thuật toán.....	59
Hình 3.18: Lưu đồ thuật toán toàn khối mix_columns	60
Hình 3.19: Lưu đồ thuật toán module mix_single_column	61
Hình 3.20: Sơ đồ khối RTL tổng thể của kiến trúc sinh khóa	62
Hình 3.21: Sơ đồ kiến trúc RTL của module sinh khóa đơn vòng key_round	64
Hình 3.22: Lưu đồ thuật toán mô tả chuỗi sinh khóa vòng tổng thể.....	65
Hình 3.23: Lưu đồ thuật toán mô tả luồng xử lý của module key_round.....	66
Hình 3.24: Sơ đồ khối RTL của module vòng tiền xử lý giải mã dec_ark10	67
Hình 3.25: Kiến trúc RTL của module vòng giải mã tiêu chuẩn inv_aes_round	68
Hình 3.26: Lưu đồ thuật toán của module vòng giải mã tiêu chuẩn.....	69
Hình 3.27: Kiến trúc RTL của module vòng giải mã kết thúc inv_aes_final_round...	70
Hình 3.28: Lưu đồ thuật toán của khối vòng giải mã kết thúc inv_aes_final_round...	71
Hình 3.29: Sơ đồ định tuyến tín hiệu RTL của khối inv_shift_rows.....	71
Hình 3.30: Lưu đồ thuật toán của khối inv_shift_rows	72
Hình 3.31: Sơ đồ khối RTL của khối inv_sub_bytes.....	73
Hình 3.32: Lưu đồ thuật toán của khối inv_sub_bytes	74
Hình 3.33: Sơ đồ khối RTL của khối inv_mix_columns.....	75
Hình 3.34: Kiến trúc RTL nội bộ của khối inv_mix_single_column hiện thực hóa thuật toán nghịch đảo.....	76

Hình 3.35: Lưu đồ thuật toán toàn khối inv_mix_columns	77
Hình 3.36: Lưu đồ thuật toán phân tách luồng dữ liệu và chuỗi tính toán xtime của module inv_mix_single_column.....	78
Hình 3.37: Sơ đồ RTL kiến trúc nội bộ của khối điều khiển trung tâm fsm_control ..	80
Hình 3.38: Sơ đồ chuyển đổi trạng thái của khối điều khiển trạng thái.....	81
Hình 3.39: Lưu đồ thuật toán luồng điều khiển tích hợp hệ thống của aes_core	83
Hình 3.40: Lưu đồ thuật toán và luồng dữ liệu của tuyến mã hóa AES-128 Pipeline.	84
Hình 3.41: Lưu đồ thuật toán và luồng dữ liệu của tuyến giải mã AES-128 Pipeline.	85
Hình 4.1: Công cụ được sử dụng để đối chiếu.....	86
Hình 4.2: Giao diện thực hiện mã hóa và giải mã kiểm tra với kết quả mô phỏng	86
Hình 4.3: Kết quả tính toán từ Tool kiểm thử cho khối Key Expansion.	88
Hình 4.4: Giải đồ xung mô phỏng chức năng khối mở rộng khóa (Key Expansion) .	88
Hình 4.5: Kết quả sinh khóa và trạng thái các vòng AES-128 trên TCL Console	88
Hình 4.6: Kết quả tính toán từ Tool kiểm thử cho khối AddRoundKey.....	90
Hình 4.7: Giải đồ xung mô phỏng chức năng khối cộng khóa vòng (AddRoundKey)	90
Hình 4.8: Kết quả khối cộng khóa vòng trên TCL Console	90
Hình 4.9: Kết quả tính toán từ Tool kiểm thử cho khối thay thế byte (SubBytes).	91
Hình 4.10: Giải đồ xung mô phỏng chức năng khối thay thế byte (SubBytes)	91
Hình 4.11: Kết quả khối thay thế byte trên TCL Console	91
Hình 4.12: Kết quả tính toán từ Tool kiểm thử cho khối hoán vị dịch hàng (ShiftRows).....	92
Hình 4.13: Giải đồ xung mô phỏng chức năng khối hoán vị dịch hàng (ShiftRows).	92
Hình 4.14: Kết quả khối hoán vị dịch hàng trên TCL Console	92
Hình 4.15: Kết quả tính toán từ Tool kiểm thử cho khối trộn cột (MixColumns).....	93
Hình 4.16: Giải đồ xung mô phỏng chức năng khối trộn cột (MixColumns).....	93
Hình 4.17: Kết quả khối trộn cột trên TCL Console.....	94

Hình 4.18: Kết quả tính toán từ Tool kiểm thử cho khối vòng mã hóa tiêu chuẩn (aes_round)	94
Hình 4.19: Giảm đồ xung mô phỏng chức năng khối vòng mã hóa tiêu chuẩn (aes_round)	95
Hình 4.20: Kết quả khối vòng mã hóa tiêu chuẩn (aes_round) trên TCL Console.....	95
Hình 4.21: Kết quả tính toán từ Tool kiểm thử cho khối vòng mã hóa cuối cùng (aes_final_round)	96
Hình 4.22: Giảm đồ xung mô phỏng chức năng khối vòng mã hóa cuối cùng (aes_final_round)	96
Hình 4.23: Kết quả khối vòng mã hóa cuối cùng (aes_final_round) trên TCL Console	96
Hình 4.24: Kết quả tính toán từ Tool kiểm thử cho khối hoán vị dịch hàng nghịch (InvShiftRows).....	98
Hình 4.25: Giảm đồ xung mô phỏng chức năng khối hoán vị dịch hàng nghịch (InvShiftRows).....	98
Hình 4.26: Kết quả khối hoán vị dịch hàng nghịch (InvShiftRows) trên TCL Console	98
Hình 4.27: Kết quả tính toán từ Tool kiểm thử cho khối thay thế byte ngược (InvSubBytes)	99
Hình 4.28: Giảm đồ xung mô phỏng chức năng khối thay thế byte ngược (InvSubBytes)	99
Hình 4.29: Kết quả khối thay thế byte ngược (InvSubBytes) trên TCL Console.....	99
Hình 4.30: Kết quả tính toán từ Tool kiểm thử cho phép cộng khóa vòng ở vòng giải mã.....	100
Hình 4.31: Kết quả tính toán từ Tool kiểm thử cho khối trộn cột ngược (InvMixColumns)	101
Hình 4.32: Giảm đồ xung mô phỏng chức năng khối trộn cột ngược (InvMixColumns)	101
Hình 4.33: Kết quả khối trộn cột ngược (InvMixColumns) trên TCL Console	102

Hình 4.34: Kết quả tính toán từ Tool kiểm thử cho khối vòng giải mã tiêu chuẩn (inv_aes_round)	102
Hình 4.35: Giảm đồ xung mô phỏng chức năng khối vòng giải mã tiêu chuẩn (inv_aes_round)	103
Hình 4.36: Kết quả khối vòng giải mã tiêu chuẩn trên TCL Console	103
Hình 4.37: Kết quả tính toán từ Tool kiểm thử cho khối vòng giải mã cuối cùng (inv_aes_final_round)	104
Hình 4.38: Giảm đồ xung mô phỏng chức năng khối vòng giải mã cuối cùng (inv_aes_final_round)	104
Hình 4.39: Kết quả khối vòng giải mã cuối cùng trên TCL Console.	104
Hình 4.40: Giảm đồ xung mô phỏng chức năng khối thanh ghi đường ống..... (Pipeline Registers)	105
Hình 4.41: Kết quả khối thanh ghi đường ống (Pipeline Registers) trên TCL Console.	105
Hình 4.42: Giảm đồ xung mô phỏng các test case chuyển đổi trạng thái của module điều khiển trung tâm (fsm_control).	107
Hình 4.43: Log kết quả kiểm thử test case 1 (Khởi tạo hệ thống)	108
Hình 4.44: Log kết quả kiểm thử test case 2 (Kích hoạt nạp khóa).....	109
Hình 4.45: Log kết quả kiểm thử test case 3 (Chờ dữ liệu hợp lệ)	109
Hình 4.46: Log kết quả kiểm thử test case 4 (Xử lý chế độ mã hóa).....	110
Hình 4.47: Log kết quả kiểm thử test case 5 (Xử lý chế độ giải mã)	110
Hình 4.48: Log kết quả kiểm thử test case 6 (Ngắt hệ thống)	111
Hình 4.49: Kết quả dạng sóng mô phỏng ban đầu	117
Hình 4.50: Kết quả dạng sóng mô phỏng cho Testcase1	119
Hình 4.51: Kết quả tính toán từ công cụ kiểm cho các round	120
Hình 4.52: Kết quả tính toán từ công cụ kiểm thử cho ngõ ra.....	120
Hình 4.53: Kết quả mô phỏng cho Testcase1 trên Tcl Console.....	120

Hình 4.54: Kết quả dạng sóng mô phỏng cho Testcase2	122
Hình 4.55: Kết quả tính toán từ công cụ kiểm cho round 10	123
Hình 4.56: Kết quả tính toán từ công cụ kiểm thử cho ngõ ra.....	123
Hình 4.57: Kết quả mô phỏng cho Testcase2 trên Tcl Consol	124
Hình 4.58: Kết quả dạng sóng mô phỏng cho Testcase3	125
Hình 4.59: Kết quả tính toán từ công cụ kiểm cho các round	127
Hình 4.60: Kết quả tính toán từ công cụ kiểm cho ngõ ra	127
Hình 4.61: Kết quả mô phỏng cho Testcase3 trên TCL Console	127
Hình 4.62: Kết quả dạng sóng mô phỏng cho Testcase4	129
Hình 4.63: Kết quả mô phỏng cho Testcase3 trên Tcl Console.....	130
Hình 4.64: Kết quả tính toán từ công cụ kiểm thử cho các round	131
Hình 4.65: Kết quả tính toán từ công cụ kiểm cho ngõ ra	131
Hình 4.66: Kết quả dạng sóng mô phỏng cho Testcase5	132
Hình 4.67: Kết quả dạng sóng mô phỏng cho Testcase6	134
Hình 4.68: Kết quả dạng sóng mô phỏng cho Testcase7 trước khi pipeline đầy.....	135
Hình 4.69: Kết quả dạng sóng mô phỏng cho Testcase7 sau khi pipeline đầy.....	136
Hình 4.70: Kết quả sử dụng tài nguyên FPGA của kiến trúc AES Pipeline.....	137
Hình 4.71: Kết quả phân tích công suất tiêu thụ của kiến trúc AES Pipeline	138
Hình 4.72: Kết quả phân tích timing của kiến trúc AES Pipeline	139

DANH MỤC BẢNG

Bảng 2.1: Bảng tóm tắt các tham số của những phiên bản aes	12
Bảng 2.2: Bảng giá trị Rcon.....	22
Bảng 3.1: Mô tả các tín hiệu của khối pipeline_regs	45
Bảng 3.2: Mô tả các tín hiệu của vòng khởi tạo (enc_ark0)	47
Bảng 3.3: Mô tả các tín hiệu của khối vòng tiêu chuẩn (aes_round).....	49
Bảng 3.4: Mô tả các tín hiệu của module vòng kết thúc (aes_final_round)	51
Bảng 3.5: Mô tả các tín hiệu của module sub_bytes.....	53
Bảng 3.6: Mô tả các tín hiệu của module shift_rows.....	55
Bảng 3.7: Mô tả các tín hiệu giao tiếp của module mix_columns.....	60
Bảng 3.8: Mô tả các tín hiệu của module tổng Key Expansion	63
Bảng 3.9: Mô tả các tín hiệu của module vòng tiền xử lý giải mã (dec_ark0)	67
Bảng 3.10: Mô tả các tín hiệu của module vòng giải mã tiêu chuẩn (inv_aes_round)	68
Bảng 3.11: Mô tả các tín hiệu của khối vòng giải mã kết thúc (inv_aes_final_round).....	70
Bảng 3.12: Mô tả cổng giao tiếp của khối inv_shift_rows	72
Bảng 3.13: Mô tả các tín hiệu của khối inv_sub_bytes	73
Bảng 3.14: Mô tả các tín hiệu giao tiếp của module inv_mix_columns.....	77
Bảng 3.15: Mô tả các tín hiệu giao tiếp của khối fsm_control	79
Bảng 3.16: Mô tả các tín hiệu của khối aes_core	82
Bảng 4.1: Kết quả mở rộng khóa vòng của thuật toán AES-128.....	87
Bảng 4.2: Quá trình biến đổi một vòng mã hóa tiêu chuẩn của AES-128.....	89
Bảng 4.3: Quá trình biến đổi một vòng giải mã tiêu chuẩn của AES-128.....	97
Bảng 4.4. Các thông số mô phỏng của lõi AES-128	112
Bảng 4.5: Danh sách các test case mô phỏng khối AES-128 pipeline	112
Bảng 4.6: Bộ dữ liệu sử dụng trong TC1	115
Bảng 4.7: Bộ dữ liệu sử dụng trong TC2.....	121

Bảng 4.8: Bộ dữ liệu sử dụng trong TC3.....	124
Bảng 4.9. Bộ dữ liệu sử dụng trong TC4.....	128
Bảng 4.10. Tổng hợp kết quả mô phỏng các test case.....	140

CHƯƠNG 1: TỔNG QUAN

1.1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số và sự phát triển mạnh mẽ của các hệ thống điện tử hiện đại, nhu cầu bảo mật thông tin ngày càng trở nên cấp thiết. Các thiết bị nhúng, hệ thống SoC (System-on-Chip), cũng như các ứng dụng trong lĩnh vực IoT, truyền thông và ô tô thông minh đều yêu cầu cơ chế bảo vệ dữ liệu hiệu quả nhằm đảm bảo tính toàn vẹn, tính bảo mật và xác thực thông tin. Trong đó, các thuật toán mã hóa đóng vai trò then chốt, là nền tảng cho các hệ thống bảo mật hiện đại.

Thuật toán mã hóa AES (Advanced Encryption Standard) hiện nay là một trong những tiêu chuẩn mã hóa đối xứng được sử dụng rộng rãi nhất trên thế giới nhờ vào mức độ an toàn cao và hiệu suất tốt[1][3]. AES-128, với độ dài khóa 128-bit, được ứng dụng phổ biến trong nhiều hệ thống thực tế như truyền thông mạng, bảo mật dữ liệu trên thiết bị di động, hệ thống nhúng và các nền tảng phần cứng chuyên dụng. Tuy nhiên, khi triển khai AES trên nền tảng phần mềm, đặc biệt đối với các hệ thống yêu cầu xử lý thời gian thực hoặc thông lượng cao, hiệu năng thường bị giới hạn bởi độ trễ xử lý và khả năng khai thác tài nguyên tính toán song song.

Do đó, việc thiết kế và hiện thực thuật toán AES trên phần cứng, cụ thể là dưới dạng lõi vi mạch (hardware core), trở thành một hướng tiếp cận hiệu quả nhằm cải thiện thông lượng, giảm độ trễ và nâng cao hiệu quả sử dụng tài nguyên hệ thống[4]. Trong các phương pháp thiết kế phần cứng, kiến trúc pipeline được xem là một giải pháp hiệu quả nhằm nâng cao thông lượng hệ thống thông qua khả năng xử lý song song nhiều khối dữ liệu. Tuy nhiên, các thiết kế AES hiện có vẫn còn tồn tại sự đánh đổi giữa thông lượng, độ trễ và mức sử dụng tài nguyên phần cứng. Việc xác định cấu trúc pipeline phù hợp để đạt hiệu năng cao nhưng vẫn đảm bảo khả năng mở rộng và hiệu quả tài nguyên vẫn là một vấn đề cần được nghiên cứu[2].

Bên cạnh đó, trong các hệ thống hiện đại như SoC hoặc các nền tảng xử lý dữ liệu tốc độ cao, việc tích hợp một lõi mã hóa hiệu năng cao, có khả năng mở rộng và tương thích với các chuẩn giao tiếp là một yêu cầu quan trọng. Việc nghiên cứu và xây dựng một kiến trúc AES-128 pipeline không chỉ mang ý nghĩa về mặt lý thuyết mà còn có giá trị thực tiễn cao, góp phần làm nền tảng cho các ứng dụng bảo mật trong các hệ thống nhúng và vi mạch số.

Xuất phát từ những yêu cầu thực tiễn đó, đề tài “**Thiết kế và tối ưu kiến trúc pipeline cho lõi AES-128**” được thực hiện nhằm xây dựng một mô hình lõi mã hóa/giải mã AES-128 trên phần cứng với kiến trúc pipeline, hướng đến việc cải thiện hiệu năng xử lý và tạo tiền đề cho khả năng tích hợp vào các hệ thống SoC trong tương lai. Đề tài không chỉ giúp củng cố kiến thức về thiết kế vi mạch số, ngôn ngữ mô tả phần cứng và kiến trúc máy tính, mà còn tạo nền tảng cho việc nghiên cứu các hệ thống bảo mật phần cứng trong tương lai.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là thiết kế và xây dựng một lõi phần cứng thực hiện mã hóa và giải mã theo chuẩn AES-128 đạt hiệu năng cao, thông qua việc sử dụng ngôn ngữ mô tả phần cứng Verilog HDL và áp dụng kiến trúc pipeline. Thiết kế hướng tới khả năng tối ưu thông lượng xử lý, giảm độ trễ và đảm bảo hiệu quả sử dụng tài nguyên trong các hệ thống số hiện đại.

Để hiện thực hóa mục tiêu chung này, đề tài tập trung vào các mục tiêu cụ thể sau:

- Thứ nhất, nghiên cứu và đề xuất kiến trúc pipeline hoàn chỉnh cho lõi AES 128, trong đó quá trình xử lý được phân chia thành nhiều giai đoạn nhằm tăng khả năng xử lý song song, đồng thời kết hợp phương pháp tính toán trước khóa vòng để giảm độ trễ hệ thống[4].
- Thứ hai, thiết kế và tối ưu các module chức năng cốt lõi của thuật toán AES, bao gồm SubBytes, ShiftRows, MixColumns, AddRoundKey và Key Expansion, đảm bảo phù hợp với triển khai phần cứng và hoạt động hiệu quả trong môi trường pipeline.
- Thứ ba, xây dựng kiến trúc datapath riêng biệt cho quá trình mã hóa và giải mã, đồng thời tích hợp các module thành một hệ thống hoàn chỉnh có cấu trúc rõ ràng và dễ mở rộng.
- Thứ tư, thực hiện mô phỏng, kiểm chứng và đánh giá hiệu năng của thiết kế dựa trên các tiêu chí như thông lượng, độ trễ và mức sử dụng tài nguyên phần cứng thông qua các công cụ tổng hợp, mà không cần triển khai trực tiếp trên phần cứng thực tế.

Thông qua các mục tiêu trên, đề tài hướng tới việc xây dựng một lõi AES-128 pipeline có hiệu năng cao, đáp ứng tiêu chuẩn thuật toán và có khả năng ứng dụng trong các hệ thống nhúng, SoC và các nền tảng xử lý dữ liệu tốc độ cao.

1.3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm:

- Thuật toán mã hóa và giải mã AES-128, bao gồm cấu trúc dữ liệu và các phép biến đổi cốt lõi như SubBytes, ShiftRows, MixColumns, AddRoundKey và Key Expansion.
- Kiến trúc thiết kế phần cứng cho thuật toán AES, đặc biệt là kiến trúc xử lý theo đường ống (pipeline) nhằm nâng cao hiệu năng hệ thống.
- Các phương pháp thiết kế datapath và tổ chức luồng dữ liệu trong hệ thống số sử dụng ngôn ngữ mô tả phần cứng Verilog HDL.
- Các tiêu chí đánh giá hiệu năng của hệ thống phần cứng như thông lượng (throughput), độ trễ (latency) và mức sử dụng tài nguyên (LUT, FF, BRAM).

Phạm vi nghiên cứu

Đề tài được triển khai trong phạm vi thiết kế và mô phỏng hệ thống phần cứng, với các giới hạn cụ thể như sau:

- Thiết kế và hiện thực lõi AES-128 với chức năng mã hóa và giải mã trên mức mô tả phần cứng (RTL) sử dụng Verilog HDL.
- Áp dụng kiến trúc pipeline cho quá trình xử lý dữ liệu, tập trung vào việc phân tích và tối ưu số lượng stage nhằm cải thiện hiệu năng.
- Thực hiện mô phỏng và kiểm chứng chức năng thông qua testbench và các bộ dữ liệu chuẩn, không triển khai trực tiếp trên phần cứng FPGA hoặc ASIC.
- Đánh giá hiệu năng hệ thống thông qua kết quả mô phỏng và tổng hợp (synthesis), bao gồm các chỉ số như tần số hoạt động ước lượng, thông lượng, độ trễ và mức sử dụng tài nguyên.
- Không xem xét các yếu tố bảo mật nâng cao như chống tấn công kênh kề (side-channel attack) hoặc các kỹ thuật tối ưu mức vật lý (physical design).
- Hệ thống được thiết kế ở mức khối độc lập, chưa tích hợp hoàn chỉnh vào một hệ thống SoC thực tế hoặc các giao thức truyền thông cụ thể.

1.4. Phương pháp nghiên cứu

Để đạt được các mục tiêu, đề tài sử dụng phương pháp nghiên cứu kết hợp giữa phân tích lý thuyết, thiết kế hệ thống phần cứng và đánh giá định lượng, được triển khai qua các bước sau:

Nghiên cứu lý thuyết

Trước hết, đề tài tiến hành tổng hợp và phân tích các tài liệu liên quan đến thuật toán mã hóa AES-128, bao gồm cấu trúc dữ liệu, các phép biến đổi cốt lõi (SubBytes, ShiftRows, MixColumns, AddRoundKey) và cơ chế sinh khóa (Key Expansion). Đồng thời, nghiên cứu các kiến trúc triển khai phần cứng cho AES, đặc biệt là kiến trúc pipeline, nhằm làm rõ các phương pháp nâng cao hiệu năng như tăng thông lượng và giảm độ trễ.

Bên cạnh đó, đề tài tìm hiểu về các nguyên lý thiết kế vi mạch số, tổ chức datapath và kỹ thuật mô tả phần cứng bằng ngôn ngữ Verilog HDL để làm nền tảng cho quá trình thiết kế.

Phương pháp thiết kế hệ thống

Đề tài áp dụng phương pháp thiết kế theo hướng top-down, bắt đầu từ mô hình tổng thể của hệ thống AES-128, sau đó phân rã thành các khối chức năng nhỏ hơn. Hệ thống được chia thành các module chính bao gồm:

- Khối xử lý mã hóa (Encryption datapath)
- Khối xử lý giải mã (Decryption datapath)
- Khối sinh khóa (Key Expansion)
- Các module biến đổi cơ bản (SubBytes, ShiftRows, MixColumns, AddRoundKey)

Kiến trúc pipeline được áp dụng bằng cách phân chia quá trình xử lý thành nhiều giai đoạn, ứng với các vòng lặp của thuật toán, kết hợp với việc chèn các thanh ghi pipeline nhằm tăng khả năng xử lý song song và cải thiện hiệu năng tổng thể.

Phương pháp triển khai

Hệ thống được hiện thực ở mức RTL (Register Transfer Level) bằng ngôn ngữ mô tả phần cứng Verilog HDL. Các module được thiết kế độc lập, sau đó tích hợp thành một hệ thống hoàn chỉnh với giao diện rõ ràng.

Quá trình phát triển bao gồm:

- Xây dựng các module chức năng theo đặc tả thuật toán
- Tổ chức luồng dữ liệu và tín hiệu điều khiển trong kiến trúc pipeline
- Tối ưu hóa thiết kế nhằm giảm độ phức tạp logic và nâng cao hiệu năng

Phương pháp kiểm chứng và đánh giá

Đề tài sử dụng phương pháp mô phỏng để kiểm chứng chức năng của hệ thống. Các testbench được xây dựng nhằm kiểm tra hoạt động của từng module và toàn hệ thống, sử dụng các bộ dữ liệu chuẩn theo tiêu chuẩn AES để đảm bảo tính chính xác.

Hiệu năng của thiết kế được đánh giá thông qua các kết quả tổng hợp và mô phỏng, dựa trên các tiêu chí:

- Thông lượng xử lý (throughput)
- Độ trễ (latency)
- Tần số hoạt động ước lượng
- Mức sử dụng tài nguyên phần cứng (LUT, FF, BRAM)

Kết quả đánh giá được sử dụng để phân tích hiệu quả của kiến trúc pipeline đề xuất và so sánh với các phương pháp thiết kế khác.

1.5. Bố cục đề tài

Nội dung của đề tài được tổ chức thành 5 chương chính, với cấu trúc như sau:

Chương 1: Tổng quan

Trình bày lý do chọn đề tài, mục tiêu nghiên cứu, đối tượng và phạm vi nghiên cứu, phương pháp thực hiện, đồng thời giới thiệu bố cục tổng thể của đồ án.

Chương 2: Cơ sở lý thuyết

Trình bày các kiến thức nền tảng liên quan đến thuật toán AES-128, bao gồm cấu trúc và nguyên lý hoạt động của các phép biến đổi, cơ chế sinh khóa, cũng như các kiến trúc triển khai phần cứng, đặc biệt là kiến trúc pipeline.

Chương 3: Thiết kế hệ thống

Trình bày quá trình thiết kế lõi AES-128 theo kiến trúc pipeline, bao gồm xây dựng sơ đồ khối tổng thể, thiết kế các module chức năng, tổ chức datapath và tích hợp hệ thống ở mức RTL.

Chương 4: Kết quả mô phỏng và đánh giá

Trình bày quá trình kiểm chứng chức năng thông qua mô phỏng, xây dựng testbench và phân tích kết quả. Đồng thời đánh giá hiệu năng của thiết kế dựa trên các tiêu chí như thông lượng, độ trễ và mức sử dụng tài nguyên.

Chương 5: Kết luận và hướng phát triển

Tổng kết các kết quả đạt được của đề tài, nêu ra những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về mật mã học

2.1.1. Các khái niệm cơ bản

Mật mã học (Cryptography) là một lĩnh vực khoa học nghiên cứu và ứng dụng các kỹ thuật toán học để bảo vệ thông tin[3]. Mục tiêu chính của mật mã học là đảm bảo an toàn cho dữ liệu trong quá trình lưu trữ và truyền tin thông qua hai quy trình cốt lõi là mã hóa và giải mã.

Mã hóa (Encryption) là quá trình biến đổi thông tin từ dạng ban đầu, có thể đọc và hiểu được, gọi là bản rõ (plaintext), thành một dạng thông tin mới không thể hiểu được nếu không có kiến thức đặc biệt, gọi là bản mã (ciphertext)[3]. Mục tiêu chính của mã hóa là đảm bảo tính bí mật cho dữ liệu, ngăn chặn những bên không được ủy quyền truy cập vào nội dung thông tin. Quá trình này được thực hiện bởi một thuật toán mã hóa, sử dụng một tham số đầu vào là khóa.

Giải mã (Decryption) là quá trình ngược lại với mã hóa, nhằm mục đích khôi phục bản rõ ban đầu từ bản mã. Quá trình giải mã chỉ có thể được thực hiện thành công bởi bên nhận được ủy quyền, người sở hữu khóa giải mã tương ứng[3]. Nếu không có khóa chính xác, bản mã sẽ vẫn là một chuỗi dữ liệu vô nghĩa.

Khóa (Key) là một mẫu thông tin (thường là một chuỗi bit) có vai trò quyết định đến kết quả đầu ra của thuật toán mã hóa và giải mã. Trong hệ mật mã đối xứng, cùng một khóa được sử dụng cho cả hai quá trình. Theo nguyên lý Kerckhoffs, tính bảo mật của một hệ mật mã không nên phụ thuộc vào việc giữ bí mật thuật toán, mà phải phụ thuộc hoàn toàn vào việc giữ bí mật của khóa[3]. Do đó, việc quản lý và trao đổi khóa một cách an toàn là yếu tố tối quan trọng, quyết định đến sự an toàn của toàn bộ hệ thống.

2.1.2. Mật mã học đối xứng và bất đối xứng

Các hệ mật mã hiện đại được chia thành hai loại chính: mã hóa đối xứng và mã hóa bất đối xứng. Trong đó, mật mã học đối xứng, hay còn được gọi là mật mã khóa bí mật, sử dụng một khóa duy nhất cho cả quá trình mã hóa và giải mã. Khóa này phải được bảo mật và chia sẻ an toàn giữa các bên tham gia. Bên gửi dùng khóa bí mật để biến đổi bản rõ thành bản mã, còn bên nhận sử dụng chính khóa đó để khôi phục dữ liệu ban đầu. Do đó, tính an toàn của hệ thống phụ thuộc trực tiếp vào việc bảo vệ và phân phối khóa. Thuật toán AES là đại diện tiêu biểu và được ứng dụng rộng rãi trong thực tế.

Ngược lại, mã hóa bất đối xứng sử dụng một cặp khóa gồm khóa công khai và khóa bí mật, giúp giải quyết vấn đề phân phối khóa an toàn nhưng có chi phí tính toán cao hơn. Trong các hệ thống hiện đại, hai phương pháp này thường được kết hợp để tận dụng ưu điểm của mỗi loại.

2.1.3. Ứng dụng trong hệ thống nhúng và SoC

Trong bối cảnh các hệ thống nhúng và SoC ngày càng được ứng dụng rộng rãi trong IoT, truyền thông không dây và các thiết bị thông minh, yêu cầu bảo mật thông tin trở nên đặc biệt quan trọng nhằm đảm bảo tính bảo mật, toàn vẹn và xác thực dữ liệu. AES-128 được xem là giải pháp phù hợp nhờ khả năng cân bằng giữa độ an toàn và hiệu năng, đồng thời đáp ứng tốt các môi trường hạn chế tài nguyên như cảm biến, thẻ thông minh và thiết bị truyền thông không dây.

AES-128 được ứng dụng rộng rãi trong mã hóa dữ liệu truyền thông, bảo mật lưu trữ và các giao thức như SSL/TLS và VPN, góp phần bảo vệ dữ liệu trong quá trình trao đổi giữa các thiết bị. Tuy nhiên, việc triển khai AES trên hệ thống nhúng vẫn gặp các thách thức về độ trễ xử lý, tiêu thụ năng lượng và tối ưu tài nguyên phần cứng.

Vì vậy, nhiều nghiên cứu hiện nay tập trung vào việc nâng cao hiệu năng AES thông qua các kỹ thuật như xử lý song song, tối ưu luồng dữ liệu và cải tiến kiến trúc thực thi. Những giải pháp này giúp mở rộng khả năng ứng dụng của AES-128 trong các hệ thống nhúng hiện đại, đặc biệt là các hệ thống yêu cầu xử lý thời gian thực và hiệu suất cao.

2.2. Thuật toán mã hóa AES (Rijndael)

2.2.1. Cấu trúc tổng quan và quy trình mã hóa AES-128

Thuật toán AES có tên gốc là Rijndael, là một thuật toán mã hóa khối đối xứng. Nó nhận đầu vào là một khối bản rõ (plaintext) có kích thước cố định 128 bit (16 byte) và một khóa bí mật. Tùy thuộc vào độ dài khóa là 128, 192 hay 256 bit, thuật toán sẽ được gọi tương ứng là AES-128, AES-192, hoặc AES-256.

Nền tảng thiết kế của AES dựa trên một cấu trúc lặp lại gọi là mạng thay thế - hoán vị SPN [3]. Cấu trúc này được xây dựng dựa trên các nguyên tắc về hỗn loạn và khuếch tán do nhà toán học Claude Shannon đề xuất, nhằm tạo ra một thuật toán mã hóa mạnh mẽ và an toàn¹.

¹ https://en.wikipedia.org/wiki/Advanced_Encryption_Standard

Một mạng SPN bao gồm nhiều vòng xử lý, trong đó mỗi vòng áp dụng các lớp biến đổi khác nhau lên dữ liệu nhằm đảm bảo tính an toàn của quá trình mã hóa. Trong thuật toán AES, các lớp biến đổi này bao gồm lớp thay thế phi tuyến tính (SubBytes), lớp khuếch tán tuyến tính (ShiftRows và MixColumns) và lớp kết hợp khóa (AddRoundKey).

Lớp thay thế phi tuyến tính SubBytes là tầng thay thế của AES, có vai trò tạo ra sự **hỗn loạn** trong bản mã. Ở tầng này, mỗi byte trong ma trận trạng thái được thay thế bằng một byte khác thông qua hộp thay thế (S-box) theo một quy tắc định sẵn. Phép biến đổi này làm phức tạp hóa mối quan hệ giữa bản mã và khóa, khiến việc tấn công bằng các phương pháp phân tích tuyến tính hoặc vi sai trở nên khó khăn hơn.

Tiếp theo là lớp khuếch tán tuyến tính, bao gồm hai phép biến đổi **ShiftRows** và **MixColumns**. Đây là lớp hoán vị có nhiệm vụ tạo ra **sự khuếch tán**, tức là lan tỏa ảnh hưởng của một bit đầu vào ra nhiều bit đầu ra. Trong đó, ShiftRows thực hiện việc hoán vị vị trí các byte trong cùng một hàng, còn MixColumns trộn lẫn dữ liệu của các byte trong cùng một cột theo một phép nhân ma trận trong trường hữu hạn $GF(2^8)$. Sự kết hợp của hai phép biến đổi này giúp che giấu các quy luật thống kê trong bản rõ và tăng cường độ phức tạp của bản mã.

Cuối cùng là lớp kết hợp khóa **AddRoundKey**, trong đó dữ liệu trong ma trận trạng thái được kết hợp với khóa vòng (round key) bằng phép toán XOR. Các khóa vòng này được sinh ra từ khóa bí mật ban đầu thông qua quá trình mở rộng khóa (key expansion). Bước này đảm bảo rằng chỉ những người nắm giữ khóa bí mật mới có thể thực hiện quá trình giải mã chính xác thông điệp.

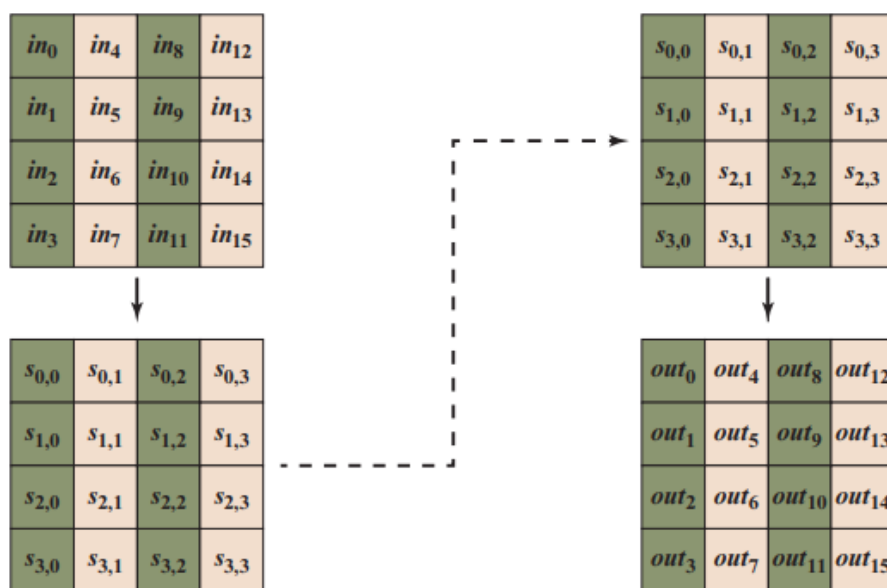
Ba lớp biến đổi trên kết hợp với nhau qua nhiều vòng xử lý tạo nên cấu trúc SPN đặc trưng của AES, giúp đảm bảo cả hai tính chất quan trọng của mật mã hiện đại là **confusion** và **diffusion**, từ đó mang lại độ an toàn cao cho thuật toán.

Biểu diễn dữ liệu: Ma trận trạng thái và lịch trình khóa

Để thực hiện các phép biến đổi một cách hiệu quả và nhất quán, thuật toán AES tổ chức dữ liệu và khóa dưới dạng các ma trận có cấu trúc rõ ràng.

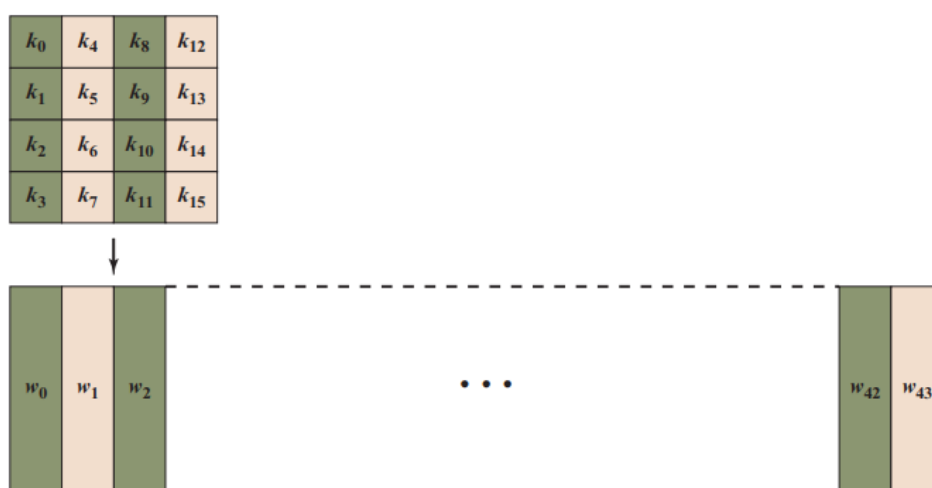
Ma trận trạng thái (State): Khối dữ liệu 128-bit đầu vào được tổ chức thành một ma trận 4x4 byte, được gọi là State. Mọi thao tác trong suốt quá trình mã hóa đều được thực hiện trên ma trận này. Việc sắp xếp 16 byte đầu vào vào ma trận được thực hiện theo thứ tự cột. Bốn byte đầu tiên của bản rõ sẽ chiếm cột đầu tiên, bốn byte tiếp theo chiếm cột thứ hai, và cứ thế tiếp tục. Sau khi vòng lặp cuối cùng hoàn tất, ma trận

State sẽ được sao chép ra để trở thành khối dữ liệu đầu ra (bản mã). Các bước này được mô tả trong Hình 2.1.



Hình 2.1: Đầu vào, ma trận trạng thái và đầu ra

Lịch trình khóa (Key Schedule): Khóa bí mật 128-bit ban đầu được biểu diễn dưới dạng ma trận 4×4 byte. Do mỗi vòng mã hóa yêu cầu một khóa vòng (Round Key) riêng, khóa gốc phải được mở rộng thông qua quá trình Key Expansion (Key Schedule). Đối với AES-128, khóa 128-bit ban đầu được mở rộng thành 44 word (176 byte), tạo ra 11 khóa vòng tương ứng với vòng 0 đến vòng 10. Các khóa vòng này được sử dụng trong quá trình mã hóa nhằm tăng cường tính bảo mật của thuật toán. Quá trình này được minh họa trong Hình 2.2.



Hình 2.2: Khóa và khóa mở rộng

Quy trình mã hóa tổng thể

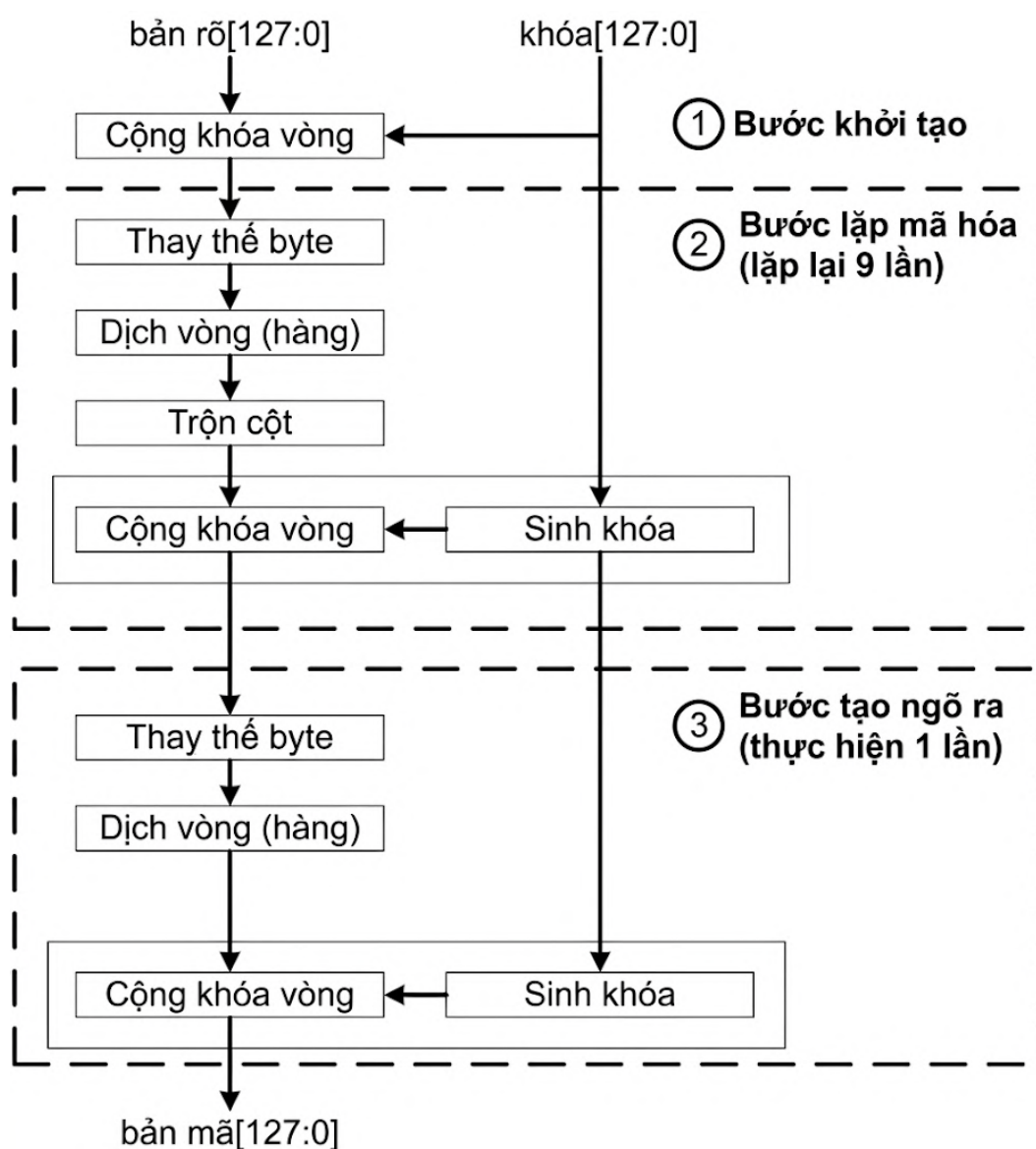
Quá trình mã hóa trong AES-128 được tổ chức theo cấu trúc gồm 10 vòng lặp xử lý, trong đó các phép biến đổi được sắp xếp theo một trật tự chặt chẽ nhằm đảm bảo tính hỗn loạn và khuếch tán của dữ liệu. Toàn bộ quy trình có thể được chia thành ba giai đoạn chính, như minh họa trong Hình 2.3.

Ở giai đoạn khởi tạo (vòng 0), thuật toán bắt đầu bằng phép biến đổi AddRoundKey [4]. Trong bước này, khối bản rõ 128 bit được ánh xạ vào ma trận trạng thái và sau đó kết hợp trực tiếp với khóa bí mật ban đầu thông qua phép XOR. Đây là bước quan trọng nhằm ràng buộc dữ liệu đầu vào với khóa, đồng thời tạo ra ma trận trạng thái ban đầu cho các vòng xử lý tiếp theo.

Tiếp theo là giai đoạn các vòng lặp chính, bao gồm từ vòng 1 đến vòng 9 [3]. Mỗi vòng lặp trong giai đoạn này đều thực thi bốn phép biến đổi theo thứ tự cố định: SubBytes, ShiftRows, MixColumns, và AddRoundKey. Trong đó, SubBytes đảm nhiệm vai trò tạo ra tính phi tuyến nhằm tăng cường độ hỗn loạn; ShiftRows và MixColumns thực hiện khuếch tán dữ liệu thông qua các phép hoán vị và biến đổi tuyến tính; còn AddRoundKey giúp kết hợp ma trận trạng thái với khóa vòng được sinh ra từ quá trình mở rộng khóa. Việc lặp lại chuỗi biến đổi này qua nhiều vòng đảm bảo rằng ảnh hưởng của từng bit trong bản rõ được lan tỏa rộng khắp, qua đó làm tăng độ phức tạp và tính an toàn của bản mã.

Vòng lặp thứ 10, cũng là vòng cuối cùng của quy trình mã hóa AES-128, thực hiện ba phép biến đổi: SubBytes, ShiftRows và AddRoundKey [3]. Phép MixColumns được lược bỏ ở vòng này nhằm đảm bảo tính đối xứng với quá trình giải mã, từ đó hỗ trợ việc khôi phục chính xác bản rõ trong giai đoạn giải mã.

Sau khi hoàn tất vòng 10, ma trận trạng thái cuối cùng được xuất ra ngoài tạo thành khối bản mã (ciphertext) hoàn chỉnh, đại diện cho dữ liệu đã được bảo vệ bằng thuật toán mã hóa khối đối xứng AES-128.



Hình 2.3: Minh họa cấu trúc tổng thể của quá trình mã hóa AES.

Các tham số chính

Thuật toán AES được đặc trưng bởi một số tham số cốt lõi quyết định mức độ bảo mật và độ phức tạp tính toán của hệ thống [3]. Các tham số này bao gồm *kích thước khối*, *kích thước khóa*, và *số vòng lặp* tương ứng của thuật toán.

Trước hết, *kích thước khối* (*Block Size*) của AES được cố định ở mức 128 bit, tương đương 16 byte. Điều này có nghĩa là trong mỗi lần mã hóa hoặc giải mã, thuật toán sẽ xử lý dữ liệu theo từng khối 128 bit. Việc duy trì kích thước khối cố định giúp đảm bảo tính thống nhất trong cấu trúc hoạt động và thiết kế của thuật toán.

Bên cạnh đó, *kích thước khóa* (*Key Size*) và *số vòng lặp* (*Nr*) là hai tham số có mối quan hệ mật thiết [4]. Đối với AES-128, kích thước khóa là 128 bit và thuật toán thực hiện 10 vòng lặp trong quá trình mã hóa. Sự kết hợp giữa độ dài khóa và số vòng

lập giúp đảm bảo khả năng khuếch tán và hỗn loạn của dữ liệu, từ đó tăng cường mức độ an toàn trước các phương pháp tấn công mật mã.

Để làm rõ mối quan hệ giữa các tham số cấu hình, Bảng 2.1 trình bày các tham số chính của AES-128.

Bảng 2.1: Bảng tóm tắt các tham số của những phiên bản aes

Tham số	AES-128
Kích thước khối (bit)	128
Kích thước khóa (bit)	128
Số từ 32-bit trong khóa (Nk)	4
Số vòng lặp (Nr)	10
Kích thước khóa vòng (bit)	128
Số từ 32-bit trong khóa mở rộng	44
Kích thước khóa mở rộng (byte)	176

Trong đó:

Nk (Number of key words): Là số lượng từ 32-bit trong khóa bí mật ban đầu.

Nr (Number of rounds): Là tổng số vòng lặp mà thuật toán sẽ thực hiện.

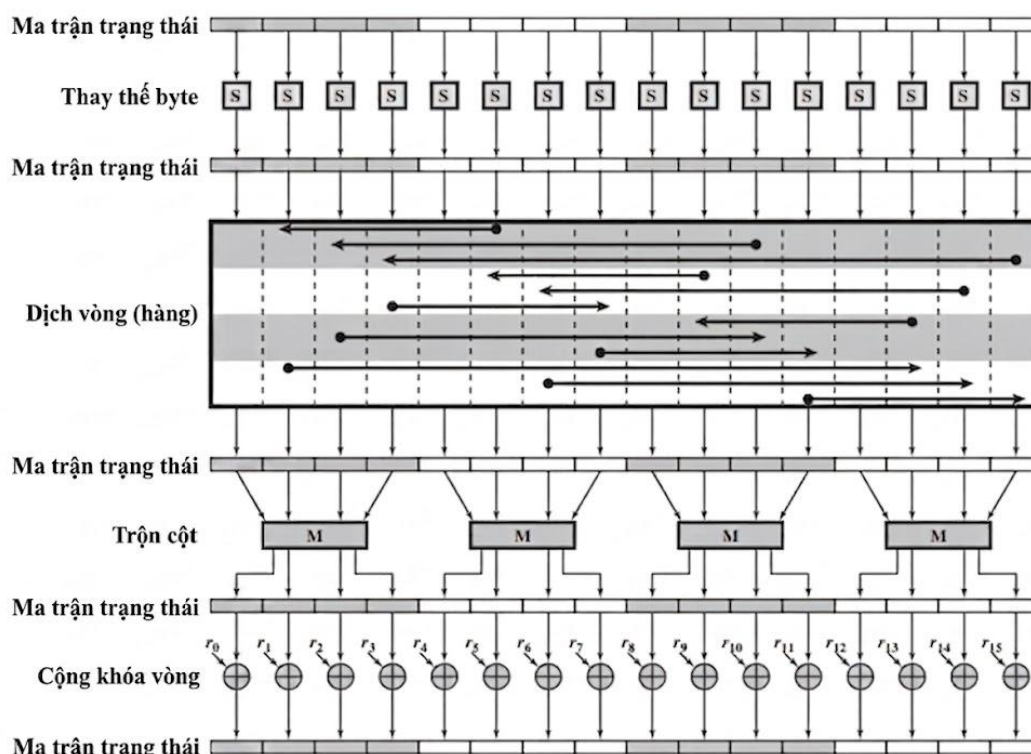
Kích thước khóa mở rộng: Là tổng dung lượng bộ nhớ cần thiết để lưu trữ tất cả các khóa vòng được sinh ra từ quá trình Key Expansion. Với AES-128, cần có 11 khóa vòng (cho Vòng 0 đến Vòng 10), mỗi khóa 128 bit (16 byte, hay 4 word), tổng cộng là $11 \times 4 = 44$ word.

2.2.2. Các phép biến đổi trong mã hóa AES-128

Như đã trình bày trong phần quy trình tổng thể, thuật toán mã hóa AES bao gồm N vòng lặp, trong đó N-1 vòng đầu tiên có cấu trúc đầy đủ và vòng cuối cùng được rút gọn bằng cách loại bỏ phép biến đổi MixColumns; mỗi vòng lặp được xây dựng từ bốn phép biến đổi riêng biệt, nhận đầu vào là ma trận trạng thái State kích thước 4×4 byte và tạo ra một ma trận State mới làm đầu vào cho vòng tiếp theo [2][3]. Phép biến đổi đầu tiên là SubBytes, sử dụng hộp S-box để thay thế từng byte trong ma trận trạng thái bằng một giá trị khác theo quy tắc phi tuyến nhằm đảm bảo mức độ hỗn loạn của thuật toán; tiếp theo là ShiftRows, một phép hoán vị trong đó các hàng của ma trận được dịch vòng sang trái theo số vị trí xác định để lan tỏa ảnh hưởng của các byte và tăng cường tính khuếch tán. Sau đó, phép biến đổi MixColumns được áp dụng để trộn lẫn các byte trong từng cột của ma trận, được thực hiện trong trường hữu hạn $GF(2^8)$ nhằm đảm bảo mỗi byte đầu ra là tổ hợp tuyến tính của bốn byte đầu vào trong cùng một cột, từ đó

tăng cường độ khuếch tán của thuật toán. Cuối cùng là AddRoundKey, bước kết hợp giữa dữ liệu hiện tại và khóa vòng thông qua phép toán XOR từng bit, trong đó khóa vòng được trích ra từ khóa mở rộng, giúp đảm bảo quá trình mã hóa hoàn toàn phụ thuộc vào khóa bí mật ban đầu.

Bốn phép biến đổi trên, khi kết hợp trong từng vòng lặp, hình thành cấu trúc mạng thay thế-hoán vị, là nền tảng đảm bảo độ an toàn của thuật toán AES. Cấu trúc của một vòng mã hóa AES đầy đủ được minh họa trong Hình 2.4.



Hình 2.4: Minh họa cấu trúc của một vòng mã hóa hoàn chỉnh.

AddRoundKey (Cộng với khóa vòng): AddRoundKey là bước cuối cùng trong mỗi vòng lặp của thuật toán AES và cũng là *bước đầu tiên* ở vòng khởi tạo [3]. Đây là phép biến đổi duy nhất trong toàn bộ quá trình mã hóa mà khóa bí mật trực tiếp tham gia vào dữ liệu.

Nguyên tắc hoạt động: ở bước này, ma trận trạng thái hiện tại (gồm 16 byte) được kết hợp với *khóa vòng* (Round Key) có cùng kích thước 128 bit. Khóa vòng được tổ chức dưới dạng 4 từ 32 bit: W_0, W_1, W_2, W_3 . Mỗi từ này tương ứng với một cột trong ma trận trạng thái. Thông qua phép toán XOR đảm bảo rằng dù kẻ tấn công có thể hiểu rõ toàn bộ cấu trúc của thuật toán, họ vẫn không thể giải mã được bản mã nếu không có khóa chính xác.

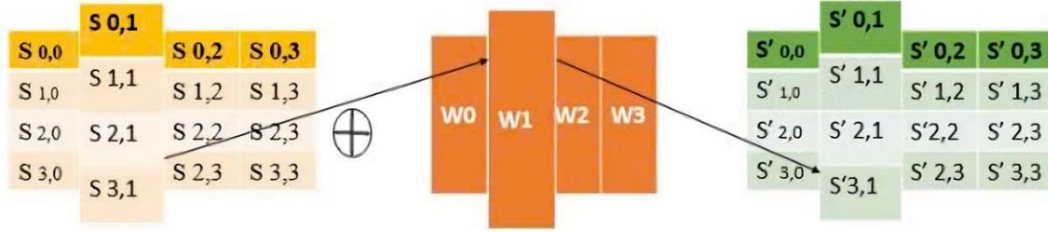
Biểu thức toán học cho phép biến đổi AddRoundKey như sau:

$$S' = S \oplus K_{\text{round}} \quad (2.1)$$

hoặc chi tiết theo cột:

$$\begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \oplus [W_0 \ W_1 \ W_2 \ W_3] \quad (2.2)$$

Nguyên lý thực hiện phép biến đổi AddRoundKey trên ma trận trạng thái và khóa vòng được minh họa trong Hình 2.5, cho thấy quá trình kết hợp dữ liệu và khóa bằng phép toán XOR theo từng byte tương ứng.



Hình 2.5: Minh họa phép biến đổi AddRoundKey

Ý nghĩa và đặc tính: Phép toán XOR đóng vai trò quan trọng trong thuật toán AES nhờ đặc tính đối xứng của nó; cụ thể, khi cùng một khóa được áp dụng hai lần bằng phép XOR, dữ liệu sẽ trở về giá trị ban đầu theo tính chất $A \oplus B \oplus B = A$, qua đó góp phần đảm bảo sự đối xứng giữa quá trình mã hóa và giải mã. Trong mỗi vòng lặp của quá trình mã hóa, một khóa vòng khác nhau được sử dụng, các khóa vòng này được tạo ra từ khóa gốc thông qua quá trình mở rộng khóa (Key Expansion), nhờ đó đảm bảo rằng ngay cả khi các khối bản rõ giống nhau thì các khối bản mã thu được vẫn hoàn toàn khác nhau. Bên cạnh đó, phép XOR là một phép toán đơn giản nhưng hiệu quả, dễ triển khai trên các nền tảng phần cứng như FPGA hoặc ASIC, đồng thời vẫn duy trì được tính đối xứng và hiệu quả trong cả hai quá trình mã hóa và giải mã.

Ví dụ minh họa: Giả sử ta có ma trận trạng thái (State) và khóa vòng (Round Key) như sau

$$\text{State} = \begin{bmatrix} 32 & 88 & 31 & e0 \\ 43 & 5a & 31 & 37 \\ f6 & 30 & 98 & 07 \\ a8 & 8d & a2 & 34 \end{bmatrix}, \text{RoundKey} = \begin{bmatrix} 2b & 28 & ab & 09 \\ 7e & ae & f7 & cf \\ 15 & d2 & 15 & 4f \\ 16 & a6 & 88 & 3c \end{bmatrix} \quad (2.3)$$

Thực hiện phép XOR từng phần tử tương ứng:

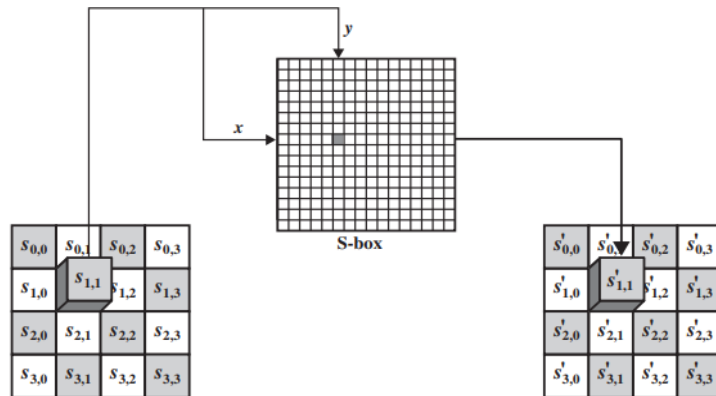
$$\begin{aligned}
 32 \oplus 2b &= 19 & 88 \oplus 28 &= a0 & 31 \oplus ab &= 9a & e0 \oplus 09 &= e9 \\
 43 \oplus 7e &= 3d & 5a \oplus ae &= f4 & 31 \oplus f7 &= c6 & 37 \oplus cf &= f8 \\
 f6 \oplus 15 &= e3 & 30 \oplus d2 &= e2 & 98 \oplus 15 &= 8d & 07 \oplus 4f &= 48 \\
 a8 \oplus 16 &= be & 8d \oplus a6 &= 2b & a2 \oplus 88 &= 2a & 34 \oplus 3c &= 08
 \end{aligned} \quad (2.4)$$

Kết quả, ta thu được ma trận trạng thái mới sau bước AddRoundKey:

$$State' = \begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix} \quad (2.5)$$

Đây chính là đầu ra của bước AddRoundKey, đồng thời cũng là đầu vào cho vòng mã hóa tiếp theo.

SubBytes (thay thế byte) - Hộp S-Box: Đây là bước thay thế phi tuyến tính duy nhất trong thuật toán AES, đóng vai trò cốt lõi trong việc tạo ra tính hỗn loạn (confusion) [4]. Trong bước này, mỗi byte của ma trận trạng thái được thay thế bằng một byte khác thông qua hộp S-Box, là một bảng tra cứu (lookup table) có kích thước 16×16 . Bảng S-Box chứa một hoán vị của toàn bộ 256 giá trị 8-bit, đảm bảo không có giá trị nào bị trùng lặp. Quá trình thay thế được thực hiện theo quy tắc: bốn bit cao của byte đầu vào được sử dụng làm chỉ số hàng, trong khi bốn bit thấp được dùng làm chỉ số cột. Hai chỉ số này xác định vị trí phần tử tương ứng trong bảng S-Box, từ đó tra ra giá trị 8-bit mới để thay thế cho byte ban đầu trong ma trận trạng thái. Phép thay thế byte trong thuật toán AES được minh họa trong Hình 2.6.



Hình 2.6: Phép thay thế byte

Bảng S-Box của AES không được xây dựng một cách ngẫu nhiên mà dựa trên cơ sở toán học chặt chẽ. Cụ thể, mỗi byte đầu vào trước tiên được thực hiện phép nghịch đảo nhân trong trường hữu hạn $GF(2^8)$ (trừ trường hợp byte bằng 0 được ánh xạ riêng),

sau đó kết quả thu được được biến đổi thông qua một phép biến đổi affine trên GF(2). Phép biến đổi này được biểu diễn như sau:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i \quad (2.6)$$

(với $0 \leq i < 80$)

trong đó, b_i là bit thứ i của byte đầu vào, và c_i là bit thứ i của hằng số $c = \{63\}$ (trong đương $\{01100011\}$ trong hệ nhị phân. Dấu phẩy trên biến (ví dụ b') biểu thị giá trị sau khi biến đổi.

Sự kết hợp giữa phép nghịch đảo trong GF(2⁸) và phép biến đổi affine giúp S-Box đạt được các đặc tính mật mã quan trọng như tính phi tuyến cao và khả năng chống lại các dạng tấn công phổ biến như tấn công vi phân và tấn công tuyến tính [3]. Cấu trúc chi tiết của bảng S-Box được trình bày trong Hình 2.7.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	34	64	5d	19	73
	9	60	31	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	05	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	27	6d	8d	d5	4a	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a8	b4	c6	a8	3d	74	1f	4b	b1	8b	9a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Hình 2.7: S - box của mã hóa AES

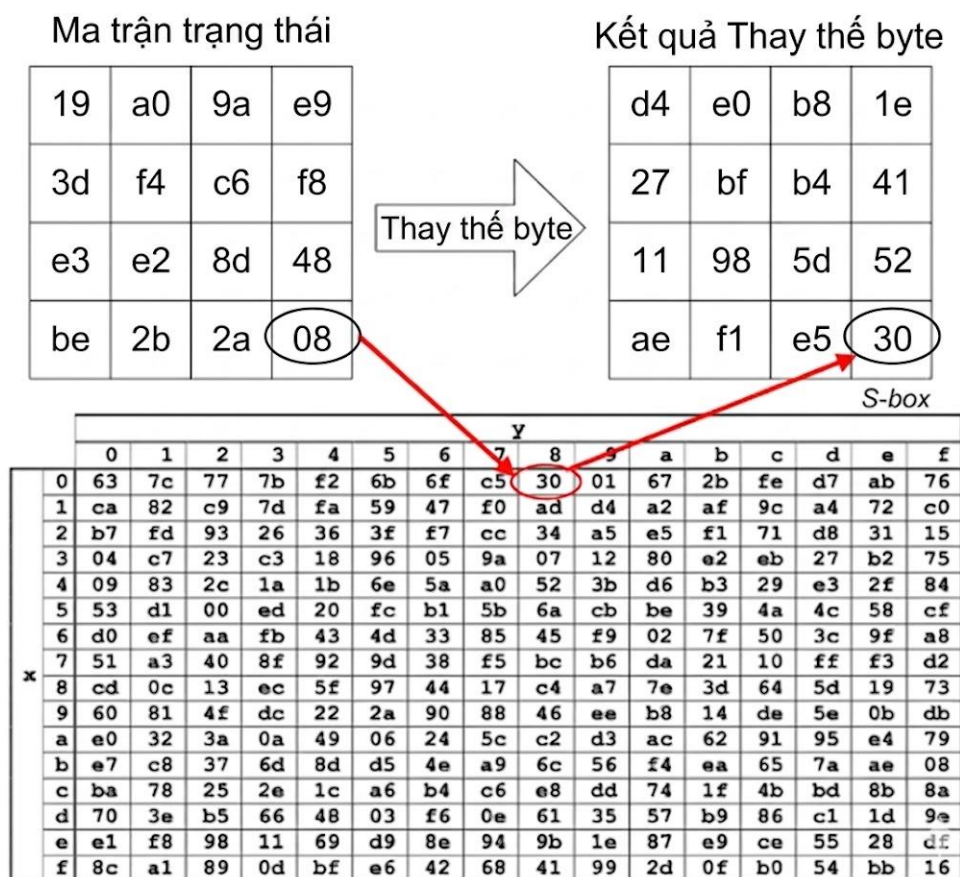
Ví dụ minh họa: Sau khi tổ chức dữ liệu ở dạng ma trận trạng thái và thực hiện phép XOR giữa từng byte trong ma trận với khóa vòng (AddRoundKey), ta thu được ma trận đầu vào cho bước SubBytes. Khi áp dụng phép biến đổi này, mỗi phần tử trong ma trận được thay thế bằng một giá trị mới thông qua bảng S-box:

$$\begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix}$$

Chẳng hạn, xét phần tử đầu tiên trong cột thứ nhất có giá trị là 19_{16} . Ta tách 4 bit cao của giá trị này làm chỉ số hàng (1) và 4 bit thấp làm chỉ số cột (9). Dựa vào bảng S-box, tại hàng 1 và cột 9 ta tra được giá trị $d4_{16}$. Do đó, $19_{16} \rightarrow d4_{16}$.

Tiếp theo, phần tử thứ hai trong cột này có giá trị $3d_{16}$. Thực hiện tương tự, hàng là 3 và cột là d, tra bảng được giá trị 27_{16} . Suy ra $3d_{16} \rightarrow 27_{16}$. Xét tiếp phần tử $e3_{16}$, hàng tương ứng là e và cột là 3, kết quả tra bảng là 11_{16} , nghĩa là $e3_{16} \rightarrow 11_{16}$. Cuối cùng, với phần tử be_{16} , hàng là b và cột là e, tra được giá trị ae_{16} , nên $be_{16} \rightarrow ae_{16}$.

Tương tự với các cột còn lại, ta được kết quả như Hình 2.8:

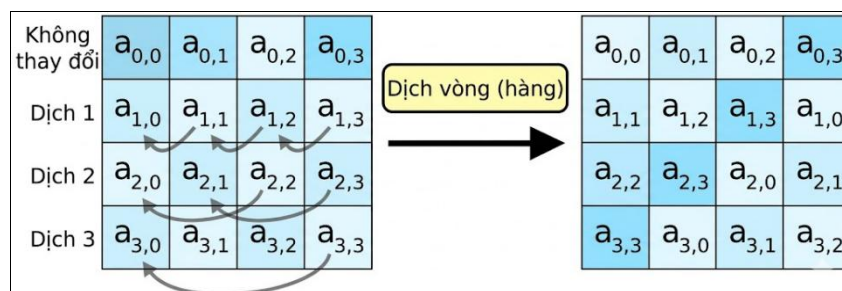


Hình 2.8: Minh họa phép biến đổi SubBytes

ShiftRows (Dịch hàng): đây là một phép hoán vị, thực hiện dịch vòng các byte trên các hàng của ma trận trạng thái để tạo ra sự khuếch tán theo chiều ngang [2]. Mục đích của phép biến đổi này là đảm bảo rằng các byte trong cùng một cột của ma trận ở vòng hiện tại sẽ được phân tán sang các cột khác nhau trong vòng tiếp theo, giúp dữ liệu được phân tán tốt hơn và tăng tính bảo mật của quá trình mã hóa [3].

Trong phép biến đổi ShiftRows, mỗi hàng của ma trận trạng thái được dịch vòng sang trái một số vị trí nhất định, tùy thuộc vào chỉ số hàng. Cụ thể, hàng thứ nhất (hàng 0) được giữ nguyên, hàng thứ hai (hàng 1) được dịch vòng sang trái một byte, hàng thứ ba (hàng 2) được dịch vòng sang trái hai byte, và hàng thứ tư (hàng 3) được dịch vòng sang trái ba byte. Các byte bị dịch ra khỏi đầu hàng sẽ được đưa trở lại cuối hàng, đảm

bảo số lượng phần tử trong mỗi hàng không thay đổi. Quá trình dịch hàng trong thuật toán AES được minh họa trong Hình 2.9.



Hình 2.9: Minh họa quá trình dịch hàng.

Ví dụ minh họa: Từ kết quả của quá trình biến đổi SubBytes, ta tiến hành dịch hàng như sau:

$$\begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix}$$

Trong bước này, hàng đầu tiên của ma trận (hàng 0) được giữ nguyên vì không thực hiện phép dịch $[d4 \ e0 \ b8 \ 1e]$

Hàng thứ hai (hàng 1) được dịch vòng sang trái một byte, tức là phần tử đầu tiên của hàng được chuyển về cuối hàng, còn các phần tử còn lại lần lượt được đẩy sang trái $[bf \ b4 \ 41 \ 27]$

Tiếp đến, hàng thứ ba (hàng 2) được dịch vòng sang trái hai byte, nghĩa là hai phần tử đầu tiên của hàng được chuyển về cuối hàng theo đúng thứ tự ban đầu $[5d \ 52 \ 11 \ 98]$

Cuối cùng, hàng thứ tư (hàng 3) được dịch vòng sang trái ba byte, tương đương với việc chỉ có phần tử cuối cùng của hàng được đưa lên đầu hàng $[30 \ ae \ f1 \ e5]$

Sau khi hoàn tất các phép dịch như trên, ta thu được một ma trận trạng thái mới,

$$\begin{bmatrix} d4 & e0 & b8 & 1e \\ bf & b4 & 41 & 27 \\ 5d & 52 & 11 & 98 \\ 30 & ae & f1 & e5 \end{bmatrix}$$

trong đó vị trí của các byte đã bị thay đổi so với ban đầu:

MixColumns (Trộn cột): Phép biến đổi này tạo ra sự khuếch tán mạnh mẽ theo chiều dọc, hoạt động độc lập trên từng cột của ma trận trạng thái [5].

Hoạt động dựa trên ma trận trạng thái theo từng cột, xem mỗi cột như một đa thức bốn số hạng, trong đó mỗi byte tương ứng với một hệ số của đa thức. Các phép toán được thực hiện trên trường hữu hạn Galois Field $GF(2^8)$ [4] để đảm bảo tính khả nghịch của quá trình mã hóa và giải mã.

Giả sử cột hiện tại của ma trận State là $s(x)$, phép trộn cột sẽ nhân đa thức này với một đa thức cố định $a(x)$, rồi lấy phần dư theo môđun x^4+1 . Đa thức cố định này được định nghĩa như sau:

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\} \quad (2.7)$$

Kết quả thu được là một cột mới $s'(x) = a(x) \oplus s(x) \bmod (x^4+1)$, tương ứng với các byte mới trong ma trận trạng thái sau khi trộn.

Biểu diễn dạng ma trận

Để thuận tiện cho việc tính toán, phép nhân đa thức này được biểu diễn lại dưới dạng *phép nhân ma trận*. Mỗi cột của ma trận trạng thái $[s_0, s_1, s_2, s_3]^T$ sẽ được nhân với một *ma trận hằng số cố định*, tạo thành cột mới $[s'_0, s'_1, s'_2, s'_3]^T$:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \times \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad (2.8)$$

Dựa trên phép nhân ma trận ở trên, mỗi byte mới trong một cột (kí hiệu là s') được tính toán từ bốn byte cũ trong cùng cột đó (kí hiệu là s) theo các công thức sau:

$$\begin{aligned} s'_{0,c} &= (\{02\} \cdot s_{0,c}) \oplus (\{03\} \cdot s_{1,c}) \oplus s_{2,c} \oplus s_{3,c} \\ s'_{1,c} &= s_{0,c} \oplus (\{02\} \cdot s_{1,c}) \oplus (\{03\} \cdot s_{2,c}) \oplus s_{3,c} \\ s'_{2,c} &= s_{0,c} \oplus s_{1,c} \oplus (\{02\} \cdot s_{2,c}) \oplus (\{03\} \cdot s_{3,c}) \\ s'_{3,c} &= (\{03\} \cdot s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \cdot s_{3,c}) \end{aligned} \quad (2.9)$$

Trong đó:

$\oplus$ là phép toán XOR.

$\cdot$ là phép nhân trong trường $GF(2^8)$. Phép nhân với $\{01\}$ là phép đồng nhất (giữ nguyên byte), nhân với $\{02\}$ tương đương dịch trái một bit (và XOR với 0x1B nếu có tràn), và nhân với $\{03\}$ là kết hợp của phép nhân với $\{02\}$ và XOR với chính nó.

Lưu ý: Bước trộn cột được bỏ qua trong vòng lặp cuối cùng của thuật toán.

Ví dụ minh họa: Từ kết quả của quá trình dịch hàng, ta tiến hành trộn cột. Giả sử lấy cột đầu tiên của ma trận trạng thái làm đầu vào $[d4 \quad bf \quad 5d \quad 32]^T$

$$\begin{bmatrix} d4 & e0 & b8 & 1e \\ bf & b4 & 41 & 27 \\ 5d & 52 & 11 & 98 \\ 30 & ae & f1 & e5 \end{bmatrix}$$

Thực hiện phép nhân ma trận trong trường $GF(2^8)$ theo từng cột, ta có:

$$\begin{bmatrix} A \\ B \\ C \\ D \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \times \begin{bmatrix} d4 \\ df \\ 5d \\ 30 \end{bmatrix} \quad (2.10)$$

$$\begin{aligned} A &= (02 \cdot d4) \oplus (03 \cdot bf) \oplus 5d \oplus 30 \\ B &= d4 \oplus (02 \cdot bf) \oplus (03 \cdot 5d) \oplus 30 \\ C &= d4 \oplus bf \oplus (02 \cdot 5d) \oplus (03 \cdot 30) \\ D &= (03 \cdot d4) \oplus bf \oplus 5d \oplus (02 \cdot 30) \end{aligned} \quad (2.11)$$

Để khôi phục byte đầu tiên của cột (phần tử A), ta áp dụng công thức tổ hợp tuyến tính:

$$A = (02 \cdot d4) \oplus (03 \cdot bf) \oplus 5d \oplus 30 \quad (2.12)$$

Trong trường $GF(2^8)$, phép nhân với hằng số $\{02\}$ tương đương với phép dịch trái 1 bit; nếu bit đầu (MSB) bằng 1, ta thực hiện XOR thêm với đa thức bất khả quy $0x1b$. Phép nhân với 03 được phân tích thành $(02 \cdot x) \oplus x$. Chi tiết các bước tính toán như sau:

- **Bước 1:** Tính toán phần tử $02 \cdot d4$

Dạng nhị phân của $d4$ là 1101 0100. Do MBS là 1, ta thực hiện dịch trái một bit và XOR với hằng số $0x1b$:

$$(02 \cdot d4) = (d4 \ll 1) \oplus 0x1b = 1010 \ 1000 \oplus 0001 \ 1011 = 1011 \ 0011 = 0xb3 \quad (2.13)$$

- **Bước 2:** Tính toán phần tử $03 \cdot bf$ theo quy tắc: $03 \cdot bf = (02 \cdot bf) \oplus bf$

Dạng nhị phân của bf là 1011 1111. Do MBS là 1, ta thực hiện dịch trái một bit và XOR với hằng số $0x1b$:

$$(02 \cdot bf) = (bf \ll 1) \oplus 0x1b = 0111 \ 1110 \oplus 0001 \ 1011 = 0110 \ 0101 = 0x65 \quad (2.14)$$

$$\text{Khi đó: } 03 \cdot bf = 65 \oplus bf = 0x65 \oplus 0xbf = 0xda$$

- **Kết quả phần tử A:**

Tiến hành XOR tất cả các kết quả từ (2.13) và (2.14), ta có:

$$A = 0xb3 \oplus 0xda \oplus 0x5d \oplus 0x30 = 0x04$$

Thực hiện các bước tính toán tương tự cho các phần tử còn lại của cột (B, C, D)

theo các phương trình:

$$B = d4 \oplus (02 \cdot bf) \oplus (03 \cdot 5d) \oplus 30 \quad (2.15)$$

$$C = d4 \oplus bf \oplus (02 \cdot 5d) \oplus (03 \cdot 30) \quad (2.16)$$

$$D = (03 \cdot d4) \oplus bf \oplus 5d \oplus (02 \cdot 30) \quad (2.17)$$

Sau khi áp dụng quy tắc dịch bit và XOR tương tự, ta sẽ nhận được kết quả lần lượt là 66_{16} , 81_{16} và $e5_{16}$.

Như vậy, ma trận thu được sau phép biến đổi trộn cột là $[04 \ 66 \ 81 \ E5]^T$.

Quá trình tính toán này được áp dụng lặp lại và độc lập cho 3 cột còn lại của ma

trận trạng thái để cho ra kết quả mã hóa như sau:

$$\begin{bmatrix} 04 & e0 & 48 & 28 \\ 66 & cb & f8 & 06 \\ 81 & 19 & d3 & 26 \\ e5 & 9a & 7a & 4c \end{bmatrix}$$

2.2.3. Quá trình sinh khóa (Key Expansion)

Trong thuật toán AES, khóa bí mật ban đầu không được sử dụng trực tiếp cho tất cả các vòng mã hóa nhằm tránh việc các vòng sử dụng cùng một khóa và làm suy giảm mức độ an toàn. Thay vào đó, từ khóa gốc, thuật toán sinh ra một tập hợp các khóa vòng (round keys) thông qua quá trình mở rộng khóa, gọi là Key Expansion hoặc Key Schedule.

Mục đích của quá trình này là tạo ra một dãy các khóa vòng độc lập, mỗi khóa vòng có độ dài 128 bit và được sử dụng cho một vòng mã hóa cụ thể. Quá trình mở rộng khóa được thiết kế sao cho mọi thay đổi nhỏ trong khóa ban đầu sẽ dẫn đến sự thay đổi đáng kể trong toàn bộ các khóa vòng, từ đó tăng cường tính bảo mật tổng thể của hệ thống.

Thuật toán mở rộng khóa dựa trên ba tham số chính: số lượng từ 32-bit trong khóa gốc N_k (bằng 4, 6 hoặc 8 tương ứng với AES-128, AES-192 và AES-256), số cột trong ma trận trạng thái N_b (luôn là 4), và số vòng lặp N_r (tương ứng là 10, 12 hoặc 14 tùy theo độ dài khóa). Tổng số từ cần được tạo ra trong toàn bộ quá trình mở rộng khóa được xác định theo công thức $N_w = N_b \times (N_r + 1)$. Chẳng hạn, với AES-128, số lượng từ được tạo ra là $N_w = 4 \times (10 + 1) = 44$ từ (word).

Trong trường hợp AES-128, khóa đầu vào có độ dài 128 bit bao gồm 16 byte và được chia thành bốn từ 32-bit, ký hiệu là W_0, W_1, W_2, W_3 . Từ bốn từ ban đầu này, thuật toán tạo ra tổng cộng 44 từ, trong đó mỗi nhóm bốn từ liên tiếp tạo thành một khóa vòng 128 bit cho từng vòng của quá trình mã hóa. Việc sinh các từ mới được thực hiện theo công thức tổng quát của thuật toán AES, trong đó cứ mỗi bốn từ lại có một từ

qua hàm biến đổi $g()$. Hàm này bao gồm ba bước con: phép dịch vòng trái 1 byte (RotWord), phép thay thế từng byte bằng giá trị trong bảng S-box (SubWord), và phép XOR byte đầu tiên của từ với hằng số vòng Rcon tương ứng. Kết quả của các phép biến đổi này được sử dụng để tạo ra các từ mới trong dãy khóa mở rộng.

Công thức tổng quát để sinh các từ khóa mới cho thuật toán AES-128 là

$$W[i] = \begin{cases} W[i-4] \oplus g(W[i-1]), & \text{nếu } i \equiv 0 \pmod{4} \\ W[i-4] \oplus W[i-1], & \text{nếu } i \not\equiv 0 \pmod{4} \end{cases} \quad (2.18)$$

Các hằng số vòng Rcon được sử dụng trong quá trình sinh khóa nhằm đảm bảo tính phi tuyến và tránh sự lặp lại trong dãy khóa mở rộng. Giá trị cụ thể của các hằng số Rcon được liệt kê trong Bảng 2.2.

Bảng 2.2: Bảng giá trị Rcon

Vòng (i)	Giá trị Rcon[i] (Hex)	Giải thích
1	01 00 00 00	Bắt đầu chuỗi hằng số, bit cao nhất = 1
2	02 00 00 00	Dịch trái giá trị Rcon[1]
3	04 00 00 00	Dịch trái giá trị Rcon[2]
4	08 00 00 00	Dịch trái giá trị Rcon[3]
5	10 00 00 00	Dịch trái giá trị Rcon[4]
6	20 00 00 00	Dịch trái giá trị Rcon[5]
7	40 00 00 00	Dịch trái giá trị Rcon[6]
8	80 00 00 00	Dịch trái giá trị Rcon[7]
9	1B 00 00 00	Khi dịch trái vượt giới hạn byte, XOR với 0x1B theo trường GF(2 ⁸)
10	36 00 00 00	Tiếp tục phép XOR để quay lại dãy ban đầu

Ví dụ minh họa: Giả sử khóa ban đầu (Key) gồm 16 byte như sau:

$$\begin{bmatrix} 2b & 28 & ab & 09 \\ 7e & ae & f7 & cf \\ 15 & d2 & 15 & 4f \\ 16 & a6 & 88 & 3c \end{bmatrix}$$

Các từ khóa ban đầu:

- $W[0] = 2b \ 7e \ 15 \ 16$
- $W[1] = 28 \ ae \ d2 \ a6$
- $W[2] = ab \ f7 \ 15 \ 88$
- $W[3] = 09 \ cf \ 4f \ 3c$

Bước 1: Tính W[4]

Ta có $i = 4$ là bội của 4 nên:

$$W[i]=W[4]=W[4-4] \oplus g(W[4-1])=W[0] \oplus g(W[3]) \quad (2.19)$$

Áp dụng hàm $g(W_3)$:

1. RotWord(W_3): xoay trái 1 byte ta được $W[3] = cf4f3c09$

2. SubWord(RotWord(W_3)) - thay từng byte bởi S-box AES:

- Sbox(cf) = 8a
- Sbox(3c) = eb
- Sbox(4f) = 84
- Sbox(09) = 01

→ 8a84eb01

3. XOR byte đầu với Rcon của vòng 1 ($Rcon[1] = 01\ 00\ 00\ 00$)

$$8a\ 84\ eb\ 01 \oplus 01\ 00\ 00\ 00 = 8b\ 84\ eb\ 01$$

→ $g(W[3]) = 8b\ 84\ eb\ 01$

$$\text{Vậy } W[4]=W[0] \oplus g(W[3])= 2b\ 7e\ 15\ 16 \oplus 8b\ 84\ eb\ 01 = a0\ fa\ fe\ 17$$

Bước 2: Tính W[5], W[6], W[7].

Ta có $i = 5, 6, 7$ không là bội của 4 nên áp dụng:

$$W[i]=W[i-4] \oplus W[i-1] \quad (2.20)$$

- $W[5] = W[1] \oplus W[4] = 28\ ae\ d2\ a6 \oplus a0\ fa\ fe\ 17 = 88\ 54\ 2c\ b1$
- $W[6] = W[2] \oplus W[5] = ab\ f7\ 15\ 88 \oplus 88\ 54\ 2c\ b1 = 23\ a3\ 39\ 39$
- $W[7] = W[3] \oplus W[6] = 09\ cf\ 4f\ 3c \oplus 23\ a3\ 39\ 39 = 2a\ 6c\ 76\ 05$

Suy ra kết quả của Round Key 1 (Khóa vòng 1) là:

$$\begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}$$

Bước 3: AddRoundKey với kết quả của Mixcolumn tại vòng 1

$$\text{Sau Mixcolumn, ta có } \begin{bmatrix} 04 & e0 & 48 & 28 \\ 66 & cb & f8 & 06 \\ 81 & 19 & d3 & 26 \\ e5 & 9a & 7a & 4c \end{bmatrix} \text{ thực hiện XOR với Round Key 1 ta}$$

sẽ có kết quả **mã hóa cuối cùng của vòng 1** là:

$$\begin{bmatrix} 04 & e0 & 48 & 28 \\ 66 & cb & f8 & 06 \\ 81 & 19 & d3 & 26 \\ e5 & 9a & 7a & 4c \end{bmatrix} \oplus \begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix} = \begin{bmatrix} a4 & 68 & 6b & 02 \\ 9c & 9f & 5b & 6a \\ 7f & 35 & ca & 50 \\ f2 & 2b & 43 & 49 \end{bmatrix} \quad (2.21)$$

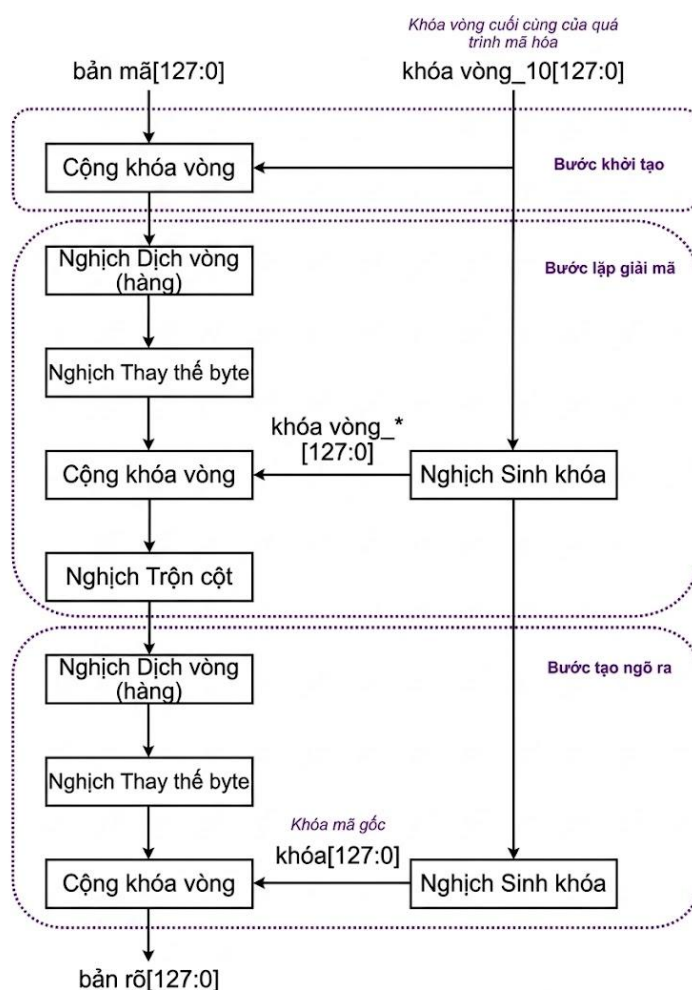
Các vòng tiếp theo (từ vòng 2 đến vòng 9) được thực hiện tương tự như vòng 1, bao gồm bốn phép biến đổi tuần tự: SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey. Riêng vòng 10 là vòng mã hóa cuối cùng trong thuật toán AES-128, có cấu trúc tương tự nhưng bỏ qua bước MixColumns, chỉ còn ba phép biến đổi:

SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey.

2.3. Giải mã trong thuật toán AES-128

2.3.1. Quy trình giải mã AES-128

Quy trình giải mã trong thuật toán AES-128 về bản chất là sự đảo ngược của quá trình mã hóa[3]. Để khôi phục lại bản rõ ban đầu, các phép biến đổi thuận được thay thế bằng các hàm nghịch đảo tương ứng, đồng thời các khóa vòng được sử dụng theo trình tự ngược lại. Các phép biến đổi cốt lõi trong giải mã bao gồm: InvShiftRows, InvSubBytes, InvMixColumns và AddRoundKey. Đặc biệt, do phép toán XOR có tính chất tự nghịch, phép biến đổi AddRoundKey được sử dụng đồng nhất trong cả hai luồng mã hóa và giải mã. Sơ đồ khối tổng thể của quy trình giải mã AES-128 được minh họa chi tiết trong Hình 2.10.



Hình 2.10: Quá trình giải mã của thuật toán AES-128

Trong kiến trúc phần cứng, bộ giải mã được triển khai trên một đường dẫn dữ liệu độc lập với bộ mã hóa và được tổ chức theo cấu trúc Pipeline nhiều tầng nhằm tối đa hóa thông lượng xử lý. Dữ liệu bản mã đầu vào được biểu diễn dưới dạng ma trận trạng thái và truyền tuần tự qua các tầng biến đổi của thuật toán AES. Các khóa vòng được tạo ra từ khối mở rộng khóa và phân phối tương ứng đến từng vòng xử lý, đảm bảo dữ liệu được xử lý liên tục theo từng chu kỳ xung nhịp và giảm độ trễ trong toàn bộ quá trình giải mã.

Quy trình giải mã AES-128 gồm ba giai đoạn chính: khởi tạo, xử lý qua đường ống và tạo ngõ ra. Ở giai đoạn khởi tạo, bản mã `cipher_text[127:0]` được kết hợp với khóa vòng cuối `round_key10[127:0]` thông qua phép biến đổi `AddRoundKey` để tạo trạng thái ban đầu.

Tiếp theo, dữ liệu đi qua các tầng pipeline tương ứng với các vòng giải mã. Tại mỗi tầng, các phép biến đổi `InvShiftRows`, `InvSubBytes`, `AddRoundKey` và `InvMixColumns` được thực hiện theo đúng trình tự của thuật toán AES. Nhờ kiến trúc Pipeline, nhiều khối dữ liệu 128-bit có thể được xử lý đồng thời ở các giai đoạn khác nhau. Sau khi đường ống được lấp đầy, hệ thống có khả năng tạo ra một khối dữ liệu đầu ra trong mỗi chu kỳ xung nhịp, giúp nâng cao đáng kể thông lượng so với kiến trúc tuần tự.

Trong suốt quá trình xử lý, các khóa vòng được cung cấp từ khối mở rộng khóa theo thứ tự đảo ngược để đảm bảo tính chính xác của quá trình giải mã. Việc sử dụng các khóa được tính toán trước không chỉ giảm độ trễ mà còn cải thiện hiệu năng tổng thể của hệ thống.

Cuối cùng, dữ liệu được xử lý qua các phép `InvShiftRows`, `InvSubBytes` và `AddRoundKey` với khóa ban đầu `key[127:0]`. Theo cấu trúc của vòng cuối AES, phép `InvMixColumns` được loại bỏ. Kết quả thu được là bản rõ `plain_text[127:0]`, hoàn tất quá trình giải mã và khôi phục dữ liệu gốc.

2.3.2. Các phép biến đổi trong quá trình giải mã

Dựa trên lưu đồ xử lý tổng thể đã được phân tích, ma trận trạng thái sẽ lần lượt đi qua các khối biến đổi nghịch đảo để khôi phục dữ liệu về trạng thái ban đầu trước khi mã hóa.

Như đã trình bày, vòng cuối của quá trình mã hóa AES kết thúc bằng phép biến đổi `AddRoundKey`. Vì vậy, khi bắt đầu giải mã, bản mã đầu vào sẽ được kết hợp với khóa vòng tương ứng thông qua phép ***AddRoundKey*** để khôi phục ma trận trạng thái ban đầu của quá trình giải mã. Sau bước này, dữ liệu tiếp tục được xử lý ngược qua các

vòng lặp với các phép biến đổi **InvShiftRows**, **InvSubBytes**, **AddRoundKey** và **InvMixColumns** theo đúng trình tự của thuật toán [4].

AddRoundKey là phép biến đổi kết hợp ma trận trạng thái (State) với khóa vòng (Round Key) thông qua phép toán XOR. Do XOR có tính chất tự nghịch $A \oplus B \oplus B = A$, **AddRoundKey** cũng chính là phép nghịch của chính nó. Vì vậy, trong quá trình giải mã, thuật toán chỉ cần thực hiện lại phép XOR với cùng khóa vòng tương ứng để loại bỏ ảnh hưởng của khóa đã được thêm vào trong giai đoạn mã hóa và khôi phục trạng thái dữ liệu trước đó. Đặc điểm này giúp đơn giản hóa và tối ưu việc triển khai AES trên cả phần cứng lẫn phần mềm.

Ví dụ minh họa: Để minh họa khả năng khôi phục dữ liệu của quá trình giải mã, ta xét ma trận trạng thái tương ứng với Vòng 1 của quá trình mã hóa. Trong đó, dữ liệu đầu vào của bước giải mã là ma trận trạng thái thu được sau khi kết thúc Vòng 1 mã hóa.

$$\text{Ma trận trạng thái đầu vào } S_{\text{in}} = \begin{bmatrix} a4 & 68 & 6b & 02 \\ 9c & 9f & 5b & 6a \\ 7f & 35 & ea & 50 \\ f2 & 2b & 43 & 49 \end{bmatrix}$$

Sử dụng nguyên bản Khóa vòng 1 (K_1) đã được sinh ra từ quá trình mở rộng

$$\text{khóa, } K_1 = \begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}$$

Tiến hành phép toán **AddRoundKey** bằng cách thực hiện XOR từng phần tử tương ứng của ma trận trạng thái với ma trận khóa vòng.

$$\begin{array}{llll} a4 \oplus a0 = 04 & 68 \oplus 88 = e0 & 6b \oplus 23 = 48 & 02 \oplus 2a = 28 \\ 9c \oplus fa = 66 & 9f \oplus 54 = cb & 5b \oplus a3 = f8 & 6a \oplus 6c = 06 \\ 7f \oplus fe = 81 & 35 \oplus 2c = 19 & ea \oplus 39 = d3 & 50 \oplus 76 = 26 \\ f2 \oplus 17 = e5 & 2b \oplus b1 = 9a & 43 \oplus 39 = 7a & 49 \oplus 05 = 4c \end{array}$$

Kết quả, ta thu được ma trận trạng thái mới sau bước **AddRoundKey**:

$$S_{\text{out}} = \begin{bmatrix} 04 & e0 & 48 & 28 \\ 66 & cb & f8 & 06 \\ 81 & 19 & d3 & 26 \\ e5 & 9a & 7a & 4c \end{bmatrix}$$

Kết quả đã trùng khớp với ma trận trạng thái sau phép biến đổi **MixColumns** của quá trình mã hóa. Điều này chứng minh rằng phép XOR với khóa vòng K_1 đã khôi phục thành công dữ liệu về trạng thái trước khi cộng khóa.

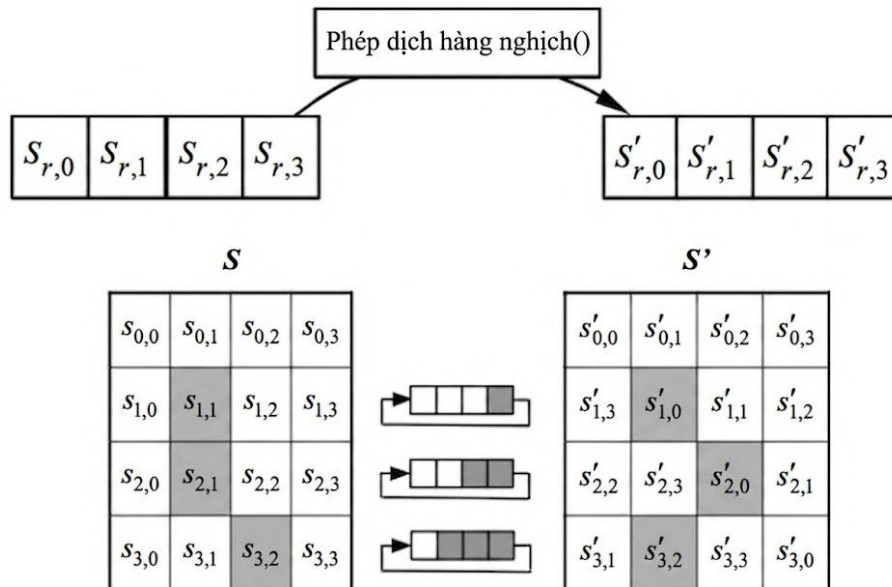
InvShiftRows là phép biến đổi nghịch của phép ShiftRows. Trong phép này, các byte trong ba hàng cuối của ma trận trạng thái được dịch vòng theo các độ dịch khác nhau. Hàng đầu tiên ($r = 0$) không bị dịch chuyển. Ba hàng còn lại được dịch vòng theo số byte là $N_b - \text{shift}(r, N_b)$, trong đó giá trị $\text{shift}(r, N_b)$ phụ thuộc vào chỉ số hàng và được xác định trong công thức $\text{shift}(1,4) = 1$; $\text{shift}(2,4) = 2$; $\text{shift}(3,4) = 3$. [2]

Cụ thể, phép biến đổi **InvShiftRows** được thực hiện theo công thức

$$s'_{r, (c+\text{shift}(r, N_b)) \bmod N_b} = s_{r,c} \quad \text{với } 0 < r < 4 \text{ và } 0 \leq c < N_b \quad (2.22)$$

Trong đó, $s_{r,c}$ và $s'_{r,c}$ lần lượt là giá trị trước và sau biến đổi tại vị trí hàng r , cột c của ma trận trạng thái [2].

Phép biến đổi này thực chất là quá trình dịch vòng sang phải các hàng của ma trận trạng thái, nhằm đảo ngược lại thao tác dịch vòng sang trái trong phép ShiftRows. Hình 2.11 minh họa chi tiết quá trình biến đổi InvShiftRows.



Hình 2.11: Minh họa phép biến đổi InvShiftRows, trong đó ba hàng cuối của ma trận trạng thái được dịch vòng sang phải với các độ dịch khác nhau.

Ví dụ minh họa: Để làm rõ quá trình hoạt động của phép biến đổi InvShiftRows trong giải mã, ta tiến hành thực hiện ngược lại ví dụ đã trình bày ở phần mã hóa (ShiftRows). Giả sử đầu vào của bước giải mã:

$$s = \begin{bmatrix} 4 & e0 & b8 & 1e \\ bf & b4 & 41 & 27 \\ 5d & 52 & 11 & 98 \\ 30 & ae & f1 & e5 \end{bmatrix} \quad (2.23)$$

Áp dụng công thức (2.22) và nguyên lý dịch vòng sang phải để đảo ngược sự khuếch tán theo chiều ngang, ta tiến hành khôi phục ma trận trạng thái từng hàng cụ thể như sau:

- Đối với hàng thứ nhất ($r = 0$), theo lý thuyết, độ dịch vòng $\text{shift}(0, 4) = 0$. Hàng này được giữ nguyên vị trí các byte **[d4 e0 b8 1e]**
- Đối với hàng thứ hai ($r = 1$), ta có, độ dịch là $\text{shift}(1, 4) = 1$. Hàng này được dịch vòng sang phải 1 byte. Nói cách khác, phần tử cuối cùng của hàng được đẩy lên vị trí đầu tiên, các phần tử còn lại lùi về sau một vị trí **[27 bf b4 41]**
- Đối với hàng thứ ba ($r = 2$), độ dịch tương ứng là $\text{shift}(2, 4) = 2$. Hàng này được dịch vòng sang phải 2 byte. Hai phần tử ở cuối hàng sẽ được chuyển lên đầu hàng theo đúng thứ tự **[11 98 5d 52]**
- Đối với hàng thứ tư ($r = 3$), độ dịch là $\text{shift}(3, 4) = 3$. Hàng này được dịch vòng sang phải 3 byte (tương đương với việc dịch vòng sang trái 1 byte). Ba phần tử ở cuối hàng được đưa lên đầu hàng **[ae f1 e5 30]**

Sau khi hoàn tất quá trình dịch vòng sang phải cho tất cả các hàng theo đúng công thức nghịch đảo, ta thu được ma trận trạng thái S hoàn chỉnh

$$s' = \begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix} \quad (2.24)$$

Ma trận kết quả hoàn toàn trùng khớp với ma trận trạng thái ban đầu trước khi thực hiện phép biến đổi **ShiftRows** trong quá trình mã hóa. Điều này chứng minh tính đúng đắn và khả năng khôi phục dữ liệu của phép biến đổi nghịch **InvShiftRows**.

InvSubBytes là phép biến đổi nghịch của quá trình thay thế từng byte, trong đó bảng tra cứu nghịch (inverse S-box) được áp dụng cho từng byte của ma trận trạng thái. Phép biến đổi này được thực hiện bằng cách áp dụng phép biến đổi affine nghịch của công thức (2.6), sau đó thực hiện phép nghịch đảo nhân trong trường hữu hạn $GF(2^8)$. Bảng inverse S-box được sử dụng trong phép biến đổi **InvSubBytes** được minh họa trong Hình 2.12.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	52	09	6a	d5	30	36	a5	38	bf	40	a3	9e	81	f3	d7	fb
	1	7c	e3	39	82	9b	2f	ff	87	34	8e	43	44	c4	de	e9	cb
	2	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	c3	4e
	3	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	d1	25
	4	72	f8	f6	64	86	68	98	16	d4	a4	5c	cc	5d	65	b6	92
	5	6c	70	48	50	fd	ed	b9	da	5e	15	46	57	a7	8d	9d	84
	6	90	d8	ab	00	8c	bc	d3	0a	f7	e4	58	05	b8	b3	45	06
	7	d0	2c	1e	8f	ca	3f	0f	02	c1	af	bd	03	01	13	8a	6b
	8	3a	91	11	41	4f	67	dc	ea	97	f2	cf	ce	f0	b4	e6	73
	9	96	ac	74	22	e7	ad	35	85	e2	f9	37	e8	1c	75	df	6e
	a	47	f1	1a	71	1d	29	c5	89	6f	b7	62	0e	aa	18	be	1b
	b	fc	56	3e	4b	c6	d2	79	20	9a	db	c0	fe	78	cd	5a	f4
	c	1f	dd	a8	33	88	07	c7	31	b1	12	10	59	27	80	ec	5f
	d	60	51	7f	a9	19	b5	4a	0d	2d	e5	7a	9f	93	c9	9c	ef
	e	a0	e0	3b	4d	ae	2a	f5	b0	c8	eb	bb	3c	83	53	99	61
	f	17	2b	04	7e	ba	77	d6	26	e1	69	14	63	55	21	0c	7d

Hình 2.12: Bảng Inverse S-box, thể hiện các giá trị thay thế tương ứng với mỗi byte đầu vào được biểu diễn ở dạng thập lục phân.

Ví dụ minh họa: Để làm rõ nguyên lý hoạt động của phép biến đổi **InvSubBytes**, ta thực hiện quá trình giải mã ngược lại ví dụ đã trình bày ở phép biến đổi **SubBytes** trong giai đoạn mã hóa. Dữ liệu đầu vào là ma trận trạng thái thu được sau khi thực hiện phép **InvShiftRows** (phương trình (2.24)). Do **InvShiftRows** đã khôi phục lại vị trí các phần tử trong ma trận, trạng thái này trùng khớp với ma trận đầu ra của bước **SubBytes** trong quá trình mã hóa.

$$S' = \begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix} \quad (2.25)$$

Khi áp dụng phép biến đổi nghịch, mỗi byte trong ma trận trạng thái sẽ được thay thế bằng một giá trị mới thông qua bảng tra cứu Inverse S-box (Hình 2.12). Quy tắc tra cứu tương tự như quá trình mã hóa: 4 bit ở nửa cao của byte đầu vào được sử dụng làm chỉ số hàng, và 4 bit ở nửa thấp được sử dụng làm chỉ số cột.

Ta tiến hành khôi phục các phần tử thuộc cột đầu tiên của ma trận trạng thái

- Đối với phần tử đầu tiên $d4_{16}$: Ta lấy 4 bit cao làm chỉ số hàng (d) và 4 bit thấp làm chỉ số cột (4). Đối chiếu vào bảng Inverse S-box, tại giao điểm của hàng d và cột 4, ta tra cứu được giá trị 19_{16} . Do đó, quá trình biến đổi là $d4_{16} \rightarrow 19_{16}$.
- Đối với phần tử thứ hai 27_{16} : Chỉ số hàng tương ứng là 2 và chỉ số cột là 7. Tra cứu trên bảng Inverse S-box ta nhận được giá trị $3d_{16}$. Suy ra, $27_{16} \rightarrow 3d_{16}$.
- Đối với phần tử thứ ba 11_{16} : Tách lấy hàng là 1 và cột là 1. Kết quả tra bảng tương ứng là $e3_{16}$, nghĩa là $11_{16} \rightarrow e3_{16}$.
- Đối với phần tử cuối cùng của cột ae_{16} : Chỉ số hàng xác định được là a và chỉ số cột là e. Tại vị trí này, bảng trả về giá trị be_{16} , nên $ae_{16} \rightarrow be_{16}$.

Áp dụng quy tắc tra cứu và thay thế tương tự cho tất cả các phần tử ở các cột còn lại của ma trận dựa trên bảng Inverse S-box, ta thu được ma trận kết quả hoàn chỉnh như sau:

$$S = \begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix} \quad (2.26)$$

Ma trận kết quả thu được ở trên hoàn toàn trùng khớp với ma trận trạng thái ban đầu trước khi thực hiện phép biến đổi thay thế byte **SubBytes** trong quá trình mã hóa. Điều này chứng minh một cách trực quan tính đúng đắn của bảng Inverse S-box và khả năng khôi phục toàn vẹn dữ liệu gốc của phép biến đổi nghịch **InvSubBytes** trong thuật toán giải mã AES.

InvMixColumns đây là phép biến đổi nghịch của **MixColumns**. Phép biến đổi này được thực hiện trên ma trận trạng thái theo từng cột, trong đó mỗi cột được xem như một đa thức bậc ba gồm bốn phần tử. Cụ thể, các cột của ma trận trạng thái được coi là các đa thức trên trường hữu hạn $GF(2^8)$ và được nhân với một đa thức cố định nghịch đảo $a^{-1}(x)$, với phép toán được thực hiện theo modulo x^4+1 . Đa thức nghịch đảo này được xác định như sau

$$a^{-1}(x) = \{0b\}x^3 + \{0d\}x^2 + \{09\}x + \{0e\} \quad (2.27)$$

Như đã trình bày trong ở quá trình **Mixcolumns**, phép biến đổi này có thể được biểu diễn dưới dạng phép nhân ma trận. Cụ thể, quá trình biến đổi có thể được mô tả thông qua phép nhân đa thức trong trường hữu hạn $GF(2^8)$, với mỗi cột của ma trận trạng thái được xem như một đa thức bậc ba. Khi đó, phép biến đổi được biểu diễn như sau

$$s'(x) = a^{-1}(x) \otimes s(x): \begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad \text{với } 0 \leq c < N_b \quad (2.28)$$

Trong đó, $s(x)$ là đa thức biểu diễn cột của ma trận trạng thái trước biến đổi, $s'(x)$ là đa thức sau biến đổi, $a^{-1}(x)$ là đa thức nghịch đảo cố định được sử dụng trong phép biến đổi **InvMixColumns**. Phép toán $\otimes$ biểu thị phép nhân đa thức trong trường hữu hạn $GF(2^8)$, được thực hiện theo modulo x^4+1 . Biểu diễn này cho thấy rõ bản chất toán học của phép biến đổi **InvMixColumns**, trong đó mỗi cột của ma trận trạng thái được biến đổi độc lập thông qua phép nhân với đa thức nghịch đảo, từ đó đảm bảo khả năng khôi phục lại dữ liệu ban đầu trong quá trình giải mã.

Kết quả của phép nhân này là bốn byte trong mỗi cột của ma trận trạng thái được thay thế bằng các giá trị mới, được tính toán thông qua tổ hợp tuyến tính của các phần tử trong cùng một cột. Cụ thể, mỗi byte đầu ra được xác định bằng cách thực hiện phép nhân trong trường hữu hạn $GF(2^8)$ giữa các hệ số của ma trận biến đổi với các byte đầu vào tương ứng, sau đó kết hợp các kết quả thông qua phép XOR.

Nói cách khác, mỗi phần tử trong cột sau biến đổi là kết quả của sự kết hợp giữa tất cả các phần tử trong cột ban đầu, theo các hệ số cố định của phép biến đổi **InvMixColumns**. Quá trình này đảm bảo rằng thông tin của từng byte đầu vào được phân tán đều trong toàn bộ cột, đồng thời cho phép khôi phục chính xác dữ liệu ban đầu khi thực hiện giải mã.

$$s'_{0,c} = (\{0e\} \cdot s_{0,c}) \oplus (\{0b\} \cdot s_{1,c}) \oplus (\{0d\} \cdot s_{2,c}) \oplus (\{09\} \cdot s_{3,c}) \quad (2.29)$$

$$s'_{1,c} = (\{09\} \cdot s_{0,c}) \oplus (\{0e\} \cdot s_{1,c}) \oplus (\{0b\} \cdot s_{2,c}) \oplus (\{0d\} \cdot s_{3,c}) \quad (2.30)$$

$$s'_{2,c} = (\{0d\} \cdot s_{0,c}) \oplus (\{09\} \cdot s_{1,c}) \oplus (\{0e\} \cdot s_{2,c}) \oplus (\{0b\} \cdot s_{3,c}) \quad (2.31)$$

$$s'_{3,c} = (\{0b\} \cdot s_{0,c}) \oplus (\{0d\} \cdot s_{1,c}) \oplus (\{09\} \cdot s_{2,c}) \oplus (\{0e\} \cdot s_{3,c}) \quad (2.32)$$

Ví dụ minh họa: Trong quá trình giải mã AES, phép biến đổi **InvMixColumns** thường được thực hiện sau bước **AddRoundKey**. Tuy nhiên, để minh họa rõ nguyên lý hoạt động và kiểm chứng tính đúng đắn của phép nghịch đảo $a^{-1}(x)$, ví dụ dưới đây sử dụng trực tiếp đầu ra của bước **MixColumns** trong quá trình mã hóa làm dữ liệu đầu vào. Nếu phép biến đổi **InvMixColumns** chính xác, ma trận trạng thái thu được sẽ khôi phục hoàn toàn dữ liệu trước khi thực hiện **MixColumns**.

Kết quả thu được của cột đầu tiên sau khi thực hiện MixColumns ở quá trình mã hóa là $[04 \ 66 \ 81 \ e5]^T$. Khi tiến hành giải mã, cột này đóng vai trò là cột đầu vào $[s_{0,c} \ s_{1,c} \ s_{2,c} \ s_{3,c}]^T$ cho phép biến đổi **InvMixColumns**.

Thực hiện phép nhân ma trận trong trường hữu hạn $GF(2^8)$ theo công thức (2.28), ta có thiết lập sau:

$$\begin{bmatrix} s'_{0,0} \\ s'_{1,0} \\ s'_{2,0} \\ s'_{3,0} \end{bmatrix} = \begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix} \begin{bmatrix} 04 \\ 66 \\ 81 \\ e5 \end{bmatrix} \quad (2.33)$$

Để khôi phục byte đầu tiên của cột $s'_{0,0}$, ta áp dụng công thức tổ hợp tuyến tính (2.29)

$$s'_{0,0} = (\{0e\} \cdot 04) \oplus (\{0b\} \cdot 66) \oplus (\{0d\} \cdot 81) \oplus (\{09\} \cdot e5) \quad (2.34)$$

Trong trường $GF(2^8)$, phép nhân với các hằng số $\{09\}$, $\{0b\}$, $\{0d\}$, $\{0e\}$ phức tạp hơn so với quá trình mã hóa (nhân với $\{02\}$, $\{03\}$). Để thực hiện, ta phân tích các hằng số này thành tổng của các lũy thừa của 2 (ví dụ: $0e = 08 \oplus 04 \oplus 02$), tương đương với các phép dịch trái 1 bit liên tiếp và thực hiện XOR với đa thức bất khả quy $0x1b$ nếu bit đầu (MSB) bằng 1.

Chi tiết các bước tính toán như sau:

Bước 1: Tính toán phần tử $(\{0e\} \cdot 04)$

Ta phân tích $0e = 08 \oplus 04 \oplus 02$. Với giá trị đầu vào là 04, các phép dịch bit không gây tràn:

- Nhân với 02: $(04 \cdot 02) = 04 \ll 1 = 08_{16}$
 - Nhân với 04: $(08 \cdot 02) = 08 \ll 1 = 10_{16}$
 - Nhân với 08: $(10 \cdot 02) = 10 \ll 1 = 20_{16}$
- $\rightarrow (\{0e\} \cdot 04) = (08 \cdot 04) \oplus (04 \cdot 04) \oplus (02 \cdot 04) = 20 \oplus 10 \oplus 8 = 38_{16}$

Bước 2: Tính toán phần tử $(\{0b\} \cdot 66)$

Ta phân tích $0b = 08 \oplus 02 \oplus 01$

- Nhân với 02: $(66 \cdot 02) = 66 \ll 1 = cc_{16}$ (không tràn bit)
- Nhân với 04: $(cc \cdot 02) = (cc \ll 1) \oplus 0x1b = 10011000_2 \oplus 00011011_2 = 83_{16}$ (tràn bit, XOR với $0x1b$)
- Nhân với 08: $(83 \cdot 02) = (83 \ll 1) \oplus 0x1b = 00000110_2 \oplus 00011011_2 = 1d_{16}$ (tràn bit, XOR với $0x1b$)

$$\rightarrow (\{0b\} \cdot 66) = (08 \cdot 66) \oplus (02 \cdot 66) \oplus 66 = 1d \oplus cc \oplus 66 = b7_{16}$$

Bước 3: Tính toán phần tử $(\{0d\} \cdot 81)$

Ta phân tích $0d = 08 \oplus 04 \oplus 01$

- Nhân với 02: $(81 \cdot 02) = (81 \ll 1) \oplus 0x1b = 00000010_2 \oplus 00011011_2 = 19_{16}$ (tràn bit)
 - Nhân với 04: $(19 \cdot 02) = 19 \ll 1 = 32_{16}$
 - Nhân với 08: $(32 \cdot 02) = 32 \ll 1 = 64_{16}$
- $\rightarrow (\{0d\} \cdot 81) = (08 \cdot 81) \oplus (04 \cdot 81) \oplus 81 = 64 \oplus 32 \oplus 81 = d7_{16}$

Bước 4: Tính toán phần tử $(\{09\} \cdot e5)$

Ta phân tích $09 = 08 \oplus 01$

- Nhân với 02: $(e5 \cdot 02) = (e5 \ll 1) \oplus 0x1b = 11001010_2 \oplus 00011011_2 = d1_{16}$ (tràn bit)
- Nhân với 04: $(d1 \cdot 02) = (d1 \ll 1) \oplus 0x1b = 10100010_2 \oplus 00011011_2 = b9_{16}$

(tràn bit)

- Nhân với 08: $(b9 \cdot 02) = (b9 \ll 1) \oplus 0x1b = 0110010_2 \oplus 00011011_2 = 69_{16}$

(tràn bit)

$$\rightarrow (\{09\} \cdot e5) = (08 \cdot e5) \oplus e5 = 69 \oplus e5 = 8c_{16}$$

Kết quả tính $s'_{0,0}$ có được bằng cách tiến hành XOR tất cả các kết quả thành phần vừa tìm được, ta có:

$$s'_{0,0} = 38 \oplus b7 \oplus d7 \oplus 8c = 8f \oplus d7 \oplus 8c = 58 \oplus 8c = d4_{16} \quad (2.35)$$

Thực hiện các bước tính toán tương tự theo công thức (2.28) cho các phần tử còn lại của cột $(s'_{1,0}, s'_{2,0}, s'_{3,0})$, ta sẽ nhận được kết quả lần lượt là bf_{16} , $5d_{16}$ và 30_{16} .

Như vậy, ma trận cột thu được sau phép biến đổi là $[d4 \quad bf \quad 5d \quad 30]^T$, hoàn toàn trùng với cột đầu vào của phép biến đổi MixColumns ở phần mã hóa. Kết quả này xác nhận rằng phép nhân với đa thức nghịch đảo $a^{-1}(x)$ đã đảo ngược chính xác tác động của phép MixColumns, qua đó khôi phục dữ liệu về trạng thái trước khi thực hiện trộn cột. Quá trình tính toán tương tự được áp dụng độc lập cho ba cột còn lại của ma trận trạng thái.

2.4. Kiến trúc phần cứng cho AES

2.4.1. Iterative architecture

Kiến trúc lặp (Iterative Architecture) là một phương pháp triển khai phần cứng phổ biến cho AES, đặc biệt phù hợp với các hệ thống có hạn chế về tài nguyên như hệ thống nhúng và thiết bị IoT. Trong kiến trúc này, một khối xử lý duy nhất được tái sử dụng qua nhiều chu kỳ xung nhịp để thực hiện toàn bộ các vòng biến đổi của thuật toán.

Thay vì triển khai phần cứng riêng cho từng vòng, kiến trúc lặp sử dụng một khối chức năng duy nhất để lần lượt thực hiện các phép biến đổi SubBytes, ShiftRows, MixColumns và AddRoundKey. Kết quả của mỗi vòng được lưu trong các thanh ghi và sử dụng làm đầu vào cho vòng tiếp theo. Toàn bộ quá trình được điều khiển bởi bộ điều khiển tuần tự, thường được xây dựng dưới dạng máy trạng thái hữu hạn (FSM).

Ưu điểm lớn nhất của kiến trúc lặp là tiết kiệm tài nguyên phần cứng nhờ khả năng tái sử dụng các khối chức năng. Điều này giúp giảm số lượng LUT, thanh ghi và mức tiêu thụ năng lượng, phù hợp với các ứng dụng yêu cầu diện tích và công suất thấp. Tuy nhiên, do các vòng xử lý được thực hiện tuần tự qua nhiều chu kỳ xung nhịp, kiến trúc này có độ trễ cao và thông lượng thấp hơn so với các kiến trúc song song như Fully Unrolled hoặc Pipelined. Đối với AES-128, thời gian xử lý phụ thuộc vào 10 vòng lặp

của thuật toán, do đó hiệu năng không tối ưu cho các ứng dụng yêu cầu xử lý thời gian thực hoặc thông lượng cao.

2.4.2. Fully unrolled architecture

Kiến trúc Fully Unrolled là phương pháp triển khai phần cứng AES theo hướng tối đa hóa khả năng xử lý song song nhằm đạt hiệu năng cao. Thay vì tái sử dụng một khối xử lý như kiến trúc Iterative, toàn bộ các vòng của thuật toán được triển khai thành các khối phần cứng độc lập và kết nối nối tiếp với nhau.

Đối với AES-128, kiến trúc này gồm 10 khối xử lý tương ứng với 10 vòng biến đổi. Mỗi khối thực hiện các phép SubBytes, ShiftRows, MixColumns và AddRoundKey, ngoại trừ vòng cuối không bao gồm MixColumns. Dữ liệu đầu vào được xử lý liên tục qua các khối, tạo thành một chuỗi xử lý hoàn toàn bằng phần cứng.

Nhờ loại bỏ việc tái sử dụng phần cứng, kiến trúc Fully Unrolled đạt thông lượng rất cao. Khi kết hợp với kỹ thuật Pipeline, hệ thống có thể xử lý liên tục nhiều khối dữ liệu với độ trễ thấp, đáp ứng tốt các ứng dụng truyền thông tốc độ cao và bảo mật hiệu năng cao.

Tuy nhiên, việc nhân bản phần cứng cho từng vòng làm tăng đáng kể mức sử dụng LUT, thanh ghi và bộ nhớ, đồng thời kéo theo mức tiêu thụ năng lượng lớn hơn. Ngoài ra, nếu không áp dụng Pipeline, đường truyền tổ hợp dài có thể làm giảm tần số hoạt động tối đa của hệ thống.

Nhìn chung, kiến trúc Fully Unrolled phù hợp với các hệ thống ưu tiên thông lượng và hiệu năng xử lý cao, trong khi không bị giới hạn nghiêm ngặt về tài nguyên phần cứng. Đây là hướng tiếp cận đối lập với kiến trúc Iterative khi chấp nhận đánh đổi tài nguyên để đạt tốc độ xử lý tối ưu.

2.4.3. Kiến trúc Pipeline

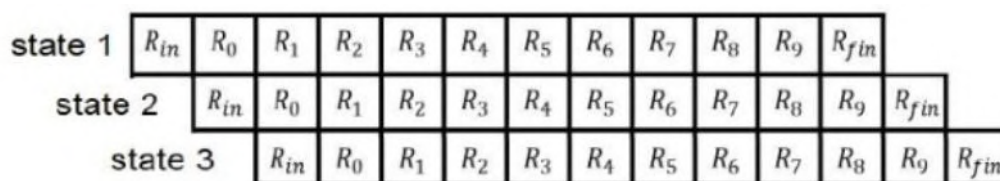
Pipeline là kỹ thuật xử lý song song trong đó một tiến trình được chia thành nhiều giai đoạn liên tiếp và các giai đoạn này được thực hiện chồng chéo theo thời gian [6]. Thay vì đợi một tác vụ hoàn thành toàn bộ quá trình xử lý, pipeline cho phép nhiều tác vụ được thực hiện đồng thời ở các giai đoạn khác nhau. Nguyên lý này tương tự dây chuyền sản xuất, khi dữ liệu mới có thể được đưa vào giai đoạn đầu trong lúc dữ liệu trước đó đang được xử lý ở các giai đoạn tiếp theo. Nhờ vậy, pipeline không làm giảm độ trễ của một tác vụ đơn lẻ nhưng giúp tăng thông lượng của hệ thống. Kỹ thuật này được ứng dụng rộng rãi trong bộ xử lý, xử lý tín hiệu số và các thuật toán mật mã như AES nhằm nâng cao hiệu năng xử lý.

Pipeline stages (các tầng pipeline) là những giai đoạn xử lý độc lập được tạo ra khi tiến trình được phân chia theo kỹ thuật pipeline. Mỗi tầng đảm nhiệm một phần công việc cụ thể và truyền kết quả cho tầng kế tiếp thông qua các thanh ghi pipeline. Khi đường ống đầy, mỗi tầng có thể xử lý một khối dữ liệu khác nhau trong cùng một chu kỳ xung nhịp, cho phép hệ thống tạo ra kết quả liên tục. Trong thuật toán AES, mỗi vòng mã hóa có thể được xem như một pipeline stage thực hiện các phép biến đổi trên trạng thái dữ liệu. Cách tổ chức này cho phép nhiều khối dữ liệu được xử lý đồng thời tại các vòng khác nhau, từ đó nâng cao đáng kể thông lượng của hệ thống.

2.4.4 Kiến trúc pipeline cho các vòng mã hóa AES

Thuật toán AES gồm nhiều vòng xử lý liên tiếp, trong đó mỗi vòng thực hiện các phép biến đổi SubBytes, ShiftRows, MixColumns và AddRoundKey trên trạng thái dữ liệu. Dựa trên đặc điểm này, kỹ thuật pipeline được áp dụng bằng cách tổ chức các vòng mã hóa thành các tầng xử lý liên tiếp, cho phép nhiều khối dữ liệu được xử lý đồng thời tại các vòng khác nhau của thuật toán [7]. Nhờ đó, mỗi vòng AES có thể được ánh xạ thành một tầng pipeline trong phần cứng hoặc được thực thi song song thông qua các luồng xử lý trong môi trường phần mềm.

Nguyên lý của kiến trúc AES Pipeline là cho phép một khối dữ liệu mới bắt đầu quá trình mã hóa ngay cả khi các khối trước đó vẫn đang được xử lý ở các vòng tiếp theo. Đối với AES-128, quá trình mã hóa gồm một vòng khởi tạo, chín vòng chính và một vòng cuối, tương ứng với mười một giai đoạn xử lý. Mô hình kiến trúc AES Pipeline được minh họa trong Hình 2.13.



Hình 2.13: Quá trình xử lý AES theo kiến trúc pipeline

Trong quá trình thực thi pipeline, tại chu kỳ xung nhịp đầu tiên, trạng thái mã hóa ban đầu sẽ trải qua vòng khởi tạo (R_{in}). Sang chu kỳ tiếp theo, vòng mã hóa thứ nhất (R_0) của trạng thái đầu tiên được xử lý đồng thời với việc bắt đầu vòng khởi tạo của trạng thái thứ hai. Quá trình này tiếp tục lặp lại theo mô hình pipeline, dẫn đến việc tại chu kỳ thứ mười hai, đầu ra của trạng thái đầu tiên đã hoàn tất trong khi hệ thống đang đồng thời xử lý các vòng mã hóa của nhiều trạng thái khác nhau. Nhờ vậy, hệ thống có khả năng sinh ra một đầu ra mới ở mỗi chu kỳ xung nhịp mà không cần chờ

toàn bộ tiến trình trước đó kết thúc, với dữ liệu đầu vào được cấp liên tục theo từng chu kỳ.

Trong AES, mỗi vòng mã hóa thực hiện bốn phép toán chính nhằm biến đổi trạng thái dữ liệu đầu vào[1]. Kết quả của mỗi vòng bao gồm hai thành phần: trạng thái dữ liệu đã được mã hóa và khóa vòng tương ứng được tạo ra để sử dụng cho vòng kế tiếp[4]. Hai thành phần này đồng thời đóng vai trò là đầu ra của vòng hiện tại và đầu vào của vòng tiếp theo, đảm bảo luồng xử lý liên tục trong kiến trúc pipeline. Điểm khác biệt cơ bản giữa AES truyền thống và AES pipeline nằm ở thời điểm khởi động một trạng thái mã hóa mới. Với phương pháp truyền thống, trạng thái tiếp theo chỉ được xử lý sau khi toàn bộ các vòng của trạng thái trước hoàn tất, dẫn đến hiệu suất thấp hơn. Ngược lại, kiến trúc AES pipeline cho phép bắt đầu mã hóa một trạng thái mới ở mỗi chu kỳ xung nhịp, từ đó cải thiện đáng kể thông lượng và rút ngắn tổng thời gian xử lý.

2.4.5 Kiến trúc pipeline cho các vòng giải mã AES

Kiến trúc pipeline cho các vòng giải mã AES được xây dựng dựa trên cùng nguyên lý với kiến trúc pipeline cho các vòng mã hóa AES. Như đã trình bày trong **Mục 2.4.4**, các vòng của thuật toán AES được tổ chức thành các tầng pipeline liên tiếp, trong đó mỗi tầng đảm nhiệm xử lý một vòng cụ thể của thuật toán. Đối với quá trình giải mã, kiến trúc pipeline được áp dụng tương tự như quá trình mã hóa, bảo đảm luồng xử lý liên tục giữa các vòng giải mã.

2.4.6 Tiêu chí đánh giá hiệu năng

Trong quá trình thiết kế và triển khai hệ thống mã hóa AES trên nền tảng phần cứng, việc xác định các tiêu chí đánh giá hiệu năng là bước quan trọng nhằm đo lường khả năng xử lý và mức độ đáp ứng của hệ thống đối với các yêu cầu thực tế. Các tiêu chí này không chỉ phản ánh tốc độ xử lý mà còn cho phép so sánh giữa các kiến trúc thiết kế khác nhau, từ đó lựa chọn giải pháp tối ưu.

Thông lượng là đại lượng then chốt biểu thị năng lực xử lý dữ liệu của hệ thống trên một đơn vị thời gian, thường được định lượng bằng đơn vị Megabit trên giây (Mbps) hoặc Gigabit trên giây (Gbps). Trong bối cảnh phát triển các ứng dụng yêu cầu băng thông lớn như truyền thông bảo mật, mạng IoT và các hệ thống nhúng thời gian thực, thông lượng trở thành chỉ số quyết định hiệu quả của một thiết kế vi mạch.

Đối với kiến trúc đường ống hóa toàn phần (fully pipelined), ưu điểm vượt trội nằm ở khả năng xử lý song song nhiều khối dữ liệu tại các tầng (stages) khác nhau của luồng thực thi. Sau khi đường ống được lấp đầy, hệ thống sẽ tiến vào trạng thái vận hành ổn

định, tại đó mỗi chu kỳ xung nhịp sẽ hoàn tất và xuất ra một khối dữ liệu đầu ra 128-bit tương ứng. Đặc tính này giúp tối ưu hóa hiệu suất phần cứng so với kiến trúc lặp (iterative architecture) vốn bị hạn chế do chỉ xử lý đơn khối tại một thời điểm.

Thông lượng lý thuyết của hệ thống được xác định dựa trên tần số hoạt động tối đa và kích thước khối dữ liệu của thuật toán AES theo công thức sau[7]:

$$T = \frac{B * Fmax * N}{d} \quad (2.36)$$

Trong đó:

- **T:** Thông lượng của hệ thống (Mbps hoặc Gbps).
- **B:** Kích thước khối dữ liệu mặc định của tiêu chuẩn AES ($B = 128\text{bits}$).
- **Fmax:** Tần số hoạt động tối đa của lõi AES sau khi tổng hợp và tối ưu hóa (MHz).
- **N:** Số giai đoạn pipeline trong kiến trúc thiết kế ($N = 10$).
- **d:** Độ trễ tính toán (Calculation delay), đại diện cho tổng số chu kỳ xung nhịp cần thiết để một khối dữ liệu đi từ đầu vào đến đầu ra (cycles).

Với kiến trúc Fully Pipeline 10 giai đoạn, mỗi nhịp clock hệ thống có thể xuất ra một khối dữ liệu hoàn chỉnh sau khi pipeline đã được lấp đầy ($N/d \approx 1$)[7]. Giả định hệ thống vận hành tại tần số $Fmax = 100\text{ MHz}$ và độ trễ tính toán lý tưởng $d = 10$ chu kỳ, thông lượng đạt được là:

$$\begin{aligned} T &= \frac{B * Fmax * N}{d} = \frac{128 * 100 * 10}{10} = 12800 \text{ Mbps} \\ &= 12.8 \text{ Gbps} \end{aligned} \quad (2.37)$$

Kết quả tính toán cho thấy thiết kế đạt mức thông lượng lý thuyết ấn tượng lên đến 12.8 Gbps, minh chứng cho khả năng đáp ứng tốt các luồng dữ liệu tốc độ cao. Tuy nhiên, cần lưu ý rằng giá trị này chỉ đạt được tối ưu khi pipeline ở trạng thái lấp đầy hoàn toàn (fully loaded). Trong giai đoạn khởi tạo ban đầu, thông lượng thực tế sẽ thấp hơn do dữ liệu cần thời gian để lan truyền qua toàn bộ các tầng xử lý trước khi hệ thống đạt trạng thái đầu ra ổn định.

Bên cạnh chỉ số về thông lượng, độ trễ là một tham số đặc biệt quan trọng dùng để đánh giá thời gian phản hồi và tính thời gian thực của hệ thống xử lý. Trong ngữ cảnh thiết kế vi mạch mã hóa, độ trễ được định nghĩa là khoảng thời gian tính từ thời điểm khối dữ liệu đầu vào (plaintext) được đưa vào cổng đầu vào cho đến khi khối dữ liệu đầu ra tương ứng (ciphertext) hoàn thành quá trình biến đổi và xuất hiện tại cổng đầu ra.

Trong kiến trúc đường ống hóa toàn phần (fully pipeline), mặc dù hệ thống có khả năng đạt được thông lượng cực cao nhờ việc xử lý song song, tuy nhiên vẫn tồn tại một khoảng thời gian trễ khởi động ban đầu. Khoảng trễ này chính là thời gian cần thiết để dữ liệu được truyền tải và xử lý tuần tự qua tất cả các tầng (stages) của đường ống trước khi lấp đầy hệ thống. Về mặt vật lý, độ trễ này phụ thuộc trực tiếp vào số tầng pipeline được triển khai, tương ứng với các vòng lặp biến đổi trong cấu trúc thuật toán AES-128.

Dựa trên tiêu chuẩn NIST FIPS-197, thuật toán AES-128 bao gồm một vòng khởi tạo (Initial Round) và 10 vòng lặp biến đổi chính. Do đó, trong một cấu trúc pipeline tiêu chuẩn 10 giai đoạn, số chu kỳ xung nhịp cần thiết để hoàn tất quá trình mã hóa cho một khối dữ liệu thường được xác định là $N = 12$ chu kỳ (bao gồm 10 chu kỳ cho các vòng lặp, 1 chu kỳ cho việc nạp khóa và 1 chu kỳ đồng bộ hóa đầu ra)[7].

Công thức tổng quát để xác định độ trễ thời gian của hệ thống được thiết lập như sau[7]:

$$\text{Latency} = \frac{d}{F_{\max}} \quad (2.38)$$

Trong đó:

- **d**: Tổng số chu kỳ xung nhịp để xử lý hoàn tất một khối dữ liệu.
- **F_{max}**: Tần số hoạt động tối đa của thiết kế đạt được sau khi tổng hợp.

Áp dụng vào thông số thực nghiệm với tần số hoạt động giả định tại $F_{\max} = 100 \text{ MHz}$ và số chu kỳ trễ là 10, ta có phép tính:

$$\text{Latency} = \frac{d}{F_{\max}} = \frac{10}{100 * 10^6} = 100 * 10^{-9} \text{ s} = 100 \text{ ns} \quad (2.39)$$

Như vậy, độ trễ của hệ thống đạt mức 100 ns. Đây là khoảng thời gian cần thiết để khối dữ liệu đầu tiên được xử lý hoàn chỉnh. Sau giai đoạn khởi động này, nhờ vào đặc tính của kiến trúc fully pipeline, hệ thống sẽ hoạt động ở trạng thái ổn định và liên tục cung cấp dữ liệu đầu ra sau mỗi chu kỳ xung nhịp, đảm bảo duy trì thông lượng tối ưu cho các ứng dụng truyền tải dữ liệu tốc độ cao.

2.5. Công cụ và nền tảng thiết kế

2.5.1. Ngôn ngữ mô tả phần cứng (Verilog)

Trong bối cảnh các hệ thống phần cứng ngày càng phức tạp, phương pháp thiết kế thủ công truyền thống dần bộc lộ nhiều hạn chế. Để đáp ứng yêu cầu thiết kế hiện đại, các công cụ thiết kế điện tử tự động (EDA) cùng các ngôn ngữ mô tả phần cứng (HDL) đã được phát triển. Trong đó, Verilog HDL là một trong những ngôn ngữ được sử dụng phổ biến nhất trong lĩnh vực thiết kế mạch số.

Verilog là ngôn ngữ mô tả phần cứng dùng để thiết kế và xác minh các hệ thống điện tử số. Ngôn ngữ này cho phép mô tả chức năng và cấu trúc của mạch ở mức luân chuyển dữ liệu giữa các thanh ghi (RTL). Từ mã Verilog, các công cụ EDA có thể thực hiện mô phỏng, kiểm chứng chức năng và tổng hợp tự động thành mạch logic để triển khai trên các nền tảng phần cứng như FPGA hoặc ASIC..

Lý do lựa chọn Verilog cho dự án:

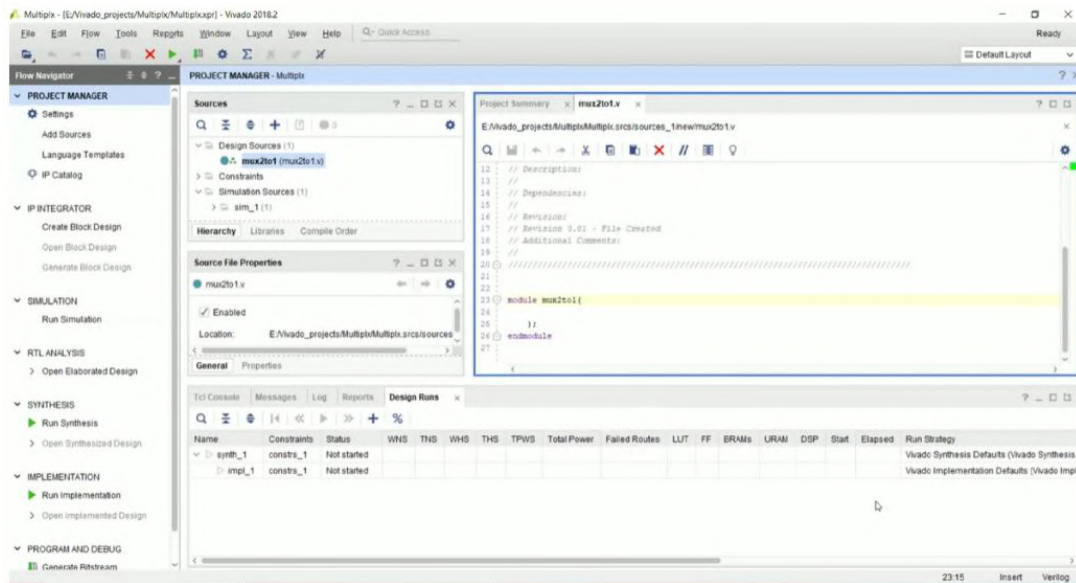
Thuật toán AES, bao gồm cả mã hóa và giải mã, được xây dựng từ các phép biến đổi có tính chất xử lý song song trên ma trận trạng thái. Các bước như SubBytes, InvSubBytes, ShiftRows, InvShiftRows, MixColumns và InvMixColumns đều cho phép thực hiện đồng thời trên nhiều phần tử dữ liệu. Verilog là ngôn ngữ mô tả phần cứng được thiết kế cho các hệ thống hoạt động song song, giúp mô tả tự nhiên các phép biến đổi này và phản ánh chính xác cách phần cứng thực thi trên chip. Vì vậy, Verilog là lựa chọn phù hợp để hiện thực hóa thuật toán AES.

Bên cạnh đó, Verilog có cú pháp tương đối đơn giản, gần gũi với ngôn ngữ C và hỗ trợ hiệu quả việc xây dựng các kiến trúc phần cứng phức tạp như Pipeline. Đồng thời, ngôn ngữ này được tích hợp rộng rãi trong các công cụ EDA, tạo thuận lợi cho quá trình mô phỏng, kiểm thử, gỡ lỗi và tổng hợp thiết kế.

Ngoài chức năng thiết kế phần cứng, Verilog còn hỗ trợ mô phỏng và xác minh hệ thống trước khi triển khai thực tế. Điều này giúp phát hiện sớm lỗi logic, giảm rủi ro phát triển, rút ngắn thời gian thiết kế và tối ưu chi phí triển khai.

2.5.2. Công cụ mô phỏng và tổng hợp

Để thực hiện quá trình thiết kế, mô phỏng và tổng hợp cho đề tài, bộ công cụ Xilinx Vivado Design Suite đã được lựa chọn làm môi trường phát triển chính để xác minh chức năng chi tiết. Giao diện tổng quan của phần mềm Vivado được minh họa trong Hình 2.14, cho thấy các khu vực chính phục vụ quá trình nhập mã RTL, mô phỏng và tổng hợp thiết kế.



Hình 2.14: Giao diện phần mềm thiết kế và mô phỏng Vivado

Vivado là một môi trường phát triển tích hợp (IDE) toàn diện do Xilinx (nay thuộc AMD) phát triển, chuyên dùng cho thiết kế và triển khai các hệ thống trên thiết bị logic khả trình (FPGA) và hệ thống trên chip (SoC)². Vivado được lựa chọn cho đề tài nhờ khả năng cung cấp một quy trình thiết kế hoàn chỉnh, bao gồm các giai đoạn từ nhập mã RTL, mô phỏng, tổng hợp logic cho đến triển khai vật lý (place and route). Việc tích hợp đầy đủ các công cụ trong cùng một môi trường giúp quá trình phát triển thiết kế diễn ra thuận lợi, đồng bộ và đáp ứng tốt yêu cầu của đề tài. Công cụ tổng hợp của Vivado cho phép chuyển đổi mã Verilog RTL thành netlist mức cổng logic một cách hiệu quả, đồng thời cung cấp các báo cáo chi tiết về mức độ sử dụng tài nguyên phần cứng như LUT và Flip-Flop, hỗ trợ quá trình đánh giá, tối ưu và đảm bảo thiết kế đáp ứng các ràng buộc kỹ thuật. Bên cạnh đó, Vivado là công cụ tiêu chuẩn trong ngành cho các thiết bị FPGA của Xilinx/AMD, sở hữu cộng đồng người dùng lớn cùng hệ thống tài liệu hướng dẫn phong phú, giúp việc học tập, tham khảo và xử lý các vấn đề trong quá trình thiết kế trở nên dễ dàng và hiệu quả hơn.

² AMD Xilinx, *Vivado Design Suite User Guide: Overview (UG892)*, AMD, 2023.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. Tổng quan hệ thống

3.1.1. Yêu cầu thiết kế

Mục tiêu của thiết kế là xây dựng một lõi IP AES-128 hiệu năng cao, hướng đến các hệ thống băng thông lớn. Để đạt được mục tiêu, hệ thống phải tuân thủ tiêu chuẩn AES-128 của NIST, với kích thước khối dữ liệu và độ dài khóa đều là 128 bit.

Về kiến trúc, hệ thống triển khai theo mô hình Fully Pipelined gồm 10 tầng xử lý tương ứng với 10 vòng tiêu chuẩn của thuật toán. Cấu trúc này cho phép đạt thông lượng tối đa 128 bit dữ liệu trên mỗi chu kỳ xung nhịp sau khi đường ống đầy.

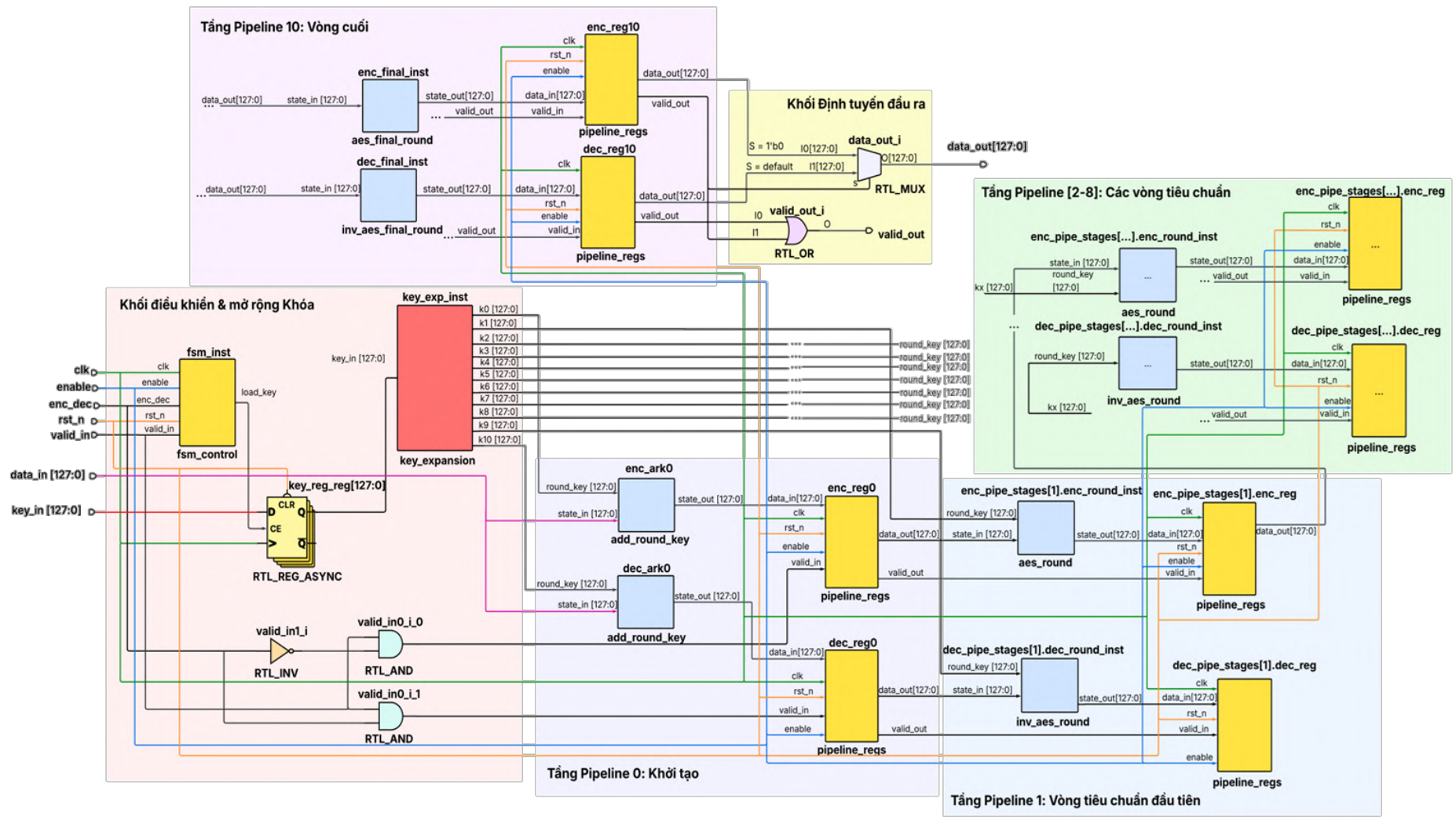
Bên cạnh đó, lõi IP được thiết kế để hỗ trợ cả hai chức năng mã hóa và giải mã thông qua hai luồng xử lý độc lập. Trong quá trình xây dựng kiến trúc, các giải pháp sử dụng tài nguyên phần cứng hiệu quả trên FPGA cũng được xem xét nhằm kiểm soát mức sử dụng LUT và các tài nguyên logic khác.

3.1.2. Sơ đồ khối tổng thể

Dựa trên các yêu cầu thiết kế, hệ thống AES-128 tổng hợp thành sơ đồ khối mức RTL với kiến trúc phân luồng. Cấu trúc chi tiết được thể hiện trong Hình 3.1.

Thiết kế được cấu thành từ 5 khối chức năng chính, đồng bộ với nhau qua hệ thống xung nhịp (clk):

- Khối điều khiển và xử lý đầu vào (Input Control & FSM): Điều phối hoạt động thông qua fsm_control, thực hiện chọn chế độ mã hóa hoặc giải mã dựa trên tín hiệu enc_dec. Khóa đầu vào key_in được lưu tạm qua thanh ghi trước khi xử lý nhằm đảm bảo đồng bộ [7].
- Khối mở rộng khóa (Key Expansion): Nhận khóa gốc key_in[127:0] và sinh đồng thời 11 khóa vòng (k0-k10) cấp cho các tầng xử lý mà không tạo thêm độ trễ [7].
- Đường truyền mã hóa: Luồng xử lý độc lập gồm khối AddRoundKey khởi tạo và 10 tầng mã hóa liên tiếp, sử dụng các khóa vòng theo thứ tự từ k0 đến k10 [7].
- Đường truyền giải mã: Hoạt động song song với đường truyền mã hóa, gồm 10 tầng giải mã sử dụng các phép biến đổi nghịch đảo và áp dụng khóa vòng theo thứ tự từ k10 đến k0 [7].
- Thanh ghi Pipeline và khối xuất dữ liệu (Pipeline Registers & Output Stage): Các thanh ghi Pipeline được chèn giữa các tầng để lưu dữ liệu trung gian và đồng bộ tín hiệu valid. Bộ chọn MUX quyết định xuất kết quả mã hóa hoặc giải mã, trong khi valid_out báo hiệu dữ liệu đầu ra đã sẵn sàng [7].



Hình 3.1: Sơ đồ khối mức RTL của hệ thống AES-128 Fully Pipelined

3.2. Thiết kế kiến trúc pipeline AES

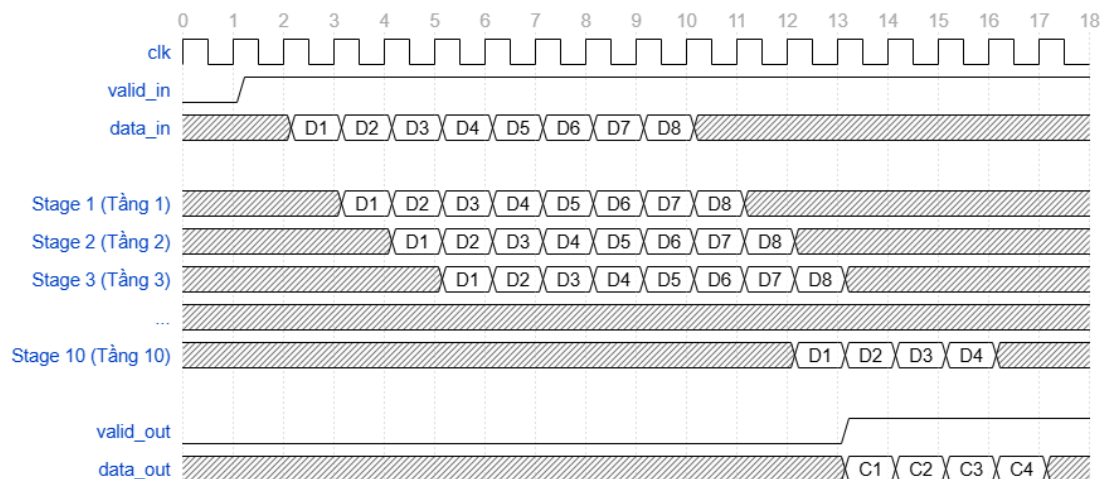
3.2.1. Mô hình và phân chia pipeline

Trong kiến trúc AES lặp vòng truyền thống, một khối xử lý vòng duy nhất được tái sử dụng cho toàn bộ 10 vòng mã hóa. Cách tiếp cận này giúp tiết kiệm tài nguyên phần cứng nhưng làm tăng độ trễ và giảm thông lượng, do hệ thống cần 10 chu kỳ xung nhịp để xử lý một khối dữ liệu.

Để khắc phục hạn chế về độ trễ và thông lượng, thiết kế áp dụng kiến trúc Fully Pipelined với 10 khối xử lý vòng độc lập được kết nối nối tiếp và ngăn cách bởi các thanh ghi pipeline. Cách tổ chức này giúp rút ngắn đường trễ tới hạn xuống còn trễ của một vòng xử lý và thời gian thiết lập của thanh ghi, từ đó tăng tần số hoạt động tối đa ($f_{\max}$) và cải thiện đáng kể thông lượng của hệ thống.

3.2.2. Tổ chức pipeline và luồng dữ liệu

Cơ chế hoạt động của kiến trúc Pipeline dựa trên nguyên lý xử lý song song theo không gian. Thay vì chờ một khối dữ liệu hoàn tất toàn bộ quá trình mã hóa rồi mới tiếp nhận dữ liệu mới, hệ thống cho phép các khối dữ liệu đầu vào (*data_in*) được nạp liên tục tại mỗi cạnh lên của xung nhịp (*clk*). Quá trình dịch chuyển dữ liệu giữa các tầng xử lý và đặc tính thời gian của kiến trúc được minh họa ở Hình 3.2.



Hình 3.2: *Giản đồ thời gian mô tả luồng dữ liệu và thông số của kiến trúc AES-128 Pipeline*

Dựa vào giản đồ thời gian, luồng dữ liệu và hiệu năng của hệ thống được phân tích qua các đặc điểm cốt lõi sau:

- **Quá trình nạp và dịch chuyển dữ liệu:** Tại chu kỳ xung nhịp số 1, tín hiệu *valid_in* được kéo lên mức logic 1, báo hiệu dữ liệu đầu vào đã sẵn sàng. Đến cạnh lên của chu kỳ số 2, khối dữ liệu đầu tiên (D1) nạp vào và lưu tại tầng xử lý thứ nhất. Sang chu kỳ số 3, D1 được chuyển sang tầng kế tiếp, đồng thời tầng thứ nhất tiếp nhận khối

dữ liệu thứ hai (D2). Quá trình này tiếp tục lặp lại ở các chu kỳ sau. Quan sát từ chu kỳ 2 đến chu kỳ 10, các khối dữ liệu (D_1 đến D_8) lần lượt được nạp vào hệ thống. Khi đường ống được lấp đầy, các tầng xử lý hoạt động đồng thời trên những khối dữ liệu khác nhau, giúp khai thác hiệu quả tài nguyên phần cứng và nâng cao khả năng xử lý song song của hệ thống.

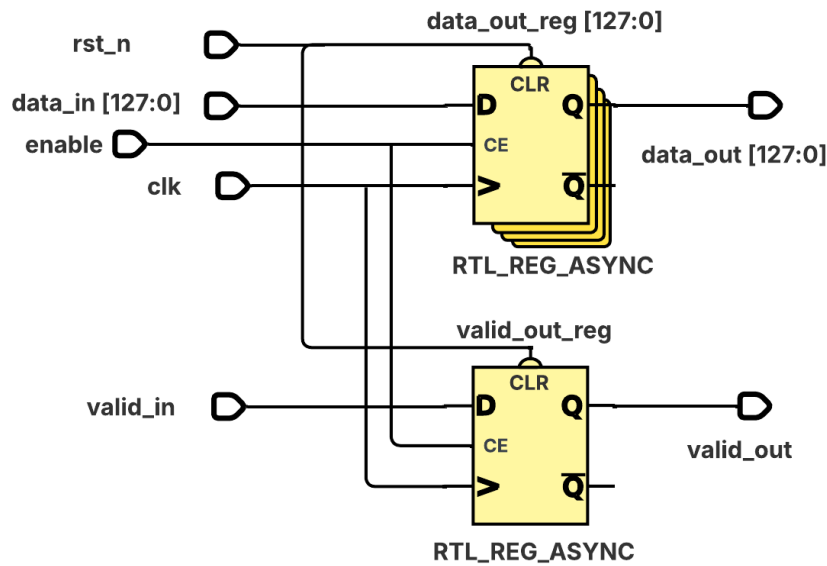
- *Phân tích độ trễ*: Quá trình của một khối dữ liệu (ví dụ D_1) bắt đầu từ khi được nạp vào ở chu kỳ 2. Nó lần lượt đi qua 10 tầng logic (Tầng 1 đến tầng 10). Đến cạnh lên của chu kỳ 13, hệ thống hoàn tất xử lý, kích hoạt cờ `valid_out` lên mức 1 và đồng thời xuất kết quả tương ứng (C_1) ra cổng `data_out`. Như vậy, hệ thống mất một khoảng thời gian cố định là 12 chu kỳ xung nhịp để hoàn tất quá trình đi từ đầu vào đến đầu ra cho một khối dữ liệu. Đây được định nghĩa là độ trễ cố định của thiết kế.

- *Phân tích thông lượng cực đại*: Ưu điểm của cấu trúc đường ống thể hiện từ chu kỳ 13 trở đi. Mặc dù hệ thống có độ trễ ban đầu là 12 chu kỳ, nhưng tại các xung nhịp tiếp theo, các kết quả ($C_2, C_3, C_4...$) liên tục được xuất ra mà không xuất hiện bất kỳ chu kỳ trống nào. Điều này minh chứng rằng hệ thống đã đạt được thông lượng cực đại: tiếp nhận 1 khối dữ liệu mới và xuất ra 1 khối kết quả (128-bit) trên mỗi chu kỳ xung nhịp, đáp ứng yêu cầu băng thông cao của các chuẩn giao tiếp hiện đại.

3.2.3. Thiết kế khối thanh ghi đường ống (Pipeline Registers)

Để phân tách các tầng xử lý trong kiến trúc Pipeline, hệ thống sử dụng module `pipeline_regs` làm thanh ghi trung gian để lưu trạng thái dữ liệu giữa các tầng[7]. Module này được dùng trong cả tuyến mã hóa và giải mã nhằm đảm bảo dữ liệu được truyền ổn định qua từng chu kỳ xung nhịp.

Về cấu trúc phần cứng, `pipeline_regs` được xây dựng từ các Flip-Flop D hoạt động đồng bộ theo cạnh lên của tín hiệu `clk` kết hợp với chân kích hoạt cho phép (Clock Enable - CE)[7]. Kiến trúc mức RTL của module được minh họa trong Hình 3.3. Trong quá trình hoạt động, tại mỗi cạnh lên của xung nhịp, module sẽ chốt dữ liệu đầu vào và chuyển sang tầng xử lý kế tiếp. Nhờ sử dụng các thanh ghi trung gian, đường trễ của toàn bộ hệ thống được chia nhỏ theo từng tầng, giúp lõi AES có thể hoạt động ổn định ở tần số cao và duy trì thông lượng xử lý liên tục.



Hình 3.3: Sơ đồ khối RTL của khối thanh ghi đường ống pipeline_regs

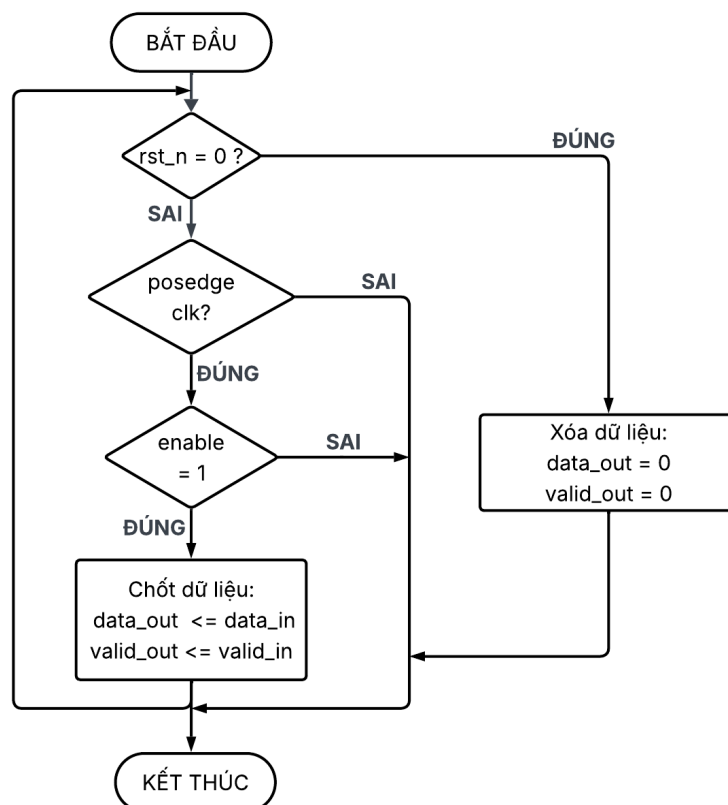
Để đảm bảo khả năng kết nối đồng bộ giữa các tầng, các tín hiệu kết nối của khối bao gồm các cổng tín hiệu được định nghĩa chi tiết trong Bảng 3.1.

Bảng 3.1: Mô tả các tín hiệu của khối pipeline_regs

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
clk	Input	1-bit	Xung nhịp hệ thống, kích hoạt việc chốt dữ liệu tại sườn lên.
rst_n	Input	1-bit	Tín hiệu reset không đồng bộ, tích cực mức thấp để xóa sạch dữ liệu.
enable	Input	1-bit	Tín hiệu kích hoạt. Thanh ghi chốt dữ liệu mới khi tín hiệu ở mức logic 1.
data_in	Input	128-bit	Tiếp nhận ma trận trạng thái từ khối tổ hợp (aes_round hoặc inv_aes_round).
valid_in	Input	1-bit	Cờ báo hiệu dữ liệu đầu vào tại tầng hiện tại đang hợp lệ.
data_out	Output	128-bit	Dữ liệu đã được "chốt" để đưa sang tầng xử lý tiếp theo ở chu kỳ sau.
valid_out	Output	1-bit	Cờ báo hiệu dữ liệu đầu ra đã sẵn sàng cho tầng kế tiếp.

Về nguyên lý hoạt động, khối pipeline_regs được xây dựng theo kiểu mạch tuần tự nhằm lưu và truyền dữ liệu giữa các tầng xử lý[7][8]. Khi tín hiệu reset rst_n ở mức 0, toàn bộ dữ liệu 128-bit và cờ trạng thái sẽ được xóa về 0 để tránh dữ liệu sai hoặc trạng thái không xác định. Trong chế độ hoạt động bình thường, tại mỗi cạnh lên của

xung nhịp clk, khối này sẽ kiểm tra mức logic của tín hiệu enable. Nếu enable = 1, hệ thống mới thực hiện chốt giá trị từ data_in sang data_out và từ valid_in sang valid_out bằng phép gán không đồng bộ. Nếu enable = 0, thanh ghi sẽ giữ nguyên giá trị cũ, tạm dừng luồng dữ liệu. Trình tự hoạt động được minh họa ở Hình 3.4.



Hình 3.4: Lưu đồ thuật toán điều khiển của khối pipeline_regs

3.3. Thiết kế khối mã hóa AES-128

Khối mã hóa là thành phần trung tâm, thực hiện quá trình biến đổi dữ liệu từ bản rõ sang bản mã 128-bit. Để phù hợp với kiến trúc Pipeline tổng thể, luồng dữ liệu được tổ chức thành nhiều tầng xử lý nối tiếp và được ngăn bởi các thanh ghi trung gian nhằm đồng bộ dữ liệu giữa các tầng, đồng thời nâng cao hiệu năng xử lý.

Quá trình xử lý bắt đầu tại vòng khởi tạo (enc_ark0), nơi dữ liệu được kết hợp với khóa ban đầu. Tiếp theo là 9 vòng xử lý tiêu chuẩn (aes_round), mỗi vòng thực hiện các phép biến đổi của thuật toán và kết thúc tại vòng cuối (aes_final_round).

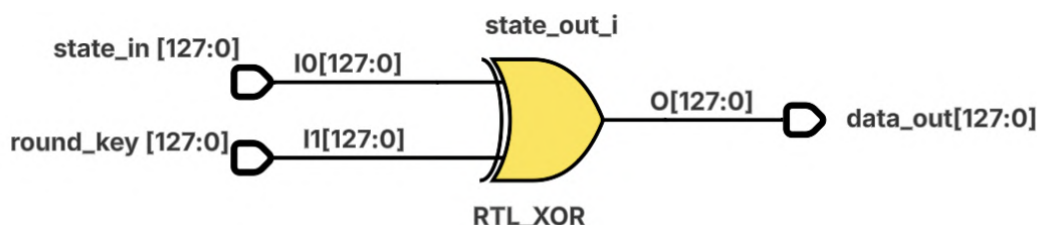
Trong toàn bộ quá trình, khối mã hóa sử dụng 11 khóa vòng được cung cấp song song từ khối Key Expansion. Việc cấp phát khóa đồng thời giúp các tầng pipeline có thể hoạt động liên tục, đảm bảo dữ liệu được xử lý nhanh và không bị gián đoạn.

3.3.1. Thiết kế các khối xử lý vòng

Để tuân thủ đúng chuẩn mã hóa AES-128 (FIPS-197), tuyến dữ liệu của khối mã hóa không được xây dựng từ một khối vòng lặp duy nhất. Thay vào đó, hệ thống được thiết kế gồm ba khối chức năng khác nhau, tương ứng với ba giai đoạn của thuật toán: vòng khởi tạo, các vòng tiêu chuẩn và vòng kết thúc.

Khối vòng khởi tạo (Initial Round)

Trước khi đi vào các vòng biến đổi phức tạp, khối dữ liệu bản rõ phải trải qua bước tiền xử lý. Như được thể hiện ở sơ đồ kiến trúc trong Hình 3.5, khối `enc_ark0` này có cấu trúc RTL đơn giản nhất. Chức năng duy nhất của nó là thực hiện phép toán XOR từng bit giữa khối dữ liệu đầu vào 128-bit và khóa khởi tạo k_0 . Thực chất, đây cũng là phép biến đổi xuất hiện trong các vòng tiêu chuẩn (`aes_round`) và vòng kết thúc (`aes_final_round`). Kết quả của bước này tạo ra trạng thái dữ liệu ban đầu, làm đầu vào cho các tầng xử lý tiếp theo trong kiến trúc pipeline.



Hình 3.5: Sơ đồ khối RTL của khối khởi tạo `enc_ark0`

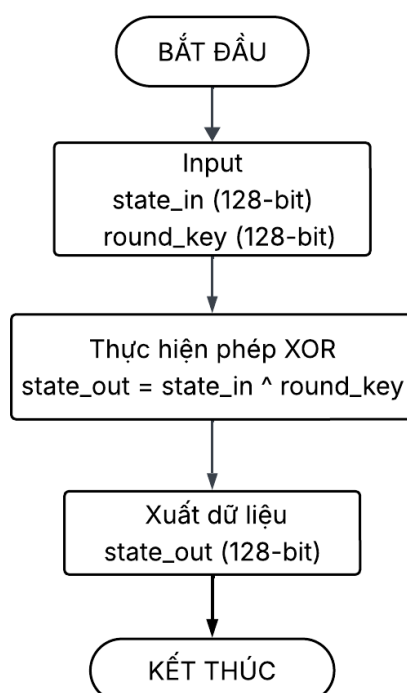
Về mặt giao tiếp phần cứng, khối là một bộ logic XOR 128-bit (`RTL_XOR`) với các ngõ vào ra được định nghĩa chi tiết như trong Bảng 3.2.

Bảng 3.2: Mô tả các tín hiệu của vòng khởi tạo (`enc_ark0`)

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>state_in</code>	Input	128-bit	Tiếp nhận khối dữ liệu bản rõ ban đầu (vòng khởi tạo), hoặc ma trận trạng thái trung gian từ phép biến đổi trước đó là trộn cột hoặc dịch hàng (nếu bên trong các vòng chính).
<code>round_key</code>	Input	128-bit	Tiếp nhận bộ khóa vòng tương ứng từ khối Key Expansion.
<code>state_out</code>	Output	128-bit	Xuất trạng thái dữ liệu sau khi thực hiện phép XOR, chuẩn bị chốt vào thanh ghi của đường ống hoặc đưa vào vòng xử lý tiếp theo.

Để tối ưu hóa tài nguyên và hiệu năng, khối được xây dựng từ mảng các cổng XOR hai ngõ vào. Nhờ được hiện thực bằng logic tổ hợp, phép biến đổi thực hiện đồng

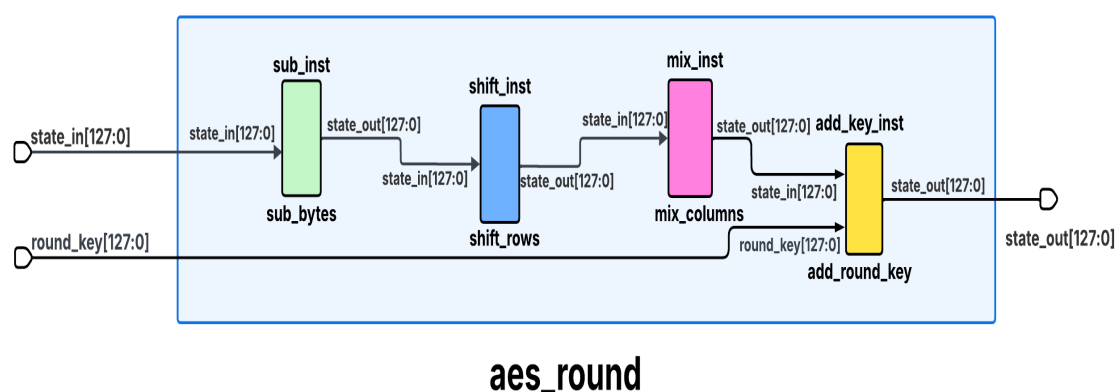
thời trên toàn bộ 128 bit dữ liệu mà không làm tăng độ trễ theo chu kỳ xung nhịp. Trình tự xử lý của khối được thể hiện qua lưu đồ thuật toán trong Hình 3.6.



Hình 3.6: Lưu đồ thuật toán của module vòng khởi tạo

Khối vòng tiêu chuẩn (aes_round)

Đây là khối quan trọng, được triển khai cho 9 vòng xử lý. Khối aes_round thực hiện đầy đủ một vòng biến đổi tiêu chuẩn của AES. Về cấu trúc, như được minh họa chi tiết trong Hình 3.7, khối gồm 4 thành phần chức năng được nối tiếp theo thứ tự cố định. Dữ liệu đầu vào lần lượt đi qua các bước: thay thế byte (sub_bytes), dịch hàng (shift_rows), trộn cột (mix_columns) và cuối cùng là cộng khóa vòng (add_round_key) với khóa tương ứng.



Hình 3.7: Kiến trúc RTL nội bộ của khối vòng tiêu chuẩn aes_round

Để đảm bảo khả năng kết nối giữa các khối trong hệ thống, khối aes_round được thiết kế với các tín hiệu rõ ràng, được mô tả chi tiết trong Bảng 3.3.

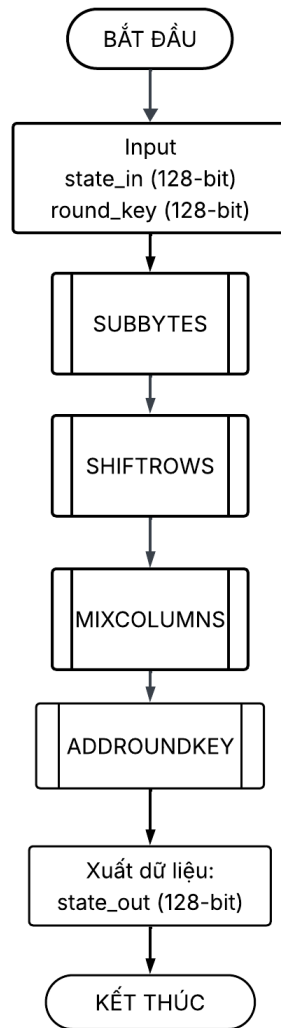
Bảng 3.3: Mô tả các tín hiệu của khối vòng tiêu chuẩn (aes_round)

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
state_in	Input	128-bit	Tiếp nhận ma trận trạng thái (State) từ tầng xử lý trước đó.
round_key	Input	128-bit	Tiếp nhận bộ khóa vòng tương ứng từ khối Key Expansion.
state_out	Output	128-bit	Xuất ma trận trạng thái kết quả sau khi hoàn tất 4 phép toán, chuẩn bị nạp vào tầng Pipeline kế tiếp.

Dựa trên sơ đồ kiến trúc RTL, dữ liệu 128-bit state_in lần lượt đi qua các khối chức năng theo thứ tự sau:

- sub_bytes: Khối dữ liệu 128-bit state_in đi vào khối sub_bytes. Tại đây, 16 byte dữ liệu được thay thế qua 16 bộ S-Box phân cứng để tạo ra tính phi tuyến.
- shift_rows: Kết quả được đưa vào khối shift_rows để thực hiện dịch vòng các hàng cấu thành nên ma trận trạng thái, giúp xáo trộn vị trí các byte.
- mix_columns: Dữ liệu tiếp tục đi vào khối mix_columns để nhân ma trận trên trường Galois $GF(2^8)$ nhằm khuếch tán dữ liệu.
- add_round_key: Tại chặng cuối, ngõ ra của bước trộn cột được đưa vào khối add_round_key. Tại đây, dữ liệu được XOR trực tiếp với tín hiệu round_key để tạo ra ngõ ra cuối cùng state_out của toàn bộ vòng.

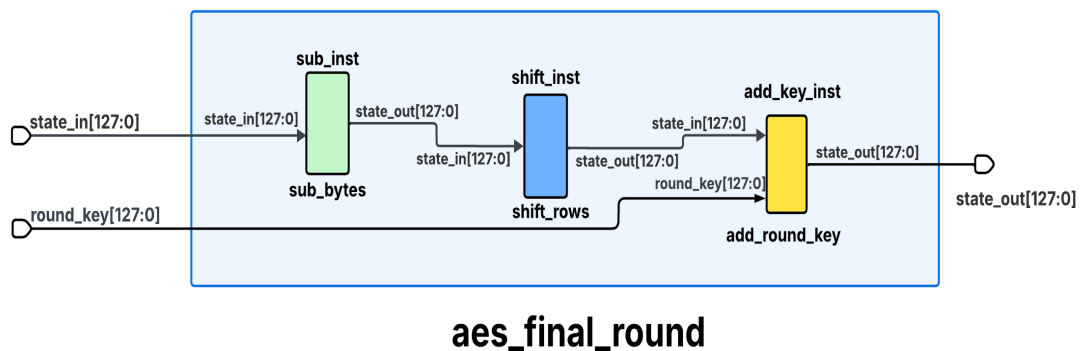
Quá trình xử lý được thực hiện liên tục trong một chu kỳ nhờ kiến trúc logic tổ hợp. Khối aes_round bao gồm các khối chức năng được kết nối nối tiếp theo đúng trình tự của thuật toán AES, cho phép dữ liệu được biến đổi trực tiếp mà không cần lưu trữ trung gian. Mỗi phép biến đổi tương ứng với một khối phân cứng riêng (sẽ được trình bày chi tiết ở **mục 3.3.2**). Trình tự xử lý tổng thể của luồng dữ liệu được mô tả trực quan trong lưu đồ thuật toán ở Hình 3.8.



Hình 3.8: Lưu đồ thuật toán của module vòng tiêu chuẩn

Khởi vòng kết thúc (*aes_final_round*)

Được bố trí ở tầng pipeline thứ 10 (tầng cuối cùng của khối mã hóa), khối *aes_final_round* có nhiệm vụ tạo ra bản mã đầu ra. So với vòng tiêu chuẩn, module này có sự tinh chỉnh đặc biệt về mặt cấu trúc. Như được minh họa chi tiết trong sơ đồ kiến trúc RTL ở Hình 3.9, khối trộn cột (*mix_columns*) đã được lược bỏ khỏi chuỗi xử lý. Cụ thể, dữ liệu chỉ đi qua ba bước là *SubBytes*, *ShiftRows* và *AddRoundKey*.



Hình 3.9: Kiến trúc RTL của module vòng kết thúc *aes_final_round*

Tương tự như các module vòng khác, `aes_final_round` duy trì chuẩn giao tiếp 128-bit nhưng mang ý nghĩa xuất dữ liệu kết thúc hệ thống, các tín hiệu được mô tả cụ thể trong Bảng 3.4.

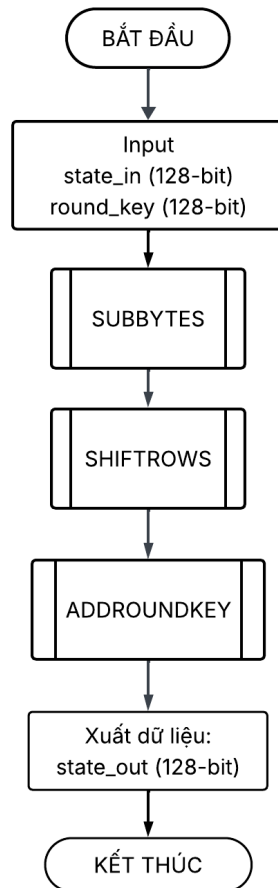
Bảng 3.4: Mô tả các tín hiệu của module vòng kết thúc (`aes_final_round`)

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>state_in</code>	Input	128-bit	Tiếp nhận ma trận trạng thái trung gian từ tầng Pipeline thứ 9.
<code>round_key</code>	Input	128-bit	Tiếp nhận khóa vòng cuối cùng (k_{10}) được cấp từ khối Key Expansion.
<code>state_out</code>	Output	128-bit	Xuất bản mã hoàn chỉnh (Ciphertext), chốt vào bộ đệm ngõ ra để hoàn tất toàn bộ tiến trình mã hóa.

Dựa trên sơ đồ RTL, do sự lược bỏ của khối `mix_columns`, luồng dữ liệu tổ hợp bên trong module vòng kết thúc được rút ngắn chỉ còn 3 chặng xử lý nối tiếp:

- `sub_inst`: Trạng thái đầu vào `state_in` trước tiên đi qua khối `sub_bytes` để thực hiện phép thay thế phi tuyến song song trên 16 byte dữ liệu thông qua các S-Box.
- `shift_inst`: Kết quả được truyền tới khối `shift_rows` để thực hiện phép dịch vòng tuần tự các hàng của ma trận trạng thái.
- `add_key_inst`: Dữ liệu được tiến hành XOR với khóa vòng k_{10} để sinh ra kết quả `state_out` cuối cùng.

Tương tự như vòng tiêu chuẩn, vòng kết thúc cũng sử dụng logic tổ hợp để duy trì bằng thông chung của đường ống. Tuy nhiên, do không thực hiện phép biến đổi MixColumns, cấu trúc của module được rút gọn, giúp giảm độ trễ so với các vòng trước. Các bước thực thi được thể hiện ở lưu đồ thuật toán trong Hình 3.10.

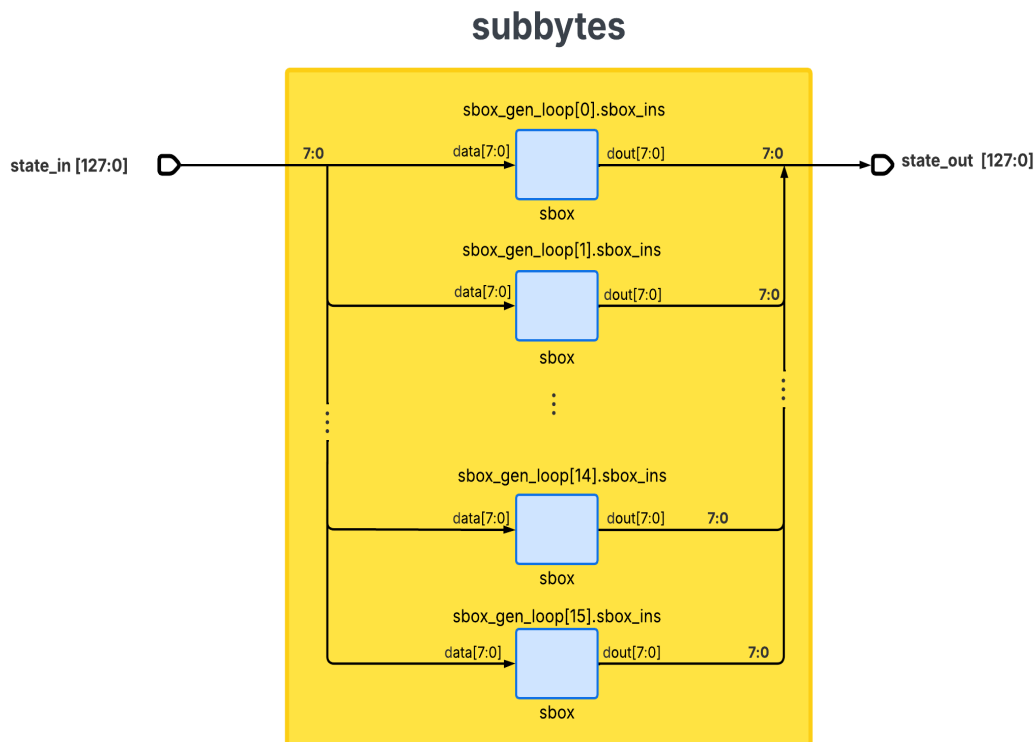


Hình 3.10: Lưu đồ thuật toán của module vòng kết thúc

3.3.2. Thiết kế chi tiết các module mã hóa

Module thay thế byte (sub_bytes): Module sub_bytes thực hiện phép biến đổi phi tuyến duy nhất trong thuật toán AES. Mỗi byte trong ma trận trạng thái 16 byte được thay thế độc lập bằng một byte khác dựa trên bảng tra S-Box.

Trong thiết kế này, S-Box không được hiện thực bằng cách tính toán trực tiếp phép nghịch đảo trong trường Galois kết hợp với biến đổi affine (vốn đòi hỏi các mạch nhân và chia phức tạp), thiết kế này hiện thực hóa S-Box dưới dạng bảng tra cứu (Look-up Table - LUT). Để xử lý toàn bộ khối dữ liệu 128-bit trong một chu kỳ, hệ thống khởi tạo 16 bộ S-Box phần cứng hoạt động hoàn toàn song song. Mỗi bộ S-Box tiếp nhận 1 byte (8-bit) ở ngõ vào và xuất ra 1 byte ở ngõ ra ngay lập tức[7]. Giải pháp này giúp đánh đổi một lượng nhỏ tài nguyên bộ nhớ (hay LUT) trên FPGA để lấy tốc độ xử lý tức thời (O(1) delay). Kiến trúc kết nối của 16 khối S-Box phần cứng này được minh họa chi tiết trong sơ đồ RTL ở Hình 3.11.



Hình 3.11: Sơ đồ khối RTL của module `sub_bytes` và kiến trúc 16 S-Box song song

Các tín hiệu giao tiếp của module `sub_bytes` được định nghĩa chi tiết trong Bảng 3.5.

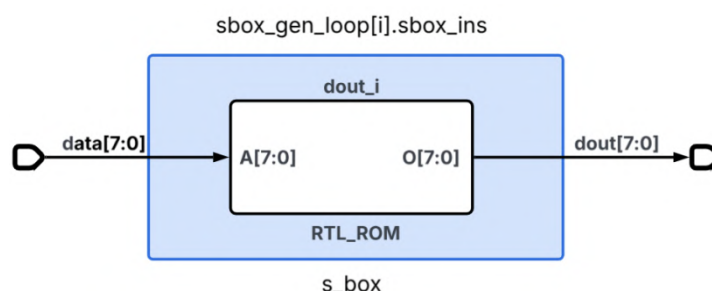
Bảng 3.5: Mô tả các tín hiệu của module `sub_bytes`

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
state_in	Input	128-bit	Ma trận trạng thái trước khi thay thế byte.
state_out	Output	128-bit	Ma trận trạng thái sau khi được ánh xạ qua 16 bộ S-Box song song.

Để thực hiện 16 phép biến đổi S-Box đồng thời trên FPGA, module được thiết kế theo hướng sinh phần cứng bằng cấu trúc generate trong Verilog. Cụ thể, một vòng lặp for với biến `i` chạy từ 0 đến 15 được sử dụng để tạo ra 16 khối S-Box giống nhau. Trong mỗi lần lặp, dữ liệu đầu vào 128-bit `state_in` được tách thành từng byte 8-bit theo vị trí $[(i*8)+7:i*8]$. Mỗi byte này được đưa vào một S-Box riêng để xử lý. Sau đó, kết quả đầu ra từ S-Box sẽ được đưa trở lại đúng vị trí tương ứng trên bus 128-bit `state_out`.

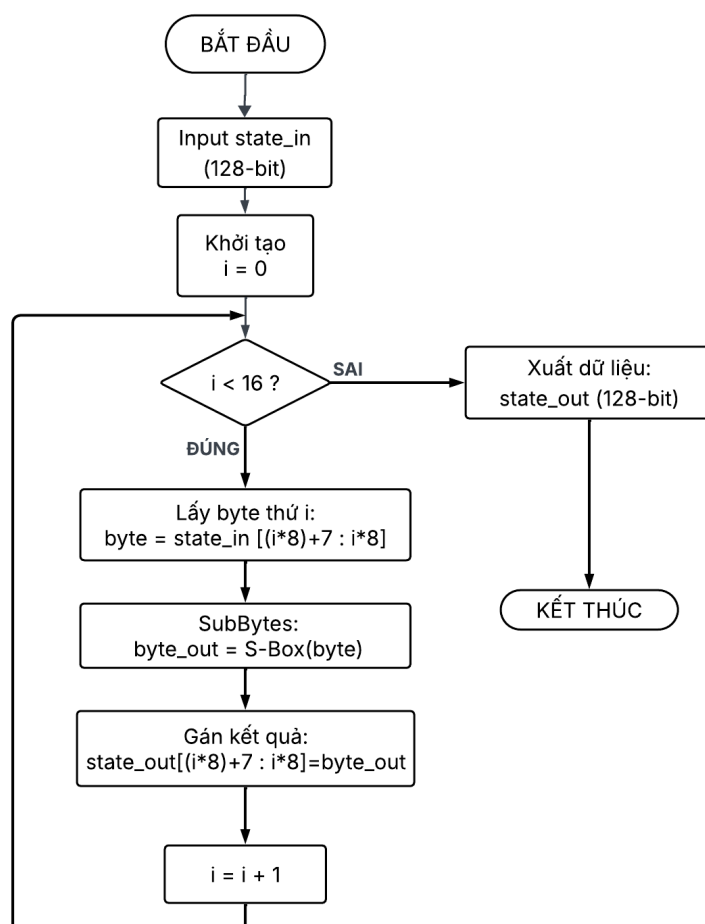
Đi sâu vào kiến trúc nội bộ của từng module S-Box riêng lẻ được sinh ra từ vòng lặp (`sbox_gen_loop[i]`), thay vì mô tả phần cứng dưới dạng các mạng lưới cổng logic phức tạp, giải pháp thiết kế được lựa chọn là định hướng hệ thống hoạt động như một khối bộ nhớ chỉ đọc (Read-Only Memory). Như được minh họa trong Hình 3.12, cốt lõi của mỗi module là một phần tử RTL_ROM. Phần tử này nhận 1 byte dữ liệu đầu vào đóng vai trò làm địa chỉ tra cứu (cổng `A[7:0]`) và ngay lập tức xuất ra giá trị đã

được ánh xạ tương ứng tại cổng dữ liệu ra (cổng O[7:0]). Việc sử dụng cấu trúc ROM phần cứng đảm bảo thời gian truy xuất là đồng đều và nhanh chóng cho mọi giá trị đầu vào.



Hình 3.12: Kiến trúc nội bộ RTL_ROM bên trong một module sbox đơn lẻ

Cách thiết kế này cho phép xử lý đồng thời cả 16 byte, đồng thời giúp mã nguồn gọn hơn và hạn chế sai sót so với việc phải viết lặp lại nhiều lần. Toàn bộ quá trình được minh họa trong lưu đồ thuật toán ở Hình 3.13.



Hình 3.13: Lưu đồ thuật toán và luồng phân tách dữ liệu của module sub_bytes

Module dịch hàng (shift_rows): Module shift_rows đóng vai trò tăng cường tính khuếch tán tuyến tính cho thuật toán bằng cách dịch vòng sang trái các byte trên từng

hàng của ma trận trạng thái. Hàng 0 giữ nguyên, hàng 1 dịch 1 byte, hàng 2 dịch 2 byte và hàng 3 dịch 3 byte.

Ở mức thiết kế RTL, phép biến đổi ShiftRows gần như không tiêu tốn tài nguyên phần cứng (không sử dụng LUT hay thanh ghi). Như được minh họa rõ nét trong sơ đồ kiến trúc RTL ở Hình 3.14, module này không chứa bất kỳ cổng tính toán logic nào, mà chỉ đơn giản là sắp xếp lại vị trí các byte thông qua việc nối dây tương ứng. Thông qua các phép gán liên tục trong Verilog, các byte trên bus dữ liệu 128-bit đầu vào được hoán đổi vị trí và nối trực tiếp sang ngõ ra. Nhờ đó, phép dịch hàng được thực hiện mà không phát sinh thêm độ trễ do cổng logic.



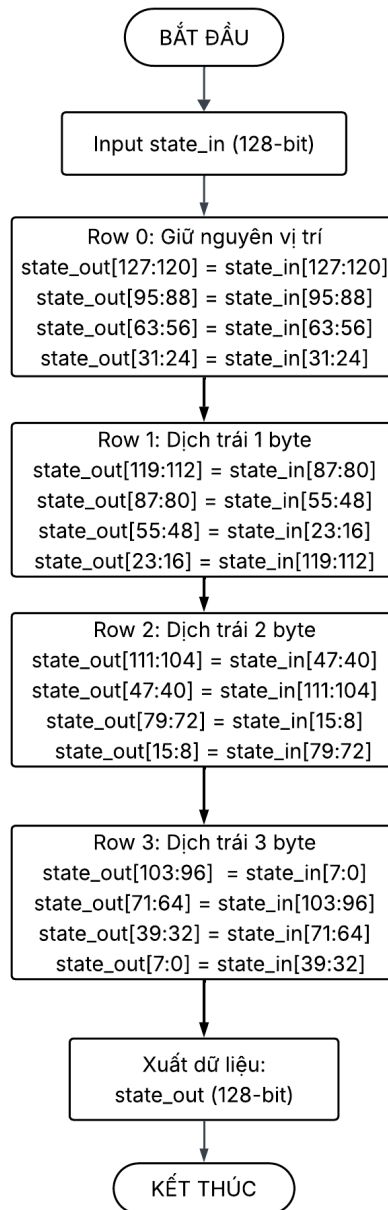
Hình 3.14: Sơ đồ định tuyến tín hiệu RTL của module *shift_rows*

Các tín hiệu giao tiếp chi tiết của module *shift_rows* được định nghĩa trong Bảng 3.6.

Bảng 3.6: Mô tả các tín hiệu của module *shift_rows*

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
state_in	Input	128-bit	Ma trận trạng thái đầu vào từ khối thay thế byte (sub_bytes).
state_out	Output	128-bit	Ma trận trạng thái sau khi đã dịch hàng (sắp xếp lại đường dây).

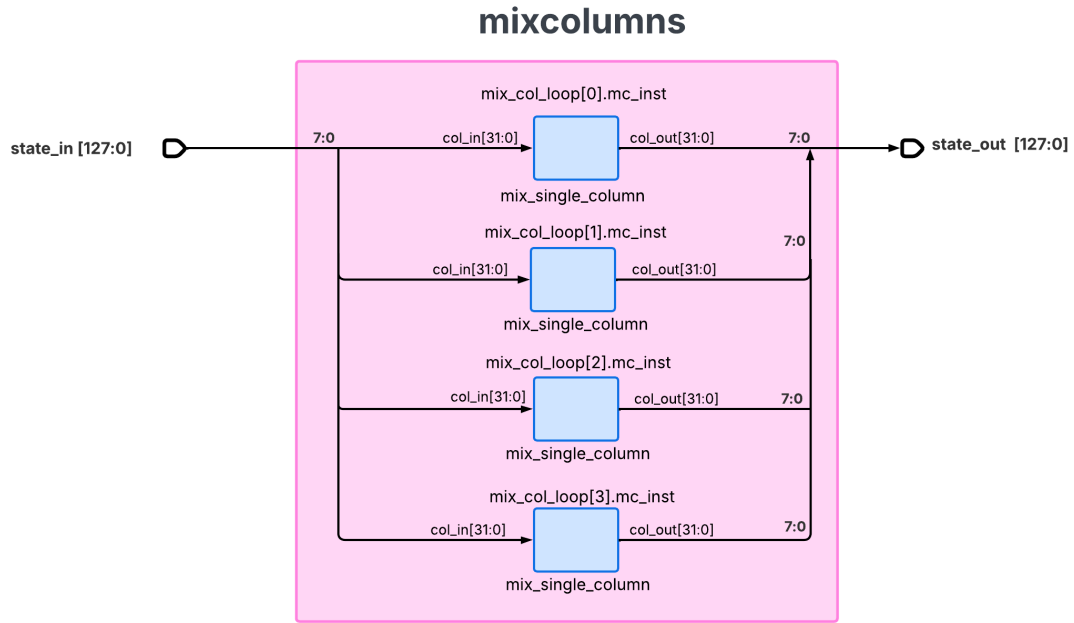
Do phép biến đổi này chỉ là hoán đổi vị trí các byte trong ma trận trạng thái, việc hiện thực được thực hiện bằng các câu lệnh gán để nối các dải 8-bit từ bus đầu vào *state_in* sang các vị trí tương ứng trên bus đầu ra *state_out*. Khối dữ liệu 128-bit được chia thành 16 byte (tương ứng ma trận 4×4). Các byte ở Hàng 0 được giữ nguyên vị trí, trong khi các byte ở Hàng 1, Hàng 2 và Hàng 3 được sắp xếp lại để thực hiện dịch vòng sang trái lần lượt 1, 2 và 3 byte. Quá trình này được minh họa trong lưu đồ thuật toán ở Hình 3.15.



Hình 3.15: Lưu đồ thuật toán của module *shift_rows*

Module trộn cột (mix_columns): *mix_columns* thực hiện phép nhân ma trận cột của trạng thái với một ma trận hằng số phân nhánh tối đa, các phép toán được thực hiện trên trường hữu hạn $GF(2^8)$. Đây là khối tạo ra độ khuếch tán mạnh nhất, đảm bảo sự thay đổi của 1 byte đầu vào sẽ ảnh hưởng đến 4 byte đầu ra.

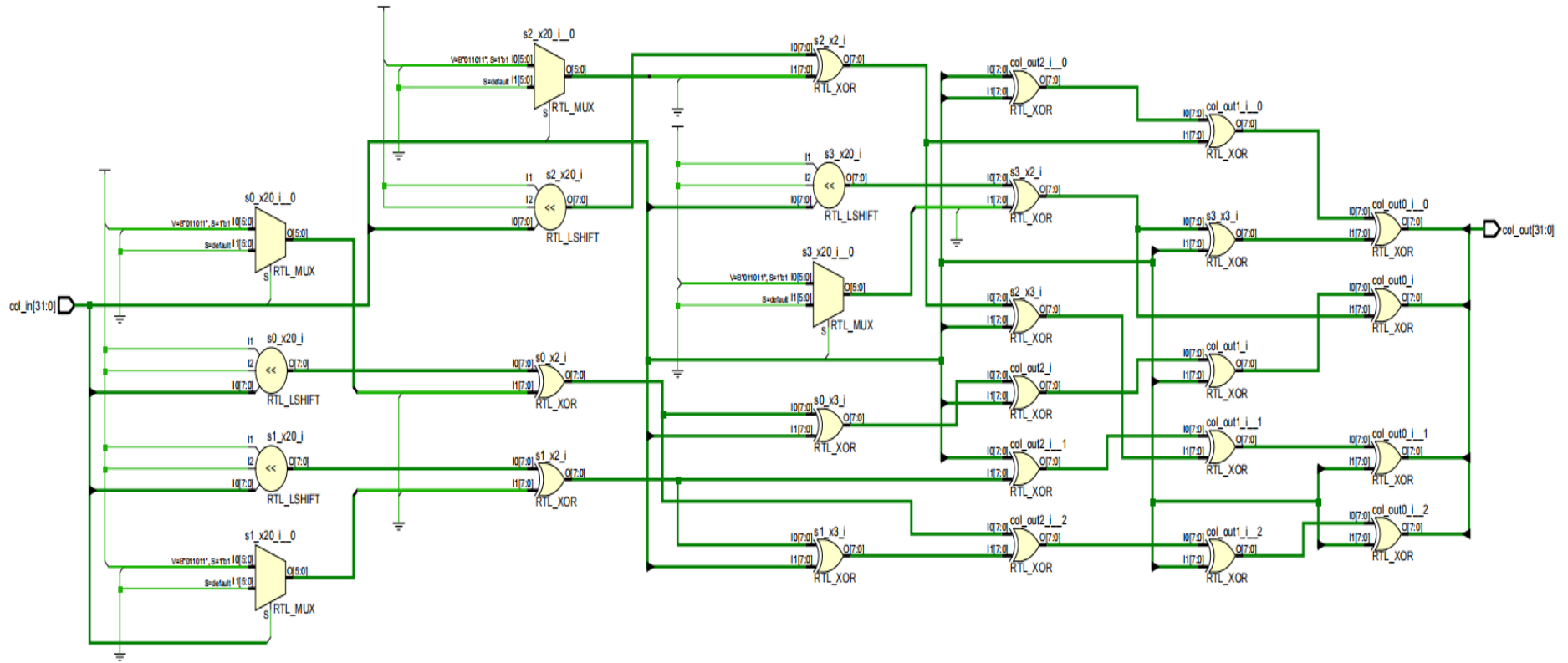
Để xử lý toàn bộ ma trận trạng thái 128-bit trong một chu kỳ, thiết kế phân tách bus dữ liệu thành 4 cột hoạt động độc lập (mỗi cột rộng 32-bit). Thông qua cấu trúc lặp generate (*mix_col_loop*), hệ thống khởi tạo 4 module con mang tên *mix_single_column* chạy song song. Kiến trúc phân chia dữ liệu và định tuyến tổng thể của khối trộn cột này được mô tả trực quan trong sơ đồ RTL ở Hình 3.16.



Hình 3.16: Sơ đồ khối RTL của module *mix_columns*.

Đi sâu vào thuật toán, trên phương diện lý thuyết, phép nhân ma trận yêu cầu các mạch nhân phần cứng rất tốn kém diện tích. Để tối ưu hóa, thiết kế phân tách các phép nhân với hằng số {02} và {03} thành các phép dịch bit và phép XOR. Cụ thể, module sử dụng một khối cơ bản gọi là *xtime*. Chức năng của *xtime* là nhân một byte với {02} trên $GF(2^8)$ bằng cách: dịch trái byte đó 1 bit, và nếu bit MSB bằng 1 thì tiến hành XOR với đa thức bất khả quy 0x1b. Từ đó, phép nhân với {03} được suy ra theo công thức: $\{03\} \cdot x = (\{02\} \cdot x) \oplus x$. Nhờ cách triển khai này, toàn bộ module *mix_columns* chỉ cần sử dụng các phép XOR và dịch bit, không cần đến bộ nhân phức tạp loại bỏ hoàn toàn bộ nhân phức tạp, giúp giảm thiểu tối đa diện tích thực thi và rút ngắn thời gian xử lý.

Kiến trúc nội bộ hiện thực hóa chuỗi logic trong mỗi module *mix_single_column* được minh họa chi tiết tại Hình 3.17. Dựa vào sơ đồ phân tích, ta có thể thấy quá trình xử lý không chứa bất kỳ bộ nhân (Multiplier) nào. Thay vào đó, toàn bộ quá trình *xtime* và nhân ma trận được tổng hợp hoàn toàn từ các khối dịch bit (RTL_LSHIFT), bộ ghép kênh để kiểm tra điều kiện MSB (RTL_MUX), kết hợp cùng cấu trúc cây cổng XOR (RTL_XOR) nhiều tầng. Cách triển khai trên giúp giảm đáng kể tài nguyên sử dụng trên FPGA, đồng thời rút ngắn thời gian xử lý so với việc dùng bộ nhân phần cứng.



Hình 3.17: Kiến trúc RTL nội bộ của khối `mix_single_column` hiện thực hóa thuật toán

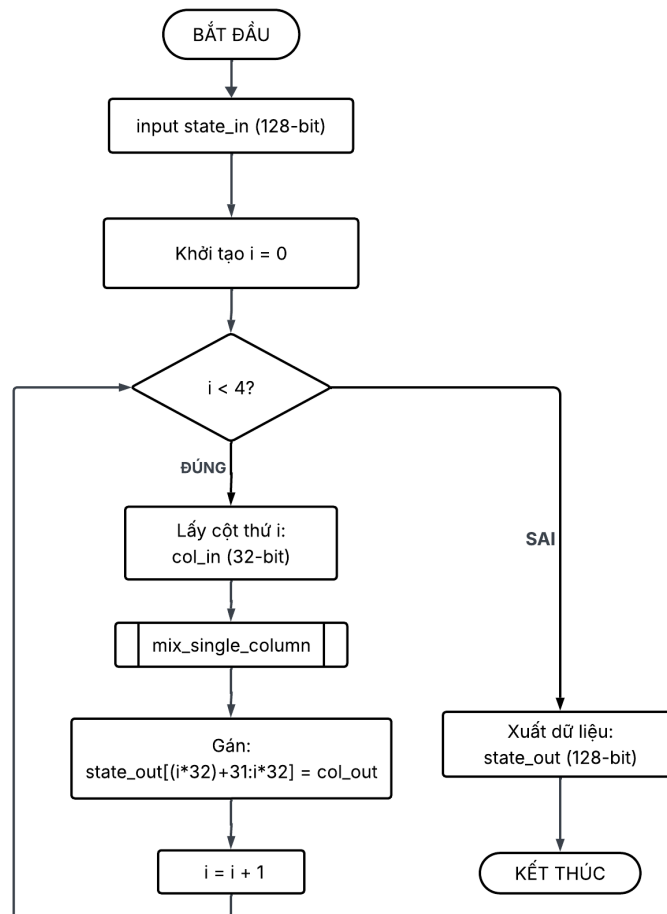
Các tín hiệu giao tiếp của module mix_columns được mô tả trong Bảng 3.7.

Bảng 3.7: Mô tả các tín hiệu giao tiếp của module mix_columns

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
state_in	Input	128-bit	Ma trận trạng thái đầu vào từ khối dịch hàng (shift_rows).
state_out	Output	128-bit	Ma trận trạng thái sau khi hoàn tất phép nhân khuếch tán trên trường $GF(2^8)$.

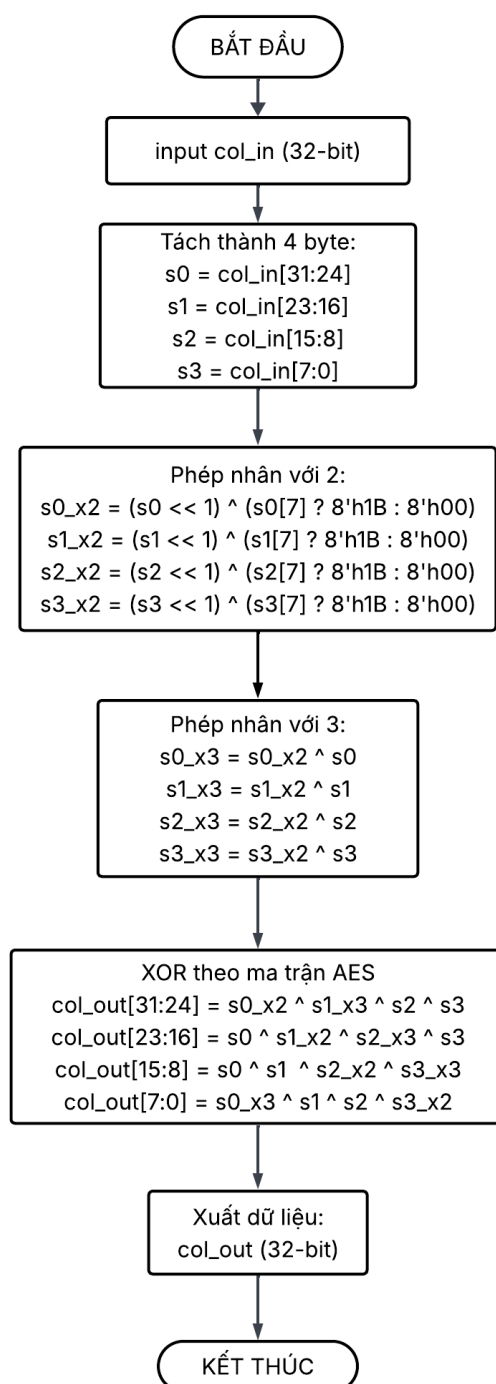
Để hiện thực phép biến đổi MixColumns trên phần cứng, thiết kế được xây dựng theo hướng chia thành hai mức xử lý: mức xử lý toàn khối và mức xử lý từng cột. Cách tiếp cận này giúp đơn giản hóa việc thiết kế và thuận tiện cho việc kiểm tra.

Ở mức toàn khối (mix_columns), dữ liệu 128-bit được chia thành 4 phần, mỗi phần 32-bit tương ứng với một cột của ma trận trạng thái. Thay vì xử lý nguyên khối 128-bit phức tạp, hệ thống sử dụng cấu trúc sinh phần cứng để cấp phát 4 dải dữ liệu này vào 4 khối xử lý cột độc lập. Các khối này sẽ hoạt động hoàn toàn song song với nhau. Luồng phân tách, gọi module xử lý và gán dữ liệu ở cấp độ này được mô tả chi tiết trong lưu đồ thuật toán tại Hình 3.18.



Hình 3.18: Lưu đồ thuật toán toàn khối mix_columns

Đi sâu vào mức xử lý bên trong mỗi khối cột (mix_single_column), mỗi cột 32-bit được tách thành 4 byte độc lập. Các byte này được xử lý theo các phép toán trong trường $GF(2^8)$, bao gồm phép nhân với $\{2\}$, $\{3\}$ và các phép XOR theo ma trận chuẩn của AES. Toàn bộ trình tự tính toán được minh họa trong lưu đồ thuật toán ở Hình 3.19.



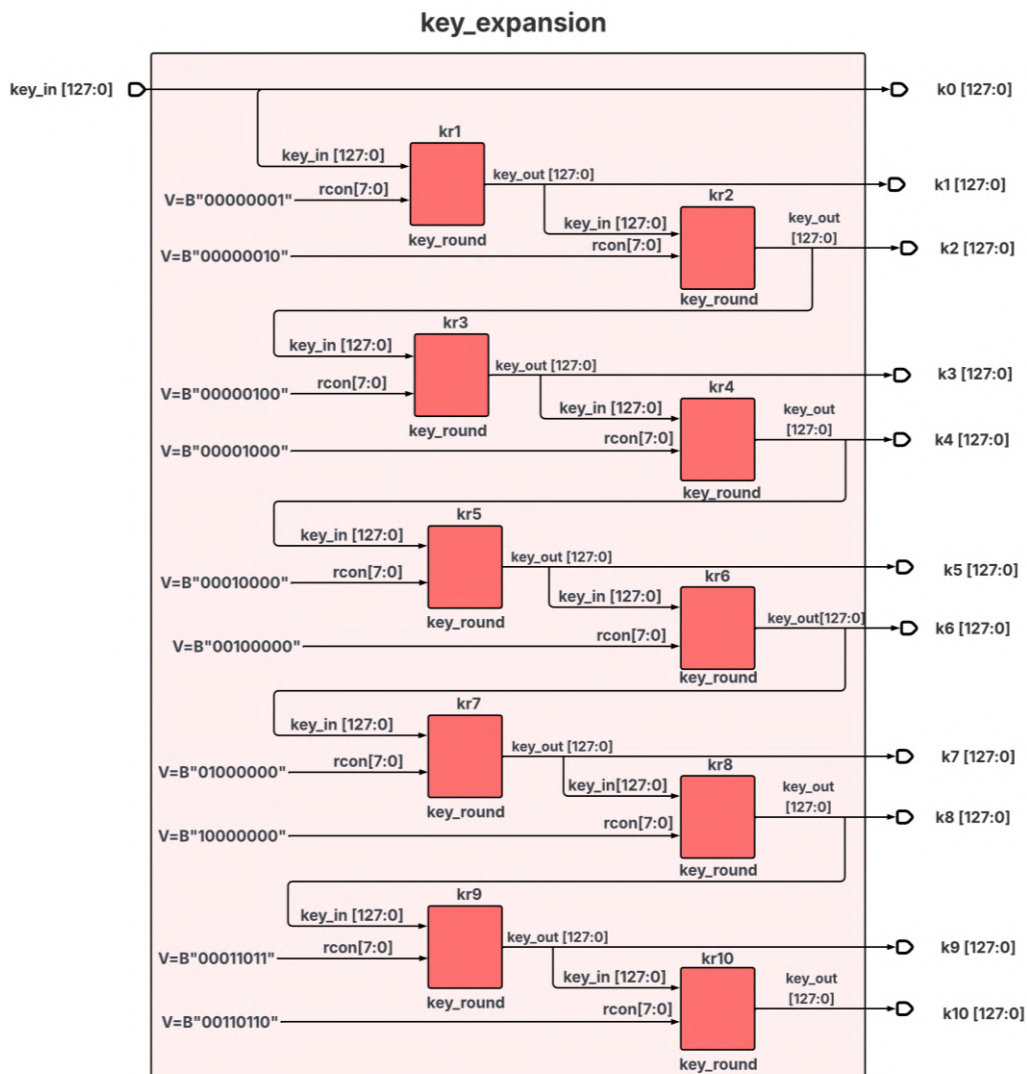
Hình 3.19: Lưu đồ thuật toán module mix_single_column

3.4. Thiết kế khối sinh khóa (Key Expansion)

3.4.1. Kiến trúc sinh khóa đồng bộ Pipeline

Để đáp ứng yêu cầu thông lượng cao của tuyến dữ liệu AES-128 được thiết kế theo kiến trúc đường ống 10 tầng, khối sinh khóa (Key Expansion) không sử dụng phương pháp tính toán lặp vòng tuần tự truyền thống - vốn đòi hỏi nhiều chu kỳ xung nhịp để sinh ra từng khóa. Thay vào đó, toàn bộ kiến trúc sinh khóa đã được trải phẳng trên không gian phân cứng[7].

Giải pháp này sử dụng logic tổ hợp để tính toán trước toàn bộ 11 khóa vòng ($k_0, k_1, \dots, k_{10}$) và cấp phát đồng thời, liên tục cho cả 10 tầng Pipeline của tuyến dữ liệu. Kiến trúc kết nối tổng thể của module được minh họa trực quan trong sơ đồ RTL tại Hình 3.20.



Hình 3.20: Sơ đồ khối RTL tổng thể của kiến trúc sinh khóa

Dựa trên sơ đồ kiến trúc RTL, hệ thống được cấu trúc từ một chuỗi gồm 10 module sinh khóa đơn vòng `key_round` (từ `kr1` đến `kr10`) mắc nối tiếp nhau. Luồng dữ liệu sinh khóa hoạt động theo cơ chế sau:

- Khóa khởi tạo (k_0): Được lấy trực tiếp từ khóa đầu vào `key_in` từ hệ thống.
- Khóa vòng từ k_1 đến k_{10} : Mỗi khóa vòng được tạo ra từ khóa của vòng trước. Cụ thể, `key_in` được đưa vào khối đầu tiên để sinh ra k_1 , sau đó k_1 tiếp tục làm đầu vào cho khối kế tiếp để tạo k_2 . Quá trình này được lặp lại tuần tự cho đến khi sinh đủ k_{10} .
- Tối ưu hóa hằng số vòng: Các giá trị `Rcon` không được tạo động bằng bộ đếm hay bộ nhớ. Thay vào đó, thiết kế đã mã hóa cứng các giá trị nhị phân `Rcon` tương ứng (ví dụ: `8'b00000001` cho vòng 1, `8'b00011011` cho vòng 9) trực tiếp vào các chân ngõ vào của từng khối `kr`. Các giá trị `Rcon` không được tạo động bằng bộ đếm hay bộ nhớ.

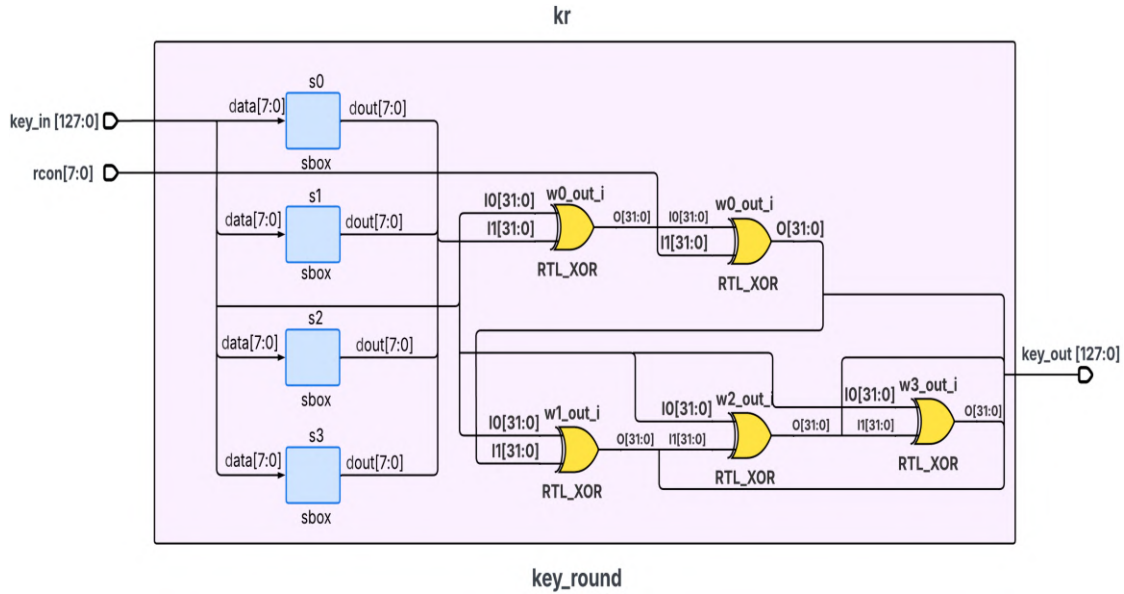
Để kết nối đồng bộ với tuyến dữ liệu chính, các tín hiệu của module `Key Expansion` tổng được định nghĩa chi tiết trong Bảng 3.8.

Bảng 3.8: Mô tả các tín hiệu của module tổng `Key Expansion`

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>key_in</code>	Input	128-bit	Tiếp nhận khóa bí mật ban đầu.
<code>k0</code>	Output	128-bit	Khóa khởi tạo, kết nối trực tiếp đến module <code>enc_ark0</code> .
<code>k1 ... k9</code>	Output	128-bit	Tập hợp 9 khóa vòng tiêu chuẩn, cấp phát tương ứng cho 9 tầng <code>aes_round</code> .
<code>k10</code>	Output	128-bit	Khóa vòng kết thúc, cấp phát riêng cho tầng <code>aes_final_round</code> .

3.4.2. Thiết kế module sinh khóa đơn vòng

Module `key_round` là khối chính trong quá trình sinh khóa, có nhiệm vụ tạo ra khóa 128-bit của vòng hiện tại từ khóa của vòng trước và hằng số. Bên trong module, các phép biến đổi như `RotWord`, `SubWord` và `XOR` được kết hợp trong cùng một luồng xử lý bằng logic tổ hợp, thay vì tách thành nhiều khối riêng để tối thiểu hóa độ trễ. Kiến trúc nội bộ của khối này được minh họa chi tiết trong sơ đồ RTL tại Hình 3.21.

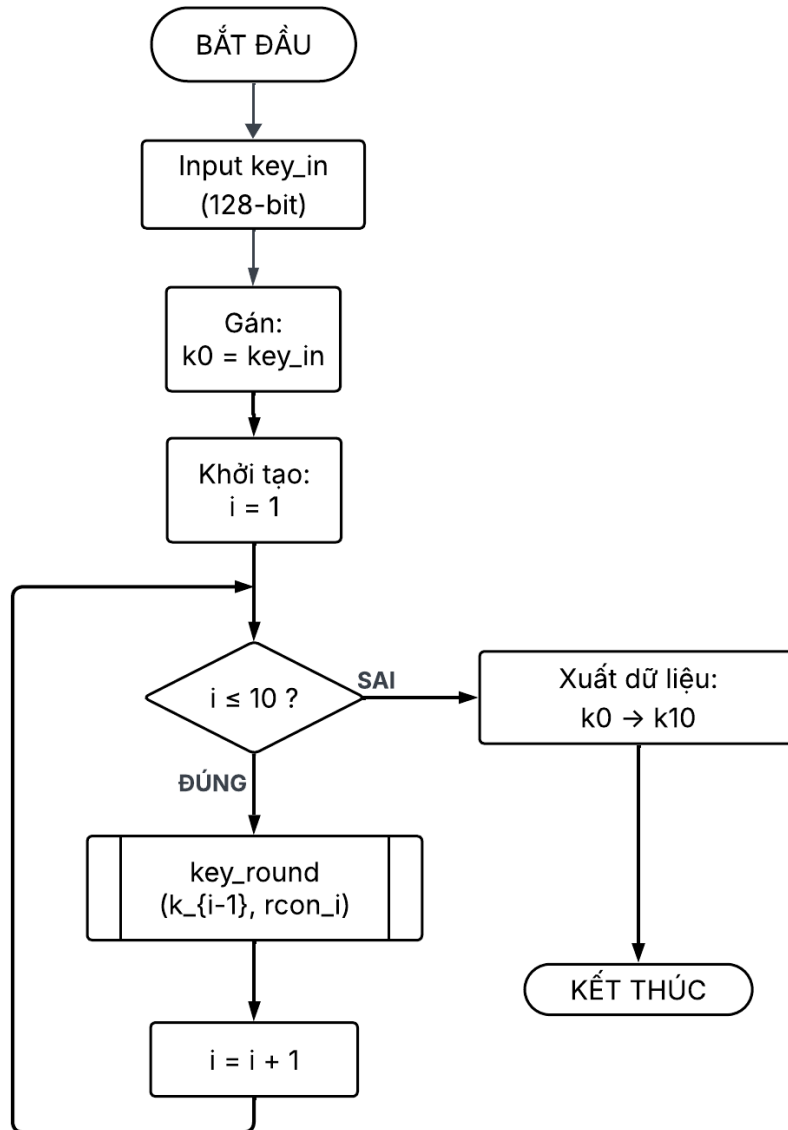


Hình 3.21: Sơ đồ kiến trúc RTL của module sinh khóa đơn vòng *key_round*

Quá trình xử lý bên trong module *key_round* được thực hiện qua các bước chính sau:

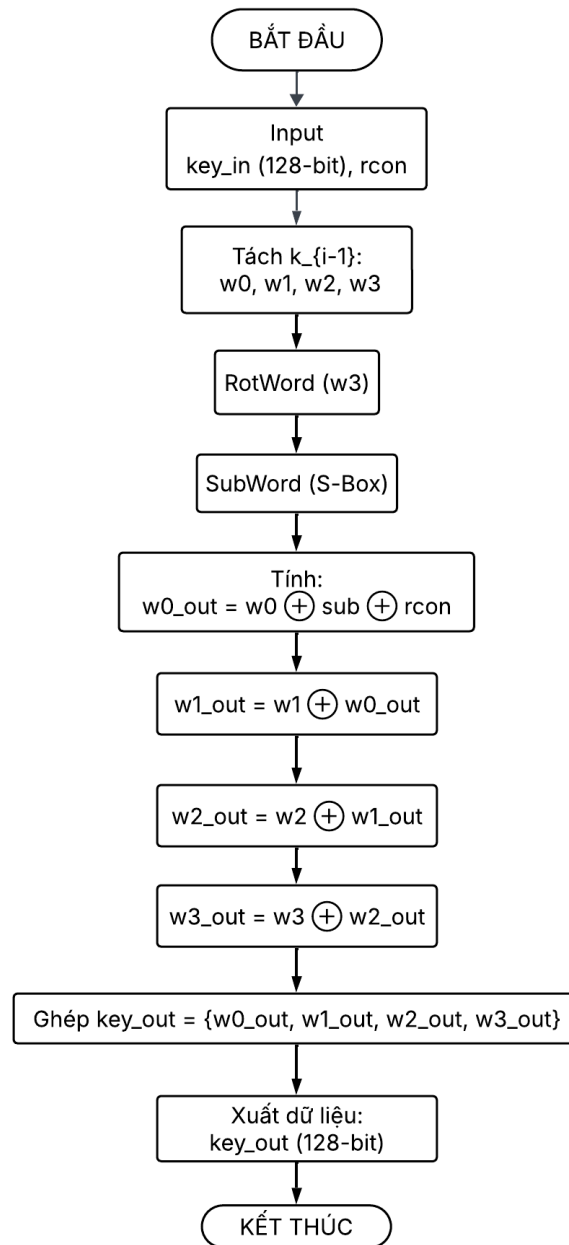
- Phân tách từ và dịch vòng (RotWord): Khóa đầu vào 128-bit *key_in* được chia thành 4 từ 32-bit: w_0 , w_1 , w_2 , w_3 . Tương tự như phép ShiftRows ở tuyến dữ liệu, phép dịch vòng RotWord trên từ w_3 hoàn toàn không tiêu tốn tài nguyên cổng logic.
- Thay thế từ (SubWord) bằng cách tái sử dụng LUT: Sau khi dịch vòng, từ w_3 (lúc này là rot_w_3) được tách thành 4 byte độc lập và đưa qua phép biến đổi phi tuyến SubWord. Như quan sát trên Hình 3.19, thiết kế đã khởi tạo 4 thực thể *sbox* (s_0 đến s_3) hoạt động song song. Một điểm tối ưu quan trọng ở đây là thiết kế tái sử dụng lại chính xác cấu trúc phần cứng của bảng tra cứu LUT (RTL_ROM) đã được thiết kế cho module *sub_bytes* trước đó, giúp tiết kiệm tài nguyên và đảm bảo tính nhất quán.
- Kết hợp và trộn khóa (XOR với Rcon): Các từ đầu ra được tính bằng các phép XOR. Từ thứ nhất (w_{0_out}) là kết quả của phép XOR giữa từ w_0 đầu vào, từ w_3 đã qua SubWord, và hằng số *rcon*. Lưu ý rằng *rcon* (8-bit) được ghép thêm 24 bit 0 để tạo thành 32-bit trước khi đưa vào cổng logic. Các từ tiếp theo (w_{1_out} , w_{2_out} , w_{3_out}) được tính toán nối tiếp theo nguyên lý: từ đầu ra hiện tại bằng từ đầu vào tương ứng XOR với từ đầu ra liền trước nó (ví dụ: $w_{1_out} = w_1 \oplus w_{0_out}$).

Luồng xử lý của khối Key Expansion được mô hình hóa qua hai lưu đồ thuật toán. Ở cấp độ hệ thống, trình tự tổng hợp chuỗi 11 khóa vòng (k_0 đến k_{10}) được khái quát hóa theo đúng logic của chuẩn AES trong Hình 3.22.



Hình 3.22: Lưu đồ thuật toán mô tả chuỗi sinh khóa vòng tổng thể

Đi sâu vào bên trong module `key_round`, Hình 3.23 mô tả chi tiết luồng xử lý dữ liệu của mạch. Trong thiết kế này, các phép biến đổi được thực hiện bằng các phép gán liên tục nhằm rút ngắn thời gian xử lý. Cụ thể, dữ liệu đầu vào được tách thành các từ, sau đó thực hiện phép xoay vòng (RotWord) bằng cách hoán đổi vị trí các byte. Tiếp theo, từ này được đưa qua các khối S-Box để thực hiện biến đổi SubWord. Cuối cùng, các từ đầu ra được tính thông qua các phép XOR với khóa ban đầu và hằng số `rcon`.



Hình 3.23: Lưu đồ thuật toán mô tả luồng xử lý của module *key_round*

3.5. Thiết kế khối giải mã AES

Quá trình giải mã AES có nhiệm vụ khôi phục bản rõ từ bản mã bằng cách thực hiện tuần tự các phép biến đổi nghịch đảo của tuyến mã hóa và sử dụng các khóa vòng theo thứ tự ngược, từ k_{10} đến k_0 .

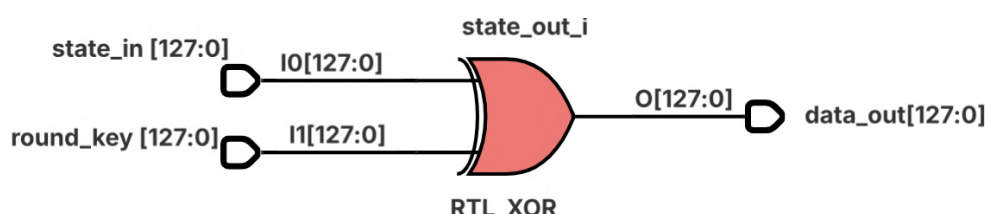
Quá trình xử lý bắt đầu tại khối tiền xử lý (*dec_ark10*), nơi bản mã được kết hợp với khóa vòng cuối k_{10} . Tiếp theo, dữ liệu lần lượt đi qua 9 vòng giải mã tiêu chuẩn (*inv_aes_round*), mỗi vòng thực hiện các phép biến đổi nghịch đảo của AES. Cuối cùng, dữ liệu được khôi phục hoàn chỉnh tại khối giải mã kết thúc (*inv_aes_final_round*) với khóa gốc k_0 . Toàn bộ 11 khóa vòng được cung cấp từ khối Key Expansion và sử dụng theo trình tự ngược trong suốt quá trình giải mã.

3.5.1. Thiết kế các khối xử lý vòng giải mã

Để duy trì thông lượng tương đương tuyến mã hóa, lõi giải mã AES-128 được triển khai theo kiến trúc pipeline 10 tầng. Quá trình khôi phục bản rõ gồm ba giai đoạn chính:

- Vòng tiền xử lý: Dữ liệu bản mã 128-bit đầu tiên được đưa qua phép toán AddRoundKey để XOR với khóa k_{10} .
- Các vòng tiêu chuẩn: Dữ liệu lần lượt đi qua các phép biến đổi InvShiftRows, InvSubBytes, AddRoundKey và InvMixColumns. Các khóa vòng k_9 đến k_1 được cung cấp trực tiếp từ khối Key Expansion cho các tầng xử lý tương ứng.
- Vòng kết thúc: Thực hiện InvShiftRows, InvSubBytes và AddRoundKey với khóa gốc k_0 . Ở vòng này, phép trộn cột nghịch đảo bị loại bỏ theo chuẩn AES.

Khối vòng tiền xử lý (Initial Round) thực hiện bước khởi tạo của quá trình giải mã. Như minh họa trong Hình 3.24, khối có cấu trúc đơn giản, chỉ bao gồm phép AddRoundKey giữa bản mã đầu vào 128-bit và khóa vòng cuối k_{10} thông qua phép toán XOR. Kết quả thu được là trạng thái dữ liệu ban đầu, làm đầu vào cho các tầng giải mã tiếp theo trong kiến trúc pipeline.



Hình 3.24: Sơ đồ khối RTL của module vòng tiền xử lý giải mã *dec_ark10*

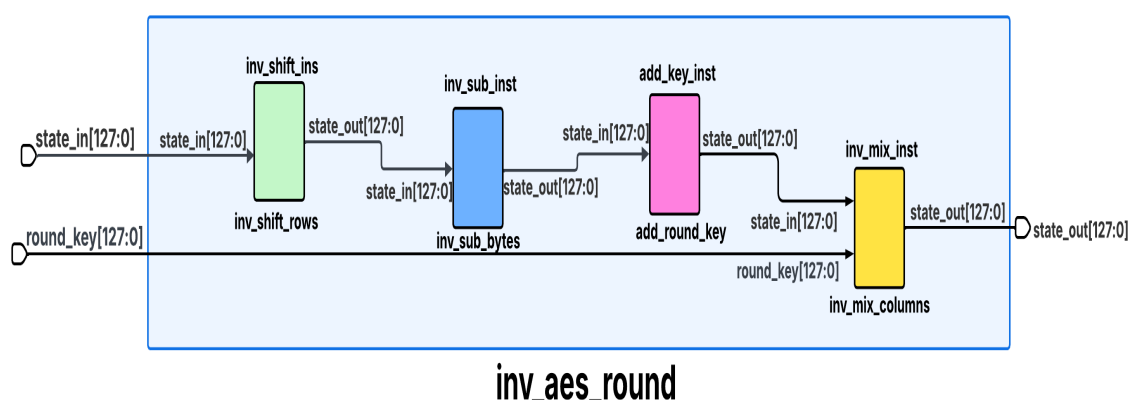
Về mặt giao tiếp, khối có kiến trúc RTL cốt lõi là một bộ logic XOR 128-bit (RTL_XOR) với các ngõ vào và ngõ ra được định nghĩa chi tiết như trong Bảng 3.9.

Bảng 3.9: Mô tả các tín hiệu của module vòng tiền xử lý giải mã (*dec_ark0*)

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
state_in	Input	128-bit	Tiếp nhận khối dữ liệu bản mã (Ciphertext) ban đầu.
round_key	Input	128-bit	Tiếp nhận khóa vòng cuối cùng (k_{10}) từ khối Key Expansion.
state_out	Output	128-bit	Xuất trạng thái dữ liệu sau khi thực hiện phép XOR, chuẩn bị đưa vào vòng giải mã tiêu chuẩn đầu tiên.

Tương tự khối `enc_ark0` ở mã hóa, khối vòng tiền xử lý giải mã cũng sử dụng công logic XOR 128-bit. Trình tự thực thi được cũng tương tự với trình tự của mã hóa qua lưu đồ giải thuật ở Hình 3.4.

Module vòng giải mã tiêu chuẩn (`inv_aes_round`) được triển khai cho 9 vòng xử lý và thực hiện đầy đủ các phép biến đổi nghịch đảo của AES. Như minh họa trong Hình 3.25, khối gồm bốn thành phần chức năng nối tiếp theo trình tự giải mã: dịch hàng nghịch (`inv_shift_rows`), thay thế byte nghịch (`inv_sub_bytes`), cộng khóa vòng (`add_round_key`) và cuối cùng là trộn cột nghịch (`inv_mix_columns`).



Hình 3.25: Kiến trúc RTL của module vòng giải mã tiêu chuẩn `inv_aes_round`

Module `inv_aes_round` được thiết kế với các tín hiệu giao tiếp được mô tả chi tiết trong Bảng 3.10.

Bảng 3.10: Mô tả các tín hiệu của module vòng giải mã tiêu chuẩn (`inv_aes_round`)

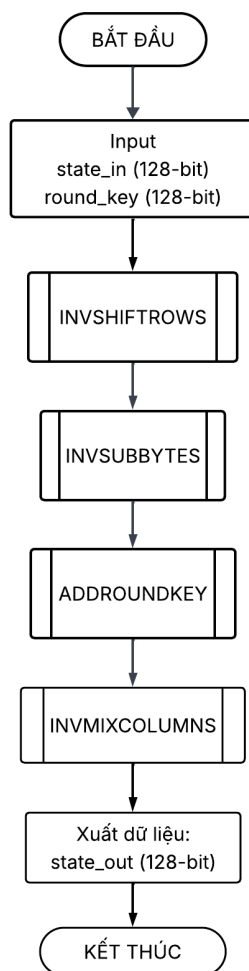
Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>state_in</code>	Input	128-bit	Tiếp nhận ma trận trạng thái từ tầng giải mã trước đó.
<code>round_key</code>	Input	128-bit	Tiếp nhận bộ khóa vòng tương ứng (từ <code>k_9</code> lùi dần về <code>k_1</code>)
<code>state_out</code>	Output	128-bit	Xuất ma trận trạng thái kết quả sau khi hoàn tất 4 phép toán nghịch đảo.

Dựa trên sơ đồ kiến trúc RTL, dữ liệu 128-bit `state_in` lần lượt đi qua các khối chức năng theo thứ tự sau:

- `inv_shift_rows`: Khối dữ liệu đi vào khối dịch hàng nghịch đảo, thực hiện dịch vòng các hàng của ma trận trạng thái sang phải nhằm khôi phục lại vị trí ban đầu của các byte.

- `inv_sub_bytes`: Dữ liệu tiếp tục đi vào khối thay thế nghịch đảo. Tại đây, 16 byte dữ liệu được ánh xạ ngược qua 16 bộ Inverse S-Box phần cứng.
- `add_round_key`: Kết quả từ S-Box nghịch đảo được tiến hành XOR trực tiếp với tín hiệu `round_key` tương ứng của vòng đó.
- `inv_mix_columns`: Tại chặng cuối, ma trận trạng thái được nhân với ma trận nghịch đảo trên trường Galois $GF(2^8)$ (với các hằng số $\{09\}$, $\{0b\}$, $\{0d\}$, $\{0e\}$) để hoàn tất quá trình giải khuếch tán dữ liệu, tạo ra ngõ ra cuối cùng `state_out`.

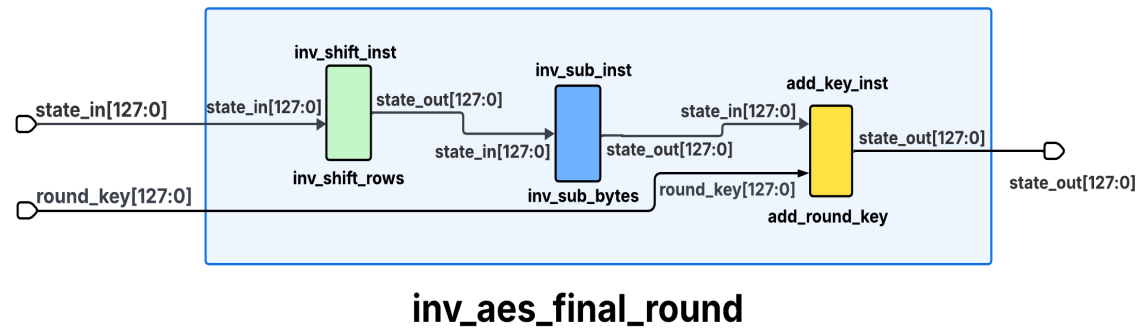
Khối `inv_aes_round` được thiết kế hoàn toàn bằng logic tổ hợp. Trong lưu đồ, các bước được biểu diễn dưới dạng các khối tiến trình định nghĩa sẵn, nhấn mạnh mỗi phép biến đổi tương ứng với một module phần cứng độc lập. Trình tự xử lý tổng thể của luồng dữ liệu được mô tả trực quan trong lưu đồ thuật toán ở Hình 3.26.



Hình 3.26: Lưu đồ thuật toán của module vòng giải mã tiêu chuẩn

Khối vòng giải mã kết thúc (`inv_aes_final_round`): Được bố trí ở tầng pipeline thứ 10, khối `inv_aes_final_round` có nhiệm vụ tạo ra bản rõ đầu ra. So với vòng giải mã tiêu chuẩn, khối trộn cột nghịch đảo (`inv_mix_columns`) đã được lược bỏ khỏi chuỗi

xử lý, như minh họa tại Hình 3.27. Cụ thể, dữ liệu chỉ đi qua ba bước: `inv_shift_rows`, `inv_sub_bytes` và `add_round_key`.



Hình 3.27: Kiến trúc RTL của module vòng giải mã kết thúc `inv_aes_final_round`

Khối `inv_aes_final_round` đảm nhiệm giai đoạn cuối của quá trình giải mã và tạo ra bản rõ hoàn chỉnh tại đầu ra. Các tín hiệu được mô tả cụ thể trong Bảng 3.11.

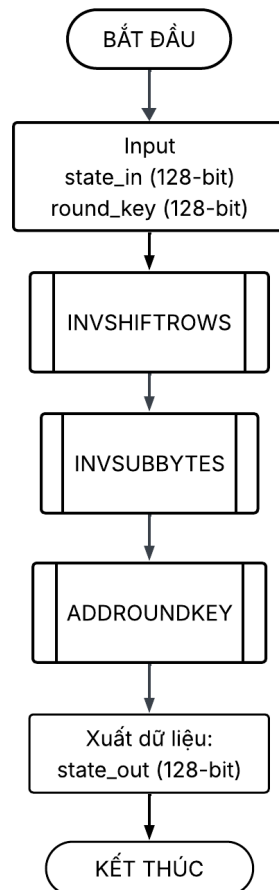
Bảng 3.11: Mô tả các tín hiệu của khối vòng giải mã kết thúc (`inv_aes_final_round`)

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>state_in</code>	Input	128-bit	Tiếp nhận ma trận trạng thái trung gian từ tầng Pipeline thứ 9.
<code>round_key</code>	Input	128-bit	Tiếp nhận khóa vòng gốc ban đầu (<code>k_0</code>)
<code>state_out</code>	Output	128-bit	Xuất bản rõ hoàn chỉnh (Plaintext), chốt vào bộ đệm ngõ ra.

Dựa trên sơ đồ RTL, do sự lược bỏ của khối `inv_mix_columns`, luồng dữ liệu tổ hợp bên trong khối vòng kết thúc được rút ngắn chỉ còn 3 chặng xử lý:

- `inv_shift_inst`: Trạng thái đầu vào `state_in` trước tiên đi qua khối `inv_shift_rows` để dịch phải tuần tự các hàng.
- `inv_sub_inst`: Kết quả được truyền tới khối `inv_sub_bytes` để thay thế phi tuyến song song qua các Inverse S-Box.
- `add_key_inst`: Dữ liệu được tiến hành XOR với khóa vòng `k_0` để khôi phục lại chính xác bản rõ `state_out` cuối cùng.

Việc không thực hiện phép biến đổi Inverse MixColumns phức tạp giúp cấu trúc của khối được rút gọn đáng kể, giảm diện tích và độ trễ trên mạch. Các bước thực thi luồng dữ liệu này được thể hiện rõ ràng trong lưu đồ thuật toán ở Hình 3.28.



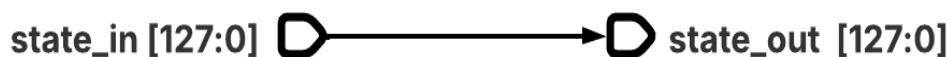
Hình 3.28: Lưu đồ thuật toán của khối vòng giải mã kết thúc *inv_aes_final_round*

3.5.2. Thiết kế chi tiết các module giải mã

Khối cộng khóa vòng (AddRoundKey): Vì bản chất của toán tử logic XOR là tự nghịch đảo ($A \oplus B = A$), khối *add_round_key* được thiết kế hoàn toàn giống với mã hóa. Khối dữ liệu 128-bit trạng thái được XOR trực tiếp với khóa vòng tương ứng.

Khối dịch hàng nghịch đảo (InvShiftRows): Khối *inv_shift_rows* có nhiệm vụ khôi phục lại vị trí ban đầu của các byte trong ma trận trạng thái sau bước *ShiftRows* ở phía mã hóa. Phép biến đổi này thực hiện dịch vòng sang phải trên từng hàng của ma trận trạng thái: hàng 0 giữ nguyên, hàng 1 dịch phải 1 byte, hàng 2 dịch phải 2 byte và hàng 3 dịch phải 3 byte.

Tương tự phía mã hóa, khối *inv_shift_rows* hầu như không tiêu tốn tài nguyên phần cứng do chỉ thực hiện hoán đổi vị trí các byte trên bus dữ liệu 128-bit bằng các phép gán trực tiếp.



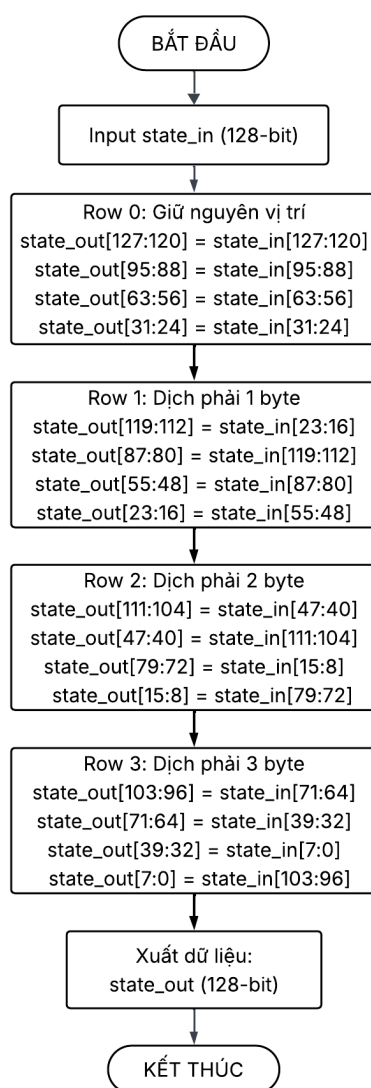
Hình 3.29: Sơ đồ định tuyến tín hiệu RTL của khối *inv_shift_rows*

Các tín hiệu của khối `inv_shift_rows` được định nghĩa trong Bảng 3.12.

Bảng 3.12: Mô tả cổng giao tiếp của khối `inv_shift_rows`

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>state_in</code>	Input	128-bit	Ma trận trạng thái đầu vào cần khôi phục vị trí hàng.
<code>state_out</code>	Output	128-bit	Ma trận trạng thái sau khi đã được dịch vòng nghịch đảo (sắp xếp lại đường dây).

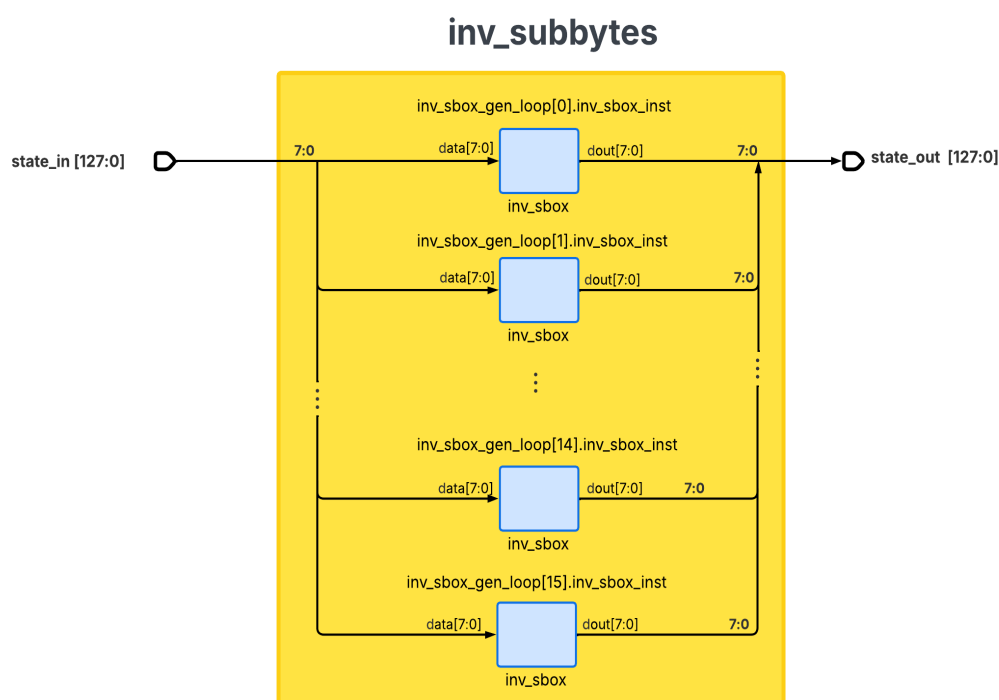
Phép biến đổi `InvShiftRows` thực chất là quá trình hoán đổi lại vị trí các byte trong ma trận trạng thái theo chiều ngược với bước `ShiftRows` ở phía mã hóa. Trong thiết kế, khối dữ liệu 128-bit `state_in` được tách thành 16 byte tương ứng với ma trận trạng thái 4×4 . Các byte thuộc hàng 0 được giữ nguyên vị trí, các byte ở hàng 1, hàng 2 và hàng 3 được dịch vòng sang phải lần lượt 1, 2 và 3 byte trước khi ghép lại thành `state_out`. Toàn bộ quá trình được minh họa trong lưu đồ thuật toán ở Hình 3.30.



Hình 3.30: Lưu đồ thuật toán của khối `inv_shift_rows`

Khối thay thế byte nghịch đảo (InvSubBytes): Khối `inv_sub_bytes` thực hiện phép biến đổi phi tuyến nghịch đảo nhằm khôi phục lại giá trị của các byte trong ma trận trạng thái.

Tương tự như thiết kế ở phần mã hóa, khối không sử dụng các mạch tính toán trực tiếp trên trường Galois mà thông qua bảng tra cứu độc lập (Inverse Look-up Table - InvS-Box). Để duy trì tốc độ xử lý tức thời ($O(1)$ delay) cho kiến trúc pipeline, hệ thống khởi tạo 16 bộ InvS-Box phân cứng hoạt động song song [7]. Kiến trúc tổng thể của 16 khối này được minh họa chi tiết trong sơ đồ RTL ở Hình 3.31.



Hình 3.31: Sơ đồ khối RTL của khối `inv_sub_bytes`

Các tín hiệu của khối `inv_sub_bytes` được định nghĩa trong Bảng 3.13.

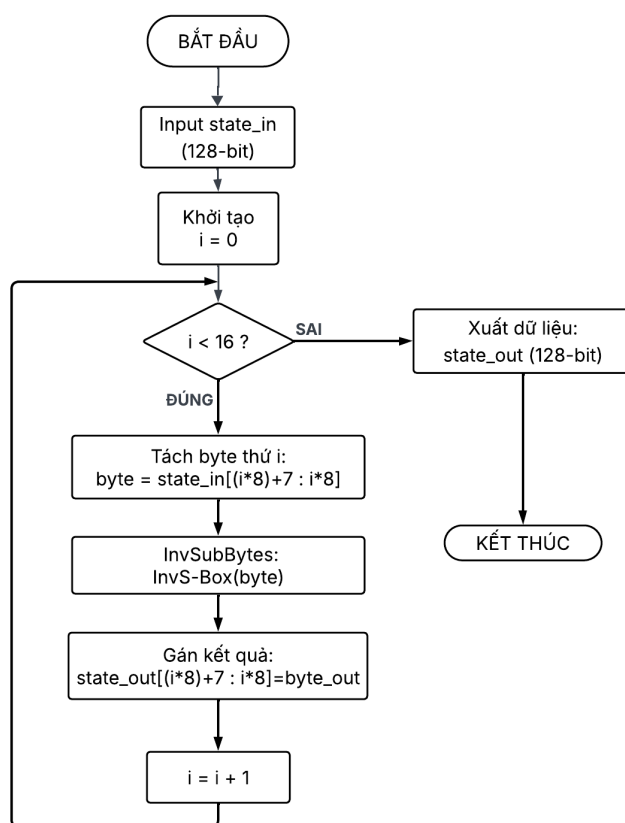
Bảng 3.13: Mô tả các tín hiệu của khối `inv_sub_bytes`

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>state_in</code>	Input	128-bit	Ma trận trạng thái trước khi thay thế byte nghịch đảo.
<code>state_out</code>	Output	128-bit	Ma trận trạng thái sau khi được ánh xạ khôi phục qua 16 bộ InvS-Box.

Để thực hiện 16 phép biến đổi đồng thời, phương pháp sinh phần cứng bằng cấu trúc generate (với biến lặp i từ 0 đến 15) tương tự ở khối mã hóa. Dữ liệu 128-bit `state_in` được tách thành 16 byte độc lập, định tuyến song song qua 16 khối con và ghép lại đúng vị trí trên bus 128-bit `state_out[7]`.

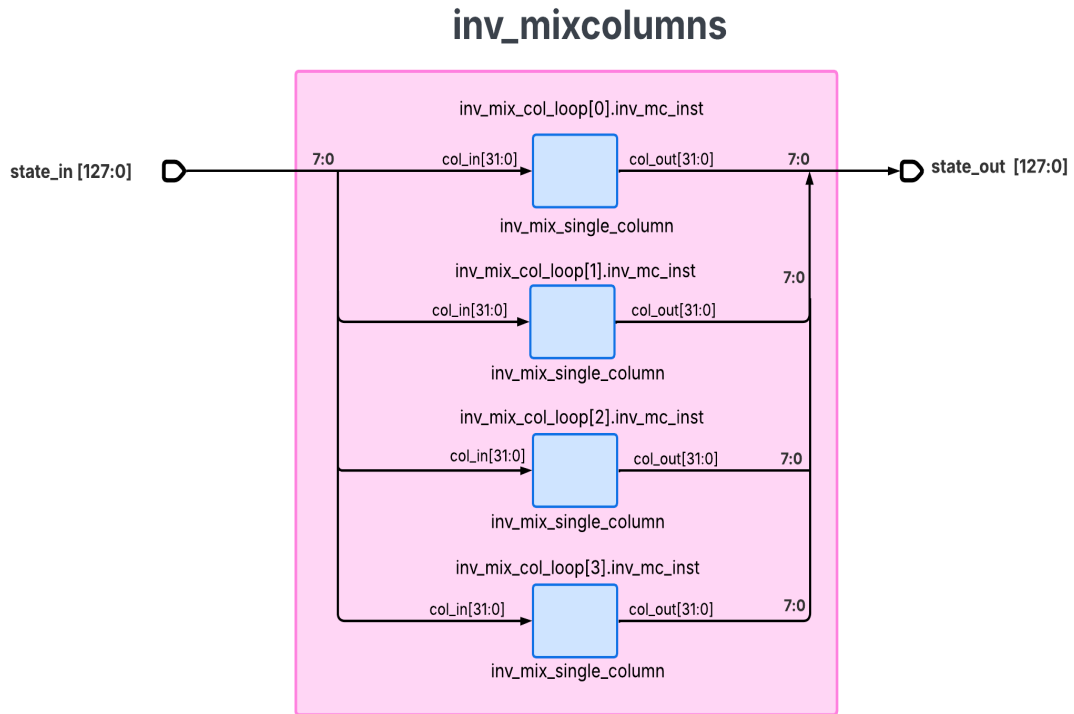
Bên trong mỗi khối `inv_sbox`, cấu trúc phần cứng tương tự như S-Box phía mã hóa. Như đã minh họa tại Hình 3.10 (mục 3.3.1), cốt lõi của khối vẫn là một khối bộ nhớ ROM dùng để lưu bảng tra cứu. Điểm khác biệt là ROM chứa 256 giá trị của bảng Inverse S-Box thay vì S-Box thuận. Mỗi byte đầu vào được sử dụng làm địa chỉ truy xuất để nhận về giá trị nghịch đảo tương ứng.

Việc xây dựng khối giải mã theo cùng cấu trúc với mã hóa giúp thiết kế giữ được tính đồng nhất. Quá trình tách dữ liệu, tra cứu song song và ghép kết quả cũng tương tự như khối `sub_bytes`, và được minh họa trong lưu đồ thuật toán ở Hình 3.32.



Hình 3.32: Lưu đồ thuật toán của khối `inv_sub_bytes`

Khối trộn cột nghịch đảo (InvMixColumns): Khối `inv_mix_columns` thực hiện phép nhân ma trận cột nghịch đảo của trạng thái với một ma trận hằng số trên trường hữu hạn $GF(2^8)$. Đây là khối thực hiện nhiệm vụ giải khuếch tán, khôi phục lại trạng thái ban đầu của dữ liệu trước khi đi qua khối MixColumns ở quá trình mã hóa. Để xử lý toàn bộ dữ liệu 128-bit trong một chu kỳ, thiết kế chia bus dữ liệu thành 4 cột độc lập (mỗi cột rộng 32-bit) và sử dụng cấu trúc generate để khởi tạo 4 khối `inv_mix_single_column` hoạt động song song. Kiến trúc tổng thể của khối trộn cột nghịch đảo này được mô tả trong sơ đồ RTL ở Hình 3.33.



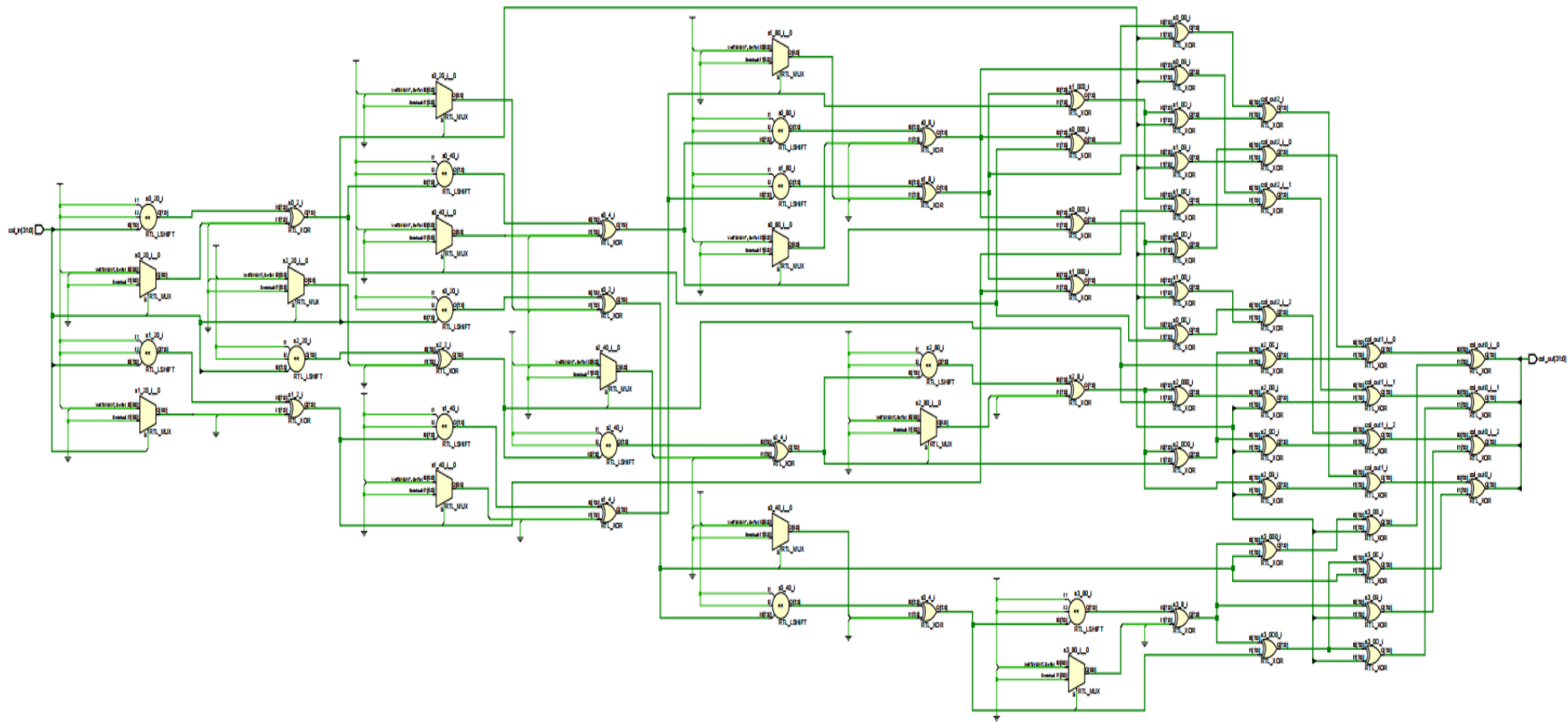
Hình 3.33: Sơ đồ khối RTL của khối *inv_mix_columns*

Đi sâu vào thuật toán, đây là khối tiêu tốn nhiều tài nguyên phần cứng nhất trong tuyến giải mã do phải thực hiện phép nhân với các hằng số phức tạp: {09}, {0b}, {0d} và {0e}. Trên phương diện lý thuyết, phép nhân ma trận này yêu cầu các mạch nhân phần cứng rất tốn kém diện tích. Để tối ưu hóa và giữ nguyên kiến trúc logic tổ hợp, thiết kế đã phân tích các hằng số này thành tổng của các lũy thừa cơ số 2:

- $\{09\} \oplus x = (x \oplus \{08\}) \oplus x$
- $\{0b\} \cdot x = (x \cdot \{08\}) \oplus (x \cdot \{02\}) \oplus x$
- $\{0d\} \cdot x = (x \cdot \{08\}) \oplus (x \cdot \{04\}) \oplus x$
- $\{0e\} \cdot x = (x \cdot \{08\}) \oplus (x \cdot \{04\}) \oplus (x \cdot \{02\})$

Dựa trên phân tích toán học này, module chỉ cần sử dụng lại khối hàm cơ bản xtime (nhân với {02}) để tạo ra các giá trị nhân {04} và nhân {08}. Nhờ đó, module *inv_mix_columns* không cần sử dụng bộ nhân phần cứng phức tạp mà chỉ gồm các phép dịch bit và XOR, phù hợp với yêu cầu xử lý tốc độ cao của hệ thống.

Kiến trúc bên trong mỗi module *inv_mix_single_column* được minh họa ở Hình 3.34. So với khối *mix_columns* phía mã hóa, mạng logic của khối giải mã phức tạp hơn do phải thực hiện thêm nhiều phép toán trên trường $GF(2^8)$. Vì vậy, số lượng khối dịch bit, bộ chọn tín hiệu và cổng XOR cũng tăng lên đáng kể. Tuy tiêu tốn nhiều tài nguyên LUT hơn trên FPGA, cách triển khai này vẫn đảm bảo khả năng xử lý song song và duy trì tốc độ hoạt động cao.



Hình 3.34: Kiến trúc RTL nội bộ của khối `inv_mix_single_column` hiện thực hóa thuật toán nghịch đảo

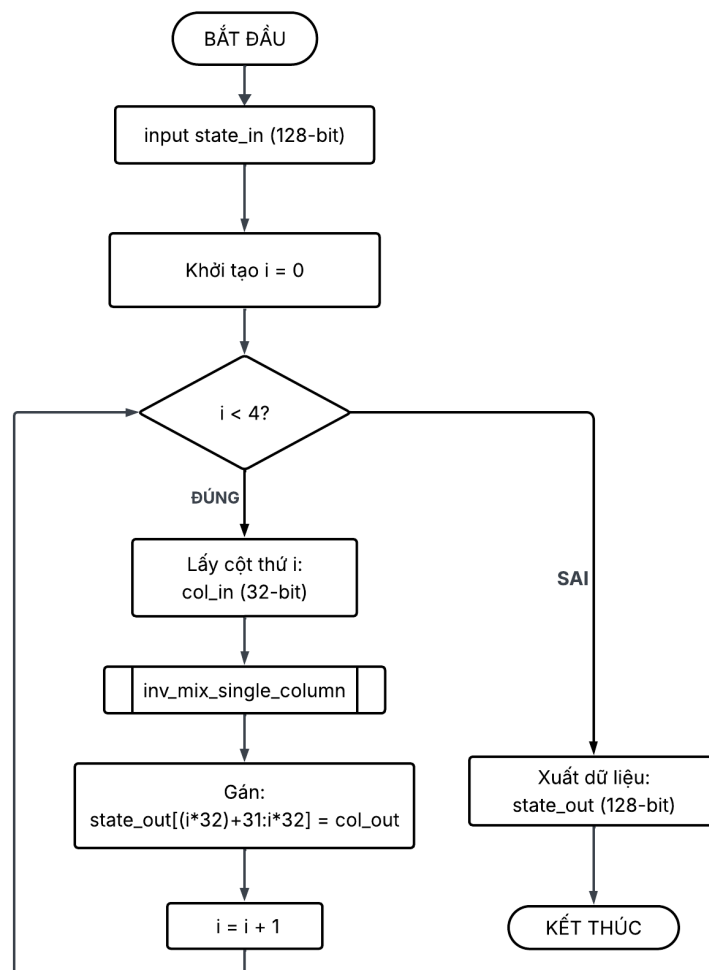
Các tín hiệu của khối `inv_mix_columns` được mô tả trong Bảng 3.14.

Bảng 3.14: Mô tả các tín hiệu giao tiếp của module `inv_mix_columns`

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
<code>state_in</code>	Input	128-bit	Mã trận trạng thái đầu vào từ khối cộng khóa vòng (<code>add_round_key</code>).
<code>state_out</code>	Output	128-bit	Mã trận trạng thái sau khi hoàn tất phép giải khuếch tán trên trường $GF(2^8)$.

Để hiện thực phép biến đổi `InvMixColumns` trên phần cứng, thiết kế tiếp tục duy trì phương pháp tiếp cận chia thành hai mức xử lý: mức xử lý toàn khối và mức xử lý từng cột.

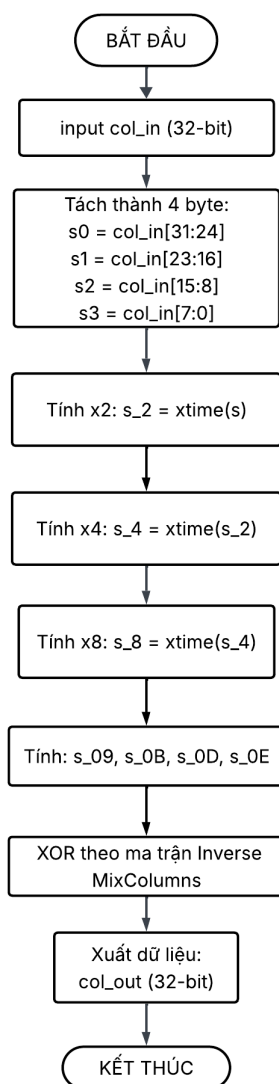
Ở mức toàn khối (`inv_mix_columns`), dữ liệu 128-bit được chia thành 4 dải 32-bit. Hệ thống sử dụng cấu trúc generate để cấp phát 4 dải dữ liệu này vào 4 khối xử lý cột độc lập hoạt động song song. Cơ chế phân tách, xử lý và ghép dữ liệu tương tự với mã hóa, được mô tả chi tiết trong lưu đồ thuật toán tại Hình 3.35.



Hình 3.35: Lưu đồ thuật toán toàn khối `inv_mix_columns`

Bên trong mỗi khối `inv_mix_single_column`, mỗi cột 32-bit được tách thành 4 byte độc lập. Các byte này lần lượt đi qua chuỗi phép tính `xtime` nhằm tạo ra các hệ số nhân `{02}`, `{04}`, `{08}`. Từ các giá trị trung gian đó, hệ thống thực hiện các phép XOR để tổ hợp theo đúng ma trận nghịch đảo chuẩn của AES và tạo ra dữ liệu đầu ra.

Luồng xử lý dữ liệu được xây dựng theo đúng trình tự của lưu đồ thuật toán ở Hình 3.36. Trước tiên, dữ liệu đầu vào `col_in` 32-bit được tách thành 4 byte độc lập gồm `s0`, `s1`, `s2` và `s3`. Tiếp theo, hệ thống sử dụng các phép tính `xtime` nối tiếp để tạo ra các giá trị nhân với `{02}`, `{04}` và `{08}`. Các kết quả này sau đó được kết hợp bằng phép XOR để tạo thành các hệ số `{09}`, `{0b}`, `{0d}` và `{0e}`. Ở bước cuối, toàn bộ các tín hiệu trung gian được đưa qua cổng XOR nhằm thực hiện phép nhân ma trận nghịch đảo và tạo ra cột dữ liệu đầu ra `col_out` 32-bit.



Hình 3.36: Lưu đồ thuật toán phân tách luồng dữ liệu và chuỗi tính toán `xtime` của module `inv_mix_single_column`

3.6. Tích hợp hệ thống

3.6.1. Thiết kế khối điều khiển trung tâm (FSM Control)

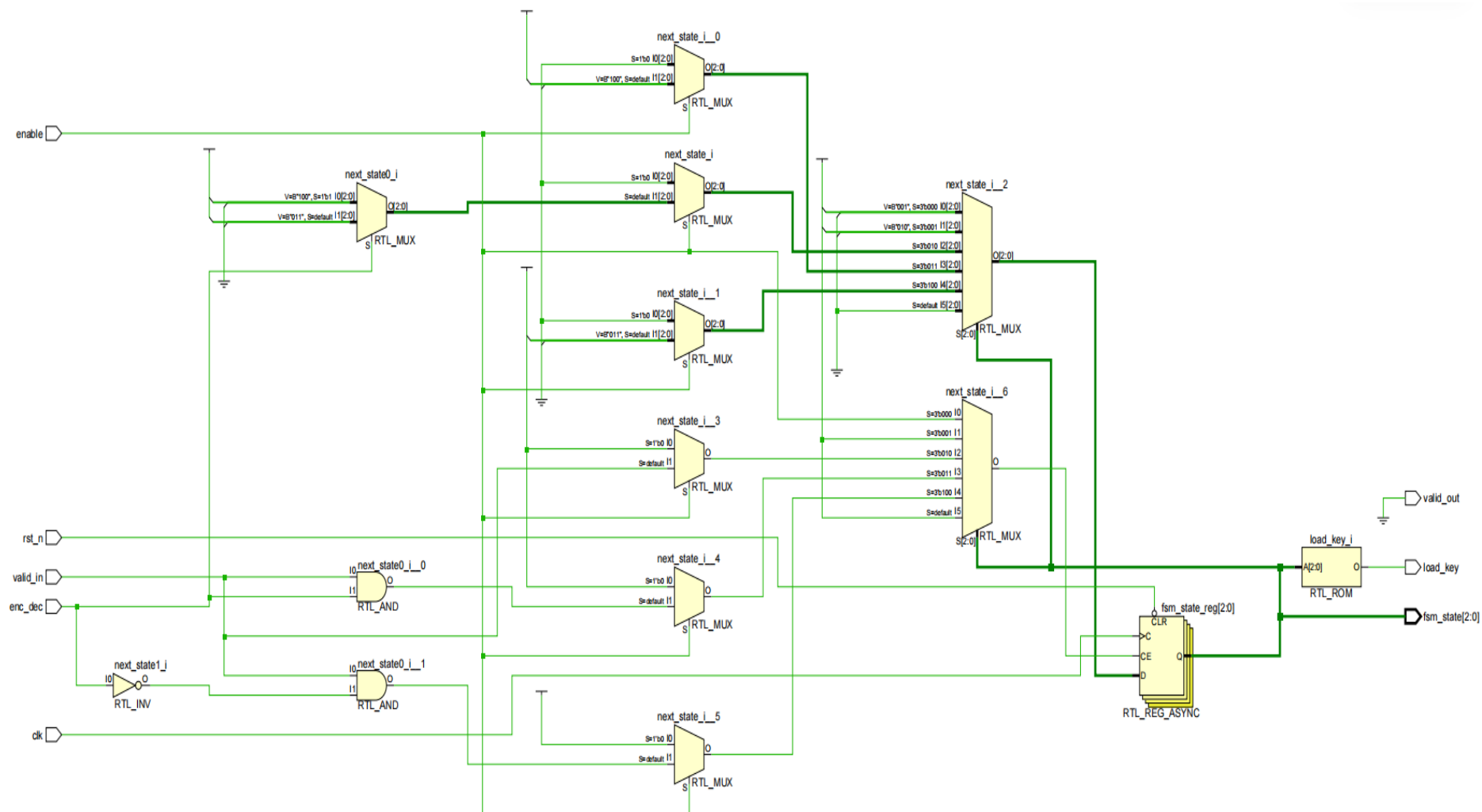
Trong một hệ thống SoC độc lập, các tuyến dữ liệu mã hóa và giải mã không thể tự vận hành mà cần một khối điều khiển trung tâm làm nhiệm vụ cấp phép, định tuyến và đồng bộ hóa thời gian. Trong thiết kế này, khối fsm_control được xây dựng dựa trên mô hình máy trạng thái hữu hạn (Finite State Machine - FSM) kết hợp với thanh ghi dịch để quản lý toàn bộ vòng đời của một tiến trình xử lý khối dữ liệu[7].

Cấu trúc phần cứng tổng thể của khối điều khiển được tổng hợp dưới dạng sơ đồ RTL tại Hình 3.37. Kiến trúc bao gồm hai khối logic chính hoạt động song song: khối logic tuần tự chịu trách nhiệm cập nhật trạng thái và dịch bit đồng bộ theo xung nhịp, cùng khối logic tổ hợp đảm nhiệm việc giải mã điều kiện rẽ nhánh.

Các tín hiệu giao tiếp của khối fsm_control được định nghĩa chi tiết trong Bảng 3.15.

Bảng 3.15: Mô tả các tín hiệu giao tiếp của khối fsm_control

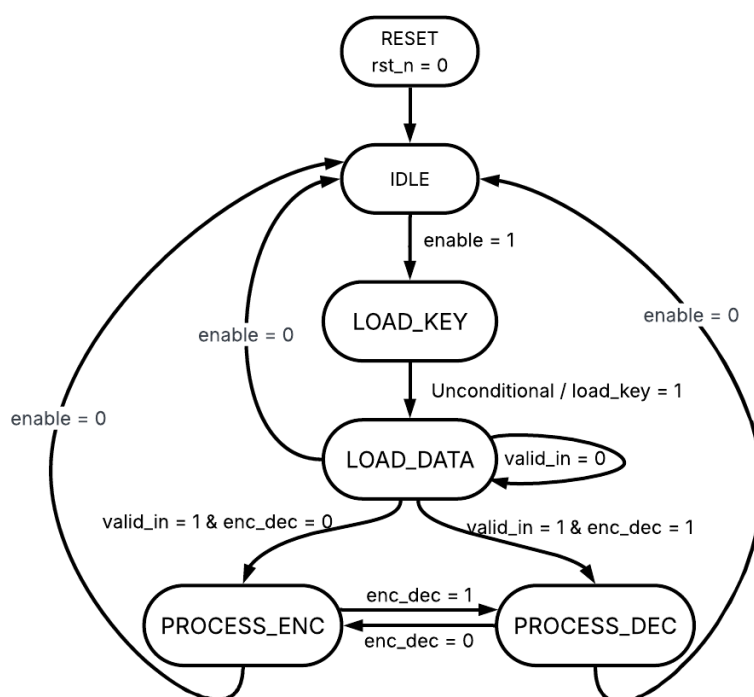
Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
clk	Input	1-bit	Xung nhịp đồng bộ hệ thống.
rst_n	Input	1-bit	Tín hiệu reset không đồng bộ, tích cực mức thấp để đưa hệ thống về trạng thái ban đầu.
enable	Input	1-bit	Tín hiệu kích hoạt cho phép toàn hệ thống bắt đầu hoạt động.
valid_in	Input	1-bit	Cờ báo hiệu có dữ liệu bản rõ/bản mã hợp lệ được đưa vào ngõ vào.
enc_dec	Input	1-bit	Tín hiệu chọn luồng xử lý: 0 cho luồng mã hóa, 1 cho luồng giải mã.
load_key	Output	1-bit	Xung kích hoạt khối Key Expansion thực hiện chốt khóa vòng.
valid_out	Output	1-bit	Cờ báo hiệu dữ liệu tại ngõ ra đã xử lý xong và sẵn sàng để đọc.
fsm_state	Output	3-bit	Xuất mã trạng thái hiện tại của FSM ra ngõ ra để giám sát và gỡ lỗi.



Hình 3.37: Sơ đồ RTL kiến trúc nội bộ của khối điều khiển trung tâm `fsm_control`

Cơ chế chuyển đổi trạng thái (FSM State Transitions):

FSM được thiết kế với bộ mã hóa trạng thái 3-bit, gồm 5 trạng thái chính để điều khiển hoạt động của hệ thống. Sơ đồ trạng thái được minh họa trong Hình 3.38.



Hình 3.38: Sơ đồ chuyển đổi trạng thái của khối điều khiển trạng thái.

Chi tiết hoạt động tại từng trạng thái được phân tích như sau:

- IDLE (3'b000): Trạng thái mặc định sau khi hệ thống được reset. Trong trạng thái này, FSM liên tục giám sát tín hiệu enable; khi enable = 1, hệ thống chuyển sang trạng thái LOAD_KEY.
- LOAD_KEY (3'b001): Trong trạng thái này, tín hiệu load_key được kích hoạt để khởi động khối Key Expansion và sinh ra toàn bộ 11 khóa vòng. Sau khi hoàn tất quá trình nạp khóa, FSM tự động chuyển sang trạng thái LOAD_DATA.
- LOAD_DATA (3'b010): Trạng thái chờ dữ liệu đầu vào. FSM giám sát tín hiệu valid_in và chỉ tiếp tục xử lý khi nhận được dữ liệu hợp lệ. Nếu enable = 0, hệ thống quay về trạng thái IDLE. Khi có khối dữ liệu hợp lệ được đưa vào (valid_in = 1), FSM sẽ kiểm tra bit chọn chế độ enc_dec. FSM kiểm tra tín hiệu enc_dec để xác định chế độ hoạt động: chuyển sang PROCESS_ENC nếu enc_dec = 0 hoặc sang PROCESS_DEC nếu enc_dec = 1.
- PROCESS_ENC (3'b011)/PROCESS_DEC (3'b100): Trạng thái xử lý dữ liệu trong chế độ mã hóa hoặc giải mã. FSM duy trì hoạt động theo tín hiệu enc_dec và cho phép chuyển đổi linh hoạt giữa hai chế độ khi đang xử lý. Nếu enable = 0, hệ thống thoát khỏi quá trình xử lý và trở về IDLE.

Giải pháp đồng bộ hóa độ trễ đường ống

Để đơn giản hóa chức năng điều khiển, khối fsm_control chỉ đảm nhiệm việc quản lý trạng thái hoạt động của hệ thống và kích hoạt quá trình nạp khóa (load_key). Tín hiệu valid_out không được tạo bên trong FSM mà được xử lý riêng tại aes_core. Vì vậy, trong khối này, valid_out được gán cố định ở mức thấp (valid_out = 1'b0;).

Do kiến trúc Pipeline AES-128 có độ trễ cố định 12 chu kỳ xung nhịp, một thanh ghi dịch 12-bit mang tên valid_shift được triển khai tại aes_core để đồng bộ tín hiệu hợp lệ với dữ liệu đầu ra. Cơ chế hoạt động gồm hai giai đoạn:

- Giai đoạn nạp tín hiệu hợp lệ: Khi FSM hoạt động ở trạng thái mã hóa (PROCESS_ENC) hoặc giải mã (PROCESS_DEC), tín hiệu valid_in được lấy mẫu tại mỗi cạnh lên của xung nhịp và nạp vào bit thấp nhất valid_shift[0] của thanh ghi dịch (valid_shift[0]). Sau mỗi chu kỳ, giá trị này tiếp tục được dịch sang vị trí kế tiếp trong thanh ghi.
- Giai đoạn đồng bộ tín hiệu đầu ra: Sau 12 chu kỳ xung nhịp, tương ứng với thời gian dữ liệu đi qua toàn bộ đường ống xử lý, giá trị hợp lệ ban đầu sẽ dịch đến bit cao nhất (valid_shift[11]). Giá trị này được sử dụng trực tiếp làm tín hiệu valid_out, đảm bảo dữ liệu đầu ra chỉ được đánh dấu hợp lệ khi đã hoàn tất quá trình xử lý.

3.6.2. Thiết kế khối aes_core

Khối aes_core đóng vai trò là thực thể mức cao nhất (Top-level Module) của toàn bộ lõi IP AES-128. Nhiệm vụ của khối này là bao quát, kết nối và đồng bộ hóa tất cả các khối thành phần thành một kiến trúc đường ống hoàn chỉnh, có khả năng giao tiếp trực tiếp với các hệ thống ngoại vi bên ngoài.

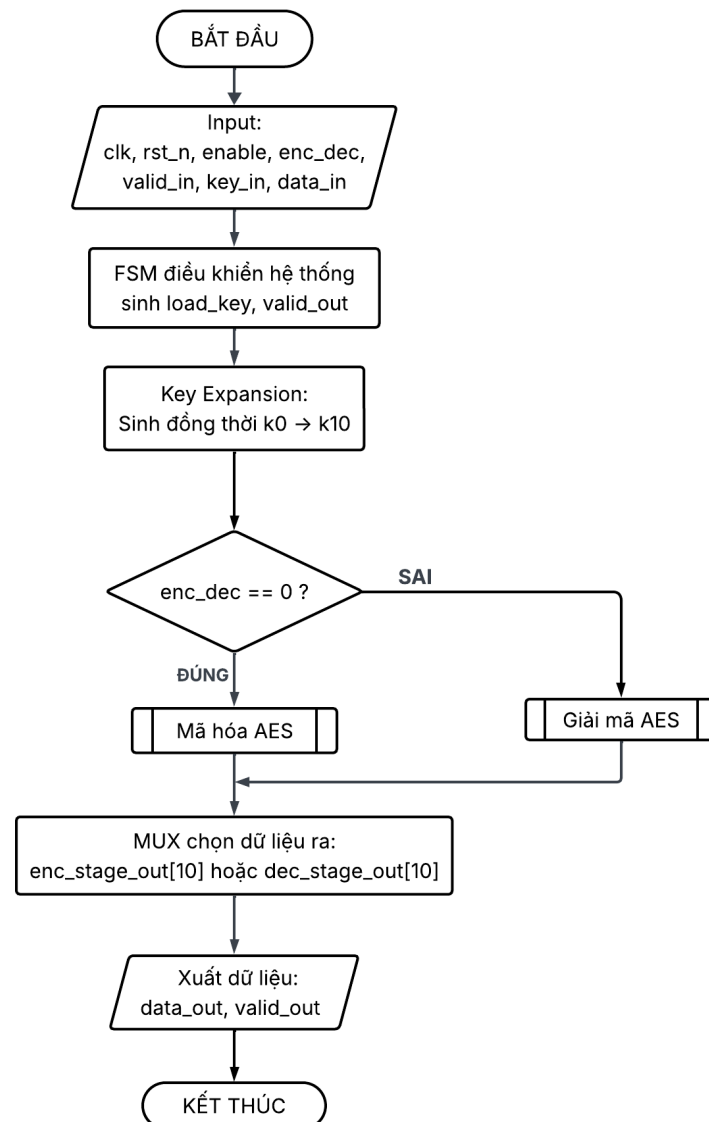
Để đáp ứng yêu cầu linh hoạt, aes_core hỗ trợ cả hai chế độ mã hóa và giải mã. Chi tiết các tín hiệu vào/ra được trình bày trong Bảng 3.16.

Bảng 3.16: Mô tả các tín hiệu của khối aes_core

Tên tín hiệu	Loại tín hiệu	Độ rộng	Mô tả chức năng
clk	Input	1-bit	Xung nhịp đồng bộ của toàn bộ lõi IP.
rst_n	Input	1-bit	Tín hiệu reset không đồng bộ (tích cực mức thấp).
enable	Input	1-bit	Xung kích hoạt cho phép FSM khởi động hệ thống.
enc_dec	Input	1-bit	Tín hiệu lựa chọn chế độ hoạt động: 0 (Mã hóa), 1 (Giải mã).

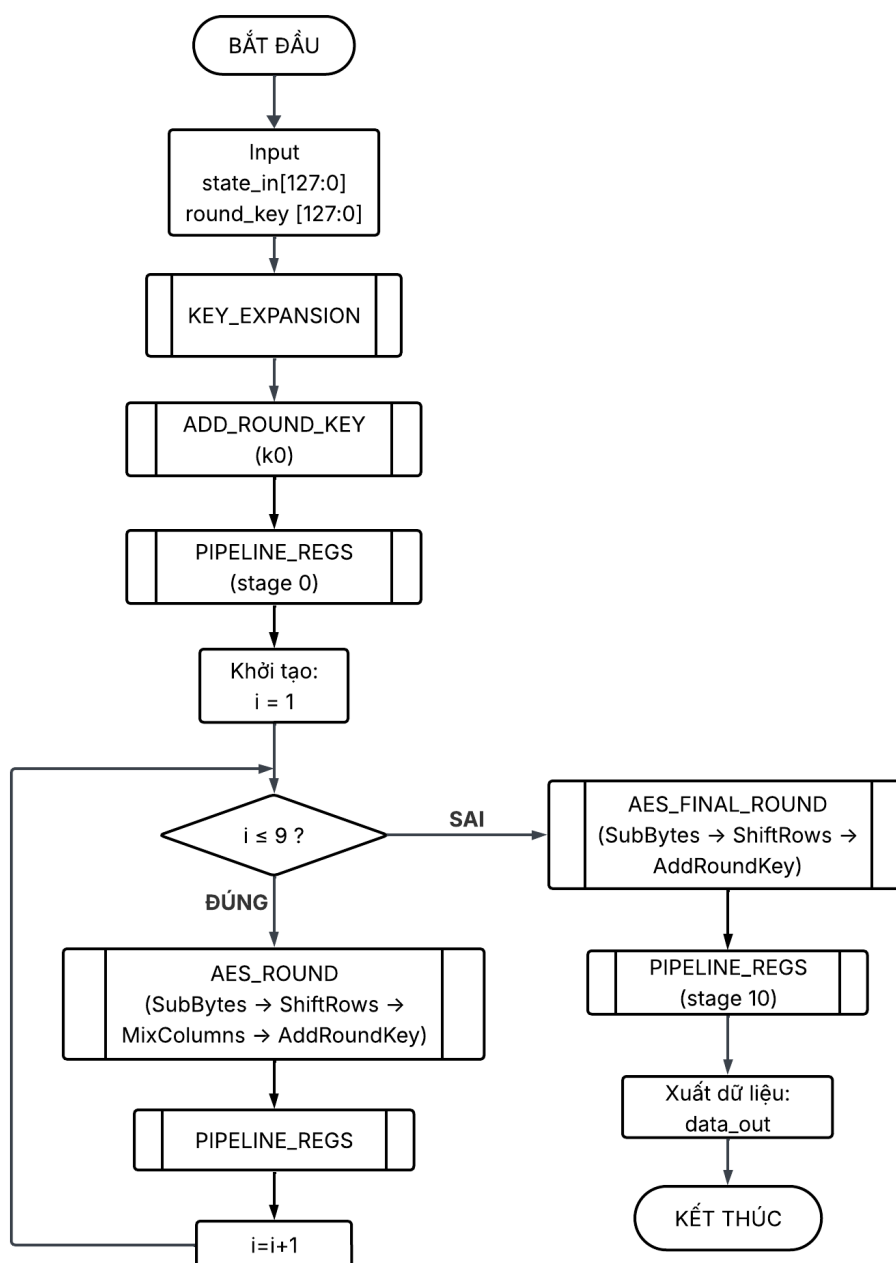
valid_in	Input	1-bit	Cờ báo hiệu dữ liệu đầu vào (data_in) đã hợp lệ để chót vào đường ống.
key_in	Input	128-bit	Khóa bí mật ban đầu cấp cho lõi AES.
data_in	Input	128-bit	Bus dữ liệu đầu vào (bản rõ hoặc bản mã).
data_out	Output	128-bit	Bus dữ liệu đầu ra (bản mã hoặc bản rõ).
valid_out	Output	1-bit	Cờ đồng bộ báo hiệu dữ liệu tại data_out đã sẵn sàng và hợp lệ.

Thay vì sử dụng cơ chế chia sẻ tài nguyên phần cứng nhằm tiết kiệm diện tích, kiến trúc của aes_core được xây dựng theo hướng tối ưu thông lượng bằng cách triển khai độc lập các tầng xử lý của thuật toán AES. Sơ đồ luồng điều khiển và quá trình khởi tạo hệ thống được minh họa trong Hình 3.37.

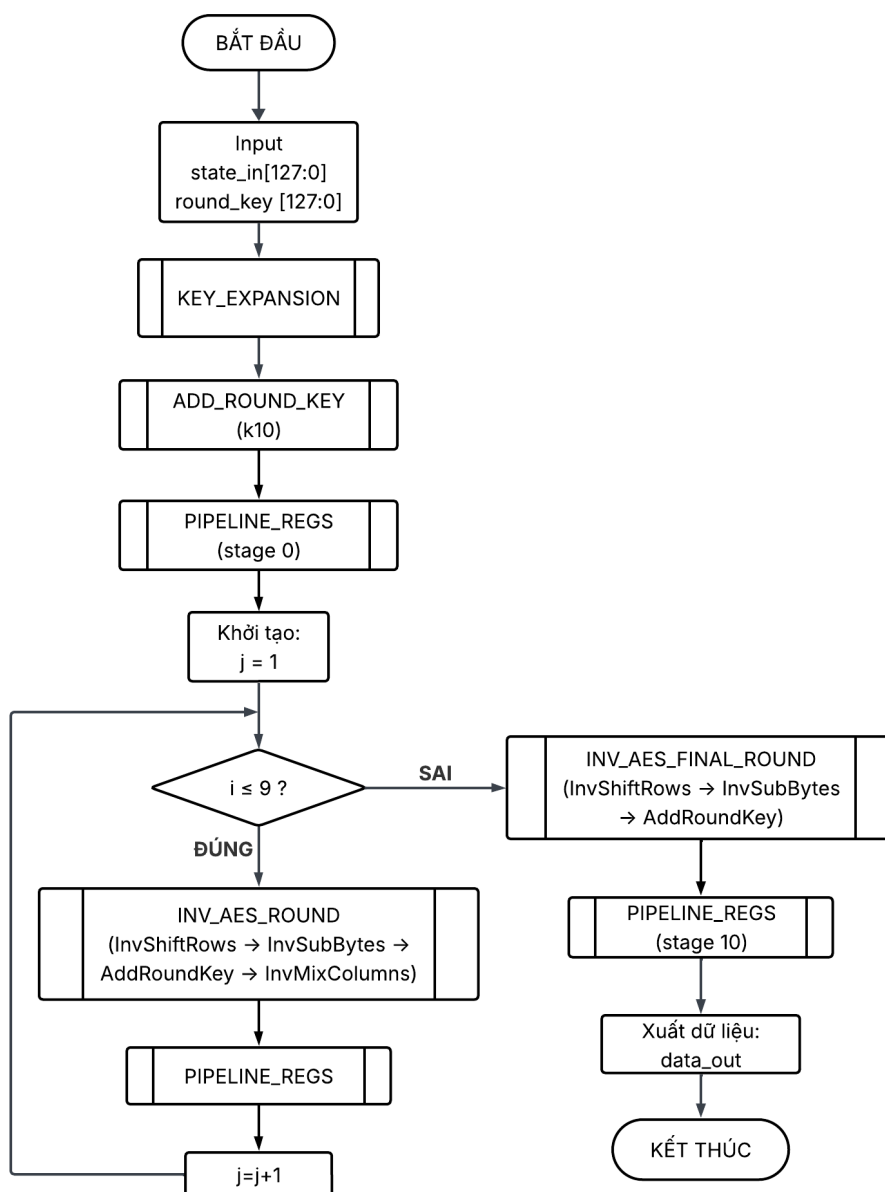


Hình 3.39: Lưu đồ thuật toán luồng điều khiển tích hợp hệ thống của aes_core

Dựa trên kiến trúc Pipeline đã xây dựng, khối aes_core bao gồm hai tuyến xử lý độc lập cho chức năng mã hóa và giải mã. Mỗi tuyến được hình thành từ các tầng biến đổi AES nối tiếp, liên kết với nhau thông qua các thanh ghi Pipeline để đồng bộ dữ liệu giữa các vòng xử lý. Hoạt động của từng tuyến được minh họa trong Hình 3.40 và Hình 3.41.



Hình 3.40: Lưu đồ thuật toán và luồng dữ liệu của tuyến mã hóa AES-128 Pipeline



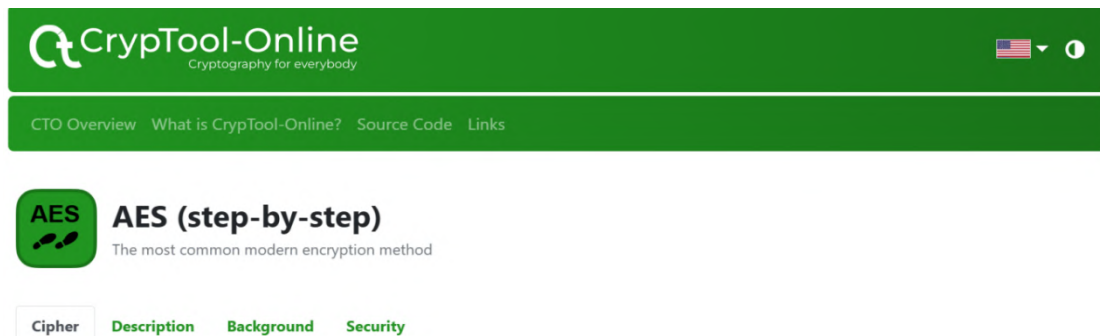
Hình 3.41: Lưu đồ thuật toán và luồng dữ liệu của tuyến giải mã AES-128 Pipeline

CHƯƠNG 4: KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ

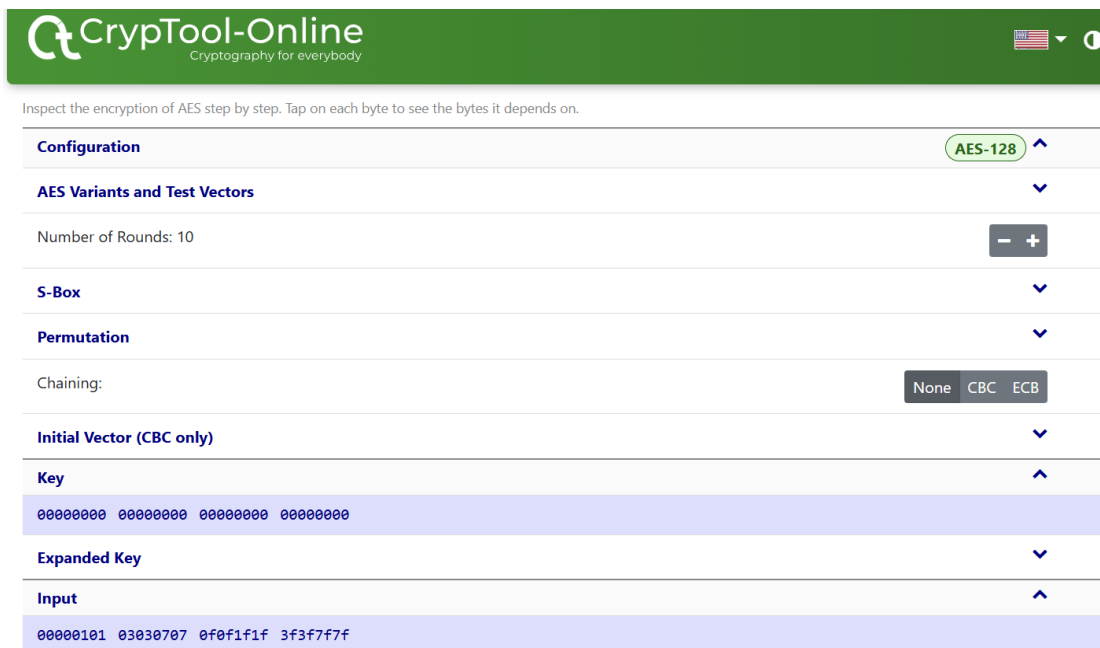
4.1. Tổng quan phương pháp kiểm thử và cấu hình

Nhằm đánh giá độ chính xác của lỗi phần cứng mã hóa và giải mã AES-128 Pipeline, một hệ thống kiểm thử tự động được xây dựng trên Xilinx Vivado nhằm đối chiếu kết quả đầu ra với dữ liệu chuẩn. Quá trình kiểm thử được thực hiện theo phương pháp khép kín, trong đó các tín hiệu được cấp liên tục để kiểm chứng cả tính đúng đắn trong xử lý dữ liệu và hiệu năng hoạt động của kiến trúc pipeline.

Để gia tăng tính toàn vẹn và độ tin cậy của kết quả mô phỏng, ở đây sử dụng công cụ hỗ trợ kiểm tra AES Step-by-Step Tool từ CrypTool (legacy.cryptool.org), cho phép quan sát giá trị round state của từng vòng dựa trên cùng plaintext và key. Kết quả này được đối chiếu với tín hiệu đầu ra từ mô phỏng. Công cụ CrypTool được sử dụng để đối chiếu giá trị round state của từng vòng và giao diện thực hiện mã hóa và thiết lập thông số kiểm tra được minh họa như trong Hình 4.1 và Hình 4.2.



Hình 4.1: Công cụ được sử dụng để đối chiếu



Hình 4.2: Giao diện thực hiện mã hóa và giải mã kiểm tra với kết quả mô phỏng

Kịch bản kiểm thử sử dụng bộ dữ liệu chuẩn hóa từ thông số thiết kế thực tế bao gồm:

- Dữ liệu đầu vào: Chuỗi ký tự định danh "DATN_NGOC_PHUONG" tương ứng với giá trị Hex 128-bit: 4441544E5F4E474F435F5048554F4E47.
- Khóa mật mã gốc: Chuỗi ký tự định danh "DATN_HCMUTE_2026" tương ứng với giá trị Hex 128-bit: 4441544E5F48434D5554455F32303236.

4.2. Kết quả mô phỏng khối sinh khóa (key expansion)

Bộ sinh khóa được thiết kế theo kiến trúc tổ hợp song song, cho phép tạo đồng thời 11 khóa vòng để cấp cho các tầng Pipeline. Khi tín hiệu load_key từ khối FSM được kích hoạt, khóa gốc được nạp và quá trình mở rộng khóa thực hiện nhằm sinh toàn bộ khóa vòng phục vụ cho mã hóa và giải mã. Giá trị mong đợi của các khóa vòng được thể hiện trong Bảng 4.1.

Bảng 4.1: Kết quả mở rộng khóa vòng của thuật toán AES-128

Khóa vòng	Hằng số rcon	Giá trị hexadecimal đầu ra lý thuyết & mô phỏng
k0 (Gốc)	-	4441544e5f48434d5554455f32303236
k1	8'h01	4162516d1e2a12204b7e577f794e6549
k2	8'h02	6c2f6adb720578fb397b2f8440354acd
k3	8'h04	fef9d7d28cfcaf29b58780adf5b2ca60
k4	8'h08	c18d07344d71a81df8f628b00d44e2d0
k5	8'h10	ca1577e38764dffe7f92f74e72d6159e
k6	8'h20	1c4c7ca39b28a35de4ba5413966c418d
k7	8'h40	0ccf213397e7826e735dd67de53197f0
k8	8'h80	4b47adeadca02f84affdf9f94acc6e09
k9	8'h1B	1bd8ac3cc77883b868857a4122491448
k10	8'h36	1622feafd15a7d17b9df07569b96131e

Để xác minh tính chính xác của bộ sinh khóa, kết quả tính toán từ công cụ kiểm thử (Hình 4.3) được dùng làm cơ sở để đối chiếu trực tiếp với xung mô phỏng trên Vivado (Hình 4.4 và Hình 4.5). Như được thể hiện trên hai hình, toàn bộ 11 khóa vòng do mạch tổ hợp sinh ra trùng khớp từng byte với dữ liệu từ công cụ kiểm thử.

Key							
4441544e	5f48434d	5554455f	32303236				
Expanded Key							
4441544e	5f48434d	5554455f	32303236	4162516d	1e2a1220	4b7e577f	
794e6549	6c2f6adb	720578fb	397b2f84	40354acd	fef9d7d2	8cfcaf29	
b58780ad	f5b2ca60	c18d0734	4d71a81d	f8f628b0	0d44e2d0	ca1577e3	
8764dffe	7f92f74e	72d6159e	1c4c7ca3	9b28a35d	e4ba5413	966c418d	
0ccf2133	97e7826e	735dd67d	e53197f0	4b47adea	dca02f84	affdf9f9	
4acc6e09	1bd8ac3c	c77883b8	68857a41	22491448	1622feaf	d15a7d17	
b9df0756	9b96131e						

Hình 4.3: Kết quả tính toán từ Tool kiểm thử cho khối Key Expansion.

Name	Value	0.000 ns	5.000 ns	10.000 ns
> key_in[127:0]	4441544e5f48434d5554455f32303236		4441544e5f48434d5554455f32303236	
> k0[127:0]	4441544e5f48434d5554455f32303236		4441544e5f48434d5554455f32303236	
> k1[127:0]	4162516d1e2a12204b7e577f794e6549		4162516d1e2a12204b7e577f794e6549	
> k2[127:0]	6c2f6adb720578fb397b2f8440354acd		6c2f6adb720578fb397b2f8440354acd	
> k3[127:0]	fef9d7d28cfcaf29b58780adf5b2ca60		fef9d7d28cfcaf29b58780adf5b2ca60	
> k4[127:0]	c18d07344d71a81df8f628b00d44e2d0		c18d07344d71a81df8f628b00d44e2d0	
> k5[127:0]	ca1577e38764dffe7f92f74e72d6159e		ca1577e38764dffe7f92f74e72d6159e	
> k6[127:0]	1c4c7ca39b28a35de4ba5413966c418d		1c4c7ca39b28a35de4ba5413966c418d	
> k7[127:0]	0ccf213397e7826e735dd67de53197f0		0ccf213397e7826e735dd67de53197f0	
> k8[127:0]	4b47adeadca02f84affdf9f94acc6e09		4b47adeadca02f84affdf9f94acc6e09	
> k9[127:0]	1bd8ac3cc77883b868857a4122491448		1bd8ac3cc77883b868857a4122491448	
> k10[127:0]	1622feafd15a7d17b9df07569b96131e		1622feafd15a7d17b9df07569b96131e	

Hình 4.4: Giản đồ xung mô phỏng chức năng khối mở rộng khóa (Key Expansion)

```

=====
TESTBENCH MODULE AES-128 KEY EXPANSION
=====

INPUT KEY:
HEX   = 4441544e5f48434d5554455f32303236
ASCII = DATN_HCMUTE_2026

GENERATED ROUND KEYS:
-----
K0 = 4441544e5f48434d5554455f32303236
K1 = 4162516d1e2a12204b7e577f794e6549
K2 = 6c2f6adb720578fb397b2f8440354acd
K3 = fef9d7d28cfcaf29b58780adf5b2ca60
K4 = c18d07344d71a81df8f628b00d44e2d0
K5 = ca1577e38764dffe7f92f74e72d6159e
K6 = 1c4c7ca39b28a35de4ba5413966c418d
K7 = 0ccf213397e7826e735dd67de53197f0
K8 = 4b47adeadca02f84affdf9f94acc6e09
K9 = 1bd8ac3cc77883b868857a4122491448
K10 = 1622feafd15a7d17b9df07569b96131e
-----

=====
TEST PASSED
=====

```

Hình 4.5: Kết quả sinh khóa và trạng thái các vòng AES-128 trên TCL Console

Dựa trên kết quả đối chiếu giữa tool kiểm thử và giản đồ xung mô phỏng RTL trên Xilinx Vivado, có thể đánh giá phân hệ mở rộng khóa AES-128 đạt độ chính xác

cao. Khi hệ thống được cấp khóa gốc DATN_HCMUTE_2026, toàn bộ 11 khóa vòng sinh ra từ k0 đến k10 đều trùng khớp hoàn toàn với giá trị lý thuyết, được xác nhận thông qua thông báo “TEST PASSED”. Kết quả này chứng minh module đã thực hiện chính xác các phép biến đổi cốt lõi của thuật toán AES như RotWord, SubWord và XOR với hằng số vòng Rcon.

Bên cạnh đó, quan sát trên waveform cho thấy các khóa vòng xuất hiện đồng thời và ổn định mà không cần thêm chu kỳ xung nhịp, thể hiện đúng đặc tính của kiến trúc logic tổ hợp. Cho phép toàn bộ khóa vòng sẵn sàng cấp trực tiếp cho các tầng Pipeline, loại bỏ độ trễ chờ khóa và góp phần nâng cao hiệu năng xử lý của hệ thống.

4.3. Kết quả mô phỏng bộ mã hóa AES-128

4.3.1. Kết quả mô phỏng vòng mã hóa tiêu chuẩn (Round 1)

Vòng 1 thực hiện tuần tự qua 4 bước: SubBytes , ShiftRows , MixColumns và AddRoundKey với khóa k1. Dữ liệu đầu vào của vòng 1 là kết quả của phép XOR khởi tạo giữa Plaintext và k0 (vòng 0). Các bước kiểm thử cho một vòng tiêu chuẩn của quá trình mã hóa được thể hiện như bảng 4.2.

Bảng 4.2: Quá trình biến đổi một vòng mã hóa tiêu chuẩn của AES-128

Công đoạn biến đổi	Giá trị chuỗi dữ liệu 128-bit tương ứng
Dữ liệu vào vòng 1 (Cộng khóa vòng k0)	00000000000060402160b1517677f7c71
Sau khối thay thế bytes	63636363636ff277472b59f085d210a3
Sau khối dịch hàng	636f59a3632b106347d263778563f2f0
Sau khối trộn cột	8df540cec866cd58f72aca96b6be12fe
Ngõ ra vòng 1 (Cộng khóa vòng k1)	cc9711a3d64cdf78bc549de9cff077b7

Để minh chứng chi tiết cho chuỗi số liệu tổng hợp trong Bảng 4.2, quá trình mô phỏng được tách thành các bước và đánh giá độc lập trên từng khối chức năng tổ hợp cốt lõi của vòng 1.

Đánh giá khối cộng khóa vòng (AddRoundKey)

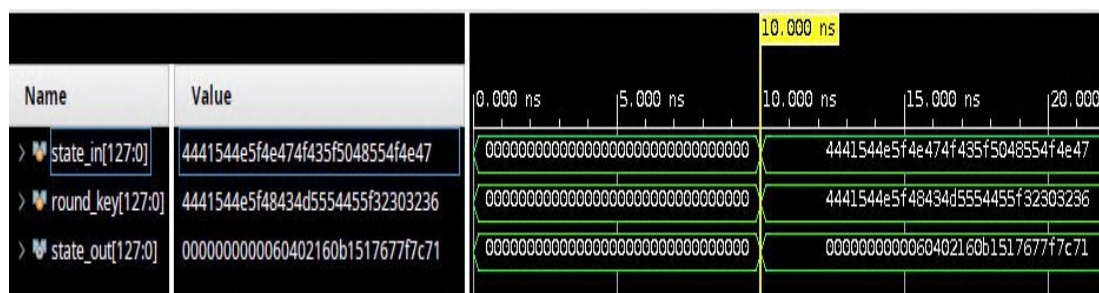
Trước khi bước vào các vòng lặp mã hóa phức tạp, dữ liệu bản rõ bắt buộc phải đi qua bước trộn khóa khởi tạo. Khối mạch tổ hợp add_round_key nhận ngõ vào là chuỗi bản rõ và thực hiện phép toán logic XOR song song toàn bộ 128-bit với khóa gốc k_0 (4441544E5F48434D5554455F32303236).

Giản đồ xung mô phỏng ghi nhận kết quả đầu ra và kết quả tính toán từ tool kiểm thử được thể hiện trong Hình 4.6, Hình 4.7 và Hình 4.8. Giá trị này hoàn toàn

trùng khớp với kết quả tính toán lý thuyết, qua đó xác nhận khối mạch thực hiện chính xác phép xử lý song song trên toàn bộ dữ liệu 128-bit. Sau đó, kết quả được chọt vào thanh ghi định thời để làm tín hiệu đầu vào (state_in) cho vòng 1.



Hình 4.6: Kết quả tính toán từ Tool kiểm thử cho khối AddRoundKey.



Hình 4.7: Giản đồ xung mô phỏng chức năng khối cộng khóa vòng (AddRoundKey)

```

=====
TESTBENCH: ADD ROUND KEY MODULE
=====
State In (DATN_NGOC_PHUONG): 4441544e5f4e474f435f5048554f4e47
Round Key (DATN_HCMUTE_2026): 4441544e5f48434d5554455f32303236
-----
Output Hex (XOR Result) : 0000000000060402160b1517677f7c71
Expected Output : 0000000000060402160b1517677f7c71
-----
>>> KET QUẢ: TEST PASSED! MATCHED EXPECTED VALUE. <<<
=====

```

Hình 4.8: Kết quả khối cộng khóa vòng trên TCL Console

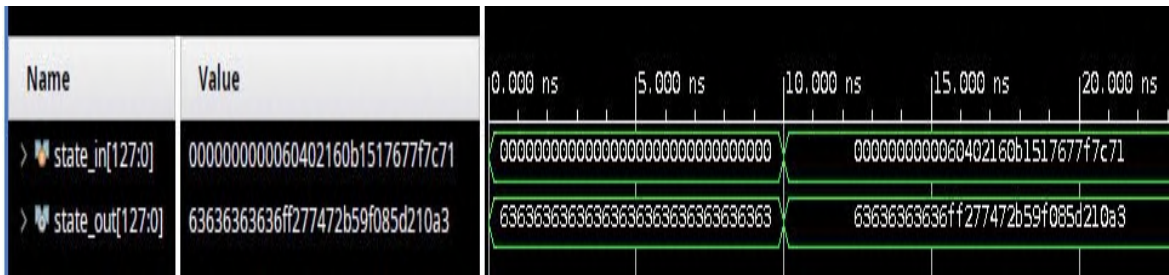
Đánh giá khối thay thế byte (SubBytes)

Khối sub_bytes thực hiện ánh xạ độc lập 16 byte dữ liệu đã trộn khóa thông qua 16 module S-Box được cấu trúc song song dưới dạng mảng LUT. Giản đồ xung mô phỏng ghi nhận kết quả đầu ra và kết quả tính toán từ tool kiểm thử được thể hiện trong Hình 4.9, Hình 4.10 và Hình 4.11. Giá trị này hoàn toàn trùng khớp với kết quả tính toán lý thuyết, qua đó xác nhận khối mạch thực hiện chính xác phép biến đổi phi tuyến

song song trên toàn bộ 16 byte dữ liệu. Sau đó, kết quả này được truyền trực tiếp làm tín hiệu đầu vào cho khối hoán vị dịch hàng (ShiftRows).



Hình 4.9: Kết quả tính toán từ Tool kiểm thử cho khối thay thế byte (SubBytes).



Hình 4.10: Giản đồ xung mô phỏng chức năng khối thay thế byte (SubBytes)

```
=====
TESTBENCH: MODULE SUB BYTES
=====
Input Hex (From AddRoundKey) : 00000000000000000000000000000000
Output Hex (Substituted)      : 63636363636363636363636363636363
Expected Output                : 63636363636363636363636363636363
-----
>>> KET QUẢ: TEST PASSED! HOẠT ĐỘNG CHÍNH XÁC. <<<
=====
```

Hình 4.11: Kết quả khối thay thế byte trên TCL Console

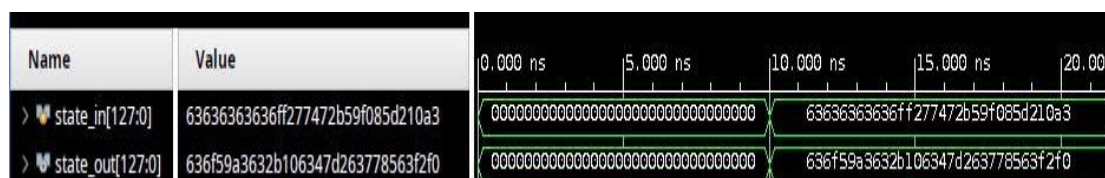
Đánh giá khối hoán vị dịch hàng (ShiftRows)

Ngay sau khi nhận luồng dữ liệu từ khối SubBytes, khối shift_rows tiến hành phép dịch vòng trái các hàng của ma trận trạng thái theo cấu trúc chuẩn AES. Giản đồ xung mô phỏng ghi nhận kết quả đầu ra và kết quả tính toán từ tool kiểm thử được thể hiện trong Hình 4.12, Hình 4.13 và Hình 4.14. Giá trị này hoàn toàn trùng khớp với kết quả tính toán lý thuyết, qua đó xác nhận khối mạch thực hiện chính xác quá trình hoán

đổi vị trí vật lý của các byte dữ liệu. Sau đó, kết quả hoán vị được đưa thẳng làm tín hiệu đầu vào cho khối trộn cột (MixColumns).

Key
4441544e 5f48434d 5554455f 32303236
Expanded Key
Input
4441544e 5f4e474f 435f5048 554f4e47
Encoding Rounds
Round 1
input to Round 1
00000000 00060402 160b1517 677f7c71
after S-Box:
63636363 636ff277 472b59f0 85d210a3
after permutation:
636f59a3 632b1063 47d26377 8563f2f0
after mult:
i ← 9 = permute[5] 0x2b = block[i]

Hình 4.12: Kết quả tính toán từ Tool kiểm thử cho khối hoán vị dịch hàng (ShiftRows)



Hình 4.13: Giả đồ xung mô phỏng chức năng khối hoán vị dịch hàng (ShiftRows)

```

=====
TESTBENCH: MODULE SHIFT ROWS
=====
Input Hex (From SubBytes) : 63636363636363636363636363636363
Output Hex (Shifted)      : 636f59a3632b106347d263778563f2f0
Expected Output           : 636f59a3632b106347d263778563f2f0
=====
>>> KET QUẢ: TEST PASSED! HOẠT ĐỘNG CHÍNH XÁC. <<<
=====

```

Hình 4.14: Kết quả khối hoán vị dịch hàng trên TCL Console

Đánh giá khối trộn cột (MixColumns)

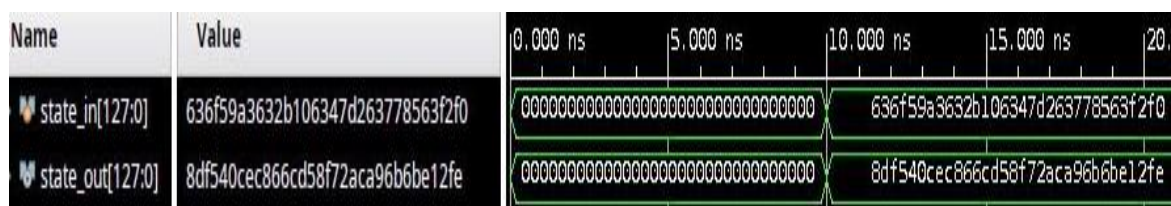
Dữ liệu từ khối dịch hàng được phân tách thành 4 cột độc lập (mỗi cột 32-bit) để đưa vào xử lý song song tại khối mix_columns. Giả đồ xung mô phỏng ghi nhận kết quả đầu ra và kết quả tính toán từ tool kiểm thử được thể hiện trong Hình 4.15, Hình 4.16 và Hình 4.17. Giá trị này hoàn toàn trùng khớp với kết quả tính toán lý thuyết, qua

đó xác nhận khối mạch thực hiện chính xác các phép nhân ma trận phức tạp trên trường hữu hạn Galois GF(28). Sau đó, kết quả được chuyển đến bước xử lý cuối cùng của vòng để thực hiện trộn khóa (k₁).

Input			
4441544e	5f4e474f	435f5048	554f4e47
Encoding Rounds			
Round 1			
input to Round 1			
00000000	00060402	160b1517	677f7c71
after S-Box:			
63636363	636ff277	472b59f0	85d210a3
after permutation:			
636f59a3	632b1063	47d26377	8563f2f0
after mult:			
8df540ce	c866cd58	f72aca96	b6be12fe
used subkey:			
4162516d	1e2a1220	4b7e577f	794e6549

$s0 \leftarrow 0x94 = 0x03 \times 0x85$
 $s3 \leftarrow 0xfb = 0x02 \times 0xf0$
 $a \leftarrow 0xf7 = s0 \oplus 0x63$
 $b \leftarrow 0x09 = 0xf2 \oplus s3$
 $0xfe = a \oplus b$

Hình 4.15: Kết quả tính toán từ Tool kiểm thử cho khối trộn cột (MixColumns)



Hình 4.16: Giảm đồ xung mô phỏng chức năng khối trộn cột (MixColumns)

```

=====
TESTBENCH: MODULE MIX_COLUMNS
=====
Input Hex (From ShiftRows) : 636f59a3632b106347d263778563f2f0
Output Hex (Mixed)         : 8df540cec866cd58f72aca96b6be12fe
Expected Output             : 8df540cec866cd58f72aca96b6be12fe
-----
>>> KET QUẢ: TEST PASSED! HOẠT ĐỘNG CHÍNH XÁC. <<<
=====

```

Hình 4.17: Kết quả khối trộn cột trên TCL Console

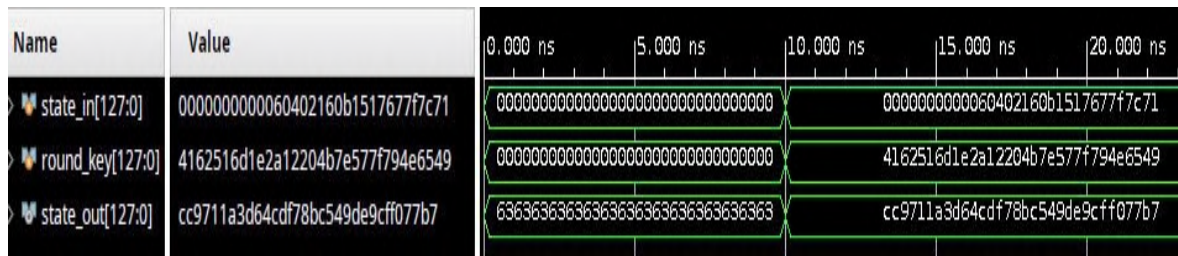
Đánh giá khối vòng mã hóa tiêu chuẩn (aes_round)

Module aes_round tích hợp toàn bộ bốn phép biến đổi của thuật toán AES gồm SubBytes, ShiftRows, MixColumns và AddRoundKey thành một chu trình xử lý hoàn chỉnh cho mỗi vòng mã hóa dữ liệu. Ở công đoạn cuối, kết quả sau phép trộn cột được thực hiện XOR với khóa vòng k1 để tạo ra dữ liệu đầu ra của Vòng 1. Giả đồ xung tổng thể cùng kết quả từ tool kiểm thử được trình bày trong Hình 4.18, Hình 4.19 và Hình 4.20 cho thấy giá trị đầu ra hoàn toàn trùng khớp với kết quả tính toán lý thuyết. Điều này xác nhận khối mạch thực hiện chính xác luồng xử lý dữ liệu tổ hợp xuyên suốt toàn bộ các tầng chức năng bên trong kiến trúc mã hóa AES-128. Sau đó, kết quả đầu ra hoàn chỉnh (state_out) được chốt vào thanh ghi Pipeline để làm tín hiệu đầu vào cho vòng mã hóa số 2.

Round 1

input to Round 1	
00000000 00060402 160b1517 677f7c71	
after S-Box:	
63636363 636ff277 472b59f0 85d210a3	
after permutation:	
636f59a3 632b1063 47d26377 8563f2f0	
after mult:	
8df540ce c866cd58 f72aca96 b6be12fe	
used subkey:	(Khóa K1)
4162516d 1e2a1220 4b7e577f 794e6549	
after mix with key:	
cc9711a3 d64cdf78 bc549de9 cff077b7	
0x78 = 0x58 ⊕ 0x20	

Hình 4.18: Kết quả tính toán từ Tool kiểm thử cho khối vòng mã hóa tiêu chuẩn (aes_round)



Hình 4.19: Giản đồ xung mô phỏng chức năng khối vòng mã hóa tiêu chuẩn (*aes_round*)

```

=====
                        TESTBENCH: AES ROUND 1
=====
State In (Pre-round XOR) : 00000000000060402160b1517677f7c71
Round Key (k1)           : 4162516d1e2a12204b7e577f794e6549
-----
Output Hex (Round 1 Out) : cc9711a3d64cdf78bc549de9cff077b7
Expected Output          : cc9711a3d64cdf78bc549de9cff077b7
-----
>>>      KET QUA: TEST PASSED!      <<<
=====

```

Hình 4.20: Kết quả khối vòng mã hóa tiêu chuẩn (*aes_round*) trên TCL Console

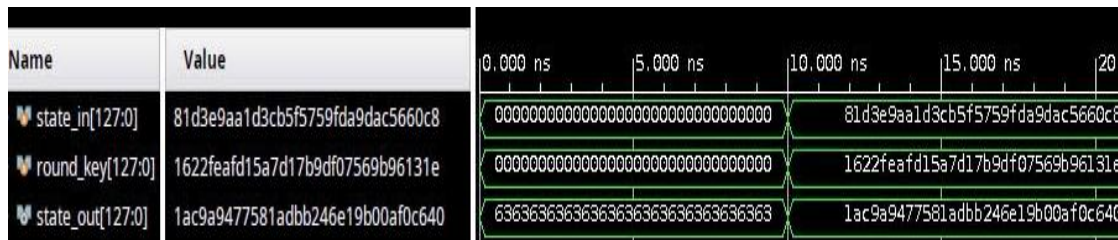
4.3.2. Mô phỏng vòng mã hóa cuối cùng

Khác với 9 vòng mã hóa trước, vòng thứ 10 được mã hóa thông qua module *aes_final_round*. Trong vòng này, dữ liệu chỉ đi qua hai phép biến đổi *SubBytes* và *ShiftRows*, sau đó được đưa trực tiếp vào khối *add_round_key* để thực hiện phép XOR với khóa vòng cuối cùng *k10* và không thực hiện phép *MixColumns*. Giản đồ xung tổng thể và kết quả từ tool kiểm thử được trình bày trong Hình 4.21, Hình 4.22 và Hình 4.23 cho thấy dữ liệu đầu ra hoàn toàn trùng khớp với kết quả tính toán lý thuyết. Điều này xác nhận khối mạch thực hiện chính xác quy trình mã hóa ở vòng cuối, tạo ra bản mã hoàn chỉnh. Sau đó, giá trị *state_out* được chốt vào thanh ghi pipeline cuối cùng và xuất ra ngoài dưới dạng dữ liệu ngõ ra chính thức (*data_out*) của khối mã hóa AES-128.

Round 10

input to Round 10
81d3e9aa 1d3cb5f5 759fda9d ac5660c8
after S-Box:
0c661eac a4ebd5e6 9ddb575e 91b1d0e8
after permutation:
0ceb57e8 a4dbd0ac 9db11ee6 9166d55e
used subkey:
1622feaf d15a7d17 b9df0756 9b96131e (Khóa K10)
after mix with key:
1ac9a947 7581adbb 246e19b0 0af0c640

Hình 4.21: Kết quả tính toán từ Tool kiểm thử cho khối vòng mã hóa cuối cùng (*aes_final_round*)



Hình 4.22: Giản đồ xung mô phỏng chức năng khối vòng mã hóa cuối cùng (*aes_final_round*)

```

=====
TESTBENCH: AES FINAL ROUND (ROUND 10)
=====
State In (Input to R10) : 81d3e9aa1d3cb5f5759fda9dac5660c8
Round Key (k10)         : 1622feafd15a7d17b9df07569b96131e
-----
Output Hex (Final Cipher): 1ac9a9477581adbb246e19b00af0c640
Expected Output         : 1ac9a9477581adbb246e19b00af0c640
-----
>>>                      KET QUA: TEST PASSED!                      <<<
=====

```

Hình 4.23: Kết quả khối vòng mã hóa cuối cùng (*aes_final_round*) trên TCL Console

4.4. Kết quả mô phỏng bộ giải mã AES-128

4.4.1. Kết quả mô phỏng vòng giải mã tiêu chuẩn (Round 1)

Khởi giải mã thực hiện quá trình biến đổi ngược nhằm khôi phục dữ liệu bản rõ từ bản mã đầu vào. Trong kiến trúc phần cứng, các khối chức năng sắp xếp theo trình tự giải mã: InvShiftRows $\rightarrow$ InvSubBytes $\rightarrow$ AddRoundKey $\rightarrow$ InvMixColumns. Quá trình kiểm chứng thực hiện trên một vòng giải mã tiêu chuẩn với dữ liệu kiểm thử và khóa vòng tương ứng, các giá trị được tổng hợp trong Bảng 4.3.

Bảng 4.3: Quá trình biến đổi một vòng giải mã tiêu chuẩn của AES-128

Công đoạn biến đổi	Giá trị chuỗi dữ liệu 128-bit tương ứng
Dữ liệu vào vòng giải mã (Cộng khóa vòng k10)	0ceb57e8a4dbd0ac9db11ee69166d55e
Sau khối dịch hàng nghịch	0c661eaca4ebd5e69ddb575e91b1d0e8
Sau khối thay thế byte nghịch	81d3e9aa1d3cb5f5759fda9dac5660c8
Sau khối cộng với khóa vòng	8df540cec866cd58f72aca96b6be12fe
Ngõ ra Vòng giải mã (Sau khối trộn cột nghịch)	cc9711a3d64cdf78bc549de9cff077b7

Để minh chứng chi tiết cho chuỗi số liệu tổng hợp trong Bảng 4.3, quá trình mô phỏng được tách thành các bước và đánh giá độc lập trên từng khối chức năng tổ hợp cốt lõi của vòng giải mã.

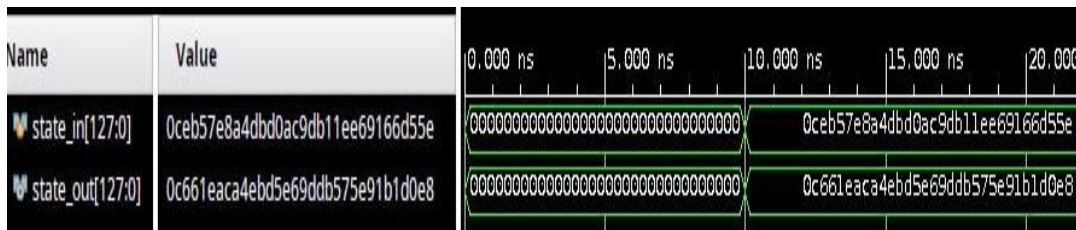
Đánh giá khối hoán vị dịch hàng nghịch (InvShiftRows)

Tại điểm bắt đầu của vòng giải mã, luồng dữ liệu 128-bit đi qua khối inv_shift_rows để thực hiện phép dịch vòng sang phải trên các hàng của ma trận trạng thái. Giảm đồ xung mô phỏng ghi nhận kết quả đầu ra và kết quả tính toán từ tool kiểm thử được thể hiện trong Hình 4.14, Hình 4.25 và Hình 4.26. Giá trị kết quả hoàn toàn trùng khớp với kết quả tính toán lý thuyết, qua đó xác nhận khối mạch thực hiện chính xác việc khôi phục vị trí các byte về nguyên trạng. Sau đó, kết quả này được truyền trực tiếp làm tín hiệu đầu vào cho khối thay thế phi tuyến ngược.

Round 10



Hình 4.24: Kết quả tính toán từ Tool kiểm thử cho khối hoán vị dịch hàng nghịch (InvShiftRows)



Hình 4.25: Giản đồ xung mô phỏng chức năng khối hoán vị dịch hàng nghịch (InvShiftRows)

```

=====
TESTBENCH: INVERSE SHIFT ROWS
=====
Input Hex (State in)       : 0ceb57e8a4dbd0ac9db11ee69166d55e
Output Hex (Shifted Right) : 0c661eaca4ebd5e69ddb575e91b1d0e8
Expected Output            : 0c661eaca4ebd5e69ddb575e91b1d0e8
>>>                        KET QUẢ: TEST PASSED!                        <<<
=====

```

Hình 4.26: Kết quả khối hoán vị dịch hàng nghịch (InvShiftRows) trên TCL Console

Đánh giá khối thay thế byte ngược (InvSubBytes)

Tiếp nối khối dịch hàng, dữ liệu được nạp vào khối inv_sub_bytes. Khối này cấu trúc từ 16 module Inverse S-Box song song, thực hiện việc tra bảng (LUTs) để đảo ngược phép biến đổi đã thực hiện lúc mã hóa. Giản đồ xung mô phỏng ghi nhận kết quả đầu ra và kết quả tính toán từ tool kiểm thử được thể hiện trong Hình 4.27, Hình 4.28 và Hình 4.29. Giá trị kết quả hoàn toàn trùng khớp với kết quả tính toán lý thuyết, qua đó xác nhận khối mạch thực hiện chính xác phép ánh xạ ngược trên trường Galois $GF(2^8)$. Sau đó, kết quả được đưa đến bước cộng khóa vòng.

Round 10

input to Round 10
0ceb57e8 a4dbd0ac 9db11ee6 9166d55e
after permutation:
0c661eac a4ebd5e6 9ddb575e 91b1d0e8
after S-Box:
81d3e9aa 1d3cb5f5 759fda9d ac5660c8
used subkey:
1bd8ac3c c77883b8 68857a41 22491448

0xe8 = S-Box[i]
 $\Rightarrow i = 0xc8$
i

Hình 4.27: Kết quả tính toán từ Tool kiểm thử cho khối thay thế byte ngược (InvSubBytes)

Name	Value	0.000 ns	5.000 ns	10.000 ns	15.000 ns	20.000 ns
state_in[127:0]	0c661eaca4ebd5e69ddb575e91b1d0e8	00000000000000000000000000000000		0c661eaca4ebd5e69ddb575e91b1d0e8		
state_out[127:0]	81d3e9aa1d3cb5f5759fda9dac5660c8	52525252525252525252525252525252		81d3e9aa1d3cb5f5759fda9dac5660c8		

Hình 4.28: Giả đồ xung mô phỏng chức năng khối thay thế byte ngược (InvSubBytes)

```

=====
TESTBENCH: INVERSE SUB BYTES
=====
Input Hex (From InvShiftRows) : 0c661eaca4ebd5e69ddb575e91b1d0e8
Output Hex (Inv-Substituted)   : 81d3e9aa1d3cb5f5759fda9dac5660c8
Expected Output                 : 81d3e9aa1d3cb5f5759fda9dac5660c8
=====
>>> KET QU: TEST PASSED! <<<
=====

```

Hình 4.29: Kết quả khối thay thế byte ngược (InvSubBytes) trên TCL Console

Trong kiến trúc giải mã AES, phép cộng khóa vòng (AddRoundKey) được thực hiện trước khối InvMixColumns, khác với trình tự của quá trình mã hóa. Về mặt phần cứng, công đoạn này tái sử dụng trực tiếp module add_round_key đã được kiểm chứng ở khối mã hóa do phép XOR 128-bit có tính chất đối xứng. Khi dữ liệu đầu ra từ khối InvSubBytes được đưa vào, mạch thực hiện phép XOR với khóa vòng giải mã tương ứng 1bd8ac3cc77883b868857a4122491448, tạo ra giá trị trung gian

9a0b4596da44364d1d1aa0dc8e1f7480. Kết quả thể hiện ở tính toán của Tool kiểm thử như Hình 30, sau đó, dữ liệu được chuyển sang khối InvMixColumns để xử lý.

Decoding Rounds			
Round 10			
input to Round 10			
0ceb57e8	a4dbd0ac	9db11ee6	9166d55e
after permutation:			
0c661eac	a4ebd5e6	9ddb575e	91b1d0e8
after S-Box:			
81d3e9aa	1d3cb5f5	759fda9d	ac5660c8
used subkey:			
1bd8ac3c	c77883b8	68857a41	22491448
after mix with key:			
9a0b4596	da44364d	1d1aa0dc	8e1f7480
after			

$$0x9a = 0x81 \oplus 0x1b$$

Hình 4.30: Kết quả tính toán từ Tool kiểm thử cho phép cộng khóa vòng ở vòng giải mã

Đánh giá khối trộn cột ngược (InvMixColumns)

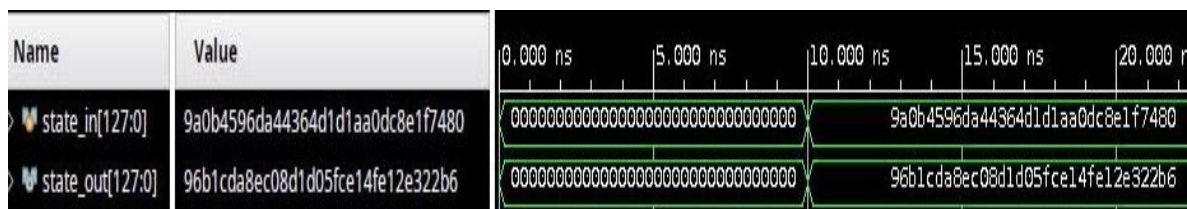
Đây là khối tiêu tốn khối lượng tính toán tổ hợp phức tạp nhất trong toàn bộ mạch giải mã, yêu cầu nhân ma trận với các hằng số đa thức lớn (0x0e, 0x0b, 0x0d, 0x09) trên trường $GF(2^8)$. Giải đồ xung mô phỏng ghi nhận kết quả đầu ra và kết quả tính toán từ tool kiểm thử được thể hiện trong Hình 4.31, Hình 4.32 và Hình 4.33. Giá trị kết quả hoàn toàn trùng khớp với kết quả tính toán lý thuyết, qua đó xác nhận khối mạch thực hiện chính xác và ổn định các phép nhân logic phức tạp.

Decoding Rounds

Round 10

input to Round 10	
0ceb57e8 a4dbd0ac 9db11ee6 9166d55e	
after permutation:	
0c661eac a4ebd5e6 9ddb575e 91b1d0e8	
after S-Box:	
81d3e9aa 1d3cb5f5 759fda9d ac5660c8	
used subkey:	
1bd8ac3c c77883b8 68857a41 22491448	
after mix with key:	
9a0b4596 da44364d 1d1aa0dc 8e1f7480	
after mult:	
96b1cda8 ec08d1d0 5fce14fe 12e322b6	$\begin{aligned} s0 &\leftarrow 0x95 = 0x0b \times 0x8e \\ s1 &\leftarrow 0x9b = 0x0d \times 0x1f \\ s2 &\leftarrow 0xf9 = 0x09 \times 0x74 \\ s3 &\leftarrow 0x41 = 0x0e \times 0x80 \\ a &\leftarrow 0x0e = s0 \oplus s1 \\ b &\leftarrow 0xb8 = s2 \oplus s3 \\ 0xb6 &= a \oplus b \end{aligned}$

Hình 4.31: Kết quả tính toán từ Tool kiểm thử cho khối trộn cột ngược (InvMixColumns)



Hình 4.32: Biểu đồ xung mô phỏng chức năng khối trộn cột ngược (InvMixColumns)


```

=====
TESTBENCH: INVERSE MIX COLUMNS
=====
Input Hex      : 9a0b4596da44364d1d1aa0dc8e1f7480
Output Hex     : 96b1cda8ec08d1d05fce14fe12e322b6
Expected Output : 96b1cda8ec08d1d05fce14fe12e322b6
=====
>>>          KET QUẢ: TEST PASSED!          <<<
=====

```

Hình 4.33: Kết quả khối trộn cột ngược (InvMixColumns) trên TCL Console

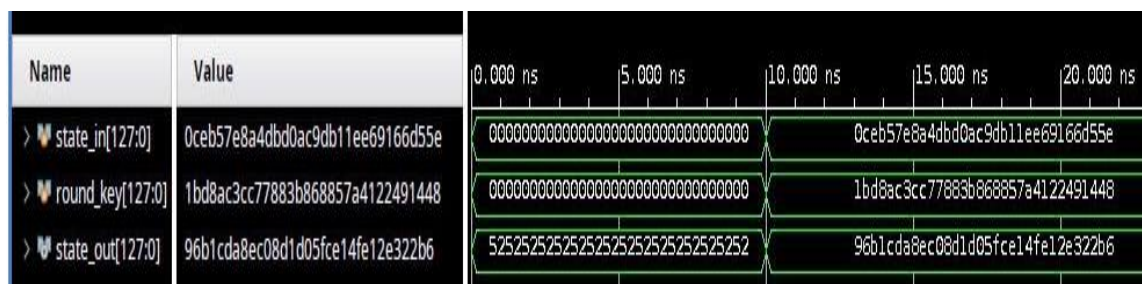
Đánh giá khối vòng giải mã tiêu chuẩn (inv_aes_round)

Module inv_aes_round tích hợp toàn bộ bốn phép biến đổi của quá trình giải mã gồm InvShiftRows, InvSubBytes, AddRoundKey và InvMixColumns thành một chu trình xử lý hoàn chỉnh. Giảm đồ xung tổng thể cùng kết quả từ tool kiểm thử được trình bày trong Hình 4.34, Hình 4.35 và Hình 4.36 cho thấy dữ liệu đầu ra (state_out) hoàn toàn trùng khớp với kết quả tính toán lý thuyết. Điều này xác nhận khối mạch thực hiện chính xác luồng xử lý dữ liệu giải mã xuyên suốt qua các tầng logic tổ hợp bên trong. Sau đó, kết quả đầu ra được chốt vào thanh ghi Pipeline để làm dữ liệu đầu vào cho các vòng giải mã tiếp theo trong hệ thống.

Round 10

input to Round 10
0ceb57e8 a4dbd0ac 9db11ee6 9166d55e
after permutation:
0c661eac a4ebd5e6 9ddb575e 91b1d0e8
after S-Box:
81d3e9aa 1d3cb5f5 759fda9d ac5660c8
used subkey:
1bd8ac3c c77883b8 68857a41 22491448
after mix with key:
9a0b4596 da44364d 1d1aa0dc 8e1f7480
after mult:
96b1cda8 ec08d1d0 5fce14fe 12e322b6

Hình 4.34: Kết quả tính toán từ Tool kiểm thử cho khối vòng giải mã tiêu chuẩn (inv_aes_round)



Hình 4.35: Giản đồ xung mô phỏng chức năng khối vòng giải mã tiêu chuẩn (*inv_aes_round*)

```

=====
TESTBENCH: INVERSE AES ROUND
=====
State In   : 0ceb57e8a4dbd0ac9db11ee69166d55e
Round Key  : 1bd8ac3cc77883b868857a4122491448
-----
Output Hex : 96b1cda8ec08d1d05fce14fe12e322b6
Expected   : 96b1cda8ec08d1d05fce14fe12e322b6
-----
>>>          KET QUẢ: TEST PASSED!          <<<
=====

```

Hình 4.36: Kết quả khối vòng giải mã tiêu chuẩn trên TCL Console

4.4.2. Mô phỏng vòng giải mã cuối cùng

Đóng vai trò là vòng cuối cùng của quá trình giải mã (*inv_aes_final_round*), dữ liệu trong vòng này chỉ đi qua hai phép biến đổi *InvShiftRows* và *InvSubBytes*, sau đó thực hiện phép XOR trực tiếp với khóa gốc *k0* và không thực hiện phép *InvMixColumns*. Giản đồ xung tổng thể và kết quả từ tool kiểm thử được trình bày trong Hình 4.37, Hình 4.38 và Hình 4.39 cho thấy giá trị đầu ra thu được tương ứng chuỗi ASCII “DATN_NGOC_PHUONG”. Kết quả này hoàn toàn trùng khớp với tính toán lý thuyết, qua đó xác nhận hệ thống phần cứng đã khôi phục chính xác bản rõ ban đầu và chứng minh tính đúng đắn của toàn bộ lõi mã hóa và giải mã AES-128.

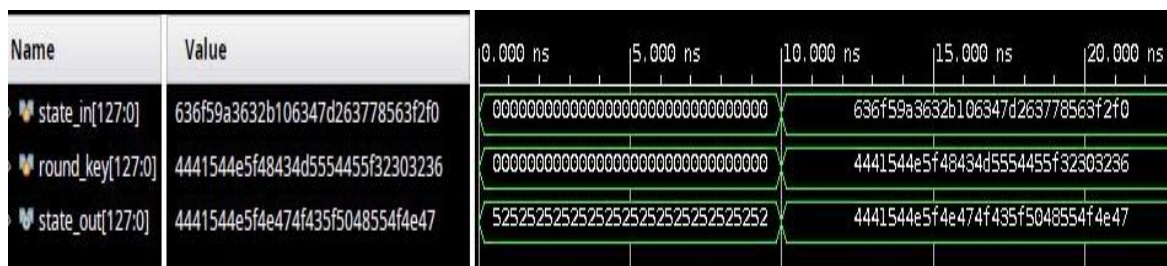
Round 1

input to Round 1
636f59a3 632b1063 47d26377 8563f2f0
after permutation:
63636363 636ff277 472b59f0 85d210a3
after S-Box:
00000000 00060402 160b1517 677f7c71
used subkey:
4441544e 5f48434d 5554455f 32303236
after mix with key:
4441544e 5f4e474f 435f5048 554f4e47

Decoded

4441544e 5f4e474f 435f5048 554f4e47

Hình 4.37: Kết quả tính toán từ Tool kiểm thử cho khối vòng giải mã cuối cùng
(*inv_aes_final_round*)



Hình 4.38: Giả đồ xung mô phỏng chức năng khối vòng giải mã cuối cùng
(*inv_aes_final_round*)

```

=====
TESTBENCH: INVERSE AES FINAL ROUND (ROUND 10)
=====
State In   : 636f59a3632b106347d263778563f2f0
Round Key  : 4441544e5f48434d5554455f32303236
-----
Output Hex : 4441544e5f4e474f435f5048554f4e47
Expected   : 4441544e5f4e474f435f5048554f4e47 (DATN_NGOC_PHUONG)
-----
>>>          KET QU: TEST PASSED!          <<<
=====

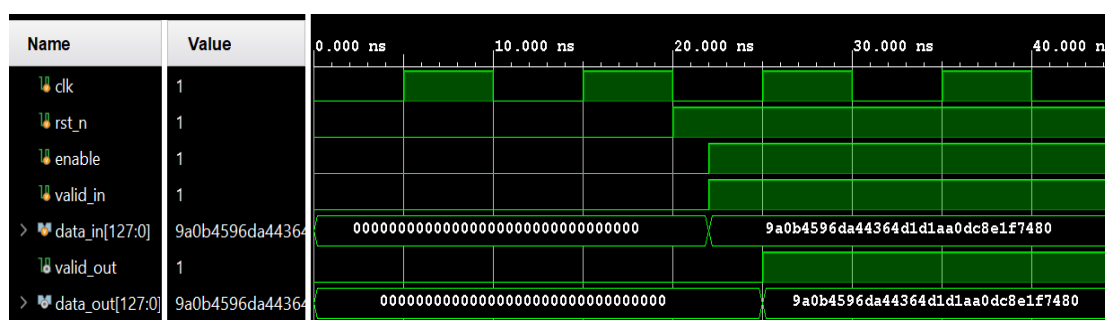
```

Hình 4.39: Kết quả khối vòng giải mã cuối cùng trên TCL Console.

4.5. Kết quả mô phỏng khối thanh ghi đường ống (Pipeline Registers)

Khối pipeline_regs đóng vai trò là các thanh ghi đồng bộ giữa các tầng xử lý trong kiến trúc Pipeline của thuật toán AES. Nhiệm vụ của khối mạch này là lưu trữ dữ liệu 128-bit (data_in) cùng tín hiệu hợp lệ (valid_in) từ tầng xử lý trước, sau đó truyền sang tầng kế tiếp tại mỗi cạnh lên của xung nhịp hệ thống.

Để kiểm chứng hoạt động của module, testbench tiến hành nạp dữ liệu thử nghiệm 9a0b4596da44364d1d1aa0dc8e1f7480 đồng thời kích hoạt các tín hiệu enable = 1 và valid_in = 1. Giản đồ xung mô phỏng được trình bày trong Hình 4.40 và Hình 4.41. Quan sát trên waveform cho thấy ngay sau cạnh lên của tín hiệu clk, toàn bộ dữ liệu 128-bit cùng cờ hợp lệ được truyền chính xác sang các ngõ ra data_out và valid_out. Thông báo “TEST PASSED” xác nhận module hoạt động đúng chức năng lưu trữ và đồng bộ dữ liệu mà không xảy ra sai lệch bit. Kết quả này cho thấy module hoàn toàn đáp ứng yêu cầu triển khai cho toàn bộ các tầng Pipeline trong kiến trúc AES-128 tốc độ cao.



Hình 4.40: Giản đồ xung mô phỏng chức năng khối thanh ghi đường ống (Pipeline Registers)

```

=====
                        TESTBENCH: PIPELINE REGISTER
=====
Input Hex      : 9a0b4596da44364d1d1aa0dc8e1f7480
Output Hex     : 9a0b4596da44364d1d1aa0dc8e1f7480
Expected Output: 9a0b4596da44364d1d1aa0dc8e1f7480
-----
>>>           KET QUA: TEST PASSED!           <<<
=====

```

Hình 4.41: Kết quả khối thanh ghi đường ống (Pipeline Registers) trên TCL Console.

4.6. Kết quả mô phỏng khối điều khiển trung tâm (FSM Control)

Module fsm_control đóng vai trò điều khiển hoạt động của toàn bộ hệ thống thông qua kiến trúc máy trạng thái hữu hạn (FSM) gồm 5 trạng thái chính: IDLE, LOAD_KEY, LOAD_DATA, PROCESS_ENC và PROCESS_DEC. Để kiểm chứng

hoạt động của FSM, kịch bản testbench được xây dựng với 6 trường hợp kiểm thử nhằm bao phủ toàn bộ quá trình vận hành của hệ thống.

Quá trình chuyển đổi trạng thái được ghi nhận như sau:

- Test case 1 - RESET SYSTEM: Khi tín hiệu reset tích cực mức thấp $\text{rst_n} = 0$ được đưa vào, mạch logic tổ hợp bỏ qua toàn bộ các kích thích khác. Hệ thống lập tức bị ép về trạng thái nghỉ $\text{fsm_state} = \text{IDLE}$ (mã trạng thái 000). Các tín hiệu điều khiển ngõ ra load_key và valid_out đều được ghim ở mức logic an toàn (0).

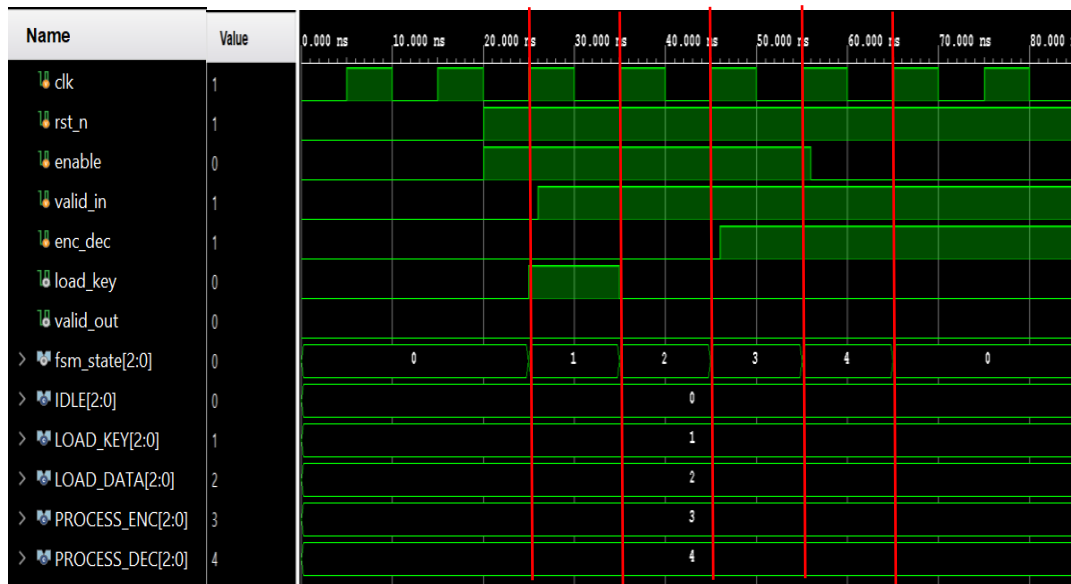
- Test case 2 - ENABLE SYSTEM: Khi hệ thống được đưa ra khỏi trạng thái reset ($\text{rst_n} = 1$) và tín hiệu $\text{enable} = 1$ được kích hoạt, FSM sẽ chuyển sang trạng thái $\text{fsm_state} = \text{LOAD_KEY}$ (mã 001) tại cạnh lên kế tiếp của xung nhịp hệ thống. Đồng thời, ngõ ra load_key được kích hoạt lên mức 1 trong đúng một chu kỳ xung nhịp để thực hiện lệnh nạp khóa gốc vào khối Key Expansion.

- Test case 3 - LOAD DATA: Sau một chu kỳ thực hiện nạp khóa, FSM tự động chuyển sang trạng thái $\text{fsm_state} = \text{LOAD_DATA}$ (mã 010) mà không cần thêm tín hiệu điều khiển từ ngõ vào. Đồng thời, tín hiệu load_key được đưa trở về mức logic 0. Tại trạng thái này, FSM sẽ duy trì chế độ chờ cho đến khi tín hiệu valid_in được kích hoạt, báo hiệu dữ liệu đầu vào đã sẵn sàng để xử lý.

- Test case 4 và Test case 5 - ENCRYPT/DECRYPT MODE: Khi hệ thống nhận được tín hiệu dữ liệu hợp lệ ($\text{valid_in} = 1$) và chế độ hoạt động được thiết lập là mã hóa ($\text{enc_dec} = 0$), FSM sẽ chuyển sang trạng thái PROCESS_ENC ($\text{fsm_state} = 3'b011$). Tại trạng thái này, dữ liệu đầu vào bắt đầu được đưa vào kiến trúc Pipeline để thực hiện quá trình mã hóa AES-128. Ngược lại, nếu tín hiệu điều khiển được thay đổi sang chế độ giải mã ($\text{enc_dec} = 1$), FSM sẽ tự động chuyển sang trạng thái PROCESS_DEC ($\text{fsm_state} = 3'b100$) tại cạnh lên tiếp theo của xung nhịp hệ thống. Kết quả này cho thấy bộ điều khiển có khả năng chuyển đổi linh hoạt giữa hai chế độ mã hóa và giải mã mà không cần khởi động lại hệ thống.

- Test case 6 - DISABLE SYSTEM: Cho dù tín hiệu valid_in vẫn duy trì ở mức 1, khi tín hiệu cấp phép enable bị đưa về mức thấp ($\text{enable} = 0$), FSM sẽ ưu tiên điều kiện điều khiển này và lập tức quay trở lại trạng thái khởi tạo IDLE.

Toàn bộ quá trình thay đổi của các tín hiệu đầu vào cùng đáp ứng trạng thái tương ứng của 6 test case kiểm thử đều được ghi nhận và thể hiện trực quan trên giản đồ xung trong Hình 4.42.



Hình 4.42: Giảns đồ xung mô phỏng các test case chuyển đổi trạng thái của module điều khiển trung tâm (fsm_control).

Giai đoạn khởi tạo hệ thống (test case 1) trong khoảng thời gian từ 0 μ s đến 15 μ s, do tín hiệu kích hoạt enable đang ở mức thấp (0), máy trạng thái được giữ cố định ở trạng thái 0 (IDLE).

Giai đoạn kích hoạt và nạp khóa (test case 2) ở khoảng thời điểm 20 μ s, tín hiệu enable được kéo lên mức 1. Ngay tại sườn lên của xung nhịp clk tiếp theo (tại mốc 25 μ s), FSM lập tức chuyển trạng thái từ IDLE (0) sang LOAD_KEY (1). Đồng thời, tín hiệu load_key được kích hoạt lên mức logic 1 trong đúng một chu kỳ xung nhịp nhằm phát lệnh nạp khóa gốc vào khối Key Expansion.

Ở giai đoạn kiểm tra trạng thái chờ (test case 3), Sau đúng một chu kỳ thực hiện nạp khóa, tại cạnh lên tiếp theo của xung nhịp hệ thống (mốc 35 μ s), FSM tự động chuyển sang trạng thái LOAD_DATA (2) mà không cần thêm tín hiệu kích thích từ ngõ vào. Đồng thời, tín hiệu load_key được đưa trở về mức logic 0. Tại thời điểm này, tín hiệu dữ liệu đầu vào bắt đầu được xác lập hợp lệ với valid_in = 1, sẵn sàng cho quá trình xử lý tiếp theo trong kiến trúc Pipeline.

Tại giai đoạn kiểm tra xử lý chế độ hoạt động (test case 4,5), do phát hiện tín hiệu valid_in = 1 đồng thời chế độ hoạt động được thiết lập là mã hóa (enc_dec = 0), tại cạnh lên của xung nhịp ở mốc 45 μ s, tại FSM chuyển chính xác sang trạng thái PROCESS_ENC (3). Từ thời điểm này, dữ liệu đầu vào bắt đầu được đưa vào kiến trúc Pipeline để thực hiện quá trình mã hóa AES-128. Tương tự với chế độ giải mã, Tại thời điểm 46 μ s, tín hiệu chọn chế độ enc_dec được chuyển sang mức logic 1 để yêu cầu

thực hiện giải mã. Ngay tại cạnh lên tiếp theo của xung nhịp hệ thống (55 μ s), FSM lập tức chuyển trạng thái từ PROCESS_ENC (3) sang PROCESS_DEC (4).

Tại khoảng thời gian 56 μ s, hệ thống tiến hành kiểm tra *chế độ dừng hoạt động* (testcase 6) bằng cách đưa tín hiệu enable về mức logic 0. Mặc dù tín hiệu dữ liệu đầu vào vẫn còn hợp lệ (valid_in = 1), FSM vẫn ưu tiên điều kiện điều khiển từ chân enable và lập tức chuyển trở về trạng thái IDLE (0) tại cạnh lên xung nhịp tiếp theo ở mốc 65 ns.

Chi tiết kết quả kiểm thử tự động của 6 test case được lấy trực tiếp từ log console của công cụ mô phỏng. Các bản ghi log này thể hiện đầy đủ trạng thái của các tín hiệu kích thích đầu vào (INPUT SIGNALS) và phản hồi thực tế của FSM tại ngõ ra (OUTPUT SIGNALS) để đối chiếu với giá trị kỳ vọng. Chuỗi kết quả từ Hình 4.43 đến Hình 4.48 cho thấy khối điều khiển trung tâm đã thực hiện chính xác các quá trình chuyển trạng thái và định hướng hoạt động trong toàn bộ các trường hợp kiểm thử. Việc hệ thống đều trả về thông báo “TEST PASSED!” ở cả 6 test case là cơ sở xác nhận module FSM Control hoạt động ổn định, chính xác và đáp ứng đầy đủ các yêu cầu thiết kế đã đề ra.

```
=====
Testcase           : TC1 - RESET SYSTEM
Time               : 20000 ns
-----
INPUT SIGNALS
rst_n              : 0
enable             : 0
valid_in           : 0
enc_dec            : 0
-----
OUTPUT SIGNALS
FSM State          :          IDLE
Expected State     :          IDLE
load_key           : 0
valid_out          : 0
Mode               : ENCRYPTION
-----
>>>                KET QUA: TEST PASSED!                <<<
=====
```

Hình 4.43: Log kết quả kiểm thử test case 1 (Khởi tạo hệ thống)

```

=====
                        TESTBENCH: FSM CONTROL
=====
Testcase           : TC2 - ENABLE SYSTEM
Time               : 26000 ns
=====
INPUT SIGNALS
rst_n              : 1
enable             : 1
valid_in           : 0
enc_dec            : 0
=====
OUTPUT SIGNALS
FSM State          :          LOAD_KEY
Expected State     :          LOAD_KEY
load_key           : 1
valid_out          : 0
Mode               : ENCRYPTION
=====
>>>                KET QUA: TEST PASSED!                <<<
=====

```

Hình 4.44: Log kết quả kiểm thử test case 2 (Kích hoạt nạp khóa)

```

=====
Testcase           : TC3 - LOAD DATA
Time               : 36000 ns
=====
INPUT SIGNALS
rst_n              : 1
enable             : 1
valid_in           : 1
enc_dec            : 0
=====
OUTPUT SIGNALS
FSM State          :          LOAD_DATA
Expected State     :          LOAD_DATA
load_key           : 0
valid_out          : 0
Mode               : ENCRYPTION
=====
>>>                KET QUA: TEST PASSED!                <<<
=====

```

Hình 4.45: Log kết quả kiểm thử test case 3 (Chờ dữ liệu hợp lệ)

```

=====
Testcase          : TC4 - ENCRYPT MODE
Time              : 46000 ns
-----

INPUT SIGNALS
rst_n             : 1
enable            : 1
valid_in          : 1
enc_dec           : 0
-----

OUTPUT SIGNALS
FSM State         :      PROCESS_ENC
Expected State    :      PROCESS_ENC
load_key          : 0
valid_out         : 0
Mode              : ENCRYPTION
-----

>>>                KET QUA: TEST PASSED!                <<<
=====

```

Hình 4.46: Log kết quả kiểm thử test case 4 (Xử lý chế độ mã hóa)

```

=====
Testcase          : TC5 - DECRYPT MODE
Time              : 56000 ns
-----

INPUT SIGNALS
rst_n             : 1
enable            : 1
valid_in          : 1
enc_dec           : 1
-----

OUTPUT SIGNALS
FSM State         :      PROCESS_DEC
Expected State    :      PROCESS_DEC
load_key          : 0
valid_out         : 0
Mode              : DECRYPTION
-----

>>>                KET QUA: TEST PASSED!                <<<
=====

```

Hình 4.47: Log kết quả kiểm thử test case 5 (Xử lý chế độ giải mã)

```

=====
Testcase           : TC6 - DISABLE SYSTEM
Time               : 66000 ns
-----

INPUT SIGNALS
rst_n              : 1
enable             : 0
valid_in           : 1
enc_dec            : 1
-----

OUTPUT SIGNALS
FSM State          :          IDLE
Expected State     :          IDLE
load_key           : 0
valid_out          : 0
Mode               : DECRYPTION
-----

>>>                KET QUA: TEST PASSED!                <<<
=====

```

Hình 4.48: Log kết quả kiểm thử test case 6 (Ngắt hệ thống)

4.7. Kết quả mô phỏng khối tích hợp hệ thống (aes_core)

4.7.1. Mục tiêu kiểm thử

Mục tiêu của quá trình mô phỏng lõi AES-128 pipeline là kiểm chứng tính đúng đắn của chức năng mã hóa và giải mã, đồng thời đánh giá hiệu quả cải thiện thông lượng xử lý của kiến trúc được đề xuất. Thiết kế cần được xác minh khả năng tích hợp và hoạt động chính xác của các khối chức năng chính gồm SubBytes, ShiftRows, MixColumns, AddRoundKey, Key Expansion, InvSubBytes, InvShiftRows, InvMixColumns và Key Expansion trong môi trường pipeline với các thanh ghi được chèn giữa các vòng xử lý AES.

Quá trình kiểm thử được thực hiện thông qua testbench sử dụng các bộ dữ liệu chuẩn và dữ liệu tự định nghĩa. Kết quả đầu ra của hệ thống được so sánh với giá trị mong đợi nhằm xác nhận tính chính xác của thuật toán AES. Đồng thời, các tín hiệu điều khiển và trạng thái như valid_in, valid_out, pipeline_valid_dbg, reset và enable được theo dõi để đánh giá quá trình truyền dữ liệu, mức độ đồng bộ giữa các tầng pipeline và khả năng điều khiển của hệ thống.

Các nội dung kiểm thử tập trung vào việc đánh giá độ chính xác của quá trình mã hóa và giải mã, xác định độ trễ xử lý, kiểm tra thông lượng khi xử lý liên tục nhiều khối dữ liệu và so sánh mức cải thiện hiệu năng so với kiến trúc AES tuần tự. Kết quả

kiểm thử là cơ sở để xác nhận tính đúng đắn của thiết kế và đánh giá hiệu quả của việc áp dụng kiến trúc pipeline trong các hệ thống yêu cầu mã hóa dữ liệu tốc độ cao..

4.7.2. Các kịch bản kiểm thử hệ thống tổng thể

Thông số mô phỏng

Quá trình mô phỏng lõi AES-128 được thực hiện với các thông số cơ bản được thiết lập trong testbench. Các thông số này bao gồm chu kỳ clock, tần số hoạt động, độ rộng dữ liệu, độ rộng khóa và chế độ thuật toán AES được sử dụng. Trong mô phỏng, tín hiệu clock được tạo với chu kỳ 10 ns, tương ứng với tần số hoạt động 100 MHz. Dữ liệu đầu vào và đầu ra của lõi AES có độ rộng 128 bit, phù hợp với kích thước khối dữ liệu tiêu chuẩn của thuật toán AES. Khóa đầu vào cũng có độ rộng 128 bit, do đó thiết kế hoạt động ở chế độ AES-128. Các thông số mô phỏng chính được trình bày như bảng 4.4

Bảng 4.4. Các thông số mô phỏng của lõi AES-128

Thông số	Giá trị
Clock period	10 ns
Frequency	100 MHz
Data width	128 bit
Key width	128 bit
AES mode	AES-128

Với chu kỳ clock 10 ns, mỗi tầng pipeline được cập nhật sau một cạnh lên của clock. Do thiết kế sử dụng kiến trúc pipeline gồm 11 stage, một khối dữ liệu cần khoảng 11 chu kỳ clock để đi từ đầu vào đến đầu ra. Tuy nhiên, sau khi pipeline được lấp đầy, lõi AES có khả năng xuất ra một khối dữ liệu 128 bit ở mỗi chu kỳ clock. Đây là cơ sở để đánh giá độ trễ và thông lượng của thiết kế trong các phân phân tích tiếp theo.

Danh sách các test case mô phỏng

Bảng dưới đây trình bày danh sách các test case được sử dụng để mô phỏng và đánh giá hoạt động của khối AES-128 pipeline. Các test case được xây dựng nhằm kiểm tra từng chức năng chính của thiết kế, bao gồm quá trình mã hóa, giải mã và kiểm tra vòng lặp kết hợp giữa mã hóa và giải mã như được thể hiện trong bảng 4.5.

Bảng 4.5: Danh sách các test case mô phỏng khối AES-128 pipeline

STT	Mã test case	Tên test case	Mục đích kiểm thử
1	TC1	Kiểm tra mã hóa AES-128 theo chuẩn FIPS	Kiểm tra chức năng mã hóa của khối AES-128 pipeline bằng bộ dữ liệu chuẩn FIPS. Kết quả mã hóa được so sánh với ciphertext kỳ vọng.
2	TC2	Kiểm tra giải mã AES-128 theo chuẩn FIPS	Kiểm tra chức năng giải mã bằng cách đưa ciphertext chuẩn FIPS vào khối giải mã và so sánh kết quả với plaintext ban đầu.
3	TC3	Kiểm tra mã hóa dữ liệu DATN	Kiểm tra khả năng mã hóa của thiết kế với bộ dữ liệu tự định nghĩa dùng trong đồ án. Ciphertext đầu ra được ghi nhận để sử dụng cho testcase giải mã tiếp theo.
4	TC4	Kiểm tra giải mã dữ liệu DATN	Kiểm tra khả năng giải mã ciphertext thu được từ TC3. Kết quả giải mã được so sánh với plaintext DATN ban đầu.
5	TC5	Kiểm tra tín hiệu enable giữ pipeline	Kiểm tra khả năng tạm dừng và tiếp tục hoạt động của pipeline khi tín hiệu enable chuyển từ mức 1 sang 0 rồi trở lại mức 1.
6	TC6	Kiểm tra reset giữa quá trình pipeline	Kiểm tra khả năng xử lý của thiết kế khi tín hiệu reset được kích hoạt trong lúc pipeline đang xử lý dữ liệu. Sau reset, đầu ra và tín hiệu hợp lệ phải được đưa về trạng thái an toàn.
7	TC7	Kiểm tra throughput của pipeline	Kiểm tra khả năng xử lý liên tục nhiều block dữ liệu của kiến trúc pipeline. Sau khi pipeline được lấp đầy, thiết kế cần cho ra một block dữ liệu sau mỗi chu kỳ clock.

Các tín hiệu quan sát

Trong quá trình mô phỏng, một số tín hiệu quan trọng của lõi AES-128 được đưa vào cửa sổ waveform để theo dõi hoạt động của hệ thống. Các tín hiệu này giúp kiểm tra quá trình reset, nạp khóa, nhận dữ liệu đầu vào, truyền dữ liệu qua pipeline và xuất kết quả đầu ra. Các tín hiệu quan sát chính bao gồm:

Nhóm tín hiệu điều khiển: Các tín hiệu điều khiển dùng để xác định trạng thái hoạt động của lõi AES-128 trong quá trình mô phỏng.

- clk: tín hiệu xung clock của hệ thống. Dữ liệu trong các thanh ghi pipeline được cập nhật tại cạnh lên của tín hiệu này.
- rst_n: tín hiệu reset tích cực mức thấp. Khi rst_n = 0, các thanh ghi dữ liệu và tín hiệu valid trong pipeline được đưa về trạng thái ban đầu.
- enable: tín hiệu cho phép lõi AES hoạt động. Khi enable = 1, dữ liệu được truyền qua các tầng pipeline. Khi enable = 0, các thanh ghi pipeline giữ nguyên giá trị hiện tại.
- enc_dec: tín hiệu chọn chế độ làm việc của lõi AES. Khi enc_dec = 0, hệ thống thực hiện mã hóa. Khi enc_dec = 1, hệ thống thực hiện giải mã.

Nhóm tín hiệu đầu vào: Các tín hiệu đầu vào dùng để cung cấp khóa, dữ liệu và báo hiệu thời điểm dữ liệu đầu vào hợp lệ.

- key_in: khóa đầu vào 128 bit được sử dụng cho quá trình mã hóa hoặc giải mã AES-128.
- data_in: dữ liệu đầu vào 128 bit. Trong chế độ mã hóa, tín hiệu này là plaintext. Trong chế độ giải mã, tín hiệu này là ciphertext.
- valid_in: tín hiệu báo dữ liệu đầu vào hợp lệ. Khi valid_in = 1, dữ liệu tại data_in được xem là hợp lệ và được đưa vào pipeline tại cạnh lên của clock.

Nhóm tín hiệu đầu ra: Các tín hiệu đầu ra dùng để kiểm tra kết quả xử lý của lõi AES.

- data_out: dữ liệu đầu ra 128 bit của lõi AES. Trong chế độ mã hóa, đây là ciphertext. Trong chế độ giải mã, đây là plaintext sau khi khôi phục.
- valid_out: tín hiệu báo dữ liệu đầu ra hợp lệ. Tín hiệu này được kích hoạt khi dữ liệu đã đi qua toàn bộ các tầng pipeline và giá trị tại data_out có thể được kiểm tra.

Nhóm tín hiệu quan sát pipeline: Do thiết kế sử dụng kiến trúc pipeline, các tín hiệu debug được bổ sung để quan sát rõ quá trình dữ liệu di chuyển qua từng stage.

- pipeline_valid_dbg: tín hiệu 11 bit cho biết trạng thái hợp lệ của dữ liệu tại từng stage pipeline. Mỗi bit tương ứng với một stage từ Stage 0 đến Stage 10.
- stage0_data_dbg: dữ liệu sau tầng Initial AddRoundKey.

- stage1_data_dbg: dữ liệu sau AES Round 1.
- stage2_data_dbg: dữ liệu sau AES Round 2.
- ...
- stage9_data_dbg: dữ liệu sau AES Round 9.
- stage10_data_dbg: dữ liệu sau Final Round, tương ứng với kết quả cuối cùng trước khi đưa ra data_out.

Cụ thể, trong chế độ mã hóa, các tín hiệu stageX_data_dbg cho phép quan sát dữ liệu sau từng vòng xử lý AES. Nhờ đó, có thể kiểm tra quá trình biến đổi dữ liệu qua các bước SubBytes, ShiftRows, MixColumns và AddRoundKey.

4.7.2.1. Kết quả mô phỏng TC1 – Mã hóa AES-128 với vector chuẩn FIPS

Mục tiêu: Test case TC1 được xây dựng nhằm kiểm chứng chức năng mã hóa của lõi AES-128 khi sử dụng bộ vector chuẩn FIPS. Trong quá trình mô phỏng, khóa 128 bit và khối plaintext 128 bit được đưa vào lõi AES để thực hiện quá trình mã hóa. Kết quả đầu ra data_out sau đó được so sánh với giá trị ciphertext mong đợi tại thời điểm tín hiệu valid_out được kích hoạt. Thông qua test case này, có thể đánh giá tính đúng đắn của quá trình mã hóa, khả năng truyền dữ liệu qua các tầng pipeline, cũng như sự đồng bộ giữa dữ liệu đầu ra và tín hiệu báo hợp lệ. Kết quả được xem là đạt yêu cầu khi data_out trùng khớp với ciphertext chuẩn tại thời điểm valid_out = 1.

Bảng 4.6: Bộ dữ liệu sử dụng trong TC1

Tín hiệu	Giá trị	Ý nghĩa
Khóa mã hóa	000102030405060708090a0b0c0d0e0f	Khóa mã hóa 128 bit
Dữ liệu đầu vào	00112233445566778899aabbccddeeff	Dữ liệu plaintext 128 bit đưa vào lõi AES
Dữ liệu đầu ra	69c4e0d86a7b0430d8cdb78070b4c55a	Ciphertext chuẩn dùng để so sánh với data_out

Trình tự thực hiện mô phỏng

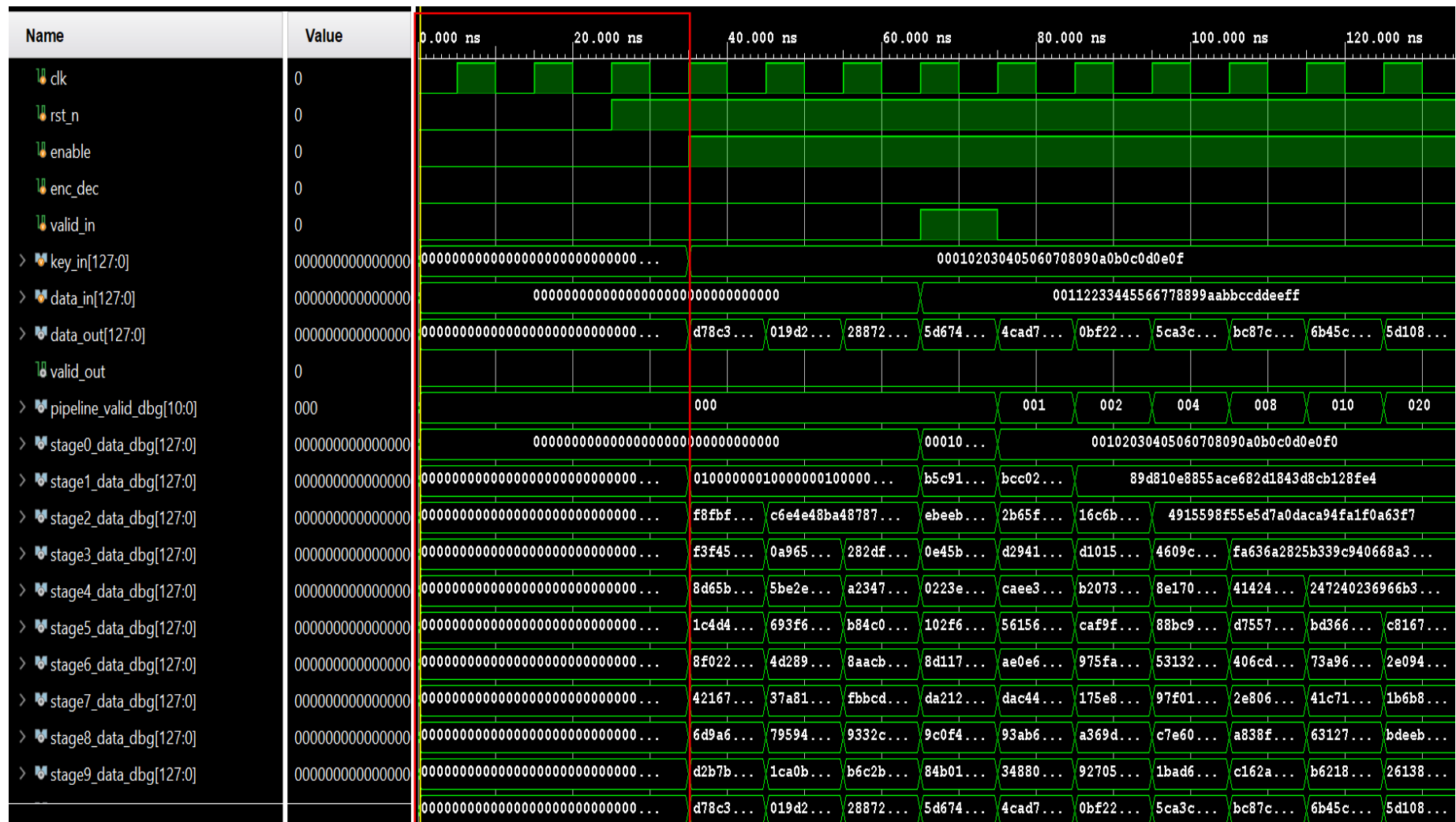
Trước tiên, Khối AES được reset để đưa hệ thống về trạng thái ban đầu. Sau đó, khóa chuẩn FIPS được nạp vào Khối và dữ liệu bản rõ 00112233445566778899AABBCCDDEEFF được đưa vào với chế độ mã hóa enc_dec = 0. Khi tín hiệu valid_out được kích hoạt, kết quả tại data_out được so sánh với bản mã chuẩn 69C4E0D86A7B0430D8CDB78070B4C55A. Nếu hai giá trị trùng khớp, Testcase 1 được đánh giá là PASS.

Phân tích dạng sóng

Quan sát hình 4.49, có thể thấy trong khoảng từ 0 ns đến 35 ns, dạng sóng thể hiện giai đoạn khởi tạo và chuẩn bị dữ liệu đầu vào cho hệ thống AES pipeline. Ban đầu tại 0 ns, tín hiệu `rst_n` = 0 nên toàn bộ hệ thống đang ở trạng thái reset. Khi đó các thanh ghi pipeline đều được xóa về giá trị 0, vì vậy các tín hiệu như `data_out`, `pipeline_valid_dbg`, cùng dữ liệu ở các stage (`stage0_data_dbg` đến `stage9_data_dbg`) đều bằng 0. Tín hiệu clock `clk` hoạt động liên tục với chu kỳ khoảng 10 ns, và do thiết kế tác động ở cạnh lên nên mọi dữ liệu chỉ được chốt tại cạnh lên của xung clock.

Đến khoảng 25 ns, tín hiệu `rst_n` được đưa lên mức 1, hệ thống thoát khỏi trạng thái reset và hoạt động bình thường. Tuy nhiên lúc này tín hiệu `enable` vẫn bằng 0, nên pipeline chưa cho phép dữ liệu lan truyền qua các stage xử lý. Vì vậy mặc dù clock vẫn chạy, toàn bộ dữ liệu bên trong hệ thống vẫn giữ nguyên trạng thái ban đầu.

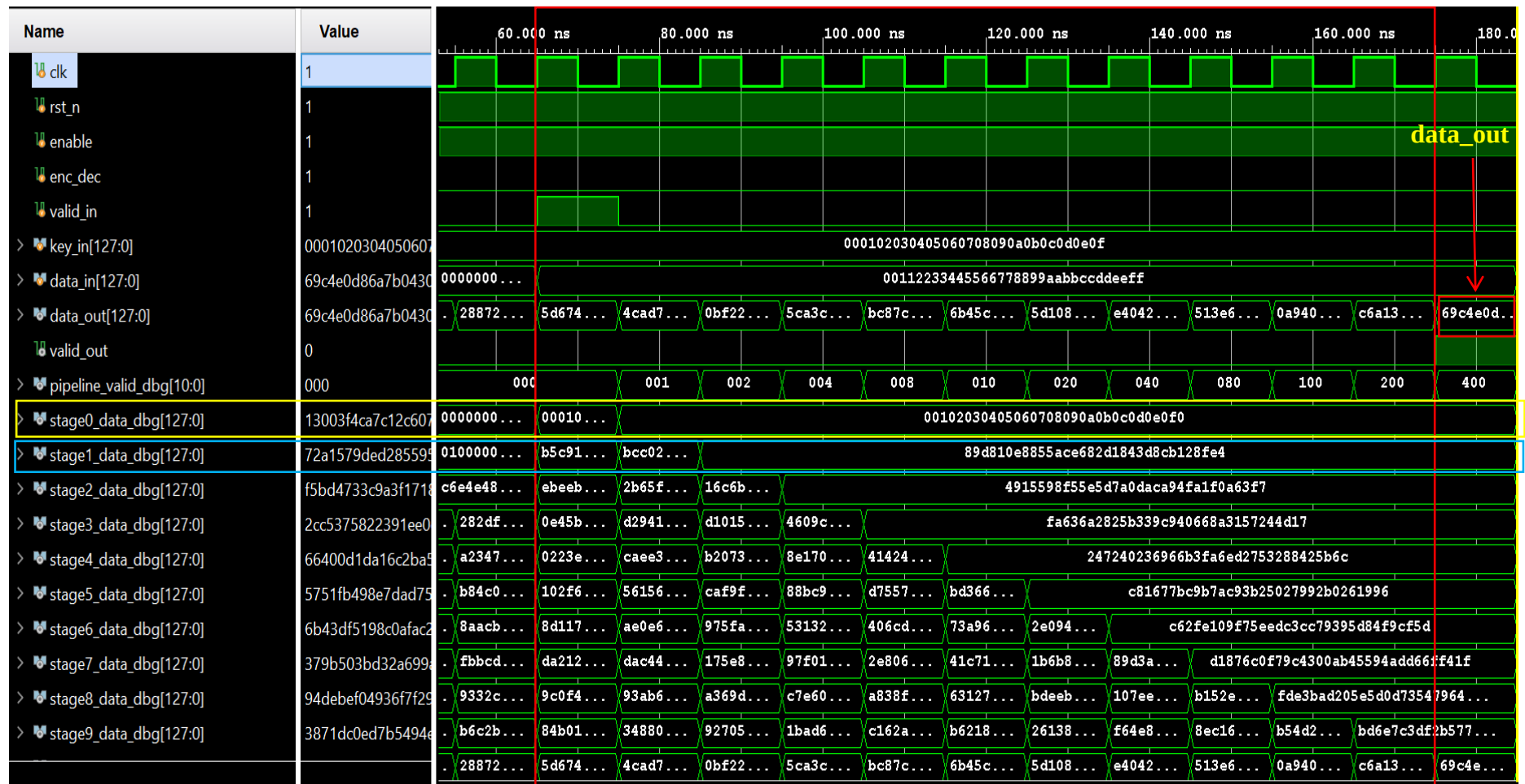
Tại khoảng 35 ns, tín hiệu `enable` chuyển sang mức 1, cho phép pipeline bắt đầu hoạt động. Trong giai đoạn này tín hiệu `enc_dec` = 0, nghĩa là hệ thống đang được cấu hình ở chế độ mã hóa AES-128. Đồng thời `valid_in` vẫn bằng 0, cho thấy dữ liệu đầu vào hợp lệ vẫn chưa được đưa vào pipeline. Các tín hiệu `key_in` và `data_in` tại thời điểm này vẫn chủ yếu là giá trị 0, nên dữ liệu ở các stage pipeline chưa có sự thay đổi đáng kể. Đây là giai đoạn hệ thống hoàn tất quá trình khởi tạo và sẵn sàng tiếp nhận plaintext cùng khóa mã hóa ở các chu kỳ tiếp theo.



Hình 4.49: Kết quả dạng sóng mô phỏng ban đầu

Trong hình 4.50, hệ thống được cấu hình ở chế độ mã hóa với tín hiệu `enc_dec = 0`. Sau khi tín hiệu `reset rst_n` được đưa lên mức logic 1 và tín hiệu `enable = 1`, pipeline bắt đầu hoạt động đồng bộ theo các cạnh lên của xung clock. Tại khoảng thời gian 65 ns, tín hiệu `valid_in` được kích hoạt trong một chu kỳ clock để báo hiệu dữ liệu đầu vào hợp lệ. Khi đó, khóa mã hóa `key_in` được nạp với giá trị 000102030405060708090a0b0c0d0e0f, đồng thời plaintext đầu vào `data_in` là 00112233445566778899aabbccddeeff. Sau khi dữ liệu được đưa vào hệ thống, tín hiệu `pipeline_valid_dbg` thay đổi tuần tự theo dạng dịch bit từ 001, 002, 004, 008, 010, 020... cho thấy dữ liệu đang được lan truyền lần lượt qua các stage của pipeline. Quan sát các tín hiệu `stage0_data_dbg` đến `stage10_data_dbg` cho thấy dữ liệu tại mỗi tầng xử lý thay đổi liên tục theo từng chu kỳ clock, phản ánh quá trình thực hiện các phép biến đổi của thuật toán AES như SubBytes, ShiftRows, MixColumns và AddRoundKey. Nhờ kiến trúc pipeline, nhiều khối dữ liệu có thể được xử lý đồng thời ở các stage khác nhau, giúp tăng thông lượng của hệ thống.

Sau khi dữ liệu đi qua toàn bộ pipeline, tại khoảng thời gian 175 ns, tín hiệu `valid_out` được kích hoạt lên mức logic 1, báo hiệu dữ liệu đầu ra đã hợp lệ. Giá trị ciphertext thu được tại `data_out` là 69c4e0d86a7b0430d8cdb78070b4c55a, hoàn toàn trùng khớp với kết quả chuẩn của thuật toán AES-128 theo bộ kiểm thử NIST.



Hình 4.50: Kết quả dạng sóng mô phỏng cho Testcase1

Kết quả mô phỏng cho thấy dữ liệu tại các tín hiệu stage_data_dbg hoàn toàn trùng khớp với kết quả từng vòng mã hóa trên công cụ phân tích AES. Cụ thể, giá trị tại stage0_data_dbg tương ứng với đầu vào của vòng 1 (Round 1) sau bước thêm khóa vòng (AddRoundKey) ban đầu 00102030405060708090a0b0c0d0e0f0 như hình 4.51.

Encoding Rounds
Round 1
input to Round 1
00102030 40506070 8090a0b0 c0d0e0f0
after S-Box:
63cab704 0953d051 cd60e0e7 ba70e18c
after permutation:
6353e08c 0960e104 cd70b751 bacad0e7
after mult:
5f726415 57f5bc92 f7be3b29 1db9f91a
used subkey:
d6aa74fd d2af72fa daa678f1 d6ab76fe
after mix with key:
89d810e8 855ace68 2d1843d8 cb128fe4

Hình 4.51: Kết quả tính toán từ công cụ kiểm cho các round

Tại stage1_data_dbg: 89d810e8855ace682d1843d8cb128fe4 trùng với kết quả “after mix with key” của Round 1 trên tool thể hiện trên hình 4.51. Tương tự, các stage tiếp theo stage2_data_dbg,... stage10_data_dbg cũng cho kết quả giống với các vòng 2,...vòng 10 của công cụ tham chiếu. Điều này chứng minh dữ liệu đã được truyền tuần tự qua từng stage pipeline và mỗi stage thực hiện một vòng mã hóa AES-128. Cuối cùng, giá trị data_out thu được là: 69c4e0d86a7b0430d8cdb78070b4c55a trùng hoàn toàn với ciphertext chuẩn của AES-128 cũng như kết quả của công cụ kiểm thử trên hình 4.52, xác nhận thiết kế pipeline hoạt động chính xác.

Encoded
69c4e0d8 6a7b0430 d8cdb780 70b4c55a
Reference
69c4e0d8 6a7b0430 d8cdb780 70b4c55a

Hình 4.52: Kết quả tính toán từ công cụ kiểm thử cho ngõ ra

```

TC1 - AES ENCRYPT FIPS
=====
[PASS] TC1 AES ENCRYPT FIPS
Time      : 176000 ns
Data Out  : 69c4e0d86a7b0430d8cdb78070b4c55a

```

Hình 4.53: Kết quả mô phỏng cho Testcase1 trên Tcl Console

4.7.2.2. Kết quả mô phỏng TC2 – Giải mã AES-128 với vector chuẩn FIPS

Mục tiêu: Test case TC2 được thực hiện nhằm kiểm tra chức năng giải mã AES-128 của lõi thiết kế bằng vector chuẩn. Trong test case này, ciphertext được đưa vào data_in và hệ thống được thiết lập ở chế độ giải mã với enc_dec = 1. Khi valid_out = 1, kết quả data_out được so sánh với plaintext ban đầu để xác nhận quá trình giải mã hoạt động chính xác. Mô phỏng sử dụng bộ dữ liệu như bảng 4.7

Bảng 4.7: Bộ dữ liệu sử dụng trong TC2

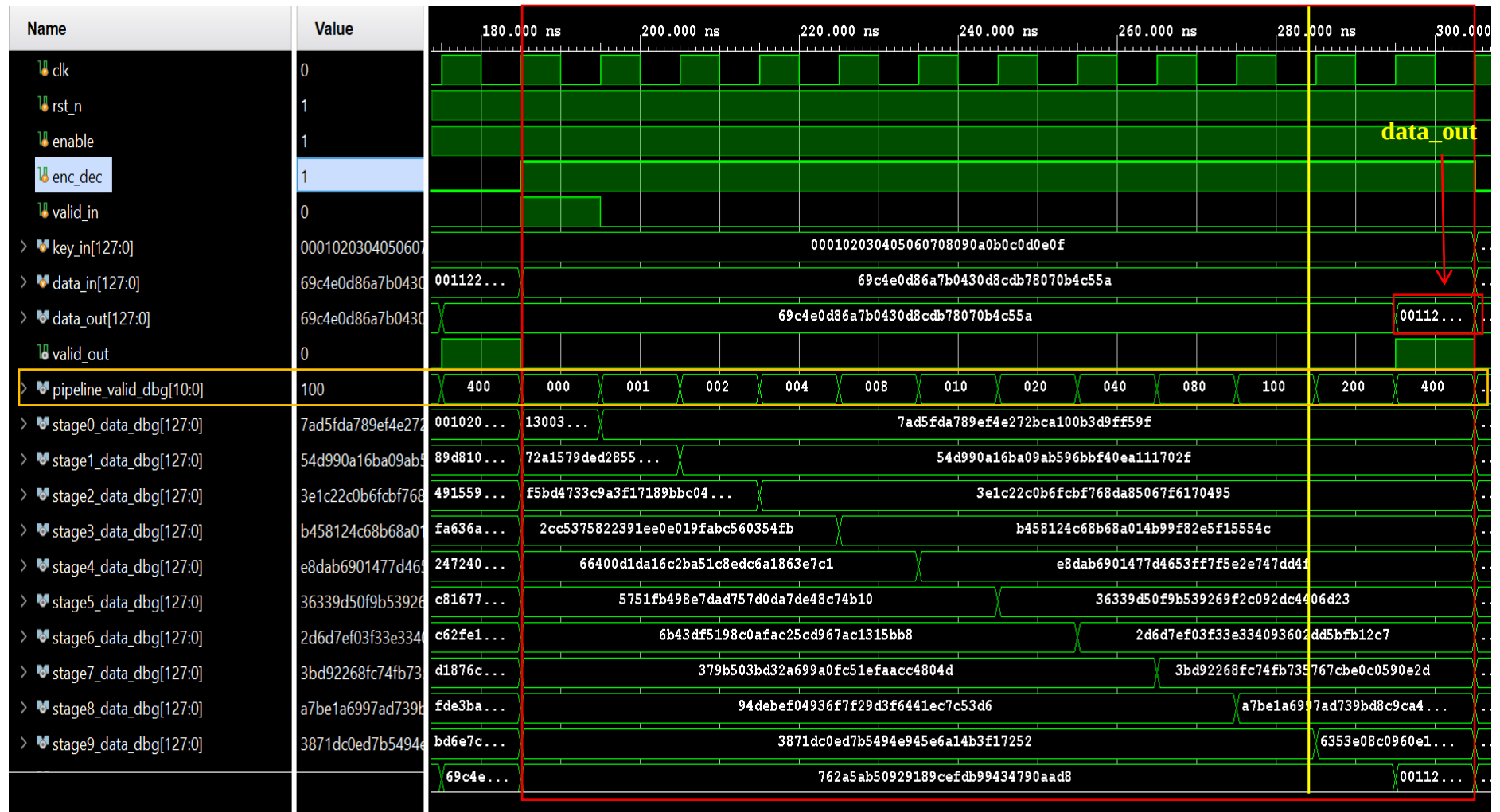
Tín hiệu	Giá trị	Ý nghĩa
Khóa giải mã	000102030405060708090a0b0c0d0e0f	Khóa giải mã 128 bit
Dữ liệu đầu vào	69c4e0d86a7b0430d8cdb78070b4c55a	Dữ liệu mã hóa đưa vào lõi AES
Dữ liệu đầu ra	00112233445566778899aabbccddeeff	Plaintext mong đợi sau khi giải mã

Trình tự thực hiện mô phỏng

Ở Testcase 2, khối aes_core kiểm tra ở chế độ giải mã. Sau khi khóa đã được nạp vào, bản mã 69c4e0d86a7b0430d8cdb78070b4c55a được đưa vào đầu vào data_in. Tín hiệu enc_dec được thiết lập bằng 1 để chọn chế độ giải mã. Sau đó, tín hiệu valid_in được kích hoạt trong một chu kỳ clock để báo rằng dữ liệu đầu vào hợp lệ. Khi tín hiệu valid_out được kích hoạt, khi đó dữ liệu tại data_out được xem là hợp lệ. Kết quả được so sánh với bản rõ 00112233445566778899aabbccddeeff. Nếu hai giá trị trùng khớp, testcase được đánh giá là PASS.

Phân tích dạng sóng

Trong hình 4.54 sau khi testcase 1 hoàn tất và ciphertext đầu ra của quá trình mã hóa trước đó được tạo thành công, dữ liệu này tiếp tục được sử dụng làm đầu vào cho testcase kế tiếp. Hệ thống vẫn hoạt động ổn định với rst_n = 1 và enable = 1, đảm bảo pipeline duy trì trạng thái xử lý liên tục mà không cần reset lại hệ thống. Tại khoảng thời gian 185 ns, tín hiệu valid_in được kích hoạt để đưa block dữ liệu mới vào pipeline. Trong testcase này, khóa mã hóa key_in vẫn được giữ nguyên với giá trị 000102030405060708090a0b0c0d0e0f, trong khi dữ liệu đầu vào data_in là ciphertext thu được từ testcase trước: 69c4e0d86a7b0430d8cdb78070b4c55a.



Hình 4.54: Kết quả dạng sóng mô phỏng cho Testcase2

Quan sát dạng sóng cho thấy tín hiệu pipeline_valid_dbg tiếp tục thay đổi tuần tự theo dạng dịch bit 001 → 002 → 004 → 008 → 010 → 020 → 040 → 080 → 100 → 200 → 400, thể hiện dữ liệu đang được lan truyền lần lượt qua các stage của pipeline. Đồng thời, các tín hiệu stage0_data_dbg đến stage9_data_dbg liên tục thay đổi giá trị sau mỗi chu kỳ clock, phản ánh hoạt động xử lý song song của kiến trúc pipeline AES. Trong suốt quá trình mô phỏng, nhiều block dữ liệu khác nhau tồn tại đồng thời tại các stage khác nhau, cho thấy pipeline hoạt động liên tục và không bị gián đoạn giữa các block dữ liệu. Sau khi hoàn tất quá trình xử lý qua toàn bộ pipeline, tại khoảng thời gian 295 ns, tín hiệu valid_out được kích hoạt lên mức logic 1, báo hiệu dữ liệu đầu ra đã hợp lệ. Giá trị tại data_out thu được là: 00112233445566778899aabbccddeeff. So sánh với kết quả từng vòng, ví dụ vòng 10 của của công cụ mô phỏng thể hiện ở hình 4.55 với stage1_data_dbg[127:0] của dạng sóng thể hiện ở hình 4.54 là hoàn toàn trùng khớp. Giá trị giải mã của công cụ mô phỏng ở hình 4.56 và kết quả của mô phỏng trên Vivado ở hình 4.57 cũng hoàn toàn trùng khớp.

Round 10	
input to Round 10	
7ad5fda7 89ef4e27 2bca100b 3d9ff59f	
after permutation:	ON
7a9f1027 89d5f50b 2beffd9f 3dca4ea7	
after S-Box:	ON
bd6e7c3d f2b5779e 0b61216e 8b10b689	
used subkey:	
549932d1 f0855768 1093ed9c be2c974e	
after mix with key:	ON
e9f74eec 023020f6 1bf2ccf2 353c21c7	
after mult:	ON
54d990a1 6ba09ab5 96bbf40e a111702f	

Hình 4.55: Kết quả tính toán từ công cụ kiểm cho round 10

Decoded
00112233 44556677 8899aabb ccddeeff

Hình 4.56: Kết quả tính toán từ công cụ kiểm thử cho ngõ ra

```

TC2 - AES DECRYPT FIPS
=====
[PASS] TC2 AES DECRYPT FIPS
Time      : 296000 ns
Data Out  : 00112233445566778899aabbccddeeff

```

Hình 4.57: Kết quả mô phỏng cho Testcase2 trên Tcl Console

4.7.2.3. Kết quả mô phỏng TC3 – Mã hóa AES-128 với dữ liệu người dùng

Mục tiêu: Kiểm tra khả năng mã hóa dữ liệu 128-bit tùy chọn của khối aes_core, không chỉ giới hạn ở bộ dữ liệu chuẩn FIPS. Trong testcase này, dữ liệu đầu vào và khóa được xây dựng dưới dạng chuỗi ký tự ASCII.

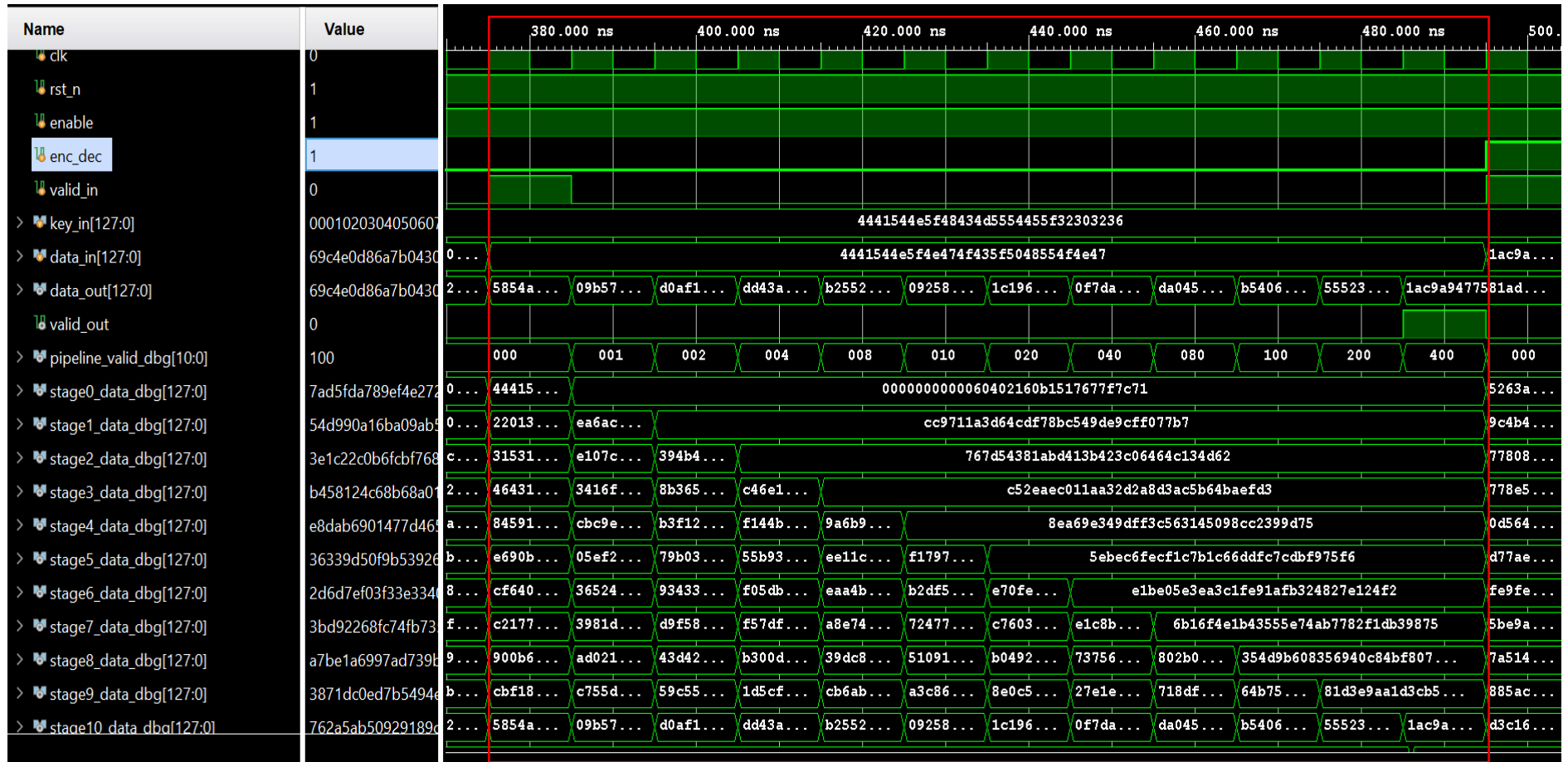
Bảng 4.8: Bộ dữ liệu sử dụng trong TC3

Nội dung	Giá trị ASCII	Giá trị Hex 128-bit
Khóa mã hóa	DATN_HCMUTE_2026	4441544e5f48434d5554455f32303236
Dữ liệu bản rõ	DATN_NGOC_PHUONG	4441544e5f4e474f435f5048554f4e47
Kết quả mong đợi	Bản mã 128-bit	1ac9a9477581adbb246e19b00af0c640

Trình tự thực hiện mô phỏng

Hệ thống được reset trước khi thực hiện mô phỏng để đưa Khối về trạng thái ban đầu. Sau đó, khóa DATN_HCMUTE_2026 được nạp vào Khối AES-128 pipeline. Dữ liệu bản rõ DATN_NGOC_PHUONG được đưa vào đầu vào data_in, đồng thời tín hiệu enc_dec được đặt bằng 0 để chọn chế độ mã hóa. Khi valid_in được kích hoạt trong một chu kỳ clock, dữ liệu đầu vào được ghi nhận và bắt đầu đi qua các tầng pipeline. Testbench chờ đến khi valid_out được đưa lên mức 1, sau đó giá trị tại data_out được ghi nhận là bản mã sau mã hóa. Bản mã này được lưu vào biến cipher_temp để sử dụng cho testcase giải mã tiếp theo.

Phân tích dạng sóng



Hình 4.58: Kết quả dạng sóng mô phỏng cho Testcase3

Trong hình 4.58, hệ thống AES-128 pipeline tiếp tục được mô phỏng nhằm đánh giá khả năng xử lý dữ liệu với bộ plaintext mới trong điều kiện pipeline hoạt động liên tục. Trong suốt quá trình mô phỏng, hệ thống duy trì trạng thái ổn định với $rst_n = 1$ và $enable = 1$, cho phép dữ liệu được đưa liên tục vào pipeline mà không cần reset lại hệ thống. Đồng thời, tín hiệu $enc_dec = 1$ cho biết hệ thống đang hoạt động ở chế độ mã hóa AES-128.

Tại khoảng thời gian 375 ns, tín hiệu $valid_in$ được kích hoạt để đưa block dữ liệu mới vào hệ thống. Trong testcase này, khóa mã hóa key_in vẫn được giữ nguyên với giá trị 000102030405060708090a0b0c0d0e0f, trong khi dữ liệu đầu vào $data_in$ được thay đổi thành 4441544e5f4e474f435f5048554f4e47, tương ứng với chuỗi ký tự ASCII “DATN_NGOC_PHUONG”. Sau khi dữ liệu được đưa vào pipeline, tín hiệu $pipeline_valid_dbg$ tiếp tục thay đổi tuần tự theo dạng dịch bit $001 \rightarrow 002 \rightarrow 004 \rightarrow 008 \rightarrow 010 \rightarrow 020 \rightarrow 040 \rightarrow 080 \rightarrow 100 \rightarrow 200 \rightarrow 400$, cho thấy dữ liệu đang được lan truyền lần lượt qua từng stage xử lý của kiến trúc pipeline AES.

Quan sát các tín hiệu $stage0_data_dbg$ đến $stage9_data_dbg$ cho thấy dữ liệu tại từng stage liên tục thay đổi sau mỗi chu kỳ clock, phản ánh quá trình thực hiện các phép biến đổi của thuật toán AES như SubBytes, ShiftRows, MixColumns và AddRoundKey tại từng round xử lý. Nhờ cơ chế pipeline, nhiều block dữ liệu có thể được xử lý đồng thời tại các stage khác nhau, giúp nâng cao thông lượng và hiệu năng của hệ thống. Sau khi dữ liệu đi qua toàn bộ pipeline, tại khoảng thời gian 485 ns, tín hiệu $valid_out$ được kích hoạt lên mức logic 1, báo hiệu dữ liệu đầu ra đã hợp lệ. Giá trị ciphertext thu được tại $data_out$ là 1ac9a9477581adbb246e19b00af0c640.

Kết quả mô phỏng cho thấy dữ liệu tại các tín hiệu $stage_data_dbg$ hoàn toàn trùng khớp với kết quả từng vòng mã hóa trên công cụ phân tích AES Online. Cụ thể, giá trị tại $stage0_data_dbg$ tương ứng với đầu vào của vòng 1 (Round 1) sau bước thêm khóa vòng (AddRoundKey) ban đầu 00000000000060402160b1517677f7c71 như hình 4.59.

Round 1			
input to Round 1			
00000000	00060402	160b1517	677f7c71
after S-Box:			
63636363	636ff277	472b59f0	85d210a3
after permutation:			
636f59a3	632b1063	47d26377	8563f2f0
after mult:			
8df540ce	c866cd58	f72aca96	b6be12fe
used subkey:			
4162516d	1e2a1220	4b7e577f	794e6549
after mix with key:			
cc9711a3	d64cdf78	bc549de9	cff077b7

Hình 4.59: Kết quả tính toán từ công cụ kiểm cho các round

Tại stage1_data_dbg: cc9711a3d64cdf78bc549de9cff077b7 trùng với kết quả “after mix with key” của Round 1 trên tool AES thể hiện trên hình 4.60 . Tương tự, các stage tiếp theo stage2_data_dbg, stage3_data_dbg, ... stage10_data_dbg cũng cho kết quả giống với các vòng Round 2, Round 3, ... của công cụ tham chiếu. Điều này chứng minh dữ liệu đã được truyền tuần tự qua từng stage pipeline và mỗi stage thực hiện đúng một vòng mã hóa AES-128. Cuối cùng, giá trị data_out thu được là: 1ac9a9477581adbb246e19b00af0c640 trùng hoàn toàn với ciphertext chuẩn của AES-128 cũng như kết quả của công cụ kiểm thử trên hình 4.60, xác nhận thiết kế pipeline hoạt động chính xác.

Encoded			
1ac9a947	7581adbb	246e19b0	0af0c640

Hình 4.60: Kết quả tính toán từ công cụ kiểm cho ngõ ra

```

TC3 - DATN ENCRYPT
=====
[PASS] TC3 DATN ENCRYPT
KEY           : DATN_HCMUTE_2026
PLAINTEXT    : DATN_NGOC_PHUONG
CIPHERTEXT   : 1ac9a9477581adbb246e19b00af0c640
Time         : 486000 ns

```

Hình 4.61: Kết quả mô phỏng cho Testcase3 trên TCL Console

4.7.2.4. Kết quả mô phỏng TC4 – Giải mã AES-128 với dữ liệu cá nhân

Mục tiêu: Kiểm tra khả năng giải mã của Khối AES-128 pipeline bằng cách sử dụng bản mã thu được từ Testcase 3 làm dữ liệu đầu vào. Kết quả mong đợi là Khối giải mã chính xác và khôi phục lại đúng bản rõ ban đầu DATN_NGOC_PHUONG, qua đó xác nhận tính đúng đắn của quá trình mã hóa và giải mã.

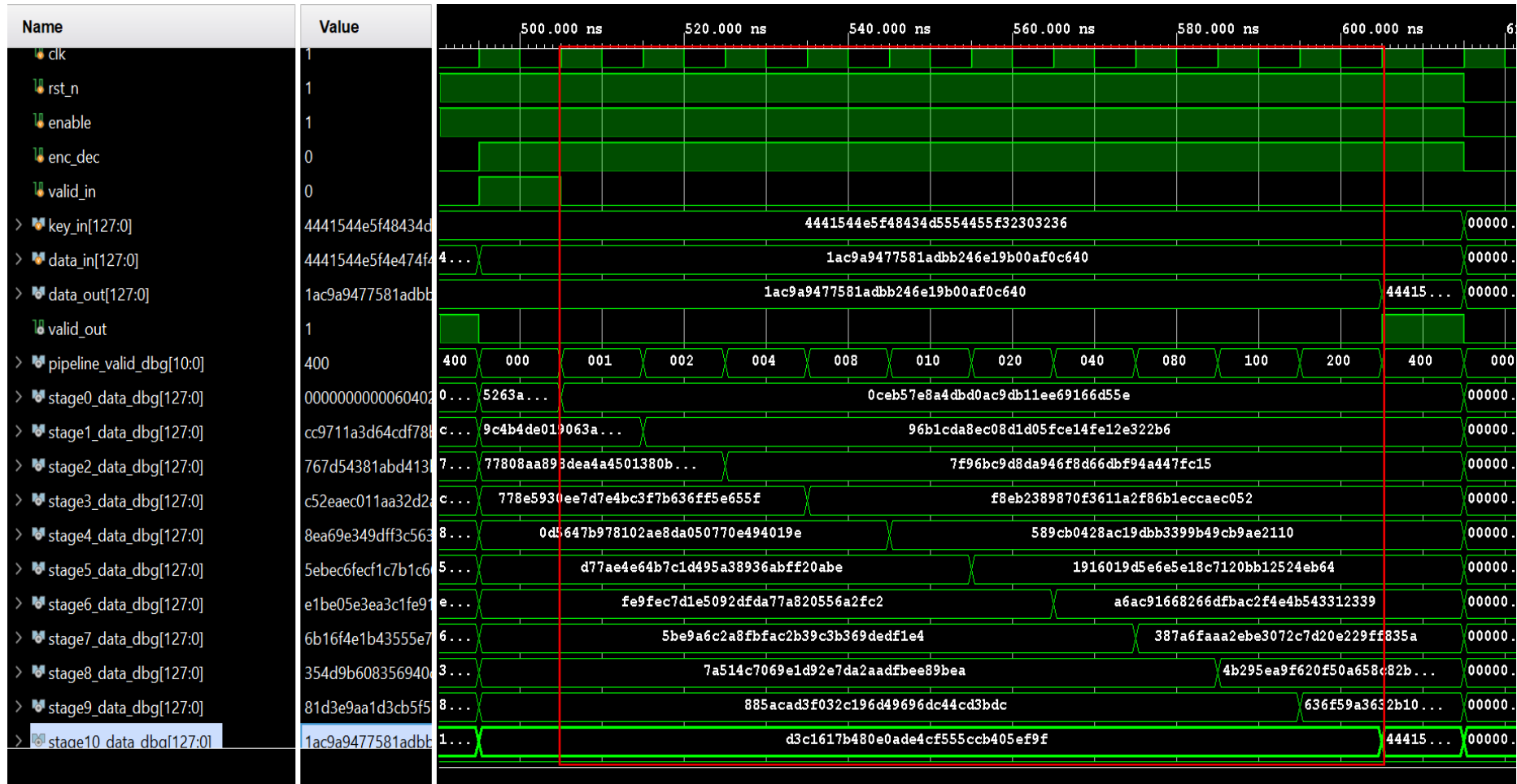
Bảng 4.9. Bộ dữ liệu sử dụng trong TC4

Nội dung	Giá trị ASCII	Giá trị Hex 128-bit
Khóa giải mã	DATN_HCMUTE_2026	4441544e5f48434d5554455f32303236
Dữ liệu bản mã đầu vào	Kết quả mã hóa từ Testcase 3	1ac9a9477581adbb246e19b00af0c640
Kết quả mong đợi	DATN_NGOC_PHUONG	4441544e5f4e474f435f5048554f4e47

Trình tự thực hiện mô phỏng

Khối tiếp tục sử dụng khóa DATN_HCMUTE_2026 đã được nạp trước đó. Bản mã thu được từ Testcase 3, được lưu trong biến cipher_temp, được đưa trở lại Khối thông qua đầu vào data_in. Tín hiệu enc_dec được thiết lập bằng 1 để chọn chế độ giải mã. Sau khi valid_in được kích hoạt, dữ liệu bản mã được đưa vào pipeline để xử lý. Khi tín hiệu valid_out được kích hoạt, kết quả tại data_out được so sánh với dữ liệu bản rõ ban đầu DATN_NGOC_PHUONG. Nếu kết quả sau giải mã trùng khớp với bản rõ ban đầu, testcase được đánh giá là PASS.

Phân tích dạng sóng



Hình 4.62: Kết quả dạng sóng mô phỏng cho Testcase4

Trong hình 4.62, hệ thống AES-128 pipeline được mô phỏng ở chế độ giải mã nhằm đánh giá khả năng khôi phục lại plaintext từ ciphertext đã được mã hóa trước đó. Trong suốt quá trình mô phỏng, hệ thống vẫn hoạt động ổn định với $rst_n = 1$ và $enable = 1$. Khác với các testcase trước, tại testcase này tín hiệu $enc_dec = 1$, cho biết hệ thống đang hoạt động ở chế độ giải mã AES-128. Tại khoảng thời gian 505 ns, tín hiệu $valid_in$ được kích hoạt để đưa dữ liệu đầu vào vào pipeline. Khi đó, khóa giải mã key_in được nạp với giá trị 4441544e5f48434d5554455f32333032336, trong khi dữ liệu đầu vào $data_in$ là ciphertext 1ac9a477581adbb246e19b00af0c640. Đây chính là ciphertext thu được từ quá trình mã hóa ở testcase trước. Sau khi dữ liệu được đưa vào hệ thống, tín hiệu $pipeline_valid_dbg$ tiếp tục thay đổi tuần tự theo dạng dịch bit 001 $\rightarrow$ 002 $\rightarrow$ 004 $\rightarrow$ 008 $\rightarrow$ 010 $\rightarrow$ 020 $\rightarrow$ 040 $\rightarrow$ 080 $\rightarrow$ 100 $\rightarrow$ 200 $\rightarrow$ 400, cho thấy dữ liệu đang được lan truyền qua từng stage của pipeline giải mã.

Quan sát các tín hiệu $stage0_data_dbg$ đến $stage9_data_dbg$ cho thấy dữ liệu tại từng stage liên tục thay đổi theo từng chu kỳ clock. Các phép biến đổi ngược của thuật toán AES gồm InvSubBytes, InvShiftRows, InvMixColumns và AddRoundKey được thực hiện lần lượt tại các round xử lý. Nhờ kiến trúc pipeline, nhiều block dữ liệu vẫn có thể được xử lý đồng thời ở các stage khác nhau, giúp hệ thống duy trì khả năng xử lý tốc độ cao ngay cả ở chế độ giải mã. Sau khi dữ liệu đi qua toàn bộ pipeline, tại khoảng thời gian gần 605 ns, tín hiệu $valid_out$ được kích hoạt lên mức logic 1, báo hiệu dữ liệu đầu ra đã hợp lệ. Giá trị tại $data_out$ thu được là 4441544e5f4e474f435f5048554f4e47 (DATN_NGOC_PHUONG).

```
-----
TC4 - DATN DECRYPT
=====
[PASS] TC4 DATN DECRYPT
Time      : 606000 ns
Data Out  : 4441544e5f4e474f435f5048554f4e47
```

Hình 4.63: Kết quả mô phỏng cho Testcase3 trên Tcl Console

Kết quả mô phỏng cho thấy dữ liệu tại các tín hiệu $stage_data_dbg$ trùng khớp với kết quả từng vòng giải mã trên công cụ kiểm thử. Cụ thể, giá trị tại $stage0_data_dbg$ tương ứng với đầu vào của vòng 10 (Round 10) sau bước cộng khóa vòng (AddRoundKey) ban đầu 0ceb57e8a4dbd0ac9db11ee69166d55e như hình 4.65.

Decoding Rounds
Round 10
input to Round 10
0ceb57e8 a4dbd0ac 9db11ee6 9166d55e
after permutation:
0c661eac a4ebd5e6 9ddb575e 91b1d0e8
after S-Box:
81d3e9aa 1d3cb5f5 759fda9d ac5660c8
used subkey:
1bd8ac3c c77883b8 68857a41 22491448
after mix with key:
9a0b4596 da44364d 1d1aa0dc 8e1f7480
after mult:
96b1cda8 ec08d1d0 5fce14fe 12e322b6

Hình 4.64: Kết quả tính toán từ công cụ kiểm thử cho các round

Tại stage1_data_dbg: 96b1cda8ec08d1d05fce14fe12e322b6 trùng với kết quả “after mult” của Round 10 trên tool AES thể hiện trên hình 4.64. Tương tự, các stage tiếp theo stage2_data_dbg, stage3_data_dbg, ... stage10_data_dbg cũng cho kết quả giống với các vòng Round 2, Round 3, ... của công cụ tham chiếu. Điều này chứng minh dữ liệu đã được truyền tuần tự qua từng stage pipeline và mỗi stage thực hiện đúng một vòng mã hóa AES-128. Cuối cùng, giá trị data_out thu được là: 4441544e5f4e474435f5048554f4e47 trùng hoàn toàn với ciphertext chuẩn của AES-128 cũng như kết quả của công cụ kiểm thử trên hình 4.65, xác nhận thiết kế pipeline hoạt động chính xác.

Decoded
4441544e 5f4e474f 435f5048 554f4e47

Hình 4.65: Kết quả tính toán từ công cụ kiểm cho ngõ ra

4.7.2.5 Kết quả mô phỏng TC5 – Kiểm tra chức năng hold pipeline bằng enable

Mục tiêu: Testcase TC5 được thực hiện nhằm kiểm tra khả năng tạm dừng và tiếp tục hoạt động của khối AES-128 pipeline thông qua tín hiệu enable. Khi enable = 0, pipeline phải giữ nguyên trạng thái hiện tại, không làm mất dữ liệu đang xử lý. Khi

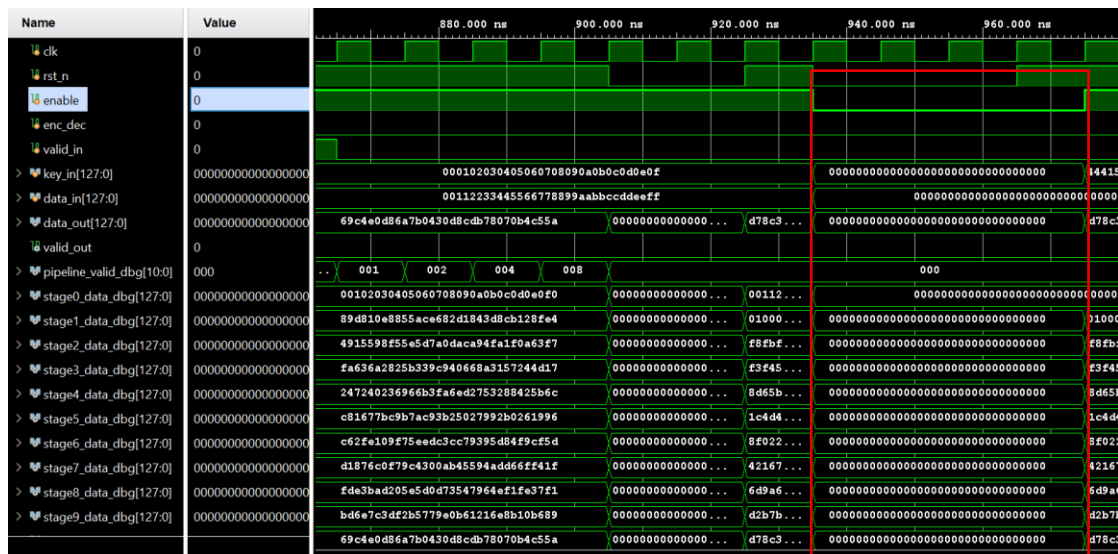
Bên cạnh đó, tín hiệu `data_out` cũng giữ nguyên giá trị trong suốt thời gian `enable = 0`, chứng minh rằng không có dữ liệu mới được xử lý hoặc xuất ra khỏi pipeline. Khi tín hiệu `enable` được đưa trở lại mức 1 tại khoảng 785 ns, pipeline tiếp tục hoạt động bình thường, dữ liệu lại tiếp tục được dịch chuyển qua các stage và tín hiệu `valid` tiếp tục lan truyền trong pipeline. Kết quả mô phỏng cho thấy cơ chế hold/stall của kiến trúc AES pipeline hoạt động chính xác, đảm bảo khả năng tạm dừng xử lý mà không làm mất dữ liệu đang tồn tại trong hệ thống.

4.7.2.6. Kết quả mô phỏng TC6 – Kiểm tra reset giữa quá trình pipeline

Mục tiêu: Testcase TC6 được thực hiện nhằm kiểm tra khả năng xử lý của khối `aes-core` khi tín hiệu `reset` được kích hoạt trong lúc hệ thống đang xử lý dữ liệu. Mục tiêu của testcase là xác nhận rằng khi `reset` xảy ra giữa quá trình pipeline, các dữ liệu đang xử lý không tiếp tục được xuất ra sai lệch, đồng thời các tín hiệu đầu ra của hệ thống được đưa về trạng thái an toàn. Cụ thể, sau khi tín hiệu `rst_n` được kéo xuống mức 0, giá trị `data_out` phải được xóa về 0 và tín hiệu `valid_out` phải ở mức 0. Điều này cho thấy pipeline đã được reset đúng cách và không còn dữ liệu không hợp lệ tồn tại ở đầu ra.

Trình tự thực hiện mô phỏng: Dữ liệu bản rõ FIPS được đưa vào Khối ở chế độ mã hóa. Sau khi dữ liệu đã đi vào pipeline trong một số chu kỳ clock nhưng chưa tạo ra kết quả đầu ra hợp lệ, tín hiệu `reset rst_n` được kích hoạt về mức 0. Mục đích của thao tác này là kiểm tra khả năng xóa trạng thái xử lý khi `reset` xảy ra giữa quá trình pipeline. Trong thời gian `reset`, các giá trị đầu ra của Khối được kiểm tra. Kết quả mong đợi là `data_out` được đưa về giá trị 0 và `valid_out` ở mức 0, cho thấy không có dữ liệu đầu ra hợp lệ sau khi `reset`. Nếu Khối xóa đúng dữ liệu đang xử lý và không phát sinh output sai, testcase được đánh giá là PASS.

Phân tích dạng sóng



Hình 4.67: Kết quả dạng sóng mô phỏng cho Testcase6

Hình 4.67 thể hiện kết quả mô phỏng của testcase TC6 dùng để kiểm tra khả năng reset hệ thống trong khi pipeline AES đang hoạt động. Ban đầu, hệ thống có $rst_n = 1$, $enable = 1$, $data_in = 00112233445566778899aabbccddeeff$ và $key = 000102030405060708090a0b0c0d0e0f$. Dữ liệu được lan truyền tuần tự qua các stage pipeline, thể hiện qua sự thay đổi liên tục của các tín hiệu stage0_data_dbg đến stage10_data_dbg. Đồng thời, tín hiệu pipeline_valid_dbg thay đổi theo dạng $001 \rightarrow 002 \rightarrow 004 \rightarrow 008$, cho thấy trạng thái valid đang được truyền qua các tầng pipeline. Kết quả đầu ra data_out xuất hiện đúng ciphertext chuẩn AES là 69c4e0d86a7b0430d8cdb78070b4c55a.

Tại khoảng thời gian gần 900 ns, tín hiệu rst_n được đưa xuống mức 0 trong khi pipeline vẫn đang chứa dữ liệu xử lý. Ngay sau đó, toàn bộ dữ liệu tại các stage pipeline đều bị xóa về giá trị 0, tín hiệu pipeline_valid_dbg trở về 000, đồng thời data_out và valid_out cũng trở về trạng thái không hợp lệ. Điều này cho thấy cơ chế reset đã tác động đồng thời lên toàn bộ các stage của pipeline và xóa hoàn toàn trạng thái xử lý hiện tại.

Sau khi tín hiệu rst_n trở lại mức 1, hệ thống khởi động lại bình thường. Dữ liệu mới tiếp tục được nạp vào pipeline và các stage bắt đầu hoạt động trở lại theo đúng nguyên lý pipeline AES. Kết quả mô phỏng chứng minh cơ chế reset giữa quá trình xử lý hoạt động chính xác, đảm bảo hệ thống có thể phục hồi ổn định và không giữ lại dữ liệu cũ sau khi reset.

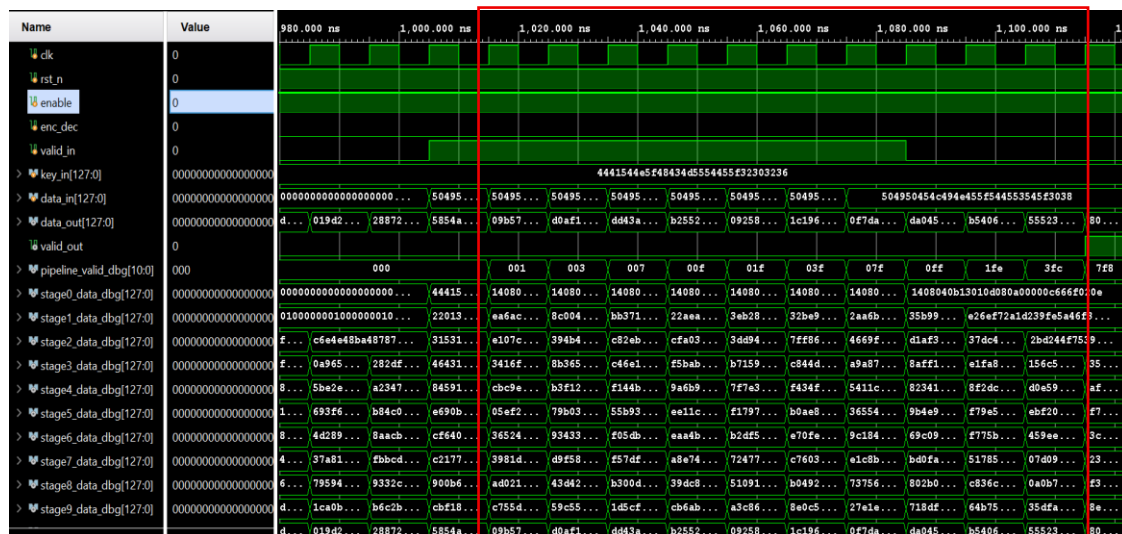
4.7.2.7. Kết quả mô phỏng TC7 – Kiểm tra throughput của pipeline

Mục tiêu: TC7 nhằm kiểm tra khả năng tăng thông lượng xử lý của kiến trúc AES-128 pipeline. Sau khi pipeline được lấp đầy, hệ thống được kỳ vọng xuất ra 1 block dữ liệu sau mỗi chu kỳ clock, từ đó chứng minh pipeline giúp xử lý dữ liệu liên tục và nhanh hơn so với kiến trúc tuần tự.

Trình tự thực hiện mô phỏng: Khối được reset và nạp khóa. Sau đó, Khối được thiết lập ở chế độ mã hóa với enc_dec = 0. Tín hiệu valid_in được giữ ở mức 1 để cho phép đưa dữ liệu liên tiếp vào pipeline. Tám khối dữ liệu lần lượt được đưa vào Khối, mỗi khối cách nhau một chu kỳ clock. Sau khi toàn bộ dữ liệu đầu vào đã được đưa vào, valid_in được đưa về mức 0. Testbench tiếp tục quan sát tín hiệu valid_out và đếm số lượng output hợp lệ được tạo ra. Nếu Khối tạo đủ 8 kết quả đầu ra và sau khi pipeline đầy, mỗi chu kỳ clock có thể xuất ra một block dữ liệu, testcase được đánh giá là PASS. Với chu kỳ clock 10 ns, thông lượng lý thuyết của thiết kế được tính là 128 bit / 10 ns = 12.8 Gbps.

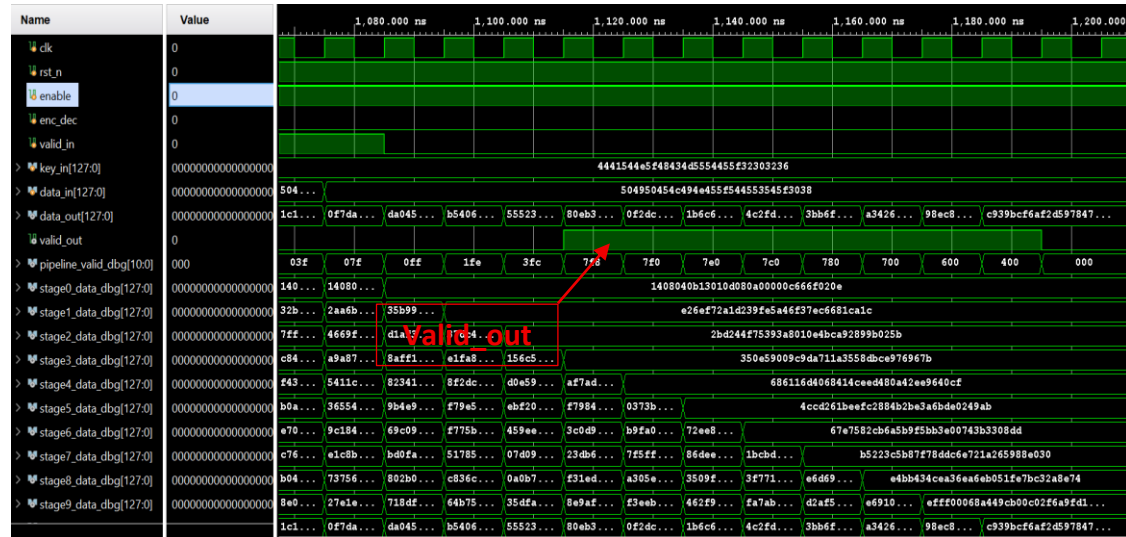
Phân tích dạng sóng

Hình 4.68 và 4.69 thể hiện kết quả mô phỏng testcase kiểm tra thông lượng của kiến trúc AES pipeline khi nhiều block dữ liệu được nạp liên tiếp vào hệ thống. Ban đầu, các block plaintext được đưa vào pipeline liên tục theo từng chu kỳ clock thông qua tín hiệu valid_in tại hình 4.69. Trong giai đoạn đầu, dữ liệu lần lượt lan truyền qua các stage từ stage0_data_dbg đến stage9_data_dbg, trong khi tín hiệu pipeline_valid_dbg tăng dần theo dạng 001 → 003 → 007 → 00F → 01F → 03F → 07F, cho thấy số lượng stage chứa dữ liệu hợp lệ đang tăng lên theo từng chu kỳ clock. Điều này thể hiện quá trình pipeline đang dần được lấp đầy.



Hình 4.68: Kết quả dạng sóng mô phỏng cho Testcase7 trước khi pipeline đầy

Sau khoảng 10 chu kỳ clock, toàn bộ pipeline đã được lấp đầy hoàn toàn như hình 4.69 và bắt đầu hoạt động ở trạng thái ổn định. Từ thời điểm này, tín hiệu valid_out được duy trì ở mức 1 liên tục và tại mỗi cạnh lên của clock đều xuất hiện một ciphertext mới tại data_out. Điều này chứng minh rằng kiến trúc pipeline cho phép xử lý song song nhiều block dữ liệu và đạt được thông lượng rất cao. Mặc dù độ trễ ban đầu của pipeline tương đối lớn do dữ liệu phải đi qua toàn bộ các stage xử lý AES, nhưng sau khi pipeline đầy, hệ thống có thể xuất liên tục một kết quả mã hóa sau mỗi chu kỳ clock.



Hình 4.69: Kết quả dạng sóng mô phỏng cho Testcase7 sau khi pipeline đầy

Ở giai đoạn cuối mô phỏng, khi không còn dữ liệu mới được nạp vào hệ thống, tín hiệu pipeline_valid_dbg giảm dần theo dạng 7F8 → 7F0 → 7E0 → 7C0 → 780 → 700 → 600 → 400 → 000, cho thấy dữ liệu đang lần lượt được đẩy hết ra khỏi pipeline. Kết quả mô phỏng này chứng minh thiết kế AES pipeline hoạt động đúng nguyên lý và đạt được mục tiêu tăng thông lượng bằng cơ chế xử lý song song theo pipeline.

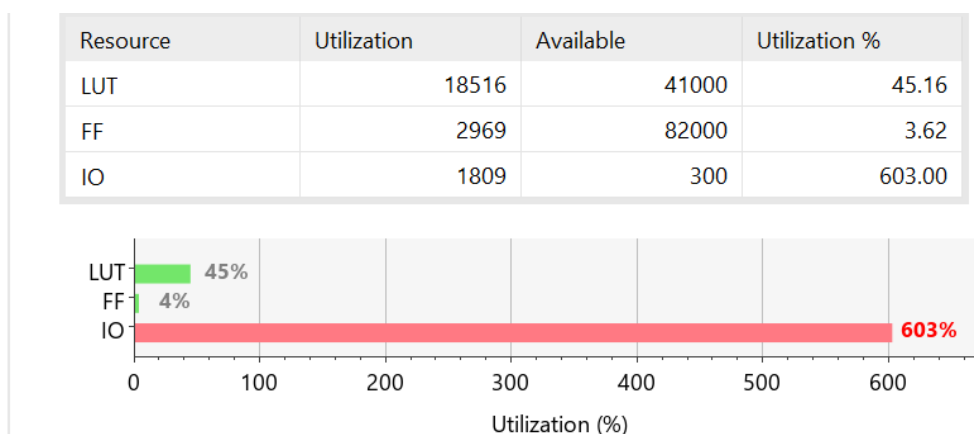
4.7.3. Đánh giá hiệu năng, thông lượng (Throughput) và tần số vận hành tối đa

Đánh giá tài nguyên phần cứng của kiến trúc AES Pipeline

Kết quả tổng hợp phần cứng của kiến trúc AES-128 Pipeline được trình bày trong Hình 4.70. Báo cáo Utilization cho thấy thiết kế sử dụng 18.516 LUT trên tổng số 41.000 LUT khả dụng của FPGA Kintex-7 XC7K70T, tương đương 45,16% tài nguyên logic. Đồng thời, thiết kế sử dụng 2.969 Flip-Flop (FF), chiếm 3,62% tổng số thanh ghi của thiết bị.

Như thể hiện trong Hình 4.70, tài nguyên được sử dụng chủ yếu là LUT. Điều này phản ánh đặc điểm của kiến trúc AES Pipeline khi các vòng xử lý được triển khai bằng phần cứng song song thay vì tái sử dụng cùng một khối xử lý cho nhiều vòng mã

hóa. Các phép biến đổi như SubBytes, ShiftRows, MixColumns và AddRoundKey được ánh xạ trực tiếp thành logic phần cứng, làm tăng số lượng LUT nhưng cho phép nhiều khối dữ liệu được xử lý đồng thời trong các tầng pipeline khác nhau.



Hình 4.70: Kết quả sử dụng tài nguyên FPGA của kiến trúc AES Pipeline

Mặc dù số lượng LUT sử dụng tương đối lớn, tỷ lệ sử dụng thanh ghi chỉ đạt 3,62%. Các Flip-Flop chủ yếu được sử dụng trong các khối pipeline register nhằm lưu trữ dữ liệu giữa các tầng xử lý. Việc bố trí các thanh ghi trung gian giúp chia nhỏ đường dữ liệu tổ hợp, giảm độ trễ trên mỗi chu kỳ clock và cải thiện khả năng đáp ứng timing của hệ thống.

Báo cáo cũng cho thấy thiết kế không sử dụng Block RAM và DSP. Điều này chứng tỏ toàn bộ thuật toán AES được hiện thực bằng tài nguyên logic của FPGA mà không cần đến các khối phần cứng chuyên dụng. Cách tiếp cận này giúp thiết kế có tính linh hoạt cao và thuận lợi cho việc mở rộng trong các nghiên cứu tiếp theo.

Ngoài ra, số lượng I/O được báo cáo là 1.809 chân, vượt quá số chân vật lý của FPGA mục tiêu. Nguyên nhân là do trong giai đoạn đánh giá và mô phỏng, các tín hiệu debug của từng tầng pipeline được đưa ra ngoài module để phục vụ việc quan sát và phân tích hoạt động của hệ thống. Do đó, giá trị I/O trong báo cáo không phản ánh cấu hình triển khai thực tế trên FPGA mà chỉ phục vụ mục đích kiểm chứng thiết kế.

Từ kết quả trên có thể thấy kiến trúc AES Pipeline đã khai thác hiệu quả tài nguyên logic của FPGA để đổi lấy khả năng xử lý song song và thông lượng cao. Mặc dù sử dụng 45,16% số lượng LUT hiện có, thiết kế vẫn còn hơn 54% tài nguyên logic và hơn 96% tài nguyên thanh ghi chưa được sử dụng, cho thấy khả năng mở rộng và tích hợp thêm các chức năng bảo mật khác trong tương lai.

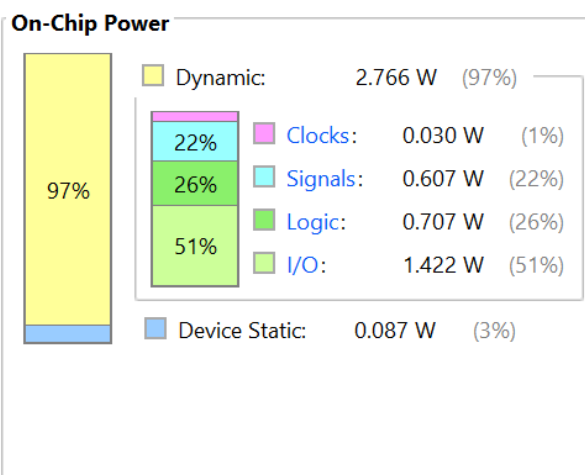
Đánh giá công suất tiêu thụ của kiến trúc AES Pipeline

Kết quả phân tích công suất sau quá trình tổng hợp được trình bày trong Hình 4.71. Báo cáo cho thấy tổng công suất tiêu thụ trên chip (Total On-Chip Power) của

thiết kế đạt 2.853W. Trong đó, công suất động (Dynamic Power) chiếm 2.766W, tương đương 97% tổng công suất tiêu thụ, trong khi công suất tĩnh (Device Static Power) chỉ chiếm 0.087W, tương đương 3%. Điều này cho thấy phần lớn năng lượng được sử dụng cho hoạt động chuyển mạch của các khối logic và tín hiệu dữ liệu trong quá trình xử lý AES.

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	2.853 W
Design Power Budget:	Not Specified
Power Budget Margin:	N/A
Junction Temperature:	30.4°C
Thermal Margin:	54.6°C (28.8 W)
Effective θ_{JA} :	1.9°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low



Hình 4.71: Kết quả phân tích công suất tiêu thụ của kiến trúc AES Pipeline

Phân tích chi tiết công suất động cho thấy thành phần tiêu thụ lớn nhất là khối I/O với mức công suất 1.422 W, chiếm 51% tổng công suất động. Tiếp theo là khối Logic tiêu thụ 0,707 W, tương đương 26%, và khối Signals tiêu thụ 0.607 W, tương đương 22%. Trong khi đó, mạng Clock chỉ tiêu thụ 0,030 W, chiếm khoảng 1% tổng công suất động. Kết quả này cho thấy mức tiêu thụ năng lượng chủ yếu đến từ hoạt động truyền dữ liệu và xử lý logic bên trong kiến trúc pipeline.

Cần lưu ý rằng giá trị công suất I/O tương đối cao là do trong quá trình đánh giá thiết kế, nhiều tín hiệu debug của các tầng pipeline được đưa ra ngoài module top-level nhằm phục vụ việc quan sát và phân tích hoạt động của hệ thống. Do đó, công suất I/O trong báo cáo hiện tại lớn hơn đáng kể so với trường hợp triển khai thực tế trên FPGA. Khi tích hợp vào hệ thống hoàn chỉnh và chỉ giữ lại các tín hiệu giao tiếp cần thiết, mức công suất này sẽ giảm đáng kể.

Báo cáo cũng cho thấy nhiệt độ môi nối (Junction Temperature) của FPGA được ước lượng ở mức 30,4°C với Thermal Margin đạt 54,6°C. Điều này cho thấy thiết kế hoạt động trong vùng nhiệt độ an toàn và chưa gây ra áp lực đáng kể lên hệ thống tản nhiệt của FPGA. Mặc dù kiến trúc AES Pipeline sử dụng số lượng phần cứng lớn hơn để đạt được khả năng xử lý song song và thông lượng cao, tổng công suất tiêu thụ của hệ thống vẫn nằm trong giới hạn cho phép của FPGA mục tiêu. Kết quả này cho thấy

kiến trúc pipeline không chỉ cải thiện hiệu năng xử lý mà còn duy trì được mức tiêu thụ năng lượng hợp lý đối với các ứng dụng mã hóa dữ liệu tốc độ cao trên nền tảng FPGA.

Tuy nhiên, cần lưu ý rằng báo cáo công suất được tạo từ Synthesized Netlist và có mức độ tin cậy (Confidence Level) ở mức thấp do chưa sử dụng dữ liệu chuyển mạch thực tế từ mô phỏng hoặc triển khai vật lý. Vì vậy, các giá trị công suất trên chủ yếu mang tính chất ước lượng và có thể thay đổi sau quá trình Implementation hoặc khi hệ thống được triển khai trên phần cứng thực tế.

Đánh giá khả năng đáp ứng thời gian của kiến trúc AES Pipeline

Kết quả phân tích timing của kiến trúc AES Pipeline được trình bày trong Hình 4.72 Báo cáo Timing Summary cho thấy tất cả các ràng buộc thời gian của thiết kế đều được đáp ứng, thể hiện qua thông báo “All user specified timing constraints are met”. Điều này chứng tỏ hệ thống có thể hoạt động ổn định tại tần số mục tiêu đã được thiết lập trong quá trình tổng hợp và phân tích.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.121 ns	Worst Hold Slack (WHS): 0.070 ns	Worst Pulse Width Slack (WPWS): 9.650 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 2970	Total Number of Endpoints: 2970	Total Number of Endpoints: 2970

All user specified timing constraints are met.

Hình 4.72: Kết quả phân tích timing của kiến trúc AES Pipeline

Giá trị Worst Negative Slack (WNS) đạt 3.121 ns, lớn hơn 0, cho thấy đường dữ liệu tới hạn vẫn còn dự trữ thời gian trước khi vi phạm ràng buộc clock. Đồng thời, Total Negative Slack (TNS) bằng 0 ns và số lượng endpoint vi phạm timing bằng 0, chứng tỏ không tồn tại bất kỳ đường dữ liệu nào không đáp ứng được yêu cầu thời gian của hệ thống.

Kết quả Hold Analysis cũng cho thấy thiết kế không xuất hiện lỗi giữ dữ liệu khi giá trị Worst Hold Slack (WHS) đạt 0,070 ns và Total Hold Slack (THS) bằng 0 ns. Bên cạnh đó, các yêu cầu về độ rộng xung clock đều được đáp ứng với Worst Pulse Width Slack (WPWS) đạt 9,650 ns và không có endpoint nào vi phạm điều kiện này.

Đối với thiết kế được cấu hình hoạt động ở tần số 100 MHz, giá trị WNS dương cho thấy kiến trúc pipeline đã giúp chia nhỏ đường dữ liệu tổ hợp thành nhiều tầng xử lý độc lập. Nhờ các thanh ghi pipeline được bố trí giữa các vòng AES, độ trễ trên mỗi chu kỳ clock được giảm đáng kể, cho phép hệ thống đáp ứng yêu cầu thời gian ở tần số

cao. Đây là một trong những ưu điểm quan trọng của kiến trúc pipeline so với các phương pháp xử lý tuần tự truyền thống.

Từ kết quả phân tích timing có thể khẳng định rằng kiến trúc AES Pipeline không chỉ đáp ứng được tần số hoạt động mục tiêu 100 MHz mà còn vẫn còn dự trữ thời gian xử lý, tạo điều kiện thuận lợi cho việc mở rộng hoặc tích hợp thêm các chức năng trong tương lai mà không ảnh hưởng đáng kể đến khả năng đáp ứng timing của hệ thống.

4.7.4. Tổng hợp kết quả các test case

Sau khi thực hiện mô phỏng các test case, kết quả cho thấy khối AES-128 pipeline hoạt động đúng theo yêu cầu thiết kế. Các chức năng mã hóa, giải mã, loopback, hold pipeline bằng tín hiệu enable, reset giữa quá trình xử lý và kiểm tra throughput đều cho kết quả đạt.

Bảng 4.10. Tổng hợp kết quả mô phỏng các test case

STT	Mã test case	Nội dung kiểm tra	Kết quả mong đợi	Kết quả
1	TC1	Mã hóa AES-128 theo chuẩn FIPS	Ciphertext đầu ra trùng với giá trị chuẩn	PASS
2	TC2	Giải mã AES-128 theo chuẩn FIPS	Plaintext đầu ra trùng với dữ liệu ban đầu	PASS
3	TC3	Mã hóa dữ liệu DATN	Tạo ra ciphertext hợp lệ từ dữ liệu đầu vào	PASS
4	TC4	Giải mã dữ liệu DATN	Khôi phục đúng plaintext ban đầu	PASS
5	TC5	Hold pipeline bằng tín hiệu enable	Pipeline tạm dừng và tiếp tục xử lý đúng	PASS
6	TC6	Reset giữa quá trình pipeline	data_out = 0 và valid_out = 0 sau reset	PASS
7	TC7	Throughput của pipeline	Sau khi pipeline đầy, mỗi clock xuất ra 1 block	PASS

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Đồ án đã hoàn thành mục tiêu nghiên cứu, thiết kế và kiểm chứng thành công lõi IP mã hóa và giải mã AES-128 bằng ngôn ngữ Verilog HDL. Thông qua quá trình phân tích thuật toán, thiết kế RTL và mô phỏng trên Xilinx Vivado, hệ thống đạt được các kết quả chính như sau:

- Xây dựng thành công kiến trúc Pipeline 10 tầng cho cả khối mã hóa và giải mã, cho phép dữ liệu được xử lý liên tục nhằm nâng cao thông lượng và đáp ứng các ứng dụng yêu cầu tốc độ cao, thời gian thực.
- Thiết kế thành công bộ điều khiển FSM để quản lý quá trình hoạt động của Pipeline, đảm bảo dữ liệu và các tín hiệu trạng thái (`valid_in`, `valid_out`) được đồng bộ chính xác theo từng chu kỳ xung nhịp.
- Hoàn thiện hệ thống kiểm thử và xác minh chức năng cho toàn bộ lõi AES-128. Kết quả mô phỏng phần cứng trùng khớp hoàn toàn với mô hình tham chiếu phần mềm, chứng minh tính đúng đắn của các khối chức năng trong quá trình mã hóa và giải mã.

5.2. Hạn chế

Mặc dù đạt được các mục tiêu đề ra, thiết kế hiện tại vẫn còn một số hạn chế:

- Kiến trúc Pipeline triển khai đầy đủ cho cả 10 vòng xử lý giúp tăng hiệu năng nhưng tiêu tốn nhiều tài nguyên phần cứng như LUT và thanh ghi trên FPGA.
- Khóa mã hóa hiện được nạp theo dạng tĩnh và truyền trực tiếp trong hệ thống, chưa tích hợp cơ chế trao đổi khóa an toàn hoặc sinh khóa động theo phiên, do đó vẫn tiềm ẩn nguy cơ rò rỉ khóa khi triển khai trên môi trường thực tế.
- Hệ thống chưa tích hợp các kỹ thuật chống tấn công kênh kề như phân tích công suất hoặc phân tích thời gian trên phần cứng.

5.3. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo các hướng sau:

- Tích hợp cơ chế trao đổi khóa an toàn như RSA, Diffie-Hellman hoặc ECDH để thực hiện sinh và phân phối khóa AES động trước khi truyền dữ liệu, góp phần nâng cao tính bảo mật của hệ thống IoT thực tế.
- Tích hợp thêm cơ chế kiểm tra tính toàn vẹn và xác thực dữ liệu như CRC, MAC hoặc nâng cấp lên chế độ AES-GCM nhằm tăng cường khả năng chống giả mạo và thay đổi dữ liệu trên đường truyền.

- Tích hợp lõi AES-128 vào các hệ thống SoC hoặc các nền tảng nhúng IoT như STM32, ESP32 để xây dựng hệ thống bảo mật dữ liệu hoàn chỉnh.
- Tối ưu tài nguyên phần cứng bằng các kỹ thuật chia sẻ tài nguyên hoặc kiến trúc lặp nhằm giảm diện tích chip và mức tiêu thụ năng lượng cho các thiết bị IoT.

TÀI LIỆU THAM KHẢO

- [1] National Institute of Standards and Technology (NIST), “Advanced Encryption Standard (AES),” FIPS PUB 197, Nov. 2001. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- [2] J. Daemen and V. Rijmen, The Design of Rijndael: AES – The Advanced Encryption Standard. Berlin, Germany: Springer, 2002. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-662-04722-4>
- [3] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. London, U.K.: Pearson, 2017. [Online]. Available: <https://www.pearson.com/en-us/subject-catalog/p/cryptography-and-network-security-principles-and-practice/P200000003015>
- [4] A. Hodjat and I. Verbauwhede, “A 21.54 Gbits/s fully pipelined AES processor on FPGA,” in Proc. 12th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM), Napa, CA, USA, 2004, pp. 308–309. [Online]. Available: <https://ieeexplore.ieee.org/document/1297656>
- [5] N. Pramstaller, C. Rechberger, and V. Rijmen, “A compact high-performance AES implementation on FPGA,” in Proc. International Conference on Information Technology: Coding and Computing (ITCC), Las Vegas, NV, USA, 2004, pp. 583–587. [Online]. Available: <https://ieeexplore.ieee.org/document/1286490>
- [6] D. Harris and S. Harris, Digital Design and Computer Architecture, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2012. [Online]. Available: <https://www.elsevier.com/books/digital-design-and-computer-architecture/harris/978-0-12-394424-5>
- [7] Gokul R and Swarnalatha A, “Pipelined AES-128 Encryption and Decryption Design Using Verilog HDL,” International Journal for Multidisciplinary Research (IJFMR), vol. 7, no. 4, Jul.–Aug. 2025. [Online]. Available: <https://www.ijfmr.com/papers/2025/4/52209.pdf>

DATN-Phuong-Ngoc

ORIGINALITY REPORT

6%	5%	2%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	www.ctu.edu.vn Internet Source	2%
2	Submitted to Ho Chi Minh University of Technology and Education Student Paper	1%
3	tailieu.vn Internet Source	1%
4	mientayvn.com Internet Source	1%
5	Ton Duc Thang University Publication	1%
6	Phenikaa University Publication	1%
7	itamateurblog.wordpress.com Internet Source	1%

Exclude quotes On
Exclude bibliography On

Exclude matches < 1%